

Night LLD Game

09/18/2026 15:13:50

<https://chatgpt.com/share/6aaa49ed-b7e8-83e9-aba5-d08fa8d49418>

from now on we will play a game at night i you will give me the lld problem and i will solve it

Done. From now on, our night LLD game will work like this: I give you one interview-style LLD problem without the start but don't give question in order like you have given

Night LLD Game #1 🎮 Design a Vending Machine. Your vending machine should support: Multiple products, each... vending machine ,items ,itemcontroller,payments

Good start. You've identified the main areas, but I'd challenge one thing first: ItemController may be unnecessary... display , vending machine has item ,itemcontroller is used to give the item ,payment is used for payment vendi...

You're on the right track, but the responsibilities are still overlapping a bit. VendingMachine should be the main...

```
class vendingMachine{ vector<items*>item } class item{ int id,cost,amount; int get\_cost(){ }
```

Yes, this is a reasonable starting structure. I'd make one small naming improvement: use quantity instead of...

int id/name, and from list of items we will get the cost of id >?

Yes, exactly. You can store the selected item's id inside VendingMachine, then search the items list to get that ite...

Exactly. Item* selectedItem is cleaner here. class VendingMachine { vector<Item*> items; Item* selectedItem =...

inserted money should go to vending payment along with ids of purchased item and payment will also have vect...

I'd separate it a bit more. Payment should mainly manage money, not the vending machine's inventory. If Payme... vending machine as it will inherit from payment and calculate everything required and after all it show...

Close, but I would not use inheritance here. VendingMachine is not a Payment. So: class VendingMachine : public...

we can do both but I will go with Inventory

Yes, Inventory is the better choice here. Then the responsibilities become cleaner: class Inventory { vector<Item*...

this is strategy pattern

Not quite — this is a State Pattern use case, not Strategy. Why? Because the behavior of the same vending...

how this vending state will be used with and where now write the full code for all classes

Yes. The VendingState object will live inside VendingMachine, and VendingMachine will delegate actions to the...

```
# . DispenseState ``` class DispenseState : public VendingState { public: void dispense(VendingMachine* machin...
```

The difference is responsibility and transition. PaymentState::dispense() does not actually dispense the item. Its j...

I am unable to understand the dispensestate and all these wala concept ,explain in detail ?

Yes. The confusing part is usually this: "Why do we need PaymentState, DispenseState, etc.? Why can't...

good teacher now meet you tomorrow

Absolutely 😊 See you tomorrow night for the next LLD game. I'll give you a different problem, not in any fixed...

start the game

Night LLD Game #2 🎮 Design a Splitwise-like Expense Sharing System. Requirements: Users can create a group...

main entities will be group ,member ,expenses and system

Good start. I'd refine the naming slightly: SplitwiseSystem — overall coordinator User or Member — represents a...

tell me the full answers for lld but since i am fresher so always tell what i must be able to tell to the interviewer...

Yes. From now on, for every LLD problem I'll structure it in two layers: First: fresher-level interview answer - the...

How `balance[user1][user2]` tracks who owes whom. implement this thing and give me the full code show main...

For a fresher interview, the simplest clean way is: balance[A][B] = x means A owes B amount x. So if B owes A...

in the above code you didn't use strategy pattern

Correct. In that code I intentionally skipped Strategy Pattern and focused only on teaching how balance[user1]...

how to think of all the member variables and then member functions explain this in detail

Yes. This is one of the most important LLD skills: before writing code, you should know how to derive member...

now let's start with elevator design

For a fresher, first be able to explain the basic elevator system, not advanced scheduling or concurrency. You...
main classes will be elevator ,floor, request ,elevator system,

Yes, that's a solid first pass. For a fresher interview, these are enough to start: class Elevator; class Floor; class...
i will create a base class request and then have two different classes ExtrenalReq and InternalReq?

Yes, that is a good design. You can say to the interviewer: "I'll keep a common base Request class and derive...
vector<*Request>requests; Direction curr; int floor; State Curr int id

Yes, that's the right idea. I'd just clean up the types/names a bit. enum Direction { UP, DOWN, IDLE }; enum...
i will choose 2 vector one for if i am going up and request is also from above then i will process it and the request...

Yes, that thinking is correct. You're essentially separating requests into two buckets: vector<int> upRequests;...

i should take request according to the direction if the user pressed the above button at floor 7 and also at floor 1...

Yes — the direction part is right, but I would not use "whoever pressed first gets the elevator" as the main rule. F...
we will only consider if its idle or moving in the same direction after that absolute distance from its current floor...

Yes, that is a very good fresher-level scheduling rule. You can tell the interviewer: "For an external request, I will...
what is the work of elevatorContoller? to give my elevator and then elevator handles its movement ?

Exactly. ElevatorController should mainly decide which elevator gets a request. It should not micromanage every...
now give the full code for elevator design

For a fresher interview, this is the level you should be able to write and explain confidently: classes, request type...

****Car Rental**** this is the lld problem now explain each step for answering this in interview and explain...

For a fresher interview, don't start by throwing classes immediately. First show that you can structure the proble...
implement important classes fully with their methods also\\

For a fresher interview, this is a good "complete enough" implementation: core entities, availability, reservation...

Parking_lot this is the lld problem now explain each step for answering this in interview and explain...

For a fresher interview, Parking Lot is one of the best LLD problems because it tests whether you can identify...

``` bool occupied; why not availabilty checker method ? ```

Because occupied is state/data, while isAvailable() or isFree() is a method that reads that state. You can absolutel...  
why do we use enums in place of classes'

We use an enum when something is mainly just a fixed set of labels/states, and a class when that thing needs its...  
I don't want to use gates ? is it fine ?

Yes, completely fine. For a fresher LLD, gates are optional unless the interviewer explicitly asks about multiple...  
so now give the final code with all implementations

For a fresher interview, I'd keep the final version simple: no gate classes, no unnecessary subclassing, and no...

you misunderstood me give the full code use strategy pattern ,only don;t use gates and now give me the full code

Yes — keeping Strategy Pattern, removing only the gate classes. Below is the complete version with: Vehicle...  
attendance tracker lld full

For a fresher interview, I'd keep an Attendance Tracker centered around these core entities: User / Student Cours...

Book myshow this is the lld problem now explain each step for answering this in interview and explain everythin...

For a fresher interview, BookMyShow should be answered in layers. Don't start with 20 classes. First show that yo...  
give full code

For a fresher interview, this is a complete but still understandable version. It includes the important parts:...

why we created showseat class and how and when to think that showseat type classes needed ?

ShowSeat exists because a physical Seat and that seat's state for a particular show are not the same thing. A...  
why make movies when we have show ?

Because Movie and Show represent two different things. Movie is the content itself: Interstellar 169 minutes...

explain each entity with this example `Movie` is the content itself: ``` Interstellar 169 minutes English Sci-Fi ```...

Using the same Interstellar at PVR example, every important BookMyShow entity can be understood like this. Use...

`Movie` can have many `Show` &#x20; why then movie does not have&#x20; vector\<show\\*>shows

It can have vector<Show\*> shows, but it does not have to. This is an important LLD point: a relationship can exist...

how you are calculating the pricing ? what if we want to give pricing with different ways

In the code I gave, pricing was very simple: I assigned the price while creating each ShowSeat. For example: if...

the best algorithm for elevator design having n floors and m lifts then best algo for lift ?\

For an interview, the best practical answer is usually not one "perfect" algorithm. A strong fresher answer is a co...

now fetch all the lld problems we have discussed till now

Thinking ● Identified and reviewed practiced LLD interview problems ✓ Worked for 11s So far, the LLD problems...

start with car rental system explain everything I should explain to interviewer and then at the last tell me the full...

Absolutely. I'll treat this like a real in-person LLD interview and show you exactly how I would expect a fresher to...

don't over engineer thinks defing pattern is good but like in brach its good to have vehicles so be it do not make...

Agreed. For fresher LLD, that's the better default: keep the object model simple, and introduce patterns only whe...

when its neccesary for having a carinventory ?

You need a separate CarInventory / VehicleInventory class only when managing vehicles itself becomes a...

Pasted text.txt give full code without vehicleInventory ?

Yes. I'll keep the same design you already had, remove VehicleInventory, and let Branch directly own...

```
``` newStart <= existingEnd && newEnd >= existingStar why newend>=existing start ? ```
```

Because for two date ranges to overlap, the new reservation must not finish before the existing reservation start...

what for 5-9 and 6-8 ?

For: Existing: 5 ----- 9 New: 6 ---- 8 they clearly overlap because the new interval is completely inside the existi...

I am just saying second condition is not essential giving case where we use only firts conditiona nd it gives awron...

Yes — the second condition is essential. If you only use: newStart <= existingEnd you get a wrong answer when...

Elevator; there are n floors and m elevators and now do the same as in previous case

For an Elevator System with n floors and m elevators, I'd handle it exactly like the Car Rental discussion: first sho...

```
``` ExternalRequest( int floor, Direction direction ) : Request(floor) this is constructor delegation and we can use...
```

In C++, this: ExternalRequest( int floor, Direction direction ) : Request(floor) is a base-class constructor call throu...

find parking lot full solution from the chat and then present it here

Yes — here is the Parking Lot full solution we settled on earlier: no EntranceGate / ExitGate ParkingLot directly...

do the same fetching for book my show

Yes — here is the BookMyShow full solution we settled on earlier, fetched back into one clean version. The design...

for parking lot get all the entiites ,class daigram functionality and tradeoffs do not give the whole code and tell...

For a fresher interview, this is exactly what you should cover for Parking Lot LLD without jumping into full code. 1....

do the same for atm design as in prev example

For a fresher interview, ATM LLD should be kept simple and structured. The most important things are:...

now full code for atm ?

Yes. I'll keep it at the same level as our Parking Lot / Car Rental versions: complete enough for an interview, but n...

Uml diagrams for Parking lot uml, movie rating system and atm

[Image: Three UML Class Diagram Panels]

vending machine lld code c++

For the Vending Machine LLD, this is the clean C++ version we discussed earlier using the State Pattern. Key...

```
``` class VendingMachine { Inventory* inventory; Payment* payment; Item* selectedItem; VendingState*...
```

Yes. This code is mainly showing State Pattern in a vending machine. The easiest way to understand it is to follow...

parking lot full code

parking lot full code

Here's the full Parking Lot LLD code in C++, matching the design we finalized earlier: #include <bits/stdc++.h>...

Pasted text.txt here why use vending state we can simply define differnt states ?

Yes, you can simply keep one enum like IDLE, PAYMENT, DISPENSING inside VendingMachine and handle everythi...

Atm lld code and full explanation

Here's the ATM LLD in the same style as Parking Lot and Vending Machine: first the interview explanation, then...
mars rover explain the question then its answer?

Absolutely. Mars Rover is a classic LLD/interview problem because it tests entity modeling, movement logic,...

``` static\_cast<Direction>() what is this > ```

static\_cast<Direction>() is C++ syntax for explicitly converting a value into the enum type Direction. In our Mars...

resolve the problem using if (direction == NORTH) &#x20; direction = EAST; else if (direction == EAST) &#x20;...

Yes — we can solve the Mars Rover problem without static\_cast and just use explicit if/else for direction changes....

why using enum not the class for direction if we have to use class then how

We use an enum for Direction because in the simple Mars Rover design, direction is just a fixed label/state: NORT...

\#include <bits/stdc++.h> using namespace std; //...

In your code, Rover has a relationship with Grid because Rover stores: Grid\* grid; So: Rover ----> Grid The rover...

parking lot uml diagram

[Image: Parking Lot UML Class Diagram]





from now on we will play a game at night i you will give me the lld problem and i will solve it

28/08/2026, 23:24:17



Done. From now on, our **night LLD game** will work like this:

1. I give you **one interview-style LLD problem** without the solution.
2. You drive the design: requirements → entities/classes → relationships → methods → patterns/SOLID → code.
3. I act like the **interviewer**: I'll question decisions, introduce requirements, and point out problems-but I won't reveal the design too early.
4. Once you're done, we'll compare it with a strong interview-ready design and discuss improvements.
5. Problems will gradually get harder: **Parking Lot → Elevator → Tic-Tac-Toe → Vending Machine → Splitwise → BookMyShow → Logger → Cache → Ride Sharing**, etc.

Since it's already night, we can start **Game #1** whenever you're ready. 😊

28/08/2026, 23:24:16



start but don't give question in order like you have given

28/08/2026, 23:25:22



## Night LLD Game #1 🎯

Design a **Vending Machine**.

Your vending machine should support:

- Multiple products, each with a **name, price, and available quantity**
- User can select a product
- User can insert money
- User can cancel the transaction
- Machine should dispense the product if enough money is inserted
- Machine should return change if required
- If the product is out of stock, it should not allow purchase
- Admin should be able to refill products

Assume for now that money can be inserted as integer denominations like 1, 2, 5, 10, 20, 50 .

Treat this like an interview.

Start by telling me:

**What are the main classes/entities you would create?**

I won't give you the design. I'll only intervene where needed.

28/08/2026, 23:25:22



vending machine ,items ,itemcontroller,payments

28/08/2026, 23:28:42



Good start. You've identified the main areas, but I'd challenge one thing first: `ItemController` **may be unnecessary in an LLD unless it has a clear responsibility.**

Right now you have:

- `VendingMachine`
- `Item`
- `ItemController`
- `Payment`

Now tell me the responsibility of each class.

For example, answer in this form:

`VendingMachine -> ?`

`Item -> ?`

`ItemController -> ?`

`Payment -> ?`

Also think about one missing concept: **where will stock/quantity of each item be maintained?**

28/08/2026, 23:28:41



```
display ,
vending machine has item ,itemcontroller is used to give the item ,payment is used
for payment
vending machine{
```

28/08/2026, 23:31:18



You're on the right track, but the responsibilities are still overlapping a bit.

`VendingMachine` should be the main coordinator. It should manage the current transaction: selected item, inserted money, payment, dispensing, cancellation, etc.

`Item` should just represent product data like `id` , `name` , `price` .

`ItemController` as “used to give the item” is a bit vague. In an interview, I would ask: **does it manage inventory, or does it physically dispense the item?** Those are two different responsibilities.

`Payment` can handle inserted amount, checking whether enough money was inserted, and calculating change.

And yes, `Display` can be a separate class if you want UI/output responsibility separated.

Now complete this part:

```
class VendingMachine {
 // what members?
};
```

Think specifically about:

- items/inventory
- selected item
- inserted money
- payment
- display

Don't write methods yet. Just members.

28/08/2026, 23:31:18



```
class vendingMachine{
 vector<item*> items;
}
class item{
 int id, cost, amount;
 int get_cost(){
 }
}
```

28/08/2026, 23:34:17



Yes, this is a reasonable starting structure.

I'd make one small naming improvement: use `quantity` instead of `amount`, because `amount` can get confused with money.

```
class Item {
 int id;
 int cost;
 int quantity;

public:
 int getCost() {
 return cost;
 }
};

class VendingMachine {
 vector<Item*> items;
};
```

Now as the interviewer, I'll ask you this:

Suppose the user selects **Coke with id = 3**.

How will `VendingMachine` know which item is currently selected?

What extra member would you add inside `VendingMachine`?

28/08/2026, 23:34:16





int id/name,  
and from list of items we will get the cost of id >?

28/08/2026, 23:36:43



Yes, exactly.

You can store the selected item's `id` inside `VendingMachine`, then search the `items` list to get that item and its cost.

For example:

```
class VendingMachine {
 vector<Item*> items;
 int selectedItemId;
};
```

Then when the user selects item `3`, you set:

```
selectedItemId = 3;
```

and find it:

```
Item* getSelectedItem() {
 for (auto item : items) {
 if (item->getId() == selectedItemId)
 return item;
 }
 return nullptr;
}
```

But interview question for you now: instead of storing only `selectedItemId`, can we directly store:

```
Item* selectedItem;
```

Which one do you think is better here, and why?

28/08/2026, 23:36:43





Exactly. `Item* selectedItem` is cleaner here.

```
class VendingMachine {
 vector<Item*> items;
 Item* selectedItem = nullptr;
};
```

Then after selection:

```
selectedItem = findItem(id);
```

and later:

```
selectedItem->getCost();
```

So you don't repeatedly search the vector.

One interview nuance: **O(1) access comes because we already stored the pointer after finding the item**. Initially, finding by ID in a `vector` is still `O(n)`.

Now the user selects Coke costing `20`.

Then they insert:

```
10
5
5
```

Where will you store this inserted money?

Would you put something like:

```
int insertedAmount;
```

inside `VendingMachine`, or should this responsibility go into your `Payment` class?

28/08/2026, 23:39:35



inserted money should go to vending payment along with ids of purchased item and payment will also have vector<Item\*> items and

28/08/2026, 23:42:46



I'd separate it a bit more.

`Payment` should mainly manage **money**, not the vending machine's inventory. If `Payment` also stores `vector<Item*> items`, then it starts taking responsibility for products too, which mixes concerns.

A cleaner direction is:

```
class Payment {
 int insertedAmount;

public:
 void addMoney(int amount);
 int getInsertedAmount();
 bool isEnough(int cost);
 int getChange(int cost);
};
```

Then `VendingMachine` keeps:

```
class VendingMachine {
 vector<Item*> items;
 Item* selectedItem;
 Payment* payment;
};
```

If you want to keep a record of what was purchased, that sounds more like a separate **Transaction** concept later:

```
class Transaction {
 int transactionId;
 Item* item;
 int paidAmount;
 int change;
};
```

Now interviewer question:

Suppose Coke costs `20`, user inserts `50`.

Who should calculate and return the `30` change: `Payment` or `VendingMachine`? And why?

28/08/2026, 23:42:50





vending machine as it will inherit from payment and calculate everything required and after all it shows separation of concerns and single responsibility principle

28/08/2026, 23:45:17



Close, but I would **not use inheritance here**.

`VendingMachine` is **not a Payment**. So:

```
class VendingMachine : public Payment
```

would violate the **“is-a” relationship**.

A vending machine **has a Payment object**, so this is composition/association:

```
class VendingMachine {
 Payment* payment;
};
```

For change calculation, `Payment` should do it because money handling is its responsibility:

```
int Payment::getChange(int cost) {
 return insertedAmount - cost;
}
```

Then `VendingMachine` coordinates the flow:

```
int change = payment->getChange(selectedItem->getCost());
```

So SRP becomes cleaner:

- `VendingMachine` → coordinates the transaction
- `Payment` → handles money-related logic
- `Item` → stores product information

Next interviewer question:

The user selects Coke, inserts enough money, and now the machine needs to actually **release one Coke and decrease quantity**.

Who should do that?

Would you put:

```
selectedItem->quantity--;
```

directly inside `VendingMachine`, or create something like an `Inventory` / `ItemController` class for stock management?

28/08/2026, 23:45:16



we can do both but I will go with Inventory

28/08/2026, 23:46:38



Yes, `Inventory` is the better choice here.

Then the responsibilities become cleaner:

```
class Inventory {
 vector<Item*> items;

public:
 Item* getItem(int id);
 bool isAvailable(int id);
 void reduceQuantity(int id);
 void refillItem(int id, int quantity);
};
```

And `VendingMachine` becomes more of a coordinator:

```
class VendingMachine {
 Inventory* inventory;
 Item* selectedItem;
 Payment* payment;
};
```

So after successful payment:

```
inventory->reduceQuantity(selectedItem->getId());
```

This keeps stock logic out of `VendingMachine`.

Now next interviewer twist:

Suppose the machine can be in these situations:

- waiting for user
- item selected
- accepting money
- dispensing item
- returning change

Would you handle this using a bunch of `if/else` checks inside `VendingMachine`, or do you see a design pattern that could fit here?

28/08/2026, 23:46:37



this is stragety pattern

28/08/2026, 23:53:06



Not quite — this is a **State Pattern** use case, not Strategy.

Why? Because the **behavior of the same vending machine changes based on its current state**.

For example:

```
IdleState
SelectedState
PaymentState
DispenseState
```

If machine is in `IdleState`, `insertMoney()` should probably be invalid.

If machine is in `PaymentState`, `insertMoney()` is valid.

That is exactly what State Pattern handles.

**Strategy Pattern** is different: it is used when you have multiple interchangeable algorithms, for example:

```
CashPaymentStrategy
CardPaymentStrategy
UPIPaymentStrategy
```

Same goal: payment. Different ways to perform it.

So interviewer follow-up:

How would you model the states?

Would you create something like:

```
class VendingState {
public:
 virtual void selectItem(...) = 0;
 virtual void insertMoney(...) = 0;
 virtual void dispense(...) = 0;
};
```

and then concrete classes like `IdleState`, `PaymentState`, etc.?

28/08/2026, 23:53:06





how this vending state will be used with and where now write the full code for all classes

28/08/2026, 23:56:17



Yes. The `VendingState` object will live **inside** `VendingMachine`, and `VendingMachine` will delegate actions to the current state.

That is the key idea:

```
User calls machine.selectItem()
 ↓
VendingMachine calls currentState->selectItem()
 ↓
IdleState / PaymentState / DispenseState handles it differently
```

Here is a complete interview-style version using simple C++.

```
#include <bits/stdc++.h>
using namespace std;

class VendingMachine;
```

## 1. Item

```
class Item {
 int id;
 string name;
 int price;
 int quantity;

public:
 Item(int id, string name, int price, int quantity) {
 this->id = id;
 this->name = name;
 this->price = price;
 this->quantity = quantity;
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }

 int getPrice() {
 return price;
 }

 int getQuantity() {
 return quantity;
 }

 void addQuantity(int count) {
 quantity += count;
 }
}
```

```

void reduceQuantity() {
 if (quantity > 0)
 quantity--;
}
};

```

## 2. Inventory

Inventory is responsible for storing and managing items.

```

class Inventory {
 vector<Item*> items;

public:
 void addItem(Item* item) {
 items.push_back(item);
 }

 Item* getItem(int id) {
 for (auto item : items) {
 if (item->getId() == id)
 return item;
 }

 return nullptr;
 }

 bool isAvailable(int id) {
 Item* item = getItem(id);

 if (item == nullptr)
 return false;

 return item->getQuantity() > 0;
 }

 void reduceQuantity(int id) {
 Item* item = getItem(id);

 if (item != nullptr)
 item->reduceQuantity();
 }

 void refillItem(int id, int quantity) {
 Item* item = getItem(id);

 if (item != nullptr)
 item->addQuantity(quantity);
 }

 void showItems() {
 cout << "\nAvailable Items:\n";

 for (auto item : items) {
 cout << item->getId()
 << " " << item->getName()
 << " Price: " << item->getPrice()
 << " Quantity: " << item->getQuantity()
 << "\n";
 }
 }
}

```



```
}
};
```

### 3. Payment

Payment handles money.

```
class Payment {
 int insertedAmount;

public:
 Payment() {
 insertedAmount = 0;
 }

 void addMoney(int amount) {
 insertedAmount += amount;
 }

 int getInsertedAmount() {
 return insertedAmount;
 }

 bool isEnough(int price) {
 return insertedAmount >= price;
 }

 int calculateChange(int price) {
 return insertedAmount - price;
 }

 void reset() {
 insertedAmount = 0;
 }
};
```

Now comes the important part.

## State Pattern

We create a base state.

```
class VendingState {
public:

 virtual void selectItem(VendingMachine* machine, int id) {
 cout << "Cannot select item in current state\n";
 }

 virtual void insertMoney(VendingMachine* machine, int amount) {
 cout << "Cannot insert money in current state\n";
 }

 virtual void dispense(VendingMachine* machine) {
 cout << "Cannot dispense in current state\n";
 }

 virtual void cancel(VendingMachine* machine) {
 cout << "Cannot cancel in current state\n";
 }
};
```



```
virtual ~VendingState() {}
};
```

Notice something important.

Every state receives:

```
VendingMachine* machine
```

Why?

Because the state may need to change machine data:

```
machine->setSelectedItem(...)
machine->setState(...)
machine->getPayment()
machine->getInventory()
```

Now define the machine.

## 4. VendingMachine

```
class VendingMachine {
 Inventory* inventory;
 Payment* payment;

 Item* selectedItem;

 VendingState* currentState;

public:

 VendingMachine() {
 inventory = new Inventory();
 payment = new Payment();

 selectedItem = nullptr;
 currentState = nullptr;
 }

 Inventory* getInventory() {
 return inventory;
 }

 Payment* getPayment() {
 return payment;
 }

 Item* getSelectedItem() {
 return selectedItem;
 }

 void setSelectedItem(Item* item) {
 selectedItem = item;
 }

 void setState(VendingState* state) {
 currentState = state;
 }

 void selectItem(int id) {
```

```

 currentState->selectItem(this, id);
 }

 void insertMoney(int amount) {
 currentState->insertMoney(this, amount);
 }

 void dispense() {
 currentState->dispense(this);
 }

 void cancel() {
 currentState->cancel(this);
 }

 void resetTransaction() {
 selectedItem = nullptr;
 payment->reset();
 }
};

```

This is where State Pattern becomes clear.

The user never does:

```

IdleState idle;
idle.selectItem(...);

```

Instead, user always talks to:

```

machine.selectItem(1);
machine.insertMoney(20);
machine.dispense();

```

`VendingMachine` internally delegates to:

```

currentState

```

## 5. IdleState

Initially machine is waiting for selection.

```

class PaymentState;

class IdleState : public VendingState {

public:

 void selectItem(VendingMachine* machine, int id) override;
};

```

We'll implement it after `PaymentState` exists.

## 6. PaymentState

After selecting an item, machine enters payment state.

```

class DispenseState;

class PaymentState : public VendingState {

```

```

public:

 void insertMoney(VendingMachine* machine, int amount) override {
 machine->getPayment()->addMoney(amount);

 cout << "Inserted: " << amount << "\n";

 Item* item = machine->getSelectedItem();

 int price = item->getPrice();
 int inserted = machine->getPayment()->getInsertedAmount();

 cout << "Total inserted: " << inserted << "\n";

 if (inserted >= price) {
 cout << "Enough money inserted\n";
 }
 }

 void cancel(VendingMachine* machine) override {
 int money =
 machine->getPayment()->getInsertedAmount();

 cout << "Transaction cancelled\n";

 if (money > 0)
 cout << "Returning: " << money << "\n";

 machine->resetTransaction();

 machine->setState(new IdleState());
 }

 void dispense(VendingMachine* machine) override;
};

```

## 7. DispenseState

```

class DispenseState : public VendingState {

public:

 void dispense(VendingMachine* machine) override {

 Item* item = machine->getSelectedItem();

 if (item == nullptr) {
 cout << "No item selected\n";
 return;
 }

 Payment* payment = machine->getPayment();

 if (!payment->isEnough(item->getPrice())) {
 cout << "Insufficient money\n";
 return;
 }

 machine->getInventory()
 }

```



```

 ->reduceQuantity(item->getId());

 cout << "Dispensing "
 << item->getName()
 << "\n";

 int change =
 payment->calculateChange(item->getPrice());

 if (change > 0) {
 cout << "Returning change: "
 << change
 << "\n";
 }

 machine->resetTransaction();

 machine->setState(new IdleState());
}
};

```

Now implement `PaymentState::dispense()`.

```

void PaymentState::dispense(VendingMachine* machine) {

 Item* item = machine->getSelectedItem();

 if (!machine->getPayment()
 ->isEnough(item->getPrice())) {

 cout << "Please insert more money\n";
 return;
 }

 machine->setState(new DispenseState());

 machine->dispense();
}

```

Notice this beautiful transition:

```

PaymentState
 ↓
DispenseState
 ↓
IdleState

```

Now implement `IdleState`.

```

void IdleState::selectItem(
 VendingMachine* machine,
 int id) {

 Inventory* inventory =
 machine->getInventory();

 Item* item =
 inventory->getItem(id);

 if (item == nullptr) {
 cout << "Invalid item\n";
 }
}

```

```

 return;
 }

 if (!inventory->isAvailable(id)) {
 cout << "Item out of stock\n";
 return;
 }

 machine->setSelectedItem(item);

 cout << "Selected: "
 << item->getName()
 << "\n";

 cout << "Price: "
 << item->getPrice()
 << "\n";

 machine->setState(new PaymentState());
}

```

## 8. Main

```

int main() {

 VendingMachine* machine =
 new VendingMachine();

 machine->getInventory()->addItem(
 new Item(1, "Coke", 20, 5)
);

 machine->getInventory()->addItem(
 new Item(2, "Pepsi", 25, 3)
);

 machine->getInventory()->addItem(
 new Item(3, "Chips", 10, 10)
);

 machine->setState(new IdleState());

 machine->getInventory()->showItems();

 machine->selectItem(1);

 machine->insertMoney(10);

 machine->insertMoney(20);

 machine->dispense();

 return 0;
}

```

Output would roughly be:

```

Available Items:
1 Coke Price: 20 Quantity: 5
2 Pepsi Price: 25 Quantity: 3
3 Chips Price: 10 Quantity: 10

Selected: Coke
Price: 20

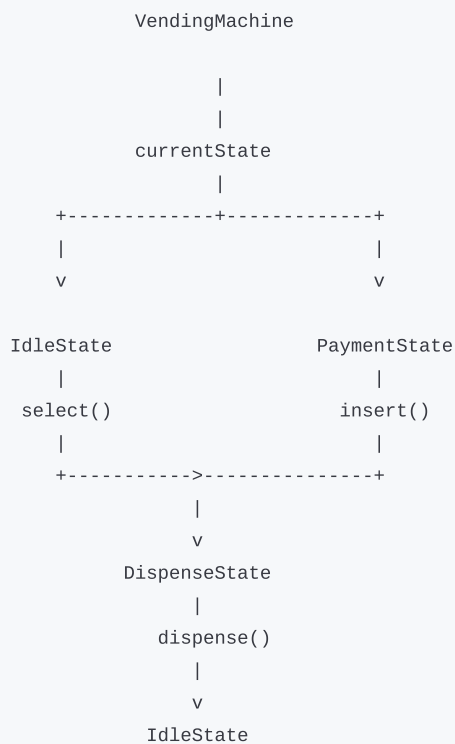
Inserted: 10
Total inserted: 10

Inserted: 20
Total inserted: 30
Enough money inserted

Dispensing Coke
Returning change: 10

```

The most important thing to understand is the flow:



So instead of doing this:

```

if (state == IDLE) {
 ...
}
else if (state == PAYMENT) {
 ...
}
else if (state == DISPENSE) {
 ...
}

```

we use polymorphism:

```

currentState->selectItem(...);

```

```
currentState->insertMoney(...);

currentState->dispense(...);
```

Each state decides what is allowed.

One important improvement I would make in a real interview: don't continuously do:

```
new IdleState()
new PaymentState()
new DispenseState()
```

because we're not deleting them here. I kept it this way because we're focusing on simple interview-style C++ rather than memory-management machinery.

The final class structure is essentially:

```
VendingMachine
|
+-- Inventory
| |
| +-- vector<Item*>
|
+-- Payment
|
+-- Item* selectedItem
|
+-- VendingState* currentState
 |
 +-- IdleState
 |
 +-- PaymentState
 |
 +-- DispenseState
```

And remember the interview distinction:

### State Pattern

*Same object behaves differently depending on its current state.*

Example:

```
Idle → Payment → Dispense → Idle
```

### Strategy Pattern

*Same operation can be performed using different algorithms.*

Example:

```
PaymentStrategy
|
+-- CashPayment
+-- CardPayment
+-- UPIPayment
```

A sophisticated vending machine could actually use **both State + Strategy patterns together**.

28/08/2026, 23:56:17



```
. DispenseState
...

class DispenseState : public VendingState {

public:

 void dispense(VendingMachine* machine) override {

 Item* item = machine->getSelectedItem();

 if (item == nullptr) {
 cout << "No item selected\n";
 return;
 }

 Payment* payment = machine->getPayment();

 if (!payment->isEnough(item->getPrice())) {
 cout << "Insufficient money\n";
 return;
 }

 machine->getInventory()
 ->reduceQuantity(item->getId());

 cout << "Dispensing "
 << item->getName()
 << "\n";

 int change =
 payment->calculateChange(item->getPrice());

 if (change > 0) {
 cout << "Returning change: "
 << change
 << "\n";
 }

 machine->resetTransaction();

 machine->setState(new IdleState());
 }
};
...

Now implement `PaymentState::dispense()`.
...

void PaymentState::dispense(VendingMachine* machine) {

 Item* item = machine->getSelectedItem();

 if (!machine->getPayment()
 ->isEnough(item->getPrice())) {

 cout << "Please insert more money\n";
 return;
 }

 machine->setState(new DispenseState());
```



```
machine->dispense();
}
what is the difference here ?
...
```

29/08/2026, 00:16:49



The difference is **responsibility and transition**.

`PaymentState::dispense()` does **not actually dispense the item**. Its job is just to check:

```
if (!machine->getPayment()->isEnough(item->getPrice()))
```

If money is insufficient, stay in `PaymentState`.

If money is enough, it changes the machine state:

```
machine->setState(new DispenseState());
```

and then calls:

```
machine->dispense();
```

Now this second call is handled by the new current state, i.e. `DispenseState`.

So the flow is:

User calls:

```
machine->dispense()
```

↓

```
currentState = PaymentState
```

↓

```
PaymentState::dispense()
```

↓

```
check whether enough money exists
```

↓ yes

```
setState(DispenseState)
```

↓

```
machine->dispense()
```

↓

```
currentState is NOW DispenseState
```

↓

```
DispenseState::dispense()
```

↓

```
reduce quantity
dispense product
calculate change
reset transaction
go back to IdleState
```

So these two methods have different jobs.

`PaymentState::dispense()` means:

*"Are we allowed to move to dispensing?"*

`DispenseState::dispense()` means:

*"Actually perform the dispensing operation."*

For example, suppose Coke costs `20` and user inserted only `10`.

```
machine->dispense();
```

goes to:

```
PaymentState::dispense()
```

and this fails:

```
if (!payment->isEnough(20)) {
 cout << "Please insert more money";
 return;
}
```

So we **never enter** `DispenseState`.

But if user inserted `30`, then:

```
PaymentState::dispense()
```

does:

```
machine->setState(new DispenseState());
machine->dispense();
```

and now:

```
DispenseState::dispense()
```

actually does:

```
inventory->reduceQuantity(...)
calculateChange(...)
resetTransaction()
setState(new IdleState())
```

One thing though: this design can be made slightly cleaner. Instead of making the user manually call:

```
machine->dispense();
```

after enough money is inserted, `PaymentState::insertMoney()` itself can transition to `DispenseState` and invoke `dispense` automatically.

Then the flow becomes:

```
select item
↓
IdleState → PaymentState

insert money
↓
enough?
↓
DispenseState

dispense
↓
IdleState
```

That version is usually cleaner for a vending machine.

29/08/2026, 00:16:49



I am unable to understand the dispensestate and all these wala concept ,explain in detail ?

29/08/2026, 00:28:06



Yes. The confusing part is usually this:

“Why do we need `PaymentState` , `DispenseState` , etc.? Why can't `VendingMachine` just do everything?”

The answer is: **we can do everything inside** `VendingMachine` , but then we end up with many `if/else` checks based on the machine's current situation. The State Pattern is just a cleaner way to organize those rules.

Suppose we do **not** use State Pattern.

Your vending machine might look like this:

```
class VendingMachine {
 int state; // 0 = idle, 1 = payment, 2 = dispense

public:

 void selectItem(int id) {

 if(state != 0) {
 cout << "Cannot select item now\n";
 return;
 }

 // select item

 state = 1;
 }

 void insertMoney(int amount) {

 if(state != 1) {
 cout << "Cannot insert money now\n";
 return;
 }

 // add money
 }

 void dispense() {

 if(state != 1) {
 cout << "Cannot dispense now\n";
 return;
 }

 // check money
 // reduce quantity
 // give item
 // return change

 state = 0;
 }
};
```

This works.

But as the vending machine grows, you may have:

```
Idle
ItemSelected
AcceptingMoney
Dispensing
ReturningChange
OutOfStock
Maintenance
```

Then every function starts becoming:

```
if(state == ...)
else if(state == ...)
else if(state == ...)
```

That becomes messy.

The **State Pattern** says:

*Instead of storing the state only as a number/string, make each state a separate object.*

So instead of:

```
int state;
```

we have:

```
VendingState* currentState;
```

And possible objects are:

```
IdleState
PaymentState
DispenseState
```

Think of these objects as representing the **current mode of the machine**.

For example:

```
Machine is waiting
↓
IdleState

User selected Coke
↓
PaymentState

Enough money received
↓
DispenseState

Item delivered
↓
IdleState
```

Now imagine the machine currently contains:

```
currentState = new PaymentState();
```

When you do:

```
machine->dispense();
```

inside `VendingMachine`, we write:

```
void dispense() {
 currentState->dispense(this);
}
```

So this:

```
machine->dispense();
```

doesn't directly dispense anything.

It means:

*"Hey current state, handle this dispense request."*

If current state is `PaymentState`:

```
currentState->dispense(this);
```

actually calls:

```
PaymentState::dispense(machine);
```

because of virtual functions.

That is the most important thing.

Think of this:

```
VendingState* currentState;
```

like:

```
currentState
|
+-----> PaymentState object
```

Then:

```
currentState->dispense(...)
```

means:

```
PaymentState::dispense(...)
```

Now let's understand why `PaymentState::dispense()` exists.

Suppose Coke costs `20`.

User has inserted only `10`.

Machine is currently:

```
PaymentState
```

Now user presses dispense.

We do:

```
machine->dispense();
```

VendingMachine forwards it:

```
currentState->dispense(this);
```

Since currentState is PaymentState, this runs:

```
void PaymentState::dispense(VendingMachine* machine) {

 Item* item = machine->getSelectedItem();

 if (!machine->getPayment()->isEnough(item->getPrice())) {

 cout << "Please insert more money\n";
 return;
 }

 ...
}
```

Money is insufficient.

So we simply stay in:

```
PaymentState
```

Nothing else happens.

Now suppose the user inserts another 10.

Total becomes 20.

Now when dispense is requested again:

```
machine->dispense();
```

we again enter:

```
PaymentState::dispense()
```

This time:

```
isEnough(20)
```

returns true.

Now we say:

```
machine->setState(new DispenseState());
```

This means:

*Payment is complete. Machine has now moved into the "dispensing mode."*

Before:

```
currentState
 |
 v
PaymentState
```

After:

```
currentState
 |
 v
DispenseState
```

Then we call:

```
machine->dispense();
```

again.

This is the part that initially looks weird.

But remember:

```
machine->dispense()
```

does this:

```
currentState->dispense(this);
```

And **currentState has changed**.

Earlier it was:

```
PaymentState
```

Now it is:

```
DispenseState
```

Therefore the second call goes here:

```
DispenseState::dispense()
```

not back to `PaymentState`.

So:

```
machine->setState(new DispenseState());
machine->dispense();
```

essentially means:

```
Change machine mode to Dispensing

then

Tell the new mode to perform its operation
```

Now `DispenseState` actually performs physical dispensing logic:

```
void dispense(VendingMachine* machine) {

 Item* item = machine->getSelectedItem();

 machine->getInventory()
 ->reduceQuantity(item->getId());
```



```

 cout << "Dispensing "
 << item->getName();

 int change =
 machine->getPayment()
 ->calculateChange(item->getPrice());

 if(change > 0)
 cout << "Return " << change;

 machine->resetTransaction();

 machine->setState(new IdleState());
}

```

Its responsibility is:

```

PaymentState
|
| money valid?
|
v

DispenseState
|
| decrease stock
| give product
| give change
|
v

IdleState

```

Think of a real vending machine.

At first screen says:

```
Select Product
```

This is:

```
IdleState
```

You press Coke.

Machine now says:

```
Price ₹20
Insert Money
```

This is:

```
PaymentState
```

You insert ₹10.

Still:

```
PaymentState
```

You insert another ₹10.

Payment finished.

Internally machine enters:

```
DispenseState
```

During this state, maybe:

```
motor rotates
product falls
inventory decreases
change returned
```

Once finished:

```
IdleState
```

screen again says:

```
Select Product
```

So these classes aren't really "new machines."

There is still only **one VendingMachine**.

The State objects just tell the machine:

*"What phase are you currently in, and what actions should be allowed in this phase?"*

A nice mental model is a traffic light.

One traffic light can be in:

```
RedState
YellowState
GreenState
```

You don't have three traffic lights.

You have:

```
TrafficLight
|
currentState
|
+-- RedState
+-- YellowState
+-- GreenState
```

Similarly:

```
VendingMachine
|
currentState
|
+-- IdleState
+-- PaymentState
+-- DispenseState
```

Now let's simplify the actual code heavily.

Base state:

```
class VendingState {
public:

 virtual void selectItem(VendingMachine* machine, int id) {
 cout << "Not allowed\n";
 }

 virtual void insertMoney(VendingMachine* machine, int money) {
 cout << "Not allowed\n";
 }

 virtual void dispense(VendingMachine* machine) {
 cout << "Not allowed\n";
 }
};
```

Notice we provide default behavior:

```
Not allowed
```

Then each state overrides only what makes sense.

IdleState :

```
class IdleState : public VendingState {
public:

 void selectItem(VendingMachine* machine, int id) override {

 Item* item =
 machine->getInventory()->getItem(id);

 if(item == nullptr) {
 cout << "Invalid item\n";
 return;
 }

 machine->setSelectedItem(item);

 cout << "Item selected\n";

 machine->setState(new PaymentState());
 }
};
```

What is allowed in IdleState ?

```
selectItem ✓
insertMoney ✗
dispense ✗
```

Because it only overrides:

```
selectItem()
```

If you call:

```
machine->insertMoney(10);
```

while in idle, base class method runs:

```
cout << "Not allowed";
```

Now `PaymentState` :

```
class PaymentState : public VendingState {
public:

 void insertMoney(
 VendingMachine* machine,
 int money
) override {

 machine->getPayment()->addMoney(money);

 cout << "Money inserted\n";
 }

 void dispense(
 VendingMachine* machine
) override {

 Item* item =
 machine->getSelectedItem();

 if(!machine->getPayment()->isEnough(
 item->getPrice())) {

 cout << "Insert more money\n";
 return;
 }

 machine->setState(new DispenseState());

 machine->dispense();
 }
};
```

Allowed here:

```
selectItem ✖
insertMoney ✔
dispense ✔
```

Now `DispenseState` :

```
class DispenseState : public VendingState {
public:

 void dispense(
 VendingMachine* machine
) override {

 Item* item =
 machine->getSelectedItem();

 cout << "Giving "
 << item->getName();

 machine->getInventory()
 ->reduceQuantity(item->getId());
 }
};
```

```

 machine->resetTransaction();

 machine->setState(new IdleState());
 }
};

```

Allowed:

```

selectItem ❌
insertMoney ❌
dispense ✅

```

And the `VendingMachine` is just delegating:

```

class VendingMachine {

 VendingState* currentState;

public:

 void selectItem(int id) {
 currentState->selectItem(this, id);
 }

 void insertMoney(int money) {
 currentState->insertMoney(this, money);
 }

 void dispense() {
 currentState->dispense(this);
 }

 void setState(VendingState* state) {
 currentState = state;
 }
};

```

So when you write:

```

machine->insertMoney(10);

```

you can mentally translate it into:

```

"Whatever my current state is,
please handle insertMoney."

```

If current state = `IdleState`:

```

Not allowed

```

If current state = `PaymentState`:

```

Accept money

```

This is **polymorphism being used to remove state-based if/else**.

Without State Pattern:

```

void insertMoney(int amount) {

```



```

if(state == IDLE) {
 cout << "Not allowed";
}

else if(state == PAYMENT) {
 payment.addMoney(amount);
}

else if(state == DISPENSE) {
 cout << "Not allowed";
}
}

```

With State Pattern:

```

void insertMoney(int amount) {
 currentState->insertMoney(this, amount);
}

```

And each state decides.

That one comparison is what you should remember for interviews:

Without State Pattern:

VendingMachine asks:  
 "What state am I in?"  
 then uses if/else.

With State Pattern:

VendingMachine says:  
 "Current state, you decide what to do."

And specifically for your `DispenseState` confusion:

```

PaymentState::dispense()

= CHECK whether we're ready to dispense
= transition to DispenseState

DispenseState::dispense()

= ACTUALLY give the item
= update inventory
= calculate change
= finish transaction
= return to IdleState

```

So their names are the same because both respond to a `dispense()` request, but they behave differently because the machine is in a different phase.

That's exactly what the State Pattern is designed for.

29/08/2026, 00:28:05



good teacher now meet you tommorow

29/08/2026, 00:38:08



Absolutely 😊 See you tomorrow night for the next LLD game. I'll give you a different problem, not in any fixed order.

29/08/2026, 00:38:07



start the game

29/08/2026, 23:43:23



## Night LLD Game #2 🎯

Design a **Splitwise-like Expense Sharing System**.

Requirements:

- Users can create a group.
- A group can have multiple members.
- A user can add an expense.
- Expense can be split in different ways:
  - equally
  - exact amounts
  - percentages
- System should track **who owes whom and how much**.
- A user should be able to view balances.
- Users should be able to settle payments.

Example:

A, B, C are in a group.

A pays ₹900 for dinner.

Equal split:

A = ₹300

B = ₹300

C = ₹300

So:

B owes A ₹300

C owes A ₹300

Treat this like an interview.

Start only with:

**What main classes/entities would you create?**

Don't worry about design patterns yet.

29/08/2026, 23:43:22







main entities will be group ,member ,expenses and system

08/09/2026, 22:30:32



Good start. I'd refine the naming slightly:

- `SplitwiseSystem` — overall coordinator
- `User` or `Member` — represents a person
- `Group` — contains members and group expenses
- `Expense` — represents one bill/transaction

One more important entity will probably emerge later: something that represents **each user's share in an expense**.

For example, if A pays ₹900 and split is:

```
A → ₹100
B → ₹300
C → ₹500
```

the `Expense` needs to know each person's individual share somehow.

But don't add it yet unless you think it is required.

Now interviewer question:

What members would you put inside these two classes?

```
class User {

};

class Group {

};
```

Only data members for now, no methods.

08/09/2026, 22:29:59





tell me the full answers for lld but since i am fresher so always tell what i must be able to tell to the interviewer before going deep

09/09/2026, 23:22:42



Yes. From now on, for every LLD problem I'll structure it in two layers:

**First: fresher-level interview answer** - the minimum you should confidently explain in 5-10 minutes.

**Then: deeper design** - patterns, extensibility, SOLID, edge cases, code, tradeoffs.

For **Splitwise**, this is what you should first be able to say to the interviewer.

### What you must explain as a fresher

*"I will model the system using `User`, `Group`, `Expense`, `Split`, and `SplitwiseSystem` classes.  
A `Group` contains users and expenses.  
An `Expense` stores who paid, total amount, and how the amount is split among users.  
Different splitting methods like `Equal`, `Exact`, and `Percentage` can be handled using the `Strategy Pattern`.  
The system will maintain balances between users so that we know who owes whom."*

That alone is already a strong starting answer.

The core entities would be:

```
class User {
 int id;
 string name;
};

class Group {
 int id;
 string name;

 vector<User*> members;
 vector<Expense*> expenses;
};

class Expense {
 int id;
 double amount;

 User* paidBy;

 vector<Split*> splits;
};

class Split {
 User* user;
 double amount;
};
```

Why do we need `Split`?

Suppose:

A paid ₹900

A share = ₹100

```
B share = ₹300
C share = ₹500
```

Instead of storing complicated maps inside `Expense`, we can represent each person's share separately:

```
Expense
|
+-- Split(A, 100)
+-- Split(B, 300)
+-- Split(C, 500)
```

So an `Expense` is basically:

```
Who paid?
How much?
Who participated?
How much should each participant pay?
```

Then we need a central class:

```
class SplitwiseSystem {
 vector<User*> users;
 vector<Group*> groups;

 // balances
};
```

The biggest part of Splitwise is actually the **balance tracking**.

Suppose A paid ₹900:

```
A = ₹300
B = ₹300
C = ₹300
```

A already paid the full ₹900.

Therefore:

```
B owes A ₹300
C owes A ₹300
```

We can store this as:

```
map<int, map<int, double>> balance;
```

Conceptually:

```
balance[B][A] = 300
balance[C][A] = 300
```

Meaning:

```
B owes A ₹300
C owes A ₹300
```

Now comes the important extensibility part.

We have three split types:

Equal  
Exact  
Percentage

Instead of doing:

```
if(type == EQUAL) {
 ...
}
else if(type == EXACT) {
 ...
}
else if(type == PERCENTAGE) {
 ...
}
```

we can use the **Strategy Pattern**.

Base strategy:

```
class SplitStrategy {
public:
 virtual vector<Split*> calculateSplit(
 double amount,
 vector<User*> users,
 vector<double> values
) = 0;

 virtual ~SplitStrategy() {}
};
```

Then:

```
class EqualSplitStrategy : public SplitStrategy {
};

class ExactSplitStrategy : public SplitStrategy {
};

class PercentageSplitStrategy : public SplitStrategy {
};
```

For example, Equal:

```
class EqualSplitStrategy : public SplitStrategy {
public:

 vector<Split*> calculateSplit(
 double amount,
 vector<User*> users,
 vector<double> values
) override {

 vector<Split*> result;

 double share = amount / users.size();

 for(auto user : users) {
 result.push_back(
 new Split(user, share)
);
 }
 }
};
```

```

 }

 return result;
}
};

```

Exact split:

Suppose:

```

Total = ₹900

A = ₹100
B = ₹300
C = ₹500

```

The strategy simply validates:

```

100 + 300 + 500 = 900

```

and creates corresponding `Split` objects.

Percentage:

```

A = 20%
B = 30%
C = 50%

```

For ₹1000:

```

A = ₹200
B = ₹300
C = ₹500

```

Now the complete high-level flow is:

```

User adds expense
 ↓
SplitwiseSystem
 ↓
select SplitStrategy
 ↓
calculate each user's share
 ↓
create Expense
 ↓
store in Group
 ↓
update balances

```

The balance update is the most important logic.

Suppose:

```

A paid ₹900

A share = ₹300
B share = ₹300
C share = ₹300

```

Loop over splits:

```
for(auto split : splits) {

 if(split->getUser() == paidBy)
 continue;

 balance[split->getUser()->getId()]
 [paidBy->getId()]
 += split->getAmount();
}
```

So:

```
B → owes A ₹300
C → owes A ₹300
```

Now imagine later B pays ₹100 to A.

Before:

```
B owes A ₹300
```

After:

```
B owes A ₹200
```

So settlement becomes:

```
void settle(
 User* from,
 User* to,
 double amount
) {

 balance[from->getId()]
 [to->getId()] -= amount;
}
```

There is one improvement you should mention in an interview.

Suppose:

```
A owes B ₹500
```

Then later B owes A ₹200.

Instead of storing both:

```
A → B = 500
B → A = 200
```

we can simplify to:

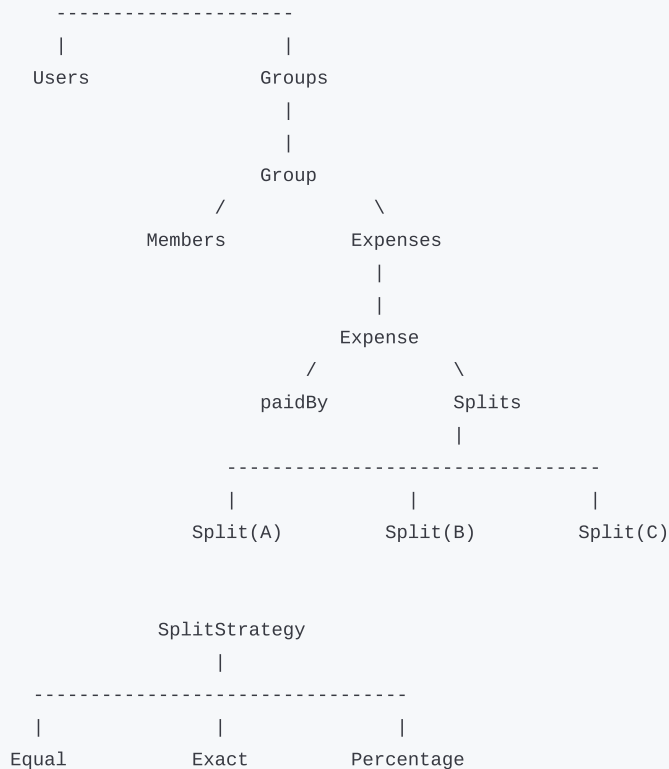
```
A → B = 300
```

This is called **balance simplification/netting**.

You don't necessarily need to implement the full optimization as a fresher unless asked, but mentioning it is good.

So the interview-level class diagram is roughly:

```
SplitwiseSystem
|
```



A simple `Expense` class could be:

```

class Expense {
 int id;
 double amount;

 User* paidBy;

 vector<Split*> splits;

public:

 Expense(
 int id,
 double amount,
 User* paidBy,
 vector<Split*> splits
) {
 this->id = id;
 this->amount = amount;
 this->paidBy = paidBy;
 this->splits = splits;
 }

 double getAmount() {
 return amount;
 }

 User* getPaidBy() {
 return paidBy;
 }

 vector<Split*> getSplits() {
 return splits;
 }
};

```

And `Split` :

```
class Split {
 User* user;
 double amount;

public:

 Split(User* user, double amount) {
 this->user = user;
 this->amount = amount;
 }

 User* getUser() {
 return user;
 }

 double getAmount() {
 return amount;
 }
};
```

The key SOLID point here is **Open/Closed Principle**.

If tomorrow we add:

```
WeightedSplit
```

we shouldn't modify a huge `if/else` .

We simply create:

```
class WeightedSplitStrategy : public SplitStrategy {
};
```

That's why Strategy Pattern fits nicely.

For a fresher interview, if you can confidently explain these five things, you're in a very good position:

1. Main classes: `User` , `Group` , `Expense` , `Split` , `SplitwiseSystem` .
2. Why `Split` exists.
3. How `balance[user1][user2]` tracks who owes whom.
4. Why Equal/Exact/Percentage can use Strategy Pattern.
5. Basic expense flow: **create expense → calculate split → update balances**.

You do **not** need to jump immediately into factories, repositories, observers, database schemas, locking, or microservices unless the interviewer asks. Start simple and extend only when requirements demand it.

09/09/2026, 23:22:09





How `balance[user1][user2]` tracks who owes whom.  
implement this thing and give me the full code show main also

10/09/2026, 15:43:57



For a fresher interview, the simplest clean way is:

`balance[A][B] = x` means **A owes B amount x**.

So if B owes A ₹300:

```
balance["B"]["A"] = 300;
```

And if later B pays A ₹100:

```
balance["B"]["A"] -= 100;
```

Here's a full simple C++ implementation with `User`, `Expense`, `Split`, `SplitwiseSystem`, and `main()`.

```
#include <bits/stdc++.h>
using namespace std;

// ----- User -----

class User {
 int id;
 string name;

public:
 User(int id, string name) {
 this->id = id;
 this->name = name;
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }
};

// ----- Split -----

class Split {
 User* user;
 double amount;

public:
 Split(User* user, double amount) {
 this->user = user;
 this->amount = amount;
 }

 User* getUser() {
 return user;
 }
};
```

```

 }

 double getAmount() {
 return amount;
 }
};

// ----- Expense -----

class Expense {
 int id;
 double totalAmount;
 User* paidBy;
 vector<Split*> splits;

public:
 Expense(
 int id,
 double totalAmount,
 User* paidBy,
 vector<Split*> splits
) {
 this->id = id;
 this->totalAmount = totalAmount;
 this->paidBy = paidBy;
 this->splits = splits;
 }

 User* getPaidBy() {
 return paidBy;
 }

 vector<Split*> getSplits() {
 return splits;
 }
};

// ----- Splitwise System -----

class SplitwiseSystem {
 vector<User*> users;

 // balance[A][B] = x
 // means A owes B x amount
 map<int, map<int, double>> balance;

public:
 void addUser(User* user) {
 users.push_back(user);
 }

 void addExpense(Expense* expense) {

 User* paidBy = expense->getPaidBy();

 vector<Split*> splits = expense->getSplits();

 for (auto split : splits) {

 User* user = split->getUser();

```

```

 // payer does not owe himself
 if (user->getId() == paidBy->getId())
 continue;

 double amount = split->getAmount();

 int debtor = user->getId();
 int creditor = paidBy->getId();

 balance[debtor][creditor] += amount;
 }
}

void settle(
 User* from,
 User* to,
 double amount
) {

 int fromId = from->getId();
 int toId = to->getId();

 if (balance[fromId][toId] < amount) {
 cout << "Invalid settlement amount\n";
 return;
 }

 balance[fromId][toId] -= amount;

 cout << from->getName()
 << " paid "
 << to->getName()
 << " Rs." << amount
 << "\n";
}

User* getUserById(int id) {

 for (auto user : users) {
 if (user->getId() == id)
 return user;
 }

 return nullptr;
}

void showBalances() {

 cout << "\n----- Balances ----- \n";

 bool found = false;

 for (auto &row : balance) {

 int debtorId = row.first;

 for (auto &entry : row.second) {

 int creditorId = entry.first;
 double amount = entry.second;

```

```

 if (amount <= 0)
 continue;

 User* debtor = getUserById(debtorId);
 User* creditor = getUserById(creditorId);

 cout << debtor->getName()
 << " owes "
 << creditor->getName()
 << " RS."
 << amount
 << "\n";

 found = true;
 }
}

if (!found) {
 cout << "No pending balances\n";
}
}
};

// ----- Main -----

int main() {

 User* A = new User(1, "A");
 User* B = new User(2, "B");
 User* C = new User(3, "C");

 SplitwiseSystem system;

 system.addUser(A);
 system.addUser(B);
 system.addUser(C);

 // A pays Rs.900
 // Equal split:
 // A = 300
 // B = 300
 // C = 300

 vector<Split*> splits;

 splits.push_back(new Split(A, 300));
 splits.push_back(new Split(B, 300));
 splits.push_back(new Split(C, 300));

 Expense* dinner =
 new Expense(
 1,
 900,
 A,
 splits
);

 system.addExpense(dinner);

 system.showBalances();
}

```

```
// B settles Rs.100 with A

system.settle(B, A, 100);

system.showBalances();

return 0;
}
```

Output:

```
----- Balances -----
B owes A Rs.300
C owes A Rs.300

B paid A Rs.100

----- Balances -----
B owes A Rs.200
C owes A Rs.300
```

The core logic is only this:

```
balance[debtor][creditor] += amount;
```

Suppose A paid ₹900 and B's share is ₹300.

Then:

```
debtor = B
creditor = A
```

so:

```
balance[B][A] += 300;
```

Meaning:

*B owes A ₹300.*

Now here's an important problem with this basic version.

Suppose first:

```
B owes A = 300
```

Later B pays ₹500 for something and A's share is ₹200.

Naively we'd get:

```
B owes A = 300
A owes B = 200
```

But ideally we should simplify that to:

```
B owes A = 100
```

That **netting logic** is the next thing I would expect an interviewer to ask.

10/09/2026, 15:43:24



in the above code you didn't use strategy pattern

10/09/2026, 15:48:27



Correct. In that code I intentionally skipped Strategy Pattern and focused only on teaching how `balance[user1][user2]` works.

For the full LLD, Strategy Pattern should be used for the **split calculation**, because Equal, Exact, and Percentage are different algorithms for the same task.

The structure should be:

```
class SplitStrategy {
public:
 virtual vector<Split*> calculateSplit(
 double totalAmount,
 vector<User*> users,
 vector<double> values
) = 0;

 virtual ~SplitStrategy() {}
};
```

Then:

```
class EqualSplitStrategy : public SplitStrategy {
public:
 vector<Split*> calculateSplit(
 double totalAmount,
 vector<User*> users,
 vector<double> values
) override {

 vector<Split*> splits;

 double each = totalAmount / users.size();

 for (auto user : users) {
 splits.push_back(new Split(user, each));
 }

 return splits;
 }
};
```

Exact split:

```
class ExactSplitStrategy : public SplitStrategy {
public:
 vector<Split*> calculateSplit(
 double totalAmount,
 vector<User*> users,
 vector<double> values
) override {

 vector<Split*> splits;

 double sum = 0;
```

```

 for (double x : values)
 sum += x;

 if (sum != totalAmount) {
 cout << "Invalid exact split\n";
 return {};
 }

 for (int i = 0; i < users.size(); i++) {
 splits.push_back(
 new Split(users[i], values[i])
);
 }

 return splits;
}
};

```

Percentage split:

```

class PercentageSplitStrategy : public SplitStrategy {
public:
 vector<Split*> calculateSplit(
 double totalAmount,
 vector<User*> users,
 vector<double> values
) override {

 vector<Split*> splits;

 double sum = 0;

 for (double percentage : values)
 sum += percentage;

 if (sum != 100) {
 cout << "Invalid percentages\n";
 return {};
 }

 for (int i = 0; i < users.size(); i++) {

 double amount =
 totalAmount * values[i] / 100.0;

 splits.push_back(
 new Split(users[i], amount)
);
 }

 return splits;
 }
};

```

Then `SplitwiseSystem` doesn't calculate splits itself.

It gets a strategy:

```

void addExpense(
 int expenseId,
 double amount,
 User* paidBy,

```

```

 vector<User*> participants,
 vector<double> values,
 SplitStrategy* strategy
) {
 vector<Split*> splits =
 strategy->calculateSplit(
 amount,
 participants,
 values
);

 Expense* expense =
 new Expense(
 expenseId,
 amount,
 paidBy,
 splits
);

 updateBalance(expense);
 }

```

And `updateBalance()` remains:

```

void updateBalance(Expense* expense) {

 User* paidBy = expense->getPaidBy();

 for (auto split : expense->getSplits()) {

 User* user = split->getUser();

 if (user->getId() == paidBy->getId())
 continue;

 balance[user->getId()]
 [paidBy->getId()]
 += split->getAmount();
 }
}

```

Then `main()` can choose strategy dynamically:

```

int main() {

 User* A = new User(1, "A");
 User* B = new User(2, "B");
 User* C = new User(3, "C");

 SplitwiseSystem system;

 system.addUser(A);
 system.addUser(B);
 system.addUser(C);

 vector<User*> participants = {A, B, C};

 SplitStrategy* strategy =
 new EqualSplitStrategy();

 system.addExpense(

```



```

 1,
 900,
 A,
 participants,
 {},
 strategy
);

 system.showBalances();
}

```

The important separation is:

```

SplitStrategy
 ↓
calculates:
"How much should each person pay?"

SplitwiseSystem
 ↓
updates:
"Who owes whom?"

```

So Strategy Pattern solves this:

```

Equal
Exact
Percentage

```

while:

```

balance[A][B]

```

solves a completely different problem:

```

debt tracking

```

That distinction is what you should tell the interviewer.

10/09/2026, 15:47:54



how to think of all the member variables and then member functions explain this in detail

10/09/2026, 15:54:42



Yes. This is one of the most important LLD skills: before writing code, you should know how to derive **member variables** and then **member functions** from the requirements.

For interviews, use this order:

**Requirement → Nouns → Classes → What data must each class remember? → Member variables → What actions can happen on that data? → Member functions**

Take Splitwise.

Requirement says: "A user can create a group, add an expense, split it, see balances, settle payments."

From this, nouns suggest classes:

```
User
Group
Expense
Split
SplitwiseSystem
SplitStrategy
```

Now for every class, ask two questions:

1. **What information must this object store permanently?**
2. **What operations are naturally owned by this object?**

That gives variables and functions.

For `User`, ask: what does a user need to remember?

At minimum:

```
class User {
 int id;
 string name;
};
```

Why these variables? Because if someone says "B owes A ₹300", we need identity. `id` uniquely identifies the user, and `name` is for display.

Now functions come from what we need to do with this data:

```
int getId();
string getName();
```

You usually do **not** put `addExpense()` inside `User` just because a user adds an expense in English. The question is: who should actually coordinate creation of an expense? Usually `SplitwiseSystem` or `Group`, not `User`.

That is a big LLD rule:

*A class should not get a method simply because its noun appeared near a verb in the requirement. Give the method to the class that owns that responsibility.*

Now `Group`.

Requirement: "A group has members and expenses."

So ask: what does a group need to remember?

```
class Group {
 int id;
 string name;

 vector<User*> members;
 vector<Expense*> expenses;
};
```

Why?

id	→ identify the group
name	→ display group name
members	→ who belongs to group
expenses	→ expenses created in this group

Now functions naturally follow from the data:

```
void addMember(User* user);
void addExpense(Expense* expense);
vector<User*> getMembers();
vector<Expense*> getExpenses();
```

Notice the pattern:

```
variable: vector<User*> members
 ↓
functions:
addMember()
getMembers()
```

Very often, member functions come naturally from the data owned by the class.

Now `Expense`.

Ask: what does one expense need to remember?

Example:

```
Dinner
₹900
Paid by A
Split among A, B, C
```

So:

```
class Expense {
 int id;
 double amount;
 User* paidBy;
 vector<Split*> splits;
};
```

Each variable corresponds directly to a requirement.

id	→ which expense?
amount	→ how much?
paidBy	→ who actually paid?
splits	→ how much should each participant bear?

Then functions:

```
int getId();
double getAmount();
User* getPaidBy();
vector<Split*> getSplits();
```

You generally don't need `calculateSplit()` here if you are using Strategy Pattern, because calculation belongs to the strategy.

This is another major rule:

*Don't put logic into a class just because the class uses the result. Put logic where the responsibility belongs.*

Now `Split`.

Ask: what is one split?

If B's share is ₹300:

```
User = B
Amount = 300
```

So:

```
class Split {
 User* user;
 double amount;
};
```

Functions:

```
User* getUser();
double getAmount();
```

Very simple.

Now `SplitStrategy`.

This one's different. It doesn't primarily represent stored data. It represents **behavior**.

The requirement says:

```
split equally
split by exact amount
split by percentage
```

So ask:

*What operation is common across all these algorithms?*

Answer:

```
calculateSplit()
```

Therefore:

```
class SplitStrategy {
public:
 virtual vector<Split*> calculateSplit(
 double amount,
 vector<User*> users,
 vector<double> values
```

```
) = 0;
};
```

Then:

```
class EqualSplitStrategy : public SplitStrategy {
public:
 vector<Split*> calculateSplit(...) override;
};
```

So sometimes classes come from **data entities**, and sometimes from **behavior variation**.

Now the most important class: `SplitwiseSystem`.

Ask: what does the system need to remember globally?

Probably:

```
class SplitwiseSystem {
 vector<User*> users;
 vector<Group*> groups;

 map<int, map<int, double>> balance;
};
```

Why?

```
users → registered users
groups → created groups
balance → who owes whom
```

Now ask: what system-level actions exist?

From requirements:

```
create/add user
create/add group
add expense
update balances
show balances
settle payment
```

So:

```
void addUser(User* user);
void addGroup(Group* group);

void addExpense(...);

void updateBalance(Expense* expense);

void showBalances();

void settle(User* from, User* to, double amount);
```

This is the full thought process.

A useful mental template for every class is:

```
Class X

1. Identity
 Does it need id/name?
```

2. Relationships  
Does it contain/reference other objects?
3. State  
What values change over time?
4. Behavior  
What operations should this class own?

Take `VendingMachine` from our previous problem.

For `VendingMachine`:

Identity? Probably no need.

Relationships?

```
Inventory* inventory;
Payment* payment;
```

State?

```
Item* selectedItem;
VendingState* currentState;
```

Behavior?

```
selectItem()
insertMoney()
dispense()
cancel()
setState()
```

Again, same process.

For interviews, I recommend saying something like this before coding:

*"I'll first identify the core entities. Then for each entity I'll decide what state it needs to own, what relationships it has with other objects, and then derive methods from the responsibilities of that class."*

That sounds structured and prevents random class design.

One more very useful rule: if you are unsure whether a variable belongs in a class, ask:

*"Who owns this data?"*

For example:

```
Who owns quantity of Coke?
```

Not `Payment`.

Probably `Inventory` / `Item`.

```
Who owns inserted money?
```

`Payment`.

```
Who owns current selected item?
```

`VendingMachine`.

Who owns group members?

Group .

Similarly, for methods ask:

*“Who should be responsible for changing this data?”*

If `Inventory` owns stock:

```
Inventory::reduceQuantity()
```

makes sense.

If `Payment` owns inserted amount:

```
Payment::addMoney()
```

makes sense.

If `Group` owns members:

```
Group::addMember()
```

makes sense.

This ownership rule alone solves a lot of LLD confusion.

For Splitwise, your final first-pass design could be:

```
class User {
 int id;
 string name;

public:
 int getId();
 string getName();
};

class Split {
 User* user;
 double amount;

public:
 User* getUser();
 double getAmount();
};

class Expense {
 int id;
 double amount;
 User* paidBy;
 vector<Split*> splits;

public:
 User* getPaidBy();
 vector<Split*> getSplits();
};

class Group {
 int id;
 string name;
```

```

 vector<User*> members;
 vector<Expense*> expenses;

public:
 void addMember(User* user);
 void addExpense(Expense* expense);
};

class SplitStrategy {
public:
 virtual vector<Split*> calculateSplit(...) = 0;
};

class SplitwiseSystem {
 vector<User*> users;
 vector<Group*> groups;
 map<int, map<int, double>> balance;

public:
 void addUser(User* user);
 void addGroup(Group* group);
 void addExpense(...);
 void updateBalance(Expense* expense);
 void settle(User* from, User* to, double amount);
 void showBalances();
};

```

The most important thing: **don't try to invent all variables and functions at once.**

Build them from the requirements one by one.

Requirement:

Group has members

Add:

```
vector<User*> members;
```

Requirement:

Add a member

Add:

```
addMember();
```

Requirement:

Expense has payer

Add:

```
User* paidBy;
```

Requirement:

System tracks debts

Add:



```
balance;
```

That is how you should think in an interview, rather than trying to memorize class diagrams.

*10/09/2026, 15:54:10*



now lets start with elevator design

10/09/2026, 16:22:09



For a fresher, first be able to explain the **basic elevator system**, not advanced scheduling or concurrency.

You should start with:

*"I'll model `ElevatorSystem` , `Elevator` , `Floor` , `Request` , and a scheduling/controller class.*

*`ElevatorSystem` manages multiple elevators.*

*Each `Elevator` maintains its current floor, direction, state, and pending requests.*

*A scheduler decides which elevator should serve an external request."*

For now, consider this version:

- Building has multiple floors.
- Multiple elevators.
- Outside each floor, user presses `UP` or `DOWN` .
- Inside elevator, user selects destination floor.
- System chooses an elevator for outside requests.
- Elevator moves and serves requested floors.

Your turn first: tell me the **main classes/entities** you would create.

Don't worry about methods or algorithms yet.

10/09/2026, 16:21:36



main classes will be elevator ,floor, request ,elevator system,

10/09/2026, 16:23:35



Yes, that's a solid first pass.

For a fresher interview, these are enough to start:

```
class Elevator;
class Floor;
class Request;
class ElevatorSystem;
```

I would probably add one more class later:

```
class ElevatorController;
```

because `ElevatorSystem` should not become responsible for every movement and scheduling decision.

Now derive the **member variables**.

For `Elevator` , ask: what does one elevator need to remember?

Likely:

```
class Elevator {
 int id;
 int currentFloor;
 Direction direction; // UP, DOWN, IDLE
 ElevatorState state; // MOVING, STOPPED
};
```

For `Floor` :

```
class Floor {
 int floorNumber;
};
```

For `Request` , we need to distinguish two cases:

```
External request:
Floor 5, UP

Internal request:
Elevator 2 wants floor 10
```

So maybe:

```
class Request {
 int sourceFloor;
 int destinationFloor;
 Direction direction;
};
```

But here is your next interviewer question:

Do you think **external request** and **internal request** should use the same `Request` class, or should we make:

```
ExternalRequest
InternalRequest
```

separate classes?

Tell me what you would choose and why.

*10/09/2026, 16:23:02*



i will create a base class request and then have two different classes ExternalReq and InternalReq?

10/09/2026, 16:26:23



Yes, that is a good design.

You can say to the interviewer:

*"I'll keep a common base `Request` class and derive `ExternalRequest` and `InternalRequest` because both represent elevator requests, but they carry different information."*

For example:

```
class Request {
protected:
 int floor;

public:
 Request(int floor) {
 this->floor = floor;
 }

 int getFloor() {
 return floor;
 }

 virtual ~Request() {}
};
```

External request needs direction:

```
class ExternalRequest : public Request {
 Direction direction;

public:
 ExternalRequest(int floor, Direction direction)
 : Request(floor) {
 this->direction = direction;
 }

 Direction getDirection() {
 return direction;
 }
};
```

Because outside the lift, user says:

Floor 5 + UP

Internal request usually only needs destination floor:

```
class InternalRequest : public Request {
public:
 InternalRequest(int destinationFloor)
 : Request(destinationFloor) {
```

```
}
};
```

Because once you're inside the elevator, you just press:

```
10
```

You don't need to say UP or DOWN explicitly.

One nuance: this inheritance is reasonable, but you could also model them as separate simple classes without inheritance. For a fresher interview, inheritance is fine if you can justify the common abstraction.

Now next step: **Elevator member variables.**

What would you store inside:

```
class Elevator {

};
```

Think about:

- current floor
- current direction
- elevator state
- pending requests
- elevator id

10/09/2026, 16:25:50



```
vector<*Request>requests;
Direction curr;
int floor;
State Curr
int id
```

10/09/2026, 16:29:49



Yes, that's the right idea. I'd just clean up the types/names a bit.

```
enum Direction {
 UP,
 DOWN,
 IDLE
};

enum ElevatorState {
 MOVING,
 STOPPED
};

class Elevator {
 int id;
 int currentFloor;
 Direction currentDirection;
 ElevatorState currentState;

 vector<Request*> requests;
};
```

Now think about whether `vector<Request*> requests` is enough.

Suppose elevator is at floor `5` and currently moving UP. Pending requests are:

```
8
10
3
7
```

If you just keep them in a normal vector, how will you decide which floor to serve next?

For example, while moving UP, ideally:

```
5 → 7 → 8 → 10
```

and then later come down to `3`.

So next interviewer question:

**Which data structure would you use for pending requests, and why?**

You can choose from `vector`, `queue`, `set`, `priority_queue`, or even two separate structures.

10/09/2026, 16:29:16



i will choose 2 vector one for if i am going up and request is also from above then i will process it and the request going down will be stored in the another vector if they are opposite of what i want ?>

10/09/2026, 16:31:32



Yes, that thinking is correct.

You're essentially separating requests into two buckets:

```
vector<int> upRequests;
vector<int> downRequests;
```

If elevator is currently moving UP:

- requests for floors above current floor can go into `upRequests`
- requests that should be handled while going DOWN later can go into `downRequests`

Example:

Elevator at floor 5, moving UP

Requests arrive:

8  
10  
3  
7

Then:

```
upRequests = {8, 10, 7}
downRequests = {3}
```

Before processing, sort `upRequests` ascending:

7, 8, 10

So elevator serves:

5 → 7 → 8 → 10

Then reverse direction and process down requests:

10 → 3

This is basically the same idea as the **SCAN/elevator disk scheduling algorithm**.

But there is one improvement: with vectors, insertion is easy, but repeatedly sorting or removing the next floor can cost extra.

A cleaner interview choice is often:

```
set<int> upRequests;
set<int, greater<int>> downRequests;
```

Then:

- `upRequests.begin()` gives nearest next floor while moving UP



- `downRequests.begin()` gives nearest next floor while moving DOWN

So:

```
class Elevator {
 int id;
 int currentFloor;

 Direction direction;
 ElevatorState state;

 set<int> upRequests;
 set<int, greater<int>> downRequests;
};
```

Example:

```
currentFloor = 5

upRequests = {7, 8, 10}
downRequests = {3, 2}
```

While going up:

```
int nextFloor = *upRequests.begin();
```

gives 7.

After reaching 7:

```
upRequests.erase(upRequests.begin());
```

next becomes 8.

For down requests:

```
int nextFloor = *downRequests.begin();
```

gives the largest floor first, which is exactly what you want while going downward.

Your reasoning was therefore correct. In an interview you can say:

*"I'll maintain separate upward and downward request collections so that an elevator can continue in its current direction and serve requests along the way before reversing."*

Now the next important question: suppose elevator is at floor 5, moving UP toward 10, and someone at floor 7 presses **DOWN**. Should this elevator stop at 7 immediately, or should that request be handled later?

10/09/2026, 16:31:00



i should take request according to the direction if the user pressed the above button at floor 7 and also at floor 10 he should also be wanting to move upwards then we should take that otherwise who ever presses first will get the elevator

10/09/2026, 16:35:43



Yes — the **direction part is right**, but I would not use “whoever pressed first gets the elevator” as the main rule.

For a fresher interview, say this:

*“If an elevator is already moving UP, I prefer assigning it requests that are above its current floor and also want to go UP. Requests in the opposite direction are usually kept pending and served when the elevator reverses direction, unless another idle/closer elevator can serve them sooner.”*

Example:

```
Elevator E1:
current floor = 5
direction = UP
destination = 10
```

Now at floor 7:

User presses UP

E1 can stop at 7, because it is already going that way:

5 → 7 → 10

But if user at floor 7 presses DOWN, then E1 is moving in the opposite direction of that passenger's desired journey.

So normally:

```
E1 continues:
5 → 10

then reverses:
10 → 7
```

and picks the DOWN passenger while coming down.

Why not simply FCFS?

Suppose:

```
E1 at 5, moving UP

t = 1: floor 2 presses UP
t = 2: floor 7 presses UP
```

If we strictly follow “first request wins,” we'd potentially send the elevator backward to 2 even though it is already moving toward 7.

That creates unnecessary direction changes:

5 → 2 → 7

Instead, elevator scheduling generally favors **continuing in the current direction**:

5 → 7

and handles floor 2 later.

So your assignment rule can start like:

```
bool canServe(Elevator* e, ExternalRequest* req) {

 if (e->getDirection() == IDLE)
 return true;

 if (e->getDirection() == UP &&
 req->getFloor() >= e->getCurrentFloor() &&
 req->getDirection() == UP)
 return true;

 if (e->getDirection() == DOWN &&
 req->getFloor() <= e->getCurrentFloor() &&
 req->getDirection() == DOWN)
 return true;

 return false;
}
```

Then among all elevators that `canServe()`, we need to choose the **best one**.

That brings us to the central part of the elevator LLD:

```
class ElevatorController {
public:
 Elevator* selectElevator(ExternalRequest* request);
};
```

Now interviewer question for you:

Suppose floor 7 presses **UP**.

```
E1 = floor 5, moving UP
E2 = floor 6, moving DOWN
E3 = floor 2, IDLE
```

Which elevator would you select, and **what factors would you use to calculate the cost of each elevator?**

10/09/2026, 16:35:13



we will only consider if its idle or moving in the same direction after that absolute distance from its current floor and it request floor

10/09/2026, 17:21:59



Yes, that is a very good fresher-level scheduling rule.

You can tell the interviewer:

*"For an external request, I will first filter elevators that are either idle or already moving in the requested direction and can reach that floor without reversing. Among those eligible elevators, I will choose the one with minimum distance from the request floor."*

For your example:

Request: floor 7, UP

E1 = floor 5, moving UP  
E2 = floor 6, moving DOWN  
E3 = floor 2, IDLE

Eligibility:

E1 → eligible, because it is moving UP and floor 7 is ahead  
E2 → not eligible, because it is moving DOWN  
E3 → eligible, because it is IDLE

Now cost:

E1 cost =  $|7 - 5| = 2$   
E3 cost =  $|7 - 2| = 5$

So choose:

E1

A simple controller could look like this:

```
class ElevatorController {
public:
 Elevator* selectElevator(
 vector<Elevator*>& elevators,
 ExternalRequest* request
) {
 Elevator* best = nullptr;
 int bestCost = INT_MAX;

 for (auto elevator : elevators) {

 bool eligible = false;

 if (elevator->getDirection() == IDLE) {
 eligible = true;
 }

 else if (
 elevator->getDirection() == UP &&
```

```

 request->getDirection() == UP &&
 elevator->getCurrentFloor() <= request->getFloor()
) {
 eligible = true;
 }

 else if (
 elevator->getDirection() == DOWN &&
 request->getDirection() == DOWN &&
 elevator->getCurrentFloor() >= request->getFloor()
) {
 eligible = true;
 }

 if (!eligible)
 continue;

 int cost =
 abs(elevator->getCurrentFloor()
 - request->getFloor());

 if (cost < bestCost) {
 bestCost = cost;
 best = elevator;
 }
}

return best;
}
};

```

But here is the next interviewer-level improvement you should know.

Distance alone is sometimes not enough.

Suppose:

```

Request = floor 7, UP

E1 = floor 5, UP
 already has stops: 6, 8, 9, 10

E2 = floor 3, IDLE

```

By absolute distance:

```

E1 = 2 floors away
E2 = 4 floors away

```

So we choose E1.

That is still reasonable, but in a more advanced design we may also consider:

```

number of pending stops
current load
direction
distance
whether request lies on current path

```

So eventually we can define:

```
cost =
 distance
 + pendingStopsPenalty
 + directionChangePenalty;
```

But as a fresher, your current rule is absolutely enough to start:

**eligible first, then minimum distance.**

Now next question: once the controller selects an elevator, how should that elevator store and process the new request? Should `ElevatorController` directly move the elevator, or should it just assign the request to `Elevator` and let `Elevator` handle its own movement?

10/09/2026, 17:21:26



what is the work of elevatorContoller?  
to give my elevator and then elevator handles its movement  
?

10/09/2026, 17:26:37



Exactly.

`ElevatorController` should mainly **decide which elevator gets a request**. It should not micromanage every floor movement.

Think of responsibilities like this:

```
ElevatorSystem
 → owns all elevators/controllers

ElevatorController
 → receives an external request
 → evaluates elevators
 → selects the best elevator
 → assigns the request to it

Elevator
 → stores its pending stops
 → decides next stop from its request queues
 → moves floor by floor
 → opens/closes door
 → changes direction when needed
```

So flow:

```
Floor 7 presses UP
 ↓
ExternalRequest(7, UP)
 ↓
ElevatorController
 ↓
selectElevator()
 ↓
chooses E1
 ↓
E1.addRequest(request)
 ↓
Elevator E1 handles movement itself
```

For example:

```
class ElevatorController {
public:
 void handleExternalRequest(
 vector<Elevator*>& elevators,
 ExternalRequest* request
) {
 Elevator* elevator =
 selectElevator(elevators, request);

 if (elevator != nullptr) {
 elevator->addRequest(request);
 }
 }
}
```

```

Elevator* selectElevator(
 vector<Elevator*>& elevators,
 ExternalRequest* request
) {
 // scheduling algorithm
}
};

```

Then `Elevator` might have:

```

class Elevator {
 int id;
 int currentFloor;
 Direction direction;

 set<int> upRequests;
 set<int, greater<int>> downRequests;

public:
 void addRequest(Request* request);

 void move();

 void moveUp();

 void moveDown();

 void openDoor();

 void closeDoor();
};

```

A very useful interview sentence is:

*“The controller is responsible for **assignment/scheduling**, while an individual elevator is responsible for **execution and movement**.”*

That separation is important because otherwise `ElevatorController` can become a giant class doing scheduling, queue management, movement, doors, state changes, etc.

There can also be an **internal request** difference. If you're already inside elevator `E1` and press floor `10`, you don't need to choose an elevator anymore. That request can go directly to `E1`:

```

InternalRequest(10)
 ↓
E1.addRequest()

```

Whereas:

```

ExternalRequest(7, UP)
 ↓
ElevatorController
 ↓
choose E1/E2/E3

```

That distinction is worth mentioning in interviews.

10/09/2026, 17:26:05





now give the full code for elevator design

10/09/2026, 17:28:43



For a fresher interview, this is the level you should be able to write and explain confidently: classes, request types, controller selection logic, two request sets, and elevator movement. No over-engineering.

```
#include <bits/stdc++.h>
using namespace std;

enum Direction {
 UP,
 DOWN,
 IDLE
};

enum ElevatorState {
 MOVING,
 STOPPED
};

// -----
// Request
// -----

class Request {
protected:
 int floor;

public:
 Request(int floor) {
 this->floor = floor;
 }

 int getFloor() {
 return floor;
 }

 virtual ~Request() {}
};

// -----
// External Request
// Example: floor 5 presses UP
// -----

class ExternalRequest : public Request {
 Direction direction;

public:
 ExternalRequest(int floor, Direction direction)
 : Request(floor) {
 this->direction = direction;
 }

 Direction getDirection() {
 return direction;
 }
}
```

```

};

// -----
// Internal Request
// Example: user inside elevator presses floor 10
// -----

class InternalRequest : public Request {

public:
 InternalRequest(int destinationFloor)
 : Request(destinationFloor) {
 }
};

// -----
// Elevator
// -----

class Elevator {
 int id;
 int currentFloor;

 Direction currentDirection;
 ElevatorState currentState;

 // ascending order
 set<int> upRequests;

 // descending order
 set<int, greater<int>> downRequests;

public:

 Elevator(int id, int currentFloor = 0) {
 this->id = id;
 this->currentFloor = currentFloor;
 this->currentDirection = IDLE;
 this->currentState = STOPPED;
 }

 int getId() {
 return id;
 }

 int getCurrentFloor() {
 return currentFloor;
 }

 Direction getDirection() {
 return currentDirection;
 }

 ElevatorState getState() {
 return currentState;
 }

 int getPendingRequestsCount() {
 return upRequests.size() + downRequests.size();
 }
}

```

```

// -----
// Adding internal request
// -----

void addInternalRequest(InternalRequest* request) {

 int floor = request->getFloor();

 cout << "Internal request for floor "
 << floor
 << " added to elevator "
 << id
 << "\n";

 if (floor > currentFloor) {
 upRequests.insert(floor);
 }
 else if (floor < currentFloor) {
 downRequests.insert(floor);
 }
 else {
 cout << "Elevator already at floor "
 << floor << "\n";
 openDoor();
 closeDoor();
 }
}

// -----
// Adding external request
// -----

void addExternalRequest(ExternalRequest* request) {

 int floor = request->getFloor();

 cout << "External request assigned to Elevator "
 << id
 << " for floor "
 << floor
 << "\n";

 /*
 Important:

 If request floor is above us,
 we need to reach that floor first.

 If below us,
 we need to eventually go down there.
 */

 if (floor > currentFloor) {
 upRequests.insert(floor);
 }
 else if (floor < currentFloor) {
 downRequests.insert(floor);
 }
 else {
 // Request is from our current floor
 openDoor();
 }
}

```

```

 closeDoor();
 }
}

// -----
// Process all requests
// -----

void processRequests() {

 while (!upRequests.empty() ||
 !downRequests.empty()) {

 /*
 * Continue in current direction
 * if possible.
 */

 if (currentDirection == UP) {

 if (!upRequests.empty()) {
 processUpRequests();
 }
 else {
 currentDirection = DOWN;
 }
 }

 else if (currentDirection == DOWN) {

 if (!downRequests.empty()) {
 processDownRequests();
 }
 else {
 currentDirection = UP;
 }
 }

 else {

 /*
 * Elevator is idle.
 *
 * Decide which direction to start.
 */

 if (!upRequests.empty()) {
 currentDirection = UP;
 }
 else if (!downRequests.empty()) {
 currentDirection = DOWN;
 }
 }
 }

 currentDirection = IDLE;
 currentState = STOPPED;

 cout << "Elevator "
 << id
 << " is now IDLE at floor "

```

```

 << currentFloor
 << "\n";
 }

private:

 // -----
 // Process next upward stop
 // -----

 void processUpRequests() {

 if (upRequests.empty())
 return;

 int nextFloor = *upRequests.begin();

 upRequests.erase(upRequests.begin());

 moveToFloor(nextFloor);
 }

 // -----
 // Process next downward stop
 // -----

 void processDownRequests() {

 if (downRequests.empty())
 return;

 int nextFloor = *downRequests.begin();

 downRequests.erase(downRequests.begin());

 moveToFloor(nextFloor);
 }

 // -----
 // Elevator physically moves
 // -----

 void moveToFloor(int destination) {

 currentState = MOVING;

 if (destination > currentFloor) {

 currentDirection = UP;

 while (currentFloor < destination) {
 currentFloor++;

 cout << "Elevator "
 << id
 << " moving UP -> floor "
 << currentFloor
 << "\n";
 }
 }
 }
}

```

```

 else if (destination < currentFloor) {

 currentDirection = DOWN;

 while (currentFloor > destination) {
 currentFloor--;

 cout << "Elevator "
 << id
 << " moving DOWN -> floor "
 << currentFloor
 << "\n";
 }
 }

 currentState = STOPPED;

 cout << "Elevator "
 << id
 << " reached floor "
 << destination
 << "\n";

 openDoor();
 closeDoor();
 }

 void openDoor() {
 cout << "Elevator "
 << id
 << " door opened at floor "
 << currentFloor
 << "\n";
 }

 void closeDoor() {
 cout << "Elevator "
 << id
 << " door closed\n";
 }
};

// -----
// Floor
// -----

class Floor {
 int floorNumber;

public:
 Floor(int floorNumber) {
 this->floorNumber = floorNumber;
 }

 int getFloorNumber() {
 return floorNumber;
 }
};

// -----
// Elevator Controller

```

```
// -----

class ElevatorController {

public:

 Elevator* selectElevator(
 vector<Elevator*>& elevators,
 ExternalRequest* request
) {

 Elevator* bestElevator = nullptr;
 int minimumCost = INT_MAX;

 for (auto elevator : elevators) {

 bool eligible = false;

 /*
 Rule 1:
 Idle elevator can serve.
 */

 if (elevator->getDirection() == IDLE) {
 eligible = true;
 }

 /*
 Rule 2:
 Elevator moving UP can serve
 if request is UP and request floor
 is ahead of elevator.
 */

 else if (
 elevator->getDirection() == UP &&
 request->getDirection() == UP &&
 elevator->getCurrentFloor()
 <= request->getFloor()
) {
 eligible = true;
 }

 /*
 Rule 3:
 Same idea for DOWN.
 */

 else if (
 elevator->getDirection() == DOWN &&
 request->getDirection() == DOWN &&
 elevator->getCurrentFloor()
 >= request->getFloor()
) {
 eligible = true;
 }

 if (!eligible)
 continue;

 int cost =
```

```

 abs(
 elevator->getCurrentFloor()
 - request->getFloor()
);

 if (cost < minimumCost) {
 minimumCost = cost;
 bestElevator = elevator;
 }
 }

 /*
 * Fallback:
 * if no elevator matched direction,
 * choose nearest elevator.
 */

 if (bestElevator == nullptr) {

 for (auto elevator : elevators) {

 int cost =
 abs(
 elevator->getCurrentFloor()
 - request->getFloor()
);

 if (cost < minimumCost) {
 minimumCost = cost;
 bestElevator = elevator;
 }
 }
 }

 return bestElevator;
}

void handleExternalRequest(
 vector<Elevator*>& elevators,
 ExternalRequest* request
) {

 Elevator* selected =
 selectElevator(
 elevators,
 request
);

 if (selected == nullptr) {
 cout << "No elevator available\n";
 return;
 }

 cout << "\nController selected Elevator "
 << selected->getId()
 << "\n";

 selected->addExternalRequest(request);
}
};

```



```

// -----
// Elevator System
// -----

class ElevatorSystem {

 vector<Elevator*> elevators;

 ElevatorController* controller;

public:

 ElevatorSystem() {
 controller =
 new ElevatorController();
 }

 void addElevator(Elevator* elevator) {
 elevators.push_back(elevator);
 }

 /*
 Called when someone presses
 UP/DOWN outside elevator.
 */

 void requestElevator(
 int floor,
 Direction direction
) {

 ExternalRequest* request =
 new ExternalRequest(
 floor,
 direction
);

 controller->handleExternalRequest(
 elevators,
 request
);
 }

 /*
 Called after passenger enters
 an already selected elevator.
 */

 void selectFloor(
 int elevatorId,
 int destinationFloor
) {

 Elevator* elevator =
 getElevatorById(elevatorId);

 if (elevator == nullptr) {
 cout << "Invalid elevator\n";
 return;
 }
 }
}

```

```

 InternalRequest* request =
 new InternalRequest(
 destinationFloor
);

 elevator->addInternalRequest(request);
 }

 void runElevator(int elevatorId) {

 Elevator* elevator =
 getElevatorById(elevatorId);

 if (elevator != nullptr) {
 elevator->processRequests();
 }
 }

private:

 Elevator* getElevatorById(int id) {

 for (auto elevator : elevators) {

 if (elevator->getId() == id)
 return elevator;
 }

 return nullptr;
 }
};

// -----
// MAIN
// -----

int main() {

 ElevatorSystem system;

 Elevator* e1 =
 new Elevator(1, 2);

 Elevator* e2 =
 new Elevator(2, 6);

 Elevator* e3 =
 new Elevator(3, 10);

 system.addElevator(e1);
 system.addElevator(e2);
 system.addElevator(e3);

 /*
 Someone on floor 5 presses UP.
 */

 cout << "\n==== External Request ==== \n";

 system.requestElevator(
 5,

```

```

 UP
);

 /*
 Assume Elevator 2 was selected.

 Passenger enters elevator 2
 and presses floor 9.
 */

 cout << "\n==== Internal Request =====\n";

 system.selectFloor(
 2,
 9
);

 /*
 Process Elevator 2's requests.
 */

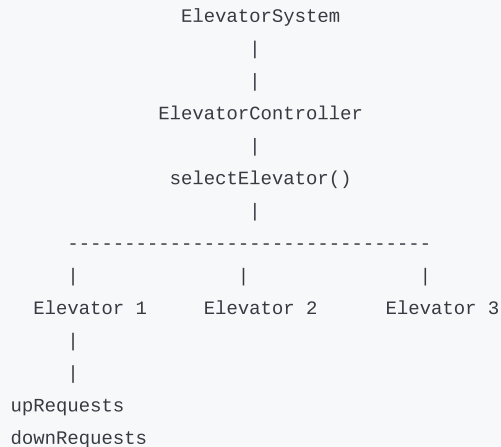
 cout << "\n==== Movement =====\n";

 system.runElevator(2);

 return 0;
}

```

The architecture you should explain is:



And requests flow differently.

For an **external request**:

```

Floor 5 presses UP
 ↓
ExternalRequest
 ↓
ElevatorSystem
 ↓
ElevatorController
 ↓
choose best elevator
 ↓
Elevator.addExternalRequest()

```

For an **internal request**:

```
Passenger inside E2 presses 9
 ↓
InternalRequest
 ↓
directly to Elevator 2
 ↓
E2.addInternalRequest()
```

The most important separation to remember is:

`ElevatorController` answers **“Which elevator?”**

while:

`Elevator` answers **“How do I serve my assigned stops?”**

And the two sets:

```
set<int> upRequests;
set<int, greater<int>> downRequests;
```

help the elevator serve floors naturally:

```
UP:
5 → 7 → 8 → 10

then DOWN:
10 → 6 → 3
```

For a fresher interview, this is enough. If the interviewer pushes deeper, the next extensions are usually **better scheduling cost, multiple simultaneous requests, door states, capacity/load, emergency mode, and Strategy Pattern for elevator-selection algorithms**.

10/09/2026, 17:28:10



**\*\*Car Rental\*\*** &#x20;  
this is the lld problem now explain each step for answering this in interview and explain everything i am supposed to do in an in person interview

12/09/2026, 16:50:17



For a fresher interview, don't start by throwing classes immediately. First show that you can **structure the problem**.

For a **Car Rental System**, I'd answer in this exact sequence.

## 1. First clarify the scope

Say something like:

*"I'll assume users can search available cars, reserve one for a time range, pick it up, return it, and make payment. I'll also assume the company has multiple branches and multiple vehicle types. For now I'll ignore GPS tracking, maintenance scheduling, insurance, and dynamic pricing unless you want me to include them."*

This is important because LLD questions are intentionally open-ended.

Then clarify the main use cases:

User registers  
User searches cars  
User selects car  
User creates reservation  
System checks availability  
User pays  
Car is picked up  
Car is returned  
Final bill is generated

## 2. Identify the main entities

From the requirements, extract nouns:

User  
Vehicle  
Car  
Branch  
Reservation  
Payment  
CarRentalSystem

You may also introduce:

VehicleInventory  
PricingStrategy

but don't over-engineer initially.

A good fresher answer:

*"My core entities will be `User`, `Vehicle`, `Branch`, `Reservation`, `Payment`, and `CarRentalSystem`. I may create a `VehicleInventory` for availability management and a pricing strategy if multiple pricing rules are required."*

### 3. Design User

Ask yourself:

*What information must a user remember?*

```
class User {
 int id;
 string name;
 string drivingLicense;
};
```

Possible methods:

```
int getId();
string getName();
```

You don't need to put `bookCar()` inside `User`. Booking should normally be coordinated by the rental system/service.

### 4. Design Vehicle

Instead of making `Car`, `SUV`, `Bike`, etc. independent, start with a base `Vehicle`.

```
enum VehicleType {
 SEDAN,
 SUV,
 HATCHBACK
};

enum VehicleStatus {
 AVAILABLE,
 RESERVED,
 RENTED,
 MAINTENANCE
};

class Vehicle {
 int id;
 string numberPlate;
 string brand;
 VehicleType type;
 VehicleStatus status;
 double pricePerDay;
};
```

Now explain:

*"Vehicle stores its identity, type, current status, and base rental rate."*

If interviewer asks whether subclasses are needed:

```
class Sedan : public Vehicle {};
class SUV : public Vehicle {};
```

You can say:

*"I'll use subclasses only if vehicle types have different behavior. If only the type differs, an enum is simpler."*

That is a good design judgment.

---

## 5. Design Branch

Suppose company has multiple locations.

```
class Branch {
 int id;
 string name;
 string location;

 vector<Vehicle*> vehicles;
};
```

Responsibilities:

```
void addVehicle(Vehicle* vehicle);
Vehicle* getVehicle(int id);
vector<Vehicle*> getAvailableVehicles();
```

Now interview explanation:

*"A branch owns or manages the vehicles physically available at that location."*

---

## 6. Design Reservation

This is the most important class.

Ask:

*What does one reservation need to remember?*

Suppose:

```
User A
Car C1
Sept 12 to Sept 15
₹2000/day
```

So:

```
enum ReservationStatus {
 CREATED,
 CONFIRMED,
 CANCELLED,
 COMPLETED
};

class Reservation {
 int id;

 User* user;
 Vehicle* vehicle;

 string startDate;
 string endDate;

 ReservationStatus status;

 double totalAmount;
};
```

Possible methods:

```
void confirm();
void cancel();
void complete();
```

Important interview point:

`VehicleStatus` and `ReservationStatus` are different.

Example:

```
VehicleStatus:
AVAILABLE / RESERVED / RENTED

ReservationStatus:
CREATED / CONFIRMED / CANCELLED / COMPLETED
```

## 7. The tricky part: availability

This is the biggest design question.

A naive solution says:

```
vehicle->status == AVAILABLE
```

But that is insufficient if bookings are for future dates.

Example:

```
Today: Sept 12

Car C1 booked:
Sept 20-22

Another user asks:
Sept 13-15
```

The car is technically available for Sept 13-15 even though it has a future reservation.

So real availability should be based on **overlapping reservations**.

Two ranges overlap if:

```
newStart <= existingEnd
AND
newEnd >= existingStart
```

Equivalent non-overlap condition:

```
newEnd < existingStart
OR
newStart > existingEnd
```

So you can tell interviewer:

*"I would not rely only on a global `AVAILABLE` flag for future bookings. I'll check the vehicle's reservations for date-range overlap."*

This is a very strong point.



## 8. Add `VehicleInventory`

Instead of putting all availability logic inside `Branch`, we can separate it:

```
class VehicleInventory {
 vector<Vehicle*> vehicles;

public:
 vector<Vehicle*> searchAvailableVehicles(
 string startDate,
 string endDate,
 VehicleType type
);
};
```

Conceptually:

```
Branch
|
+-- VehicleInventory
 |
 +-- vehicles
```

Now SRP is cleaner:

```
Branch
 → represents location

VehicleInventory
 → manages/searches vehicles
```

## 9. Payment

Keep it simple initially.

```
enum PaymentStatus {
 PENDING,
 SUCCESS,
 FAILED
};

class Payment {
 int id;
 double amount;
 PaymentStatus status;

public:
 bool pay();
};
```

If interviewer asks about:

```
Card
UPI
Cash
Wallet
```

then use Strategy Pattern:

```
class PaymentStrategy {
public:
 virtual bool pay(double amount) = 0;
};
```

Then:

```
class CardPayment : public PaymentStrategy {};
class UPIPayment : public PaymentStrategy {};
class CashPayment : public PaymentStrategy {};
```

This is where **Strategy Pattern** naturally fits.

## 10. Pricing

Initially:

```
price = pricePerDay * numberOfDays;
```

But interviewer may ask:

*“What if SUVs have different pricing? Weekend pricing? Luxury pricing?”*

Then introduce:

```
class PricingStrategy {
public:
 virtual double calculatePrice(
 Vehicle* vehicle,
 int days
) = 0;
};
```

For example:

```
class NormalPricingStrategy : public PricingStrategy {};
class WeekendPricingStrategy : public PricingStrategy {};
class LuxuryPricingStrategy : public PricingStrategy {};
```

Again, Strategy Pattern because:

*same operation, different algorithms.*

## 11. CarRentalSystem

Now create the coordinator.

```
class CarRentalSystem {
 vector<User*> users;
 vector<Branch*> branches;
 vector<Reservation*> reservations;

public:

 vector<Vehicle*> searchVehicles(...);

 Reservation* createReservation(
 User* user,
```

```

 Vehicle* vehicle,
 string startDate,
 string endDate
);

 void cancelReservation(int reservationId);

 void pickupVehicle(int reservationId);

 void returnVehicle(int reservationId);
};

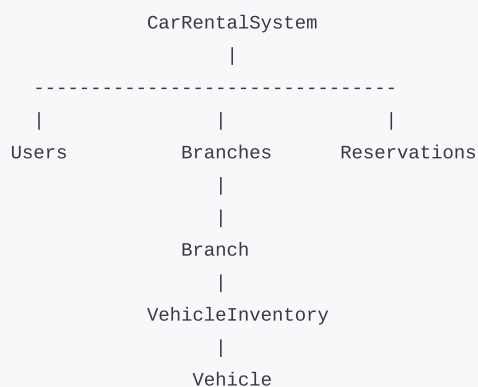
```

This class coordinates the workflow.

It should not itself contain all pricing, payment, or inventory algorithms.

## 12. Full basic class structure

In an interview, you can draw:



Reservation

```

|
+-- User
|
+-- Vehicle
|
+-- start/end time
|
+-- status

```

PaymentStrategy

```

|
+-- CardPayment
+-- UPIPayment

```

PricingStrategy

```

|
+-- NormalPricing
+-- WeekendPricing

```

## 13. Main flow you should explain

Suppose user wants an SUV from Sept 15-18.

Flow:

```
User searches:
SUV, Sept 15-18
 ↓
CarRentalSystem
 ↓
Branch / VehicleInventory
 ↓
find vehicles with no overlapping reservation
 ↓
return available SUVs
 ↓
user selects vehicle
 ↓
create Reservation
 ↓
calculate price
 ↓
payment
 ↓
reservation CONFIRMED
```

At pickup:

```
Reservation CONFIRMED
 ↓
vehicle.status = RENTED
```

At return:

```
calculate final charges
 ↓
complete payment
 ↓
reservation.status = COMPLETED
 ↓
vehicle.status = AVAILABLE
```

---

## 14. What should you say about SOLID?

Keep this concise.

### SRP

```
Vehicle
→ vehicle information

Reservation
→ booking information

VehicleInventory
→ availability/search

PaymentStrategy
→ payment behavior

PricingStrategy
→ pricing behavior
```

## OCP

If new payment method comes:

```
class CryptoPayment : public PaymentStrategy
```

without changing existing payment logic.

Same for new pricing methods.

## DIP

CarRentalSystem should depend on:

```
PaymentStrategy*
PricingStrategy*
```

rather than concrete:

```
CardPayment
WeekendPricing
```

---

## 15. Common interviewer follow-ups

You should be ready for these.

**“Two people try to reserve same car simultaneously?”**

You say:

*“The availability check and reservation creation must be atomic. In a real system I would use locking or a database transaction so only one reservation succeeds.”*

**“What if user returns late?”**

Store actual return time and calculate extra charges.

**“What if vehicle gets damaged?”**

Add damage charges during return processing.

**“Can pickup and dropoff branches differ?”**

Reservation can store:

```
Branch* pickupBranch;
Branch* dropoffBranch;
```

**“Can one reservation contain multiple cars?”**

For normal car rental, one reservation usually maps to one vehicle. If required, introduce `ReservationItem`.

**“How do you prevent double booking?”**

Date-overlap check + transactional locking.

---

## 16. What you should actually write on the board

You don't need 300 lines of code immediately.

Start with something like:

```
class User;
class Vehicle;
class Branch;
```

```
class Reservation;
class PaymentStrategy;
class PricingStrategy;
class CarRentalSystem;
```

Then fill the important members.

```
class Vehicle {
 int id;
 VehicleType type;
 VehicleStatus status;
 double pricePerDay;
};

class Reservation {
 int id;
 User* user;
 Vehicle* vehicle;
 Date startDate;
 Date endDate;
 ReservationStatus status;
 double amount;
};

class Branch {
 int id;
 vector<Vehicle*> vehicles;
};

class CarRentalSystem {
 vector<Branch*> branches;
 vector<Reservation*> reservations;
};
```

Then explain the flow.

Only write detailed methods if interviewer asks.

## 17. Best interview sequence

Memorize this process, not the exact classes:

1. Clarify requirements  
↓
2. Identify core entities  
↓
3. Find data each entity owns  
↓
4. Derive member functions from responsibilities  
↓
5. Define relationships  
↓
6. Walk through one complete use case  
↓
7. Identify changing behavior  
↓
8. Apply pattern only where useful  
↓
9. Discuss edge cases  
↓
10. Discuss concurrency/extensibility if asked

For **Car Rental**, your 30-second opening answer can be:

*"I'll model the core system using `User` , `Vehicle` , `Branch` , `Reservation` , and `CarRentalSystem` . A branch manages vehicles, a reservation connects a user with a vehicle for a time interval, and availability is determined using reservation date overlap rather than only a simple status flag. I'll keep payment and pricing extensible using Strategy Pattern if multiple implementations are required. Then I'll walk through search, booking, pickup, and return."*

That is exactly the kind of structured answer a fresher should be able to give before going deep.

12/09/2026, 16:50:18



implement important classes fully with their methods also\\

12/09/2026, 18:16:16



For a fresher interview, this is a good “complete enough” implementation: core entities, availability, reservation creation/cancel, pricing, payment, pickup, and return. I’ll keep it simple C++ and avoid over-engineering.

```
#include <bits/stdc++.h>
using namespace std;

enum VehicleType {
 SEDAN,
 SUV,
 HATCHBACK
};

enum VehicleStatus {
 AVAILABLE,
 RESERVED,
 RENTED,
 MAINTENANCE
};

enum ReservationStatus {
 CREATED,
 CONFIRMED,
 CANCELLED,
 COMPLETED
};

enum PaymentStatus {
 PENDING,
 SUCCESS,
 FAILED
};

// -----
// User
// -----

class User {
 int id;
 string name;
 string drivingLicense;

public:
 User(int id, string name, string license) {
 this->id = id;
 this->name = name;
 this->drivingLicense = license;
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }
}
```



```

 }

 string getLicense() {
 return drivingLicense;
 }
};

// -----
// Vehicle
// -----

class Vehicle {
 int id;
 string numberPlate;
 string brand;
 VehicleType type;
 VehicleStatus status;
 double pricePerDay;

public:
 Vehicle(
 int id,
 string numberPlate,
 string brand,
 VehicleType type,
 double pricePerDay
) {
 this->id = id;
 this->numberPlate = numberPlate;
 this->brand = brand;
 this->type = type;
 this->pricePerDay = pricePerDay;
 this->status = AVAILABLE;
 }

 int getId() {
 return id;
 }

 string getNumberPlate() {
 return numberPlate;
 }

 string getBrand() {
 return brand;
 }

 VehicleType getType() {
 return type;
 }

 VehicleStatus getStatus() {
 return status;
 }

 double getPricePerDay() {
 return pricePerDay;
 }

 void setStatus(VehicleStatus status) {
 this->status = status;
 }

```

```

 }
};

// -----
// Pricing Strategy
// -----

class PricingStrategy {
public:
 virtual double calculatePrice(
 Vehicle* vehicle,
 int days
) = 0;

 virtual ~PricingStrategy() {}
};

class NormalPricingStrategy : public PricingStrategy {
public:
 double calculatePrice(
 Vehicle* vehicle,
 int days
) override {
 return vehicle->getPricePerDay() * days;
 }
};

// -----
// Payment Strategy
// -----

class PaymentStrategy {
public:
 virtual bool pay(double amount) = 0;
 virtual ~PaymentStrategy() {}
};

class UPIPayment : public PaymentStrategy {
public:
 bool pay(double amount) override {
 cout << "Paid Rs." << amount << " using UPI\n";
 return true;
 }
};

class CardPayment : public PaymentStrategy {
public:
 bool pay(double amount) override {
 cout << "Paid Rs." << amount << " using Card\n";
 return true;
 }
};

// -----
// Payment
// -----

class Payment {
 int id;
 double amount;
 PaymentStatus status;
};

```

```

public:
 Payment(int id, double amount) {
 this->id = id;
 this->amount = amount;
 this->status = PENDING;
 }

 bool processPayment(PaymentStrategy* strategy) {
 bool success = strategy->pay(amount);

 if (success)
 status = SUCCESS;
 else
 status = FAILED;

 return success;
 }

 PaymentStatus getStatus() {
 return status;
 }
};

// -----
// Reservation
// -----

class Reservation {
 int id;
 User* user;
 Vehicle* vehicle;

 int startDay;
 int endDay;

 ReservationStatus status;
 double totalAmount;

public:
 Reservation(
 int id,
 User* user,
 Vehicle* vehicle,
 int startDay,
 int endDay,
 double totalAmount
) {
 this->id = id;
 this->user = user;
 this->vehicle = vehicle;
 this->startDay = startDay;
 this->endDay = endDay;
 this->totalAmount = totalAmount;
 this->status = CREATED;
 }

 int getId() {
 return id;
 }
}

```

```

 User* getUser() {
 return user;
 }

 Vehicle* getVehicle() {
 return vehicle;
 }

 int getStartDay() {
 return startDay;
 }

 int getEndDay() {
 return endDay;
 }

 double getTotalAmount() {
 return totalAmount;
 }

 ReservationStatus getStatus() {
 return status;
 }

 void confirm() {
 status = CONFIRMED;
 }

 void cancel() {
 status = CANCELLED;
 }

 void complete() {
 status = COMPLETED;
 }
};

// -----
// Vehicle Inventory
// -----

class VehicleInventory {
 vector<Vehicle*> vehicles;

public:
 void addVehicle(Vehicle* vehicle) {
 vehicles.push_back(vehicle);
 }

 vector<Vehicle*> getVehicles() {
 return vehicles;
 }

 Vehicle* getVehicleById(int id) {
 for (auto vehicle : vehicles) {
 if (vehicle->getId() == id)
 return vehicle;
 }

 return nullptr;
 }
}

```

```

};

// -----
// Branch
// -----

class Branch {
 int id;
 string name;
 string location;

 VehicleInventory* inventory;

public:
 Branch(
 int id,
 string name,
 string location
) {
 this->id = id;
 this->name = name;
 this->location = location;
 this->inventory = new VehicleInventory();
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }

 string getLocation() {
 return location;
 }

 VehicleInventory* getInventory() {
 return inventory;
 }

 void addVehicle(Vehicle* vehicle) {
 inventory->addVehicle(vehicle);
 }
};

// -----
// Car Rental System
// -----

class CarRentalSystem {
 vector<User*> users;
 vector<Branch*> branches;
 vector<Reservation*> reservations;

 int nextReservationId = 1;
 int nextPaymentId = 1;

public:

 void addUser(User* user) {

```

```

 users.push_back(user);
 }

 void addBranch(Branch* branch) {
 branches.push_back(branch);
 }

 bool isVehicleAvailable(
 Vehicle* vehicle,
 int startDay,
 int endDay
) {
 if (vehicle->getStatus() == MAINTENANCE)
 return false;

 for (auto reservation : reservations) {

 if (reservation->getVehicle()->getId()
 != vehicle->getId()) {
 continue;
 }

 if (reservation->getStatus() == CANCELLED)
 continue;

 int existingStart =
 reservation->getStartDay();

 int existingEnd =
 reservation->getEndDay();

 bool overlap =
 startDay <= existingEnd &&
 endDay >= existingStart;

 if (overlap)
 return false;
 }

 return true;
 }

 vector<Vehicle*> searchVehicles(
 Branch* branch,
 VehicleType type,
 int startDay,
 int endDay
) {
 vector<Vehicle*> result;

 for (auto vehicle :
 branch->getInventory()->getVehicles()) {

 if (vehicle->getType() != type)
 continue;

 if (isVehicleAvailable(
 vehicle,
 startDay,
 endDay)) {

```

```

 result.push_back(vehicle);
 }
}

return result;
}

Reservation* createReservation(
 User* user,
 Vehicle* vehicle,
 int startDay,
 int endDay,
 PricingStrategy* pricingStrategy
) {
 if (!isVehicleAvailable(
 vehicle,
 startDay,
 endDay)) {

 cout << "Vehicle not available\n";
 return nullptr;
 }

 int days =
 endDay - startDay + 1;

 double amount =
 pricingStrategy->calculatePrice(
 vehicle,
 days
);

 Reservation* reservation =
 new Reservation(
 nextReservationId++,
 user,
 vehicle,
 startDay,
 endDay,
 amount
);

 reservations.push_back(reservation);

 cout << "Reservation created for "
 << user->getName()
 << ", amount = Rs."
 << amount
 << "\n";

 return reservation;
}

bool makePayment(
 Reservation* reservation,
 PaymentStrategy* paymentStrategy
) {
 if (reservation == nullptr)
 return false;

 Payment* payment =

```

```

 new Payment(
 nextPaymentId++,
 reservation->getTotalAmount()
);

 bool success =
 payment->processPayment(
 paymentStrategy
);

 if (success) {
 reservation->confirm();
 reservation->getVehicle()
 ->setStatus(RESERVED);

 cout << "Reservation confirmed\n";
 }

 return success;
}

void pickupVehicle(
 Reservation* reservation
) {
 if (reservation == nullptr)
 return;

 if (reservation->getStatus()
 != CONFIRMED) {

 cout << "Reservation not confirmed\n";
 return;
 }

 reservation->getVehicle()
 ->setStatus(RENTED);

 cout << "Vehicle picked up by "
 << reservation->getUser()->getName()
 << "\n";
}

void returnVehicle(
 Reservation* reservation
) {
 if (reservation == nullptr)
 return;

 if (reservation->getVehicle()->getStatus()
 != RENTED) {

 cout << "Vehicle is not currently rented\n";
 return;
 }

 reservation->getVehicle()
 ->setStatus(AVAILABLE);

 reservation->complete();

 cout << "Vehicle returned successfully\n";
}

```



```

 cout << "Reservation completed\n";
 }

 void cancelReservation(
 Reservation* reservation
) {
 if (reservation == nullptr)
 return;

 if (reservation->getStatus()
 == COMPLETED) {

 cout << "Completed reservation cannot be cancelled\n";
 return;
 }

 reservation->cancel();

 reservation->getVehicle()
 ->setStatus(AVAILABLE);

 cout << "Reservation cancelled\n";
 }

 void showReservations() {

 cout << "\n----- Reservations ----- \n";

 for (auto reservation : reservations) {

 cout << "Reservation ID: "
 << reservation->getId()
 << "\n";

 cout << "User: "
 << reservation->getUser()->getName()
 << "\n";

 cout << "Vehicle: "
 << reservation->getVehicle()
 ->getNumberPlate()
 << "\n";

 cout << "Amount: Rs."
 << reservation->getTotalAmount()
 << "\n\n";
 }
 }
};

// -----
// MAIN
// -----

int main() {

 CarRentalSystem system;

 // Users
 User* user1 =
 new User(

```

```

 1,
 "Anirudh",
 "DL123"
);

system.addUser(user1);

// Branch
Branch* branch =
 new Branch(
 1,
 "Prayagraj Branch",
 "Civil Lines"
);

system.addBranch(branch);

// Vehicles
Vehicle* car1 =
 new Vehicle(
 1,
 "UP70AB1234",
 "Honda City",
 SEDAN,
 2000
);

Vehicle* car2 =
 new Vehicle(
 2,
 "UP70XY9876",
 "Hyundai Creta",
 SUV,
 3000
);

branch->addVehicle(car1);
branch->addVehicle(car2);

// Pricing strategy
PricingStrategy* pricing =
 new NormalPricingStrategy();

// Search SUV from day 10 to 12
vector<Vehicle*> available =
 system.searchVehicles(
 branch,
 SUV,
 10,
 12
);

cout << "Available SUVs: "
 << available.size()
 << "\n";

if (!available.empty()) {

 Vehicle* selected =
 available[0];

```

```

// Create reservation
Reservation* reservation =
 system.createReservation(
 user1,
 selected,
 10,
 12,
 pricing
);

// Payment strategy
PaymentStrategy* payment =
 new UPIPayment();

// Pay and confirm
system.makePayment(
 reservation,
 payment
);

// Pickup
system.pickupVehicle(
 reservation
);

// Return
system.returnVehicle(
 reservation
);
}

system.showReservations();

return 0;
}

```

The most important relationships to explain are:

```

CarRentalSystem
|
+-- Users
+-- Branches
+-- Reservations

Branch
|
+-- VehicleInventory
 |
 +-- Vehicles

Reservation
|
+-- User
+-- Vehicle
+-- start/end date
+-- amount
+-- status

```

The central workflow is:

```
searchVehicles()
 ↓
isVehicleAvailable()
 ↓
createReservation()
 ↓
PricingStrategy
 ↓
makePayment()
 ↓
reservation CONFIRMED
 ↓
pickupVehicle()
 ↓
vehicle RENTED
 ↓
returnVehicle()
 ↓
reservation COMPLETED
```

One interview caveat: in a production system, I would **not rely on** `VehicleStatus = RESERVED` **alone for future bookings**, because a car may have multiple non-overlapping future reservations. The real source of truth for booking availability is the reservation date-overlap check. The status is mainly useful for the vehicle's current operational state like `RENTED` or `MAINTENANCE`.

Also, if the interviewer says “two users book the same car at exactly the same time,” this in-memory code is not enough. You should say the **availability check + reservation creation must be atomic**, usually using a transaction/locking mechanism.

12/09/2026, 18:16:17



Parking\_lot; this is the LLD problem now explain each step for answering this in interview and explain everything i am supposed to do in an in person interview

12/09/2026, 18:36:12



For a fresher interview, Parking Lot is one of the best LLD problems because it tests whether you can identify entities, ownership, states, and extensibility without overcomplicating things.

I'd answer it in this exact order.

## 1. Clarify requirements first

Say something like:

*"I'll assume the parking lot supports multiple floors, different parking spot types, vehicle entry and exit, ticket generation, spot allocation, fee calculation, and payment. I'll keep reservation, valet, EV charging, and dynamic pricing out unless you want them."*

Then confirm the main flow:

```
Vehicle enters
→ system finds a compatible free spot
→ ticket is generated
→ vehicle parks
→ vehicle exits
→ fee is calculated
→ payment happens
→ spot becomes free
```

That already shows structure.

## 2. Identify core entities

From the requirements, pick the nouns:

```
ParkingLot
ParkingFloor
ParkingSpot
Vehicle
Ticket
EntranceGate
ExitGate
Payment
```

You may also need:

```
ParkingSpotManager
PricingStrategy
PaymentStrategy
```

But don't start with 15 classes.

A good fresher answer is:

*"My main entities will be ParkingLot, ParkingFloor, ParkingSpot, Vehicle, Ticket, and gate classes. I'll use a spot manager for allocation and pricing/payment strategies only if multiple rules are required."*

### 3. Design Vehicle

Ask: what does a vehicle need to remember?

```
enum VehicleType {
 BIKE,
 CAR,
 TRUCK
};

class Vehicle {
 string number;
 VehicleType type;
};
```

That's enough initially.

If interviewer asks for subclassing:

```
class Car : public Vehicle {};
class Bike : public Vehicle {};
```

you can say:

*"I'll only subclass if vehicle types have different behavior. If only the type differs, enum is simpler."*

That is a good design choice.

### 4. Design ParkingSpot

This is one of the most important classes.

A spot needs to know:

```
enum SpotType {
 BIKE_SPOT,
 CAR_SPOT,
 TRUCK_SPOT
};

class ParkingSpot {
 int id;
 SpotType type;
 bool occupied;
 Vehicle* vehicle;
};
```

Methods naturally follow:

```
bool canPark(Vehicle* vehicle);
void parkVehicle(Vehicle* vehicle);
void removeVehicle();
bool isFree();
```

This is a very good place to explain ownership:

*"ParkingSpot owns the current parking state of one physical slot."*

### 5. Compatibility rule

You should explicitly define which vehicle can park where.

For a simple version:

```
BIKE → BIKE_SPOT
CAR → CAR_SPOT
TRUCK → TRUCK_SPOT
```

Or maybe allow smaller vehicles in larger spots if interviewer wants.

Then `canPark()` handles this.

Example:

```
bool canPark(Vehicle* vehicle) {
 if (occupied)
 return false;

 if (vehicle->getType() == BIKE &&
 type == BIKE_SPOT)
 return true;

 if (vehicle->getType() == CAR &&
 type == CAR_SPOT)
 return true;

 if (vehicle->getType() == TRUCK &&
 type == TRUCK_SPOT)
 return true;

 return false;
}
```

## 6. Design `ParkingFloor`

A floor contains many spots.

```
class ParkingFloor {
 int floorNumber;
 vector<ParkingSpot*> spots;
};
```

Methods:

```
void addSpot(ParkingSpot* spot);
ParkingSpot* findFreeSpot(Vehicle* vehicle);
int getAvailableCount(SpotType type);
```

So:

```
ParkingFloor
|
+-- many ParkingSpot
```

## 7. Design `ParkingLot`

This is the high-level coordinator/owner.

```
class ParkingLot {
 int id;
 string name;
```



```
vector<ParkingFloor*> floors;
};
```

Possible methods:

```
ParkingSpot* findSpot(Vehicle* vehicle);
void addFloor(ParkingFloor* floor);
```

You can say:

*"ParkingLot owns floors, floors own spots."*

That relationship is important.

## 8. Design Ticket

When a vehicle enters, we need to remember enough information to later calculate fee and release the same spot.

```
enum TicketStatus {
 ACTIVE,
 PAID,
 CLOSED
};

class Ticket {
 int id;
 Vehicle* vehicle;
 ParkingSpot* spot;
 long long entryTime;
 TicketStatus status;
};
```

Important fields:

```
vehicle
spot
entry time
ticket id
status
```

At exit, we use ticket to know:

- who parked
- where they parked
- how long they stayed

## 9. Entry flow

This is something you should definitely walk through verbally.

```
Vehicle arrives at EntryGate
 ↓
ParkingLot finds compatible free spot
 ↓
spot.parkVehicle(vehicle)
 ↓
Ticket created
 ↓
Ticket returned to user
```



You can say:

*“EntryGate coordinates the entry operation but doesn’t itself own parking allocation logic.”*

That keeps responsibilities cleaner.

## 10. EntranceGate

```
class EntranceGate {
 int id;
 ParkingLot* parkingLot;

public:
 Ticket* generateTicket(Vehicle* vehicle);
};
```

Conceptually:

```
Ticket* generateTicket(Vehicle* vehicle) {
 ParkingSpot* spot =
 parkingLot->findSpot(vehicle);

 if (spot == nullptr)
 return nullptr;

 spot->parkVehicle(vehicle);

 return new Ticket(...);
}
```

## 11. ExitGate

At exit:

```
Ticket scanned
 ↓
calculate duration
 ↓
calculate fee
 ↓
take payment
 ↓
spot.removeVehicle()
 ↓
ticket CLOSED
```

Class:

```
class ExitGate {
 int id;

public:
 void processExit(Ticket* ticket);
};
```

## 12. Pricing

Initially keep it simple:

```
Bike = ₹10/hour
Car = ₹20/hour
Truck = ₹30/hour
```

Then if interviewer asks for different pricing models, use Strategy Pattern.

```
class PricingStrategy {
public:
 virtual double calculateFee(
 Ticket* ticket,
 long long exitTime
) = 0;
};
```

Possible strategies:

```
HourlyPricing
FlatPricing
WeekendPricing
```

This is a natural Strategy Pattern use case.

### 13. Payment

Again, same pattern if multiple methods exist:

```
class PaymentStrategy {
public:
 virtual bool pay(double amount) = 0;
};
```

Implementations:

```
CashPayment
CardPayment
UPIPayment
```

### 14. Spot allocation strategy

Another place where Strategy Pattern can fit.

Initially:

*"Pick first available compatible spot."*

Later interviewer might ask:

*"What if I want nearest spot to gate?"*

Then:

```
class ParkingSpotStrategy {
public:
 virtual ParkingSpot* findSpot(
 vector<ParkingFloor*>& floors,
 Vehicle* vehicle
) = 0;
};
```

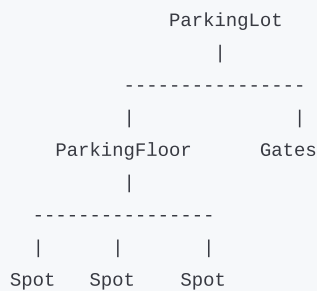
Implementations:

```
FirstAvailableStrategy
NearestSpotStrategy
LowestFloorStrategy
```

Again: same goal, different algorithm.

## 15. Class diagram you should draw

Something like:



```
Vehicle
|
Ticket
|
+-- Vehicle
+-- ParkingSpot
+-- entryTime
+-- status
```

```
EntranceGate
|
+-- create Ticket
```

```
ExitGate
|
+-- PricingStrategy
+-- PaymentStrategy
```

## 16. Most important member variables

If interviewer asks you to start coding, write these first:

```
class Vehicle {
 string number;
 VehicleType type;
};

class ParkingSpot {
 int id;
 SpotType type;
 bool occupied;
 Vehicle* vehicle;
};

class ParkingFloor {
 int floorNumber;
```

```

 vector<ParkingSpot*> spots;
};

class Ticket {
 int id;
 Vehicle* vehicle;
 ParkingSpot* spot;
 long long entryTime;
 TicketStatus status;
};

class ParkingLot {
 vector<ParkingFloor*> floors;
};

```

This is enough to show you understand the model.

## 17. How to derive methods

Use the same rule we discussed earlier:

*Who owns the data, and who should be responsible for changing it?*

**ParkingSpot** owns occupancy:

```

parkVehicle()
removeVehicle()

```

**ParkingFloor** owns spots:

```

addSpot()
findFreeSpot()

```

**ParkingLot** owns floors:

```

addFloor()
findSpot()

```

**Ticket** owns its status:

```

close()
markPaid()

```

**EntranceGate** coordinates entry:

```

generateTicket()

```

**ExitGate** coordinates exit:

```

processExit()

```

That's how you should think.

## 18. SOLID points

For fresher interviews, keep it brief.

**SRP**

ParkingSpot → spot state  
ParkingFloor → spots on one floor  
Ticket → one parking session  
PricingStrategy → fee calculation  
PaymentStrategy → payment handling

## OCP

New pricing model:

```
class WeekendPricing : public PricingStrategy
```

without modifying existing exit logic.

## DIP

ExitGate should depend on:

```
PricingStrategy*
PaymentStrategy*
```

not directly on:

```
UPIPayment
HourlyPricing
```

## 19. Important follow-up questions

Be ready for these.

**“What if parking lot is full?”**

Return no spot / show FULL.

**“What if two cars try to take the last spot?”**

Allocation + spot marking must be atomic. In production, use locking/transaction.

**“How do you show available spots?”**

Maintain counts per floor/type, or compute from spots.

**“How do you find nearest spot efficiently?”**

Maintain ordered/free-spot structures per type/floor instead of scanning all spots.

**“Can bike park in car spot?”**

Make compatibility rule configurable.

**“What if a ticket is lost?”**

Use vehicle number / active parking session lookup.

**“What if payment fails?”**

Do not release the spot or close ticket until successful payment.

## 20. Best 30-second opening answer

Say this:

*“I’ll model the system using `ParkingLot`, `ParkingFloor`, `ParkingSpot`, `Vehicle`, `Ticket`, `EntranceGate`, and `ExitGate`. A parking lot owns multiple floors, each floor owns parking spots, and a ticket represents one active parking session. On entry, the system finds a compatible free spot and creates a ticket. On exit, it calculates the fee, processes payment, frees the spot, and closes the ticket. If multiple allocation, pricing, or payment rules are required, I’ll use Strategy Pattern for those varying behaviors.”*

That's a very solid fresher answer.

The interview sequence you should follow is:

```
clarify requirements
→ identify entities
→ define ownership
→ write important member variables
→ derive methods
→ walk through entry flow
→ walk through exit flow
→ add strategy patterns only if required
→ discuss edge cases/concurrency
```

Don't start by saying "I'll use Factory, Observer, Singleton..." Patterns should come after the requirements show where they help.

*12/09/2026, 18:36:13*



```
...
bool occupied; why not availability checker method ?
...
```

12/09/2026, 18:41:10



Because `occupied` is **state/data**, while `isAvailable()` or `isFree()` is a **method that reads that state**.

You can absolutely have both:

```
class ParkingSpot {
 bool occupied;

public:
 bool isAvailable() {
 return !occupied;
 }
};
```

Why keep `occupied` then? Because the object needs to **remember** whether the spot is currently occupied.

The method itself cannot magically know that unless some state exists underneath.

Think of it like this:

```
occupied = true
```

means actual stored condition.

And:

```
isAvailable()
```

means:

*"Tell me whether this spot is available based on the current condition."*

So:

```
bool isAvailable() {
 return !occupied;
}
```

is better than making `occupied` public.

You can even avoid `occupied` completely and derive availability from:

```
Vehicle* vehicle;
```

For example:

```
class ParkingSpot {
 Vehicle* vehicle = nullptr;

public:
 bool isAvailable() {
 return vehicle == nullptr;
 }
};
```

```
}
};
```

This is actually cleaner because now you avoid storing duplicate state:

```
occupied
vehicle
```

Otherwise these could become inconsistent:

```
occupied = false
vehicle = car123
```

which makes no sense.

So for this design, I would prefer:

```
class ParkingSpot {
 int id;
 SpotType type;
 Vehicle* vehicle;

public:
 bool isAvailable() {
 return vehicle == nullptr;
 }

 void parkVehicle(Vehicle* v) {
 vehicle = v;
 }

 void removeVehicle() {
 vehicle = nullptr;
 }
};
```

That is a better interview answer:

*"I don't necessarily need a separate `occupied` flag because availability can be derived from whether `vehicle` is null. This avoids duplicate state and inconsistency."*

That is the cleaner design.

12/09/2026, 18:41:11





We use an `enum` when something is mainly just a **fixed set of labels/states**, and a class when that thing needs its own **data + behavior**.

For example, in Parking Lot:

```
enum VehicleType {
 BIKE,
 CAR,
 TRUCK
};
```

This works well because `VehicleType` is just telling us which category a vehicle belongs to. It doesn't need separate behavior by itself.

Similarly:

```
enum TicketStatus {
 ACTIVE,
 PAID,
 CLOSED
};
```

These are just states.

If we made classes instead:

```
class BikeType {};
class CarType {};
class TruckType {};
```

that would usually be overkill if all we need is a label.

The rule is:

```
Use enum when:
- finite known set of values
- mostly labels/states
- no unique behavior needed

Use class when:
- object has its own fields
- object has behavior
- different variants behave differently
```

Example where class is better:

```
class PaymentStrategy {
public:
 virtual bool pay(double amount) = 0;
};
```

Then:

```
class UPIPayment : public PaymentStrategy {
public:
```

```

 bool pay(double amount) override {
 // UPI-specific logic
 return true;
 }
};

class CardPayment : public PaymentStrategy {
public:
 bool pay(double amount) override {
 // card-specific logic
 return true;
 }
};

```

Here we should not just do:

```

enum PaymentType {
 UPI,
 CARD
};

```

if each payment method has different processing behavior.

A very good interview sentence is:

*"I use enums when I only need to represent a fixed category or state. If different variants need different behavior or additional data, I prefer classes or polymorphism."*

For Parking Lot specifically:

```

VehicleType
SpotType
TicketStatus

```

are good enums.

But things like:

```

PricingStrategy
PaymentStrategy
ParkingSpotStrategy

```

are better modeled as classes because behavior varies.

12/09/2026, 18:43:41



I don't want to use gates ? is it fine ?

12/09/2026, 18:45:41



Yes, completely fine.

For a fresher LLD, **gates are optional** unless the interviewer explicitly asks about multiple entry/exit gates or gate-specific responsibilities.

You can keep the design simpler:

```
class ParkingLot {
 vector<ParkingFloor*> floors;
 vector<Ticket*> activeTickets;

public:
 Ticket* parkVehicle(Vehicle* vehicle);
 double unparkVehicle(Ticket* ticket);
};
```

Then the flow becomes:

```
Vehicle enters
↓
ParkingLot::parkVehicle()
↓
find free compatible spot
↓
park vehicle
↓
create ticket
```

And exit:

```
ParkingLot::unparkVehicle()
↓
calculate fee
↓
payment
↓
free spot
↓
close ticket
```

This is actually a very reasonable fresher design because it keeps focus on the important parts:

- Vehicle
- ParkingSpot
- ParkingFloor
- ParkingLot
- Ticket
- pricing/payment if required

You should only introduce EntranceGate / ExitGate if the interviewer asks something like:

*"There are multiple entrances and exits. How would you model them?"*

Then you can extend the design later.

A good interview answer would be:

*"For the initial version I'll keep entry and exit operations inside `ParkingLot` to avoid unnecessary classes. If the requirement includes multiple physical gates or gate-specific behavior, I can introduce `EntranceGate` and `ExitGate` later."*

That shows you understand **YAGNI** too: don't add abstractions before they're needed.

12/09/2026, 18:45:43



so now give the final code with all implementations

12/09/2026, 18:49:39



For a fresher interview, I'd keep the final version simple: no gate classes, no unnecessary subclassing, and no patterns unless behavior actually varies.

```
#include <bits/stdc++.h>
using namespace std;

// =====
// ENUMS
// =====

enum VehicleType {
 BIKE,
 CAR,
 TRUCK
};

enum SpotType {
 BIKE_SPOT,
 CAR_SPOT,
 TRUCK_SPOT
};

enum TicketStatus {
 ACTIVE,
 CLOSED
};

// =====
// VEHICLE
// =====

class Vehicle {
 string vehicleNumber;
 VehicleType type;

public:
 Vehicle(string vehicleNumber, VehicleType type) {
 this->vehicleNumber = vehicleNumber;
 this->type = type;
 }

 string getVehicleNumber() {
 return vehicleNumber;
 }

 VehicleType getType() {
 return type;
 }
};

// =====
// PARKING SPOT
// =====
```

```

class ParkingSpot {
 int id;
 SpotType type;

 // nullptr means spot is free
 Vehicle* vehicle;

public:
 ParkingSpot(int id, SpotType type) {
 this->id = id;
 this->type = type;
 this->vehicle = nullptr;
 }

 int getId() {
 return id;
 }

 SpotType getType() {
 return type;
 }

 bool isAvailable() {
 return vehicle == nullptr;
 }

 bool canPark(Vehicle* v) {
 if (!isAvailable())
 return false;

 if (v->getType() == BIKE && type == BIKE_SPOT)
 return true;

 if (v->getType() == CAR && type == CAR_SPOT)
 return true;

 if (v->getType() == TRUCK && type == TRUCK_SPOT)
 return true;

 return false;
 }

 bool parkVehicle(Vehicle* v) {
 if (!canPark(v))
 return false;

 vehicle = v;

 cout << "Vehicle "
 << v->getVehicleNumber()
 << " parked at spot "
 << id
 << "\n";

 return true;
 }

 void removeVehicle() {
 if (vehicle == nullptr) {

```

```

 cout << "Spot already empty\n";
 return;
 }

 cout << "Vehicle "
 << vehicle->getVehicleNumber()
 << " removed from spot "
 << id
 << "\n";

 vehicle = nullptr;
}

Vehicle* getVehicle() {
 return vehicle;
}
};

// =====
// PARKING FLOOR
// =====

class ParkingFloor {
 int floorNumber;
 vector<ParkingSpot*> spots;

public:
 ParkingFloor(int floorNumber) {
 this->floorNumber = floorNumber;
 }

 int getFloorNumber() {
 return floorNumber;
 }

 void addSpot(ParkingSpot* spot) {
 spots.push_back(spot);
 }

 ParkingSpot* findFreeSpot(Vehicle* vehicle) {

 for (auto spot : spots) {

 if (spot->canPark(vehicle))
 return spot;
 }

 return nullptr;
 }

 int getAvailableCount(SpotType type) {

 int count = 0;

 for (auto spot : spots) {

 if (spot->getType() == type &&
 spot->isAvailable()) {

 count++;
 }
 }
 }
};

```

```

 }

 return count;
}

void displayAvailability() {

 int bike = 0;
 int car = 0;
 int truck = 0;

 for (auto spot : spots) {

 if (!spot->isAvailable())
 continue;

 if (spot->getType() == BIKE_SPOT)
 bike++;

 else if (spot->getType() == CAR_SPOT)
 car++;

 else if (spot->getType() == TRUCK_SPOT)
 truck++;

 }

 cout << "\nFloor " << floorNumber << ":\n";
 cout << "Bike spots available : " << bike << "\n";
 cout << "Car spots available : " << car << "\n";
 cout << "Truck spots available : " << truck << "\n";

}

};

// =====
// TICKET
// =====

class Ticket {
 int id;

 Vehicle* vehicle;
 ParkingSpot* spot;
 ParkingFloor* floor;

 long long entryTime;

 TicketStatus status;

public:
 Ticket(
 int id,
 Vehicle* vehicle,
 ParkingSpot* spot,
 ParkingFloor* floor,
 long long entryTime
) {
 this->id = id;
 this->vehicle = vehicle;
 this->spot = spot;
 this->floor = floor;
 this->entryTime = entryTime;
 }

```



```

 this->status = ACTIVE;
 }

 int getId() {
 return id;
 }

 Vehicle* getVehicle() {
 return vehicle;
 }

 ParkingSpot* getSpot() {
 return spot;
 }

 ParkingFloor* getFloor() {
 return floor;
 }

 long long getEntryTime() {
 return entryTime;
 }

 TicketStatus getStatus() {
 return status;
 }

 void closeTicket() {
 status = CLOSED;
 }
};

// =====
// PRICING
// =====

class PricingStrategy {
public:
 virtual double calculateFee(
 Ticket* ticket,
 long long exitTime
) = 0;

 virtual ~PricingStrategy() {}
};

class HourlyPricingStrategy : public PricingStrategy {

public:

 double calculateFee(
 Ticket* ticket,
 long long exitTime
) override {

 long long duration =
 exitTime - ticket->getEntryTime();

 int hours = max(
 1LL,
 (duration + 3599) / 3600

```

```

);

 VehicleType type =
 ticket->getVehicle()->getType();

 int rate = 0;

 if (type == BIKE)
 rate = 10;

 else if (type == CAR)
 rate = 20;

 else if (type == TRUCK)
 rate = 30;

 return hours * rate;
}
};

// =====
// PAYMENT
// =====

class PaymentStrategy {
public:
 virtual bool pay(double amount) = 0;

 virtual ~PaymentStrategy() {}
};

class CashPayment : public PaymentStrategy {
public:

 bool pay(double amount) override {

 cout << "Paid Rs."
 << amount
 << " using Cash\n";

 return true;
 }
};

class UPIPayment : public PaymentStrategy {
public:

 bool pay(double amount) override {

 cout << "Paid Rs."
 << amount
 << " using UPI\n";

 return true;
 }
};

// =====
// PARKING LOT
// =====

```

```

class ParkingLot {

 vector<ParkingFloor*> floors;

 vector<Ticket*> activeTickets;

 PricingStrategy* pricingStrategy;

 int nextTicketId;

public:

 ParkingLot(PricingStrategy* pricingStrategy) {

 this->pricingStrategy =
 pricingStrategy;

 nextTicketId = 1;
 }

 void addFloor(ParkingFloor* floor) {
 floors.push_back(floor);
 }

 Ticket* parkVehicle(Vehicle* vehicle) {

 ParkingSpot* selectedSpot = nullptr;
 ParkingFloor* selectedFloor = nullptr;

 for (auto floor : floors) {

 ParkingSpot* spot =
 floor->findFreeSpot(vehicle);

 if (spot != nullptr) {

 selectedSpot = spot;
 selectedFloor = floor;

 break;
 }
 }

 if (selectedSpot == nullptr) {

 cout << "No parking spot available\n";

 return nullptr;
 }

 selectedSpot->parkVehicle(vehicle);

 long long entryTime =
 time(nullptr);

 Ticket* ticket =
 new Ticket(
 nextTicketId++,
 vehicle,
 selectedSpot,
 selectedFloor,

```

```

 entryTime
);

 activeTickets.push_back(ticket);

 cout << "\nTicket generated\n";
 cout << "Ticket ID : "
 << ticket->getId()
 << "\n";

 cout << "Floor : "
 << selectedFloor->getFloorNumber()
 << "\n";

 cout << "Spot : "
 << selectedSpot->getId()
 << "\n";

 return ticket;
}

bool unparkVehicle(
 Ticket* ticket,
 PaymentStrategy* paymentStrategy
) {

 if (ticket == nullptr) {
 cout << "Invalid ticket\n";
 return false;
 }

 if (ticket->getStatus() == CLOSED) {

 cout << "Ticket already closed\n";

 return false;
 }

 long long exitTime =
 time(nullptr);

 double fee =
 pricingStrategy->calculateFee(
 ticket,
 exitTime
);

 cout << "\nParking fee = Rs."
 << fee
 << "\n";

 bool paymentSuccess =
 paymentStrategy->pay(fee);

 if (!paymentSuccess) {

 cout << "Payment failed\n";

 return false;
 }
}

```

```

 ticket->getSpot()
 ->removeVehicle();

 ticket->closeTicket();

 cout << "Ticket closed successfully\n";

 return true;
 }

 void displayAvailability() {

 cout << "\n===== Parking Availability =====\n";

 for (auto floor : floors) {
 floor->displayAvailability();
 }
 }

 Ticket* getTicketById(int id) {

 for (auto ticket : activeTickets) {

 if (ticket->getId() == id)
 return ticket;
 }

 return nullptr;
 }
};

// =====
// MAIN
// =====

int main() {

 PricingStrategy* pricing =
 new HourlyPricingStrategy();

 ParkingLot parkingLot(pricing);

 // -----
 // Floor 1
 // -----

 ParkingFloor* floor1 =
 new ParkingFloor(1);

 floor1->addSpot(
 new ParkingSpot(
 101,
 BIKE_SPOT
)
);

 floor1->addSpot(
 new ParkingSpot(
 102,
 BIKE_SPOT
)
);

```

```

);

 floor1->addSpot(
 new ParkingSpot(
 103,
 CAR_SPOT
)
);

 floor1->addSpot(
 new ParkingSpot(
 104,
 CAR_SPOT
)
);

 floor1->addSpot(
 new ParkingSpot(
 105,
 TRUCK_SPOT
)
);

 // -----
 // Floor 2
 // -----

 ParkingFloor* floor2 =
 new ParkingFloor(2);

 floor2->addSpot(
 new ParkingSpot(
 201,
 BIKE_SPOT
)
);

 floor2->addSpot(
 new ParkingSpot(
 202,
 CAR_SPOT
)
);

 floor2->addSpot(
 new ParkingSpot(
 203,
 TRUCK_SPOT
)
);

 parkingLot.addFloor(floor1);
 parkingLot.addFloor(floor2);

 // -----
 // Vehicles
 // -----

 Vehicle* bike =
 new Vehicle(
 "UP70-BIKE-01",

```

```

 BIKE
);

 Vehicle* car =
 new Vehicle(
 "UP70-CAR-01",
 CAR
);

 Vehicle* truck =
 new Vehicle(
 "UP70-TRUCK-01",
 TRUCK
);

 parkingLot.displayAvailability();

 // -----
 // Park vehicles
 // -----

 Ticket* bikeTicket =
 parkingLot.parkVehicle(bike);

 Ticket* carTicket =
 parkingLot.parkVehicle(car);

 Ticket* truckTicket =
 parkingLot.parkVehicle(truck);

 parkingLot.displayAvailability();

 // -----
 // Exit
 // -----

 PaymentStrategy* upi =
 new UPIPayment();

 parkingLot.unparkVehicle(
 carTicket,
 upi
);

 parkingLot.displayAvailability();

 return 0;
}

```

The class responsibility split is:

```

Vehicle
→ vehicle information

ParkingSpot
→ one parking position
→ knows whether it is free
→ parks/removes vehicle

ParkingFloor
→ owns spots

```

→ finds a compatible free spot

Ticket

→ represents one parking session

→ connects Vehicle + Spot + entry time

PricingStrategy

→ calculates parking fee

PaymentStrategy

→ handles different payment methods

ParkingLot

→ coordinates the overall parking/unparking flow

The entry flow is:

```
parkingLot.parkVehicle(car)
 ↓
search floors
 ↓
floor.findFreeSpot(car)
 ↓
spot.canPark(car)
 ↓
spot.parkVehicle(car)
 ↓
create Ticket
 ↓
return Ticket
```

The exit flow:

```
parkingLot.unparkVehicle(ticket)
 ↓
calculateFee()
 ↓
paymentStrategy->pay()
 ↓
spot.removeVehicle()
 ↓
ticket.closeTicket()
```

And notice the cleaner `ParkingSpot` design we discussed:

```
Vehicle* vehicle;
```

instead of keeping both:

```
bool occupied;
Vehicle* vehicle;
```

Availability is derived:

```
bool isAvailable() {
 return vehicle == nullptr;
}
```

That avoids duplicated state.



For an actual fresher interview, you usually do **not** need to write this entire program. The most important classes to code fully would be `ParkingSpot` , `ParkingFloor` , `Ticket` , and `ParkingLot` ; then explain `PricingStrategy` and `PaymentStrategy` verbally if time is limited.

12/09/2026, 18:49:40



you misunderstood me give the full code use strategy pattern ,only don;t use gates and now give me the full code

12/09/2026, 18:50:38



Yes — keeping **Strategy Pattern**, removing only the gate classes.

Below is the complete version with:

- `Vehicle`
- `ParkingSpot`
- `ParkingFloor`
- `Ticket`
- `ParkingSpotStrategy`
- `PricingStrategy`
- `PaymentStrategy`
- `ParkingLot`
- `main()`

```
#include <bits/stdc++.h>
using namespace std;

// =====
// ENUMS
// =====

enum VehicleType {
 BIKE,
 CAR,
 TRUCK
};

enum SpotType {
 BIKE_SPOT,
 CAR_SPOT,
 TRUCK_SPOT
};

enum TicketStatus {
 ACTIVE,
 CLOSED
};

// =====
// VEHICLE
// =====

class Vehicle {
 string number;
 VehicleType type;

public:
 Vehicle(string number, VehicleType type) {
 this->number = number;
 }
};
```

```

 this->type = type;
 }

 string getNumber() {
 return number;
 }

 VehicleType getType() {
 return type;
 }
};

// =====
// PARKING SPOT
// =====

class ParkingSpot {
 int id;
 SpotType type;
 Vehicle* vehicle;

public:
 ParkingSpot(int id, SpotType type) {
 this->id = id;
 this->type = type;
 this->vehicle = nullptr;
 }

 int getId() {
 return id;
 }

 SpotType getType() {
 return type;
 }

 Vehicle* getVehicle() {
 return vehicle;
 }

 bool isAvailable() {
 return vehicle == nullptr;
 }

 bool canPark(Vehicle* v) {

 if (!isAvailable())
 return false;

 if (v->getType() == BIKE &&
 type == BIKE_SPOT)
 return true;

 if (v->getType() == CAR &&
 type == CAR_SPOT)
 return true;

 if (v->getType() == TRUCK &&
 type == TRUCK_SPOT)
 return true;
 }
};

```

```

 return false;
 }

 bool parkVehicle(Vehicle* v) {

 if (!canPark(v))
 return false;

 vehicle = v;

 return true;
 }

 void removeVehicle() {
 vehicle = nullptr;
 }
};

// =====
// PARKING FLOOR
// =====

class ParkingFloor {
 int floorNumber;
 vector<ParkingSpot*> spots;

public:
 ParkingFloor(int floorNumber) {
 this->floorNumber = floorNumber;
 }

 int getFloorNumber() {
 return floorNumber;
 }

 void addSpot(ParkingSpot* spot) {
 spots.push_back(spot);
 }

 vector<ParkingSpot*>& getSpots() {
 return spots;
 }

 void displayAvailability() {

 cout << "\nFloor " << floorNumber << "\n";

 for (auto spot : spots) {

 cout << "Spot "
 << spot->getId()
 << " -> ";

 if (spot->isAvailable())
 cout << "FREE";
 else
 cout << "OCCUPIED";

 cout << "\n";
 }
 }
};

```

```

};

// =====
// TICKET
// =====

class Ticket {
 int id;

 Vehicle* vehicle;
 ParkingSpot* spot;
 ParkingFloor* floor;

 long long entryTime;

 TicketStatus status;

public:
 Ticket(
 int id,
 Vehicle* vehicle,
 ParkingSpot* spot,
 ParkingFloor* floor,
 long long entryTime
) {
 this->id = id;
 this->vehicle = vehicle;
 this->spot = spot;
 this->floor = floor;
 this->entryTime = entryTime;
 this->status = ACTIVE;
 }

 int getId() {
 return id;
 }

 Vehicle* getVehicle() {
 return vehicle;
 }

 ParkingSpot* getSpot() {
 return spot;
 }

 ParkingFloor* getFloor() {
 return floor;
 }

 long long getEntryTime() {
 return entryTime;
 }

 TicketStatus getStatus() {
 return status;
 }

 void close() {
 status = CLOSED;
 }
};

```

```

// =====
// PARKING SPOT STRATEGY
// =====

class ParkingSpotStrategy {

public:

 virtual pair<ParkingFloor*, ParkingSpot*>
 findSpot(
 vector<ParkingFloor*>& floors,
 Vehicle* vehicle
) = 0;

 virtual ~ParkingSpotStrategy() {}
};

// =====
// FIRST AVAILABLE STRATEGY
// =====

class FirstAvailableSpotStrategy
 : public ParkingSpotStrategy {

public:

 pair<ParkingFloor*, ParkingSpot*>
 findSpot(
 vector<ParkingFloor*>& floors,
 Vehicle* vehicle
) override {

 for (auto floor : floors) {

 for (auto spot : floor->getSpots()) {

 if (spot->canPark(vehicle)) {

 return {
 floor,
 spot
 };
 }
 }
 }

 return {nullptr, nullptr};
 }
};

// =====
// NEAREST / LOWEST FLOOR STRATEGY
// =====

class LowestFloorSpotStrategy
 : public ParkingSpotStrategy {

public:

 pair<ParkingFloor*, ParkingSpot*>

```

```

findSpot(
 vector<ParkingFloor*>& floors,
 Vehicle* vehicle
) override {

 ParkingFloor* selectedFloor = nullptr;
 ParkingSpot* selectedSpot = nullptr;

 int bestFloor = INT_MAX;

 for (auto floor : floors) {

 for (auto spot : floor->getSpots()) {

 if (!spot->canPark(vehicle))
 continue;

 if (floor->getFloorNumber()
 < bestFloor) {

 bestFloor =
 floor->getFloorNumber();

 selectedFloor = floor;
 selectedSpot = spot;
 }
 }
 }

 return {
 selectedFloor,
 selectedSpot
 };
}

};

// =====
// PRICING STRATEGY
// =====

class PricingStrategy {

public:

 virtual double calculateFee(
 Ticket* ticket,
 long long exitTime
) = 0;

 virtual ~PricingStrategy() {}
};

// =====
// HOURLY PRICING
// =====

class HourlyPricingStrategy
 : public PricingStrategy {

public:

```

```

double calculateFee(
 Ticket* ticket,
 long long exitTime
) override {

 long long duration =
 exitTime -
 ticket->getEntryTime();

 int hours =
 max(
 1LL,
 (duration + 3599) / 3600
);

 int rate = 0;

 VehicleType type =
 ticket->getVehicle()->getType();

 if (type == BIKE)
 rate = 10;

 else if (type == CAR)
 rate = 20;

 else if (type == TRUCK)
 rate = 30;

 return hours * rate;
}

};

// =====
// FLAT PRICING STRATEGY
// =====

class FlatPricingStrategy
 : public PricingStrategy {

public:

 double calculateFee(
 Ticket* ticket,
 long long exitTime
) override {

 VehicleType type =
 ticket->getVehicle()->getType();

 if (type == BIKE)
 return 50;

 if (type == CAR)
 return 100;

 return 200;
 }

};

// =====

```



```

// PAYMENT STRATEGY
// =====

class PaymentStrategy {

public:

 virtual bool pay(
 double amount
) = 0;

 virtual ~PaymentStrategy() {}
};

// =====
// UPI PAYMENT
// =====

class UPIPayment
 : public PaymentStrategy {

public:

 bool pay(double amount) override {

 cout << "Paid Rs."
 << amount
 << " using UPI\n";

 return true;
 }
};

// =====
// CARD PAYMENT
// =====

class CardPayment
 : public PaymentStrategy {

public:

 bool pay(double amount) override {

 cout << "Paid Rs."
 << amount
 << " using Card\n";

 return true;
 }
};

// =====
// CASH PAYMENT
// =====

class CashPayment
 : public PaymentStrategy {

public:

```

```

bool pay(double amount) override {

 cout << "Paid Rs."
 << amount
 << " using Cash\n";

 return true;
}

};

// =====
// PARKING LOT
// =====

class ParkingLot {

 vector<ParkingFloor*> floors;
 vector<Ticket*> tickets;

 ParkingSpotStrategy* spotStrategy;
 PricingStrategy* pricingStrategy;

 int nextTicketId;

public:

 ParkingLot(
 ParkingSpotStrategy* spotStrategy,
 PricingStrategy* pricingStrategy
) {
 this->spotStrategy =
 spotStrategy;

 this->pricingStrategy =
 pricingStrategy;

 nextTicketId = 1;
 }

 // -----
 // Add floor
 // -----

 void addFloor(
 ParkingFloor* floor
) {
 floors.push_back(floor);
 }

 // -----
 // Change spot strategy at runtime
 // -----

 void setSpotStrategy(
 ParkingSpotStrategy* strategy
) {
 spotStrategy = strategy;
 }

 // -----
 // Change pricing strategy at runtime

```

```

// -----

void setPricingStrategy(
 PricingStrategy* strategy
) {
 pricingStrategy = strategy;
}

// -----
// Park vehicle
// -----

Ticket* parkVehicle(
 Vehicle* vehicle
) {

 pair<ParkingFloor*, ParkingSpot*>
 result =
 spotStrategy->findSpot(
 floors,
 vehicle
);

 ParkingFloor* floor =
 result.first;

 ParkingSpot* spot =
 result.second;

 if (spot == nullptr) {

 cout << "No compatible parking spot available\n";

 return nullptr;
 }

 bool parked =
 spot->parkVehicle(vehicle);

 if (!parked) {

 cout << "Unable to park vehicle\n";

 return nullptr;
 }

 long long entryTime =
 time(nullptr);

 Ticket* ticket =
 new Ticket(
 nextTicketId++,
 vehicle,
 spot,
 floor,
 entryTime
);

 tickets.push_back(ticket);

 cout << "\nVehicle "

```

```

 << vehicle->getNumber()
 << " parked successfully\n";

 cout << "Ticket ID: "
 << ticket->getId()
 << "\n";

 cout << "Floor: "
 << floor->getFloorNumber()
 << "\n";

 cout << "Spot: "
 << spot->getId()
 << "\n";

 return ticket;
}

// -----
// Unpark vehicle
// -----

bool unparkVehicle(
 Ticket* ticket,
 PaymentStrategy* paymentStrategy
) {

 if (ticket == nullptr) {

 cout << "Invalid ticket\n";

 return false;
 }

 if (ticket->getStatus()
 == CLOSED) {

 cout << "Ticket already closed\n";

 return false;
 }

 long long exitTime =
 time(nullptr);

 double fee =
 pricingStrategy
 ->calculateFee(
 ticket,
 exitTime
);

 cout << "\nParking fee = Rs."
 << fee
 << "\n";

 bool success =
 paymentStrategy->pay(fee);

 if (!success) {

```

```

 cout << "Payment failed\n";

 return false;
 }

 ticket
 ->getSpot()
 ->removeVehicle();

 ticket->close();

 cout << "Vehicle "
 << ticket
 ->getVehicle()
 ->getNumber()
 << " removed successfully\n";

 cout << "Ticket closed\n";

 return true;
}

// -----
// Display availability
// -----

void displayAvailability() {

 cout << "\n==== PARKING STATUS ==== \n";

 for (auto floor : floors) {
 floor->displayAvailability();
 }
}

// -----
// Find ticket
// -----

Ticket* getTicketById(
 int ticketId
) {

 for (auto ticket : tickets) {

 if (ticket->getId()
 == ticketId) {

 return ticket;
 }
 }

 return nullptr;
}

};

// =====
// MAIN
// =====

int main() {

```

```

// -----
// Create floors
// -----

ParkingFloor* floor1 =
 new ParkingFloor(1);

ParkingFloor* floor2 =
 new ParkingFloor(2);

// -----
// Add spots to Floor 1
// -----

floor1->addSpot(
 new ParkingSpot(
 101,
 BIKE_SPOT
)
);

floor1->addSpot(
 new ParkingSpot(
 102,
 CAR_SPOT
)
);

floor1->addSpot(
 new ParkingSpot(
 103,
 CAR_SPOT
)
);

floor1->addSpot(
 new ParkingSpot(
 104,
 TRUCK_SPOT
)
);

// -----
// Add spots to Floor 2
// -----

floor2->addSpot(
 new ParkingSpot(
 201,
 BIKE_SPOT
)
);

floor2->addSpot(
 new ParkingSpot(
 202,
 CAR_SPOT
)
);

```

```

floor2->addSpot(
 new ParkingSpot(
 203,
 TRUCK_SPOT
)
);

// -----
// Strategies
// -----

ParkingSpotStrategy* spotStrategy =
 new FirstAvailableSpotStrategy();

PricingStrategy* pricingStrategy =
 new HourlyPricingStrategy();

// -----
// Parking lot
// -----

ParkingLot parkingLot(
 spotStrategy,
 pricingStrategy
);

parkingLot.addFloor(floor1);
parkingLot.addFloor(floor2);

// -----
// Create vehicles
// -----

Vehicle* bike =
 new Vehicle(
 "UP70-BIKE-01",
 BIKE
);

Vehicle* car1 =
 new Vehicle(
 "UP70-CAR-01",
 CAR
);

Vehicle* car2 =
 new Vehicle(
 "UP70-CAR-02",
 CAR
);

Vehicle* truck =
 new Vehicle(
 "UP70-TRUCK-01",
 TRUCK
);

// -----
// Initial availability
// -----

```

```

parkingLot.displayAvailability();

// -----
// Park vehicles
// -----

Ticket* bikeTicket =
 parkingLot.parkVehicle(
 bike
);

Ticket* carTicket1 =
 parkingLot.parkVehicle(
 car1
);

Ticket* carTicket2 =
 parkingLot.parkVehicle(
 car2
);

Ticket* truckTicket =
 parkingLot.parkVehicle(
 truck
);

// -----
// Show updated status
// -----

parkingLot.displayAvailability();

// -----
// User exits and pays with UPI
// -----

PaymentStrategy* upi =
 new UPIPayment();

parkingLot.unparkVehicle(
 carTicket1,
 upi
);

// -----
// Show availability again
// -----

parkingLot.displayAvailability();

// -----
// Change allocation strategy dynamically
// -----

ParkingSpotStrategy* lowestFloor =
 new LowestFloorSpotStrategy();

parkingLot.setSpotStrategy(
 lowestFloor
);

```



```
// -----
// Change pricing dynamically
// -----

PricingStrategy* flatPricing =
 new FlatPricingStrategy();

parkingLot.setPricingStrategy(
 flatPricing
);

Vehicle* car3 =
 new Vehicle(
 "UP70-CAR-03",
 CAR
);

Ticket* carTicket3 =
 parkingLot.parkVehicle(
 car3
);

PaymentStrategy* card =
 new CardPayment();

parkingLot.unparkVehicle(
 carTicket3,
 card
);

return 0;
}
```

The three places where Strategy Pattern is being used are:

```
ParkingSpotStrategy
|
+-- FirstAvailableSpotStrategy
+-- LowestFloorSpotStrategy
```

This answers:

***Which parking spot should I allocate?***

Then:

```
PricingStrategy
|
+-- HourlyPricingStrategy
+-- FlatPricingStrategy
```

This answers:

***How should I calculate parking price?***

And:

```
PaymentStrategy
|
+-- UPIPayment
```

```
+-- CardPayment
+-- CashPayment
```

This answers:

### ***How should the customer pay?***

The most important Strategy Pattern idea here is that `ParkingLot` does **not** contain logic like:

```
if (pricingType == HOURLY) {
 ...
}
else if (pricingType == FLAT) {
 ...
}
```

Instead:

```
pricingStrategy->calculateFee(...);
```

Similarly, it doesn't know how UPI or card actually works:

```
paymentStrategy->pay(fee);
```

And it doesn't know which allocation algorithm is being used:

```
spotStrategy->findSpot(floors, vehicle);
```

That is the clean final design without gates.

12/09/2026, 18:50:39



For a fresher interview, I'd keep an **Attendance Tracker** centered around these core entities:

```
User / Student
Course
Session / ClassSession
AttendanceRecord
AttendanceTracker / AttendanceSystem
```

If the interviewer wants multiple ways of marking attendance—manual, QR, biometric—you can then add Strategy Pattern.

The first thing you should say is:

*"I'll model a course with enrolled students and class sessions. For every class session, the system stores one attendance record per student, such as Present, Absent, or Late. The main system coordinates enrollment, session creation, marking attendance, and generating attendance percentages."*

A clean design is:

```
#include <bits/stdc++.h>
using namespace std;

enum AttendanceStatus {
 PRESENT,
 ABSENT,
 LATE
};

// =====
// STUDENT
// =====

class Student {
 int id;
 string name;

public:
 Student(int id, string name) {
 this->id = id;
 this->name = name;
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }
};

// =====
// COURSE
// =====
```

```

class Course {
 int id;
 string name;
 vector<Student*> students;

public:
 Course(int id, string name) {
 this->id = id;
 this->name = name;
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }

 void enrollStudent(Student* student) {
 students.push_back(student);
 }

 vector<Student*>& getStudents() {
 return students;
 }

 bool isStudentEnrolled(int studentId) {

 for (auto student : students) {
 if (student->getId() == studentId)
 return true;
 }

 return false;
 }
};

```

```

// =====
// ATTENDANCE RECORD
// =====

```

```

class AttendanceRecord {
 Student* student;
 AttendanceStatus status;

public:
 AttendanceRecord(
 Student* student,
 AttendanceStatus status
) {
 this->student = student;
 this->status = status;
 }

 Student* getStudent() {
 return student;
 }

 AttendanceStatus getStatus() {

```

```

 return status;
 }

 void setStatus(AttendanceStatus status) {
 this->status = status;
 }
};

// =====
// CLASS SESSION
// =====

class ClassSession {
 int id;
 Course* course;
 string date;

 vector<AttendanceRecord*> records;

public:
 ClassSession(
 int id,
 Course* course,
 string date
) {
 this->id = id;
 this->course = course;
 this->date = date;
 }

 int getId() {
 return id;
 }

 Course* getCourse() {
 return course;
 }

 string getDate() {
 return date;
 }

 void addRecord(AttendanceRecord* record) {
 records.push_back(record);
 }

 vector<AttendanceRecord*>& getRecords() {
 return records;
 }

 AttendanceRecord* getRecordForStudent(
 int studentId
) {
 for (auto record : records) {

 if (record->getStudent()->getId()
 == studentId) {

 return record;
 }
 }
 }
}

```

```

 return nullptr;
 }
};

// =====
// ATTENDANCE MARKING STRATEGY
// =====

class AttendanceMarkingStrategy {
public:
 virtual bool markAttendance(
 ClassSession* session,
 Student* student,
 AttendanceStatus status
) = 0;

 virtual ~AttendanceMarkingStrategy() {}
};

// =====
// MANUAL ATTENDANCE
// =====

class ManualAttendanceStrategy
 : public AttendanceMarkingStrategy {
public:

 bool markAttendance(
 ClassSession* session,
 Student* student,
 AttendanceStatus status
) override {

 AttendanceRecord* existing =
 session->getRecordForStudent(
 student->getId()
);

 if (existing != nullptr) {
 existing->setStatus(status);
 return true;
 }

 session->addRecord(
 new AttendanceRecord(
 student,
 status
)
);

 return true;
 }
};

// =====
// QR ATTENDANCE STRATEGY
// Just simulating validation here
// =====

```

```

class QRAttendanceStrategy
: public AttendanceMarkingStrategy {

public:

 bool markAttendance(
 ClassSession* session,
 Student* student,
 AttendanceStatus status
) override {

 cout << "QR validated for "
 << student->getName()
 << "\n";

 AttendanceRecord* existing =
 session->getRecordForStudent(
 student->getId()
);

 if (existing != nullptr) {
 existing->setStatus(status);
 return true;
 }

 session->addRecord(
 new AttendanceRecord(
 student,
 status
)
);

 return true;
 }
};

// =====
// ATTENDANCE SYSTEM
// =====

class AttendanceSystem {

 vector<Student*> students;
 vector<Course*> courses;
 vector<ClassSession*> sessions;

 AttendanceMarkingStrategy* markingStrategy;

 int nextSessionId = 1;

public:

 AttendanceSystem(
 AttendanceMarkingStrategy* strategy
) {
 this->markingStrategy = strategy;
 }

 void setMarkingStrategy(
 AttendanceMarkingStrategy* strategy
) {

```

```

 markingStrategy = strategy;
 }

 void addStudent(Student* student) {
 students.push_back(student);
 }

 void addCourse(Course* course) {
 courses.push_back(course);
 }

 void enrollStudent(
 Course* course,
 Student* student
) {
 course->enrollStudent(student);
 }

 ClassSession* createSession(
 Course* course,
 string date
) {
 ClassSession* session =
 new ClassSession(
 nextSessionId++,
 course,
 date
);

 sessions.push_back(session);

 cout << "Session created for "
 << course->getName()
 << " on "
 << date
 << "\n";

 return session;
 }

 bool markAttendance(
 ClassSession* session,
 Student* student,
 AttendanceStatus status
) {
 if (!session->getCourse()
 ->isStudentEnrolled(
 student->getId()
)) {

 cout << "Student not enrolled in course\n";
 return false;
 }

 return markingStrategy
 ->markAttendance(
 session,
 student,
 status
);
 }

```



```

);
}

double getAttendancePercentage(
 Course* course,
 Student* student
) {

 int totalSessions = 0;
 int attendedSessions = 0;

 for (auto session : sessions) {

 if (session->getCourse()->getId()
 != course->getId()) {

 continue;
 }

 totalSessions++;

 AttendanceRecord* record =
 session->getRecordForStudent(
 student->getId()
);

 if (record != nullptr &&
 (
 record->getStatus() == PRESENT ||
 record->getStatus() == LATE
)) {

 attendedSessions++;
 }
 }

 if (totalSessions == 0)
 return 0.0;

 return
 (attendedSessions * 100.0)
 / totalSessions;
}

void showSessionAttendance(
 ClassSession* session
) {

 cout << "\nAttendance for "
 << session->getCourse()->getName()
 << " on "
 << session->getDate()
 << "\n";

 for (auto record :
 session->getRecords()) {

 cout << record->getStudent()
 << record->getName()
 << " : ";
 }
}

```

```

 if (record->getStatus() == PRESENT)
 cout << "PRESENT";

 else if (record->getStatus() == ABSENT)
 cout << "ABSENT";

 else
 cout << "LATE";

 cout << "\n";
 }
}
};

// =====
// MAIN
// =====

int main() {

 Student* s1 =
 new Student(
 1,
 "Anirudh"
);

 Student* s2 =
 new Student(
 2,
 "Rahul"
);

 Student* s3 =
 new Student(
 3,
 "Aman"
);

 Course* cpp =
 new Course(
 101,
 "C++ Programming"
);

 AttendanceMarkingStrategy* manual =
 new ManualAttendanceStrategy();

 AttendanceSystem system(
 manual
);

 system.addStudent(s1);
 system.addStudent(s2);
 system.addStudent(s3);

 system.addCourse(cpp);

 system.enrollStudent(cpp, s1);
 system.enrollStudent(cpp, s2);
 system.enrollStudent(cpp, s3);

```

```

ClassSession* session1 =
 system.createSession(
 cpp,
 "13-09-2026"
);

system.markAttendance(
 session1,
 s1,
 PRESENT
);

system.markAttendance(
 session1,
 s2,
 ABSENT
);

system.markAttendance(
 session1,
 s3,
 LATE
);

system.showSessionAttendance(
 session1
);

ClassSession* session2 =
 system.createSession(
 cpp,
 "14-09-2026"
);

system.markAttendance(
 session2,
 s1,
 PRESENT
);

system.markAttendance(
 session2,
 s2,
 PRESENT
);

system.markAttendance(
 session2,
 s3,
 ABSENT
);

cout << "\nAttendance percentages:\n";

cout << s1->getName()
 << " = "
 << system.getAttendancePercentage(
 cpp,
 s1
)
 << "%\n";

```

```

cout << s2->getName()
 << " = "
 << system.getAttendancePercentage(
 cpp,
 s2
)
 << "\n";

cout << s3->getName()
 << " = "
 << system.getAttendancePercentage(
 cpp,
 s3
)
 << "\n";

// Switching strategy dynamically

AttendanceMarkingStrategy* qr =
 new QRAttendanceStrategy();

system.setMarkingStrategy(qr);

ClassSession* session3 =
 system.createSession(
 cpp,
 "15-09-2026"
);

system.markAttendance(
 session3,
 s1,
 PRESENT
);

return 0;
}

```

## What you should explain in the interview

The class relationships are:

```

AttendanceSystem
|
+---- Students
|
+---- Courses
|
+---- ClassSessions

Course
|
+---- enrolled Students

ClassSession
|
+---- Course
|

```

```

+---- AttendanceRecords

AttendanceRecord
|
+---- Student
|
+---- PRESENT / ABSENT / LATE

```

The most important concept is that **attendance belongs to a session, not directly to a student**.

You should not do something like:

```

class Student {
 bool present;
};

```

because a student can be present in Monday's class and absent in Tuesday's class.

Instead:

```

Session 1
 Student A → PRESENT
 Student B → ABSENT

Session 2
 Student A → ABSENT
 Student B → PRESENT

```

That's why we need:

```

ClassSession
 ↓
vector<AttendanceRecord*>

```

Strategy Pattern here answers:

### ***How is attendance being marked?***

```

AttendanceMarkingStrategy
|

| |
Manual QR

```

Later you could add:

```

class BiometricAttendanceStrategy
: public AttendanceMarkingStrategy {
};

```

without changing `AttendanceSystem`.

For a fresher, the five things you must be able to explain are:

- `Course` owns enrolled students.
- `ClassSession` represents one actual class occurrence.
- `AttendanceRecord` connects one student with one session and a status.
- `AttendanceSystem` coordinates enrollment, sessions, marking, and reports.

- Strategy Pattern allows manual/QR/biometric attendance methods to vary independently.

13/09/2026, 00:01:49



Book myshow

this is the lld problem now explain each step for answering this in interview and explain everything i am supposed to do in an in person interview

13/09/2026, 13:34:58



For a fresher interview, BookMyShow should be answered in layers. Don't start with 20 classes. First show that you understand the booking flow and the one genuinely tricky part: **seat locking / preventing double booking**.

## What you must be able to tell the interviewer first

Start with this:

*"I'll model the core system using `User`, `Movie`, `Theatre`, `Screen`, `Show`, `Seat`, `Booking`, and `Payment`. A theatre has screens, each screen has seats, and a show represents a movie playing on a particular screen at a particular time. Users search shows, select available seats, temporarily lock them, make payment, and on successful payment the booking is confirmed. The important part is preventing two users from booking the same seat simultaneously."*

That is already a strong opening.

## 1. Clarify the requirements

Say something like:

*"I'll assume users can search movies by city, see theatres and shows, choose seats, create a booking, pay, and receive confirmation. I'll also assume a seat can be temporarily locked during payment so another user cannot take it. I'll ignore food ordering, coupons, reviews, and recommendations unless required."*

Main flow:

```
User selects city
 ↓
searches movie
 ↓
selects theatre/show
 ↓
views seats
 ↓
selects seats
 ↓
system locks seats temporarily
 ↓
booking created
 ↓
payment
 ↓
booking confirmed
```

## 2. Identify main entities

From the requirements:

```
User
Movie
Theatre
Screen
```

```
Seat
Show
Booking
Payment
BookMyShowSystem
```

Possible later additions:

```
SeatLock
PricingStrategy
PaymentStrategy
```

Don't add them before explaining why.

### 3. User

Very simple:

```
class User {
 int id;
 string name;
};
```

Possible methods:

```
int getId();
string getName();
```

The user should not own booking logic.

### 4. Movie

Ask:

*What information does one movie need?*

```
class Movie {
 int id;
 string title;
 int durationMinutes;
 string language;
};
```

You can add genre/rating later.

### 5. Theatre

A theatre is a physical location containing multiple screens.

```
class Theatre {
 int id;
 string name;
 string city;

 vector<Screen*> screens;
};
```

Methods:



```
void addScreen(Screen* screen);
vector<Screen*> getScreens();
```

Relationship:

```
Theatre
|
+-- Screen 1
+-- Screen 2
+-- Screen 3
```

## 6. Screen

A screen contains seats.

```
class Screen {
 int id;
 string name;

 vector<Seat*> seats;
};
```

Methods:

```
void addSeat(Seat* seat);
vector<Seat*> getSeats();
```

## 7. Seat

This is where people often make a mistake.

You should not store:

```
bool booked;
```

directly inside a physical `Seat`.

Why?

Because the same physical seat can be:

```
available for 3 PM show
booked for 6 PM show
available for 9 PM show
```

So booking status depends on the **Show**, not just the physical seat.

Physical seat can be:

```
enum SeatType {
 REGULAR,
 PREMIUM,
 RECLINER
};

class Seat {
 int id;
 string row;
 int number;
```

```
SeatType type;
};
```

This is an important interview point.

Say:

*"A physical seat should not contain a global booked flag because availability varies per show."*

## 8. Show

This is one of the most important classes.

A Show means:

```
Movie X
Screen 2
6:00 PM
```

So:

```
class Show {
 int id;

 Movie* movie;
 Theatre* theatre;
 Screen* screen;

 long long startTime;
 long long endTime;
};
```

Then show-specific seat availability must be tracked.

You have two choices.

Simple version:

```
map<int, SeatStatus> seatStatus;
```

Better abstraction:

```
map<int, ShowSeat*> showSeats;
```

For a fresher interview, this is clean:

```
enum SeatStatus {
 AVAILABLE,
 LOCKED,
 BOOKED
};

class ShowSeat {
 Seat* seat;
 SeatStatus status;
 double price;
};
```

Now:

```
Seat
→ physical seat information
```

```
ShowSeat
→ status/price of that seat for one specific show
```

This distinction is very important.

## 9. Why `ShowSeat` exists

Suppose physical seat `A1` exists.

For Show 1:

```
A1 → BOOKED
```

For Show 2:

```
A1 → AVAILABLE
```

That means status belongs to:

```
Show + Seat
```

not just `Seat`.

So:

```
class ShowSeat {
 Seat* seat;
 SeatStatus status;
 double price;
};
```

## 10. `Booking`

A booking connects:

```
User
Show
Selected Seats
Amount
Status
```

So:

```
enum BookingStatus {
 CREATED,
 PAYMENT_PENDING,
 CONFIRMED,
 CANCELLED,
 FAILED
};

class Booking {
 int id;

 User* user;
 Show* show;

 vector<ShowSeat*> seats;

 double amount;
```

```
BookingStatus status;
};
```

Methods:

```
void confirm();
void cancel();
void fail();
```

## 11. Seat locking — the most important part

This is the main BookMyShow interview question.

Suppose:

```
User A selects A1
User B selects A1 at same time
```

If both are allowed to pay, we have double booking.

So flow should be:

```
seat AVAILABLE
↓
User A selects it
↓
seat becomes LOCKED
↓
User A gets limited time to pay
↓
payment success
↓
seat becomes BOOKED
```

If payment fails or lock expires:

```
LOCKED
↓
AVAILABLE
```

This is the key state transition:

```
AVAILABLE → LOCKED → BOOKED
|
↓
AVAILABLE
```

You should definitely say this to the interviewer.

## 12. `SeatLock`

You can model the lock explicitly:

```
class SeatLock {
 ShowSeat* showSeat;
 User* lockedBy;
 long long lockTime;
 int lockDurationSeconds;
};
```

Methods:

```
bool isExpired();
```

Or keep locking logic inside a manager/service.

For fresher interviews, a separate `SeatLock` is reasonable if concurrency is discussed.

---

## 13. `Payment`

Keep it simple initially:

```
enum PaymentStatus {
 PENDING,
 SUCCESS,
 FAILED
};

class Payment {
 int id;
 double amount;
 PaymentStatus status;
};
```

If multiple payment methods:

```
class PaymentStrategy {
public:
 virtual bool pay(double amount) = 0;
};
```

Then:

```
class UPIPayment : public PaymentStrategy {};
class CardPayment : public PaymentStrategy {};
```

Strategy Pattern fits naturally here.

---

## 14. Pricing

Different seat types may cost differently:

```
Regular = ₹200
Premium = ₹300
Recliner = ₹500
```

You can simply store:

```
double price;
```

inside `ShowSeat` .

Why `ShowSeat` and not `Seat` ?

Because the same physical seat may have different pricing for:

```
weekday
weekend
morning
night
premium show
```

So price can vary by show.

That is another good interview point.

---

## 15. Main coordinator

Create:

```
class BookMyShowSystem {
 vector<Movie*> movies;
 vector<Theatre*> theatres;
 vector<Show*> shows;
 vector<Booking*> bookings;
};
```

Core methods:

```
vector<Movie*> searchMovies(string city);

vector<Show*> getShows(
 Movie* movie,
 string city
);

vector<ShowSeat*> getAvailableSeats(
 Show* show
);

Booking* createBooking(
 User* user,
 Show* show,
 vector<int> seatIds
);

bool makePayment(
 Booking* booking,
 PaymentStrategy* paymentStrategy
);

void cancelBooking(
 Booking* booking
);
```

---

## 16. Booking flow

You should walk through one full example.

Suppose user wants two seats for a 6 PM show.

```
User selects movie
 ↓
system finds shows
 ↓
user selects 6 PM show
 ↓
system displays ShowSeats
 ↓
A1, A2 selected
 ↓
check both AVAILABLE
 ↓
```

```

lock A1, A2
↓
create Booking(PAYMENT_PENDING)
↓
calculate amount
↓
payment

```

If payment succeeds:

```

A1/A2 → BOOKED
Booking → CONFIRMED

```

If payment fails:

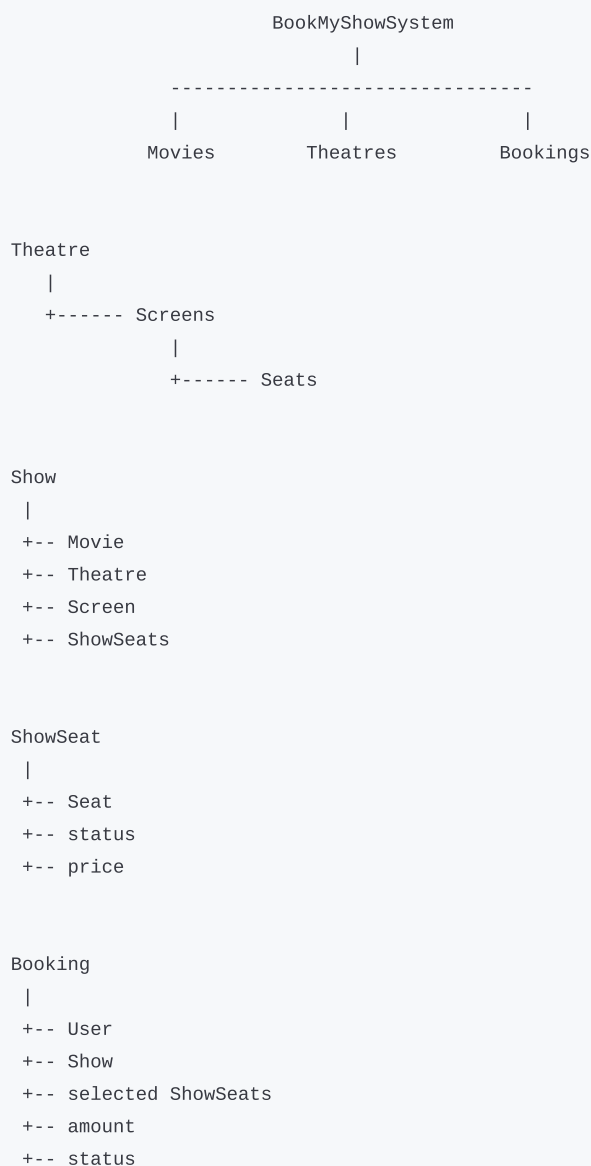
```

A1/A2 → AVAILABLE
Booking → FAILED

```

## 17. Class diagram

Draw something like:



That diagram is enough for most fresher interviews.

---

## 18. Most important member variables

If interviewer asks you to code, start with:

```
class Seat {
 int id;
 string row;
 int number;
 SeatType type;
};

class ShowSeat {
 Seat* seat;
 SeatStatus status;
 double price;
};

class Show {
 int id;
 Movie* movie;
 Screen* screen;

 vector<ShowSeat*> showSeats;

 long long startTime;
};

class Booking {
 int id;

 User* user;
 Show* show;

 vector<ShowSeat*> seats;

 double amount;
 BookingStatus status;
};
```

These are the heart of the design.

---

## 19. How to derive the methods

Use our usual rule:

*Who owns the data, and who should change it?*

ShowSeat owns status:

```
lock();
book();
release();
```

Show owns show seats:

```
getAvailableSeats();
getShowSeat(id);
```

Booking owns booking state:



```
confirm();
cancel();
fail();
```

BookMyShowSystem coordinates:

```
createBooking();
makePayment();
cancelBooking();
```

---

## 20. Strategy Pattern

You do not need Strategy Pattern everywhere.

Good places:

### Payment strategy

```
class PaymentStrategy {
public:
 virtual bool pay(double amount) = 0;
};
```

Implementations:

```
UPIPayment
CardPayment
WalletPayment
```

### Pricing strategy, if interviewer asks

```
class PricingStrategy {
public:
 virtual double calculatePrice(
 ShowSeat* seat,
 Show* show
) = 0;
};
```

Possible implementations:

```
NormalPricing
WeekendPricing
SurgePricing
```

Don't introduce strategy just for the sake of patterns.

---

## 21. Concurrency / double booking

Be ready for this question:

*"Two users select A1 at exactly the same time. What happens?"*

Strong fresher answer:

*"Seat availability check and locking must be atomic. Only one user should successfully transition the seat from AVAILABLE to LOCKED. In a real system I'd enforce this using a database transaction, row-level lock, or atomic compare-and-set operation."*

*The lock should also expire after a timeout.”*

That is probably the most important advanced point in BookMyShow.

## 22. Why not directly mark selected seat as BOOKED?

Because the user hasn't paid yet.

You need:

```
AVAILABLE
↓
LOCKED
↓
payment
↓
BOOKED
```

Otherwise someone can select seats and never pay, blocking them forever.

## 23. What if payment fails?

Say:

*“Booking becomes FAILED and all temporarily locked seats are released back to AVAILABLE.”*

Simple.

## 24. Cancellation

If confirmed booking is cancelled:

```
Booking → CANCELLED
ShowSeats → AVAILABLE
refund may be initiated
```

Depending on business rules.

## 25. Common interviewer follow-ups

Be ready for:

**“Why does Show have seat status instead of Seat?”**

Because availability changes per show.

**“Why do we need locking?”**

To avoid double booking during payment.

**“What if the lock expires?”**

Release seat to AVAILABLE.

**“How do you calculate price?”**

Use `ShowSeat.price`, or `PricingStrategy` for more complex rules.

**“Can same movie have multiple shows?”**

Yes:

```
Movie
|
+-- Show 10 AM
```

```
+-- Show 2 PM
+-- Show 6 PM
```

### “Can same screen have overlapping shows?”

No. Show scheduling should validate time overlap.

### “What if payment succeeds but booking confirmation fails?”

In a real system, use transactional/reconciliation logic; either retry confirmation or refund.

---

## 26. SOLID

Keep it short.

### SRP

```
Seat
→ physical seat info

ShowSeat
→ seat state for one show

Show
→ movie screening info

Booking
→ booking state

PaymentStrategy
→ payment behavior
```

### OCP

New payment method:

```
class WalletPayment : public PaymentStrategy
```

without changing booking logic.

### DIP

Main system depends on:

```
PaymentStrategy*
```

not directly on:

```
UPIPayment
```

---

## 27. What you should actually do in the interview

Use this exact sequence:

1. Clarify requirements
2. Identify main classes
3. Explain relationships
4. Identify tricky seat-status issue
5. Introduce ShowSeat
6. Explain seat locking

7. Define Booking
8. Walk through one booking flow
9. Add PaymentStrategy if needed
10. Discuss concurrency

The biggest mistake is jumping immediately into code.

---

## 28. Your 30-second opening answer

You can say:

*"I'll model BookMyShow using `User` , `Movie` , `Theatre` , `Screen` , `Seat` , `Show` , `ShowSeat` , and `Booking` . A theatre contains screens, screens contain physical seats, and a show represents one movie screening at a specific time. Since seat availability differs per show, I won't store booking status directly in `Seat` ; I'll use `ShowSeat` with states like `AVAILABLE`, `LOCKED`, and `BOOKED`. During booking, seats are temporarily locked, payment is processed, and on success they become booked. This prevents double booking and cleanly separates physical seat data from show-specific state."*

If you can explain that confidently, you've covered the most important part of BookMyShow LLD as a fresher.

13/09/2026, 13:34:57

[give full code](#)

13/09/2026, 13:58:28



For a fresher interview, this is a complete but still understandable version. It includes the important parts: `ShowSeat` , seat locking, booking states, payment strategy, and cancellation.

```
#include <bits/stdc++.h>
using namespace std;

// =====
// ENUMS
// =====

enum SeatType {
 REGULAR,
 PREMIUM,
 RECLINER
};

enum SeatStatus {
 AVAILABLE,
 LOCKED,
 BOOKED
};

enum BookingStatus {
 CREATED,
 PAYMENT_PENDING,
 CONFIRMED,
 CANCELLED,
 FAILED
};

// =====
// USER
// =====

class User {
 int id;
 string name;

public:
 User(int id, string name) {
 this->id = id;
 this->name = name;
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }
};

// =====
```

```

// MOVIE
// =====

class Movie {
 int id;
 string title;
 int durationMinutes;
 string language;

public:
 Movie(
 int id,
 string title,
 int durationMinutes,
 string language
) {
 this->id = id;
 this->title = title;
 this->durationMinutes = durationMinutes;
 this->language = language;
 }

 int getId() {
 return id;
 }

 string getTitle() {
 return title;
 }

 int getDurationMinutes() {
 return durationMinutes;
 }

 string getLanguage() {
 return language;
 }
};

// =====
// SEAT
// Physical seat only
// No booked/available state here
// =====

class Seat {
 int id;
 string row;
 int number;
 SeatType type;

public:
 Seat(
 int id,
 string row,
 int number,
 SeatType type
) {
 this->id = id;
 this->row = row;
 this->number = number;
 }
};

```

```

 this->type = type;
 }

 int getId() {
 return id;
 }

 string getRow() {
 return row;
 }

 int getNumber() {
 return number;
 }

 SeatType getType() {
 return type;
 }

 string getSeatName() {
 return row + to_string(number);
 }
};

// =====
// SCREEN
// =====

class Screen {
 int id;
 string name;

 vector<Seat*> seats;

public:
 Screen(int id, string name) {
 this->id = id;
 this->name = name;
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }

 void addSeat(Seat* seat) {
 seats.push_back(seat);
 }

 vector<Seat*>& getSeats() {
 return seats;
 }
};

// =====
// THEATRE
// =====

```

```

class Theatre {
 int id;
 string name;
 string city;

 vector<Screen*> screens;

public:
 Theatre(
 int id,
 string name,
 string city
) {
 this->id = id;
 this->name = name;
 this->city = city;
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }

 string getCity() {
 return city;
 }

 void addScreen(Screen* screen) {
 screens.push_back(screen);
 }

 vector<Screen*>& getScreens() {
 return screens;
 }
};

// =====
// SHOW SEAT
// Seat state for one specific show
// =====

class ShowSeat {
 Seat* seat;
 SeatStatus status;
 double price;

 User* lockedBy;
 long long lockTime;
 int lockDurationSeconds;

public:
 ShowSeat(
 Seat* seat,
 double price
) {
 this->seat = seat;
 this->price = price;
 }
};

```



```

 this->status = AVAILABLE;

 this->lockedBy = nullptr;
 this->lockTime = 0;

 this->lockDurationSeconds = 300;
 }

 Seat* getSeat() {
 return seat;
 }

 double getPrice() {
 return price;
 }

 SeatStatus getStatus() {
 /*
 * If lock expired,
 * automatically release it.
 */

 if (status == LOCKED &&
 isLockExpired()) {

 releaseLock();
 }

 return status;
 }

 bool isAvailable() {
 return getStatus() == AVAILABLE;
 }

 bool lockSeat(User* user) {

 if (!isAvailable())
 return false;

 status = LOCKED;

 lockedBy = user;
 lockTime = time(nullptr);

 return true;
 }

 bool isLockedBy(User* user) {

 if (getStatus() != LOCKED)
 return false;

 return lockedBy != nullptr &&
 lockedBy->getId() == user->getId();
 }

 void bookSeat(User* user) {

 if (!isLockedBy(user))
 return;
 }

```

```

 status = BOOKED;

 lockedBy = nullptr;
 lockTime = 0;
 }

 void releaseLock() {

 if (status == LOCKED) {
 status = AVAILABLE;

 lockedBy = nullptr;
 lockTime = 0;
 }
 }

 void makeAvailable() {

 status = AVAILABLE;

 lockedBy = nullptr;
 lockTime = 0;
 }

private:

 bool isLockExpired() {

 if (status != LOCKED)
 return false;

 long long now =
 time(nullptr);

 return
 now - lockTime
 >= lockDurationSeconds;
 }
};

// =====
// SHOW
// =====

class Show {
 int id;

 Movie* movie;
 Theatre* theatre;
 Screen* screen;

 long long startTime;

 vector<ShowSeat*> showSeats;

public:
 Show(
 int id,
 Movie* movie,
 Theatre* theatre,

```

```

 Screen* screen,
 long long startTime
) {
 this->id = id;
 this->movie = movie;
 this->theatre = theatre;
 this->screen = screen;
 this->startTime = startTime;

 initializeShowSeats();
 }

 int getId() {
 return id;
 }

 Movie* getMovie() {
 return movie;
 }

 Theatre* getTheatre() {
 return theatre;
 }

 Screen* getScreen() {
 return screen;
 }

 long long getStartTime() {
 return startTime;
 }

 vector<ShowSeat*>& getShowSeats() {
 return showSeats;
 }

 ShowSeat* getShowSeatById(
 int seatId
) {
 for (auto showSeat : showSeats) {

 if (showSeat
 ->getSeat()
 ->getId()
 == seatId) {

 return showSeat;
 }
 }

 return nullptr;
 }

 vector<ShowSeat*> getAvailableSeats() {

 vector<ShowSeat*> result;

 for (auto seat : showSeats) {

 if (seat->isAvailable()) {

```

```

 result.push_back(seat);
 }
}

return result;
}

private:

void initializeShowSeats() {

 for (auto seat :
 screen->getSeats()) {

 double price = 0;

 if (seat->getType() == REGULAR)
 price = 200;

 else if (seat->getType() == PREMIUM)
 price = 300;

 else if (seat->getType() == RECLINER)
 price = 500;

 showSeats.push_back(
 new ShowSeat(
 seat,
 price
)
);
 }
}

};

// =====
// BOOKING
// =====

class Booking {
 int id;

 User* user;
 Show* show;

 vector<ShowSeat*> seats;

 double amount;

 BookingStatus status;

public:
 Booking(
 int id,
 User* user,
 Show* show,
 vector<ShowSeat*> seats,
 double amount
) {
 this->id = id;
 this->user = user;

```

```

 this->show = show;
 this->seats = seats;
 this->amount = amount;

 this->status = PAYMENT_PENDING;
 }

 int getId() {
 return id;
 }

 User* getUser() {
 return user;
 }

 Show* getShow() {
 return show;
 }

 vector<ShowSeat*>& getSeats() {
 return seats;
 }

 double getAmount() {
 return amount;
 }

 BookingStatus getStatus() {
 return status;
 }

 void confirm() {
 status = CONFIRMED;
 }

 void cancel() {
 status = CANCELLED;
 }

 void fail() {
 status = FAILED;
 }
};

// =====
// PAYMENT STRATEGY
// =====

class PaymentStrategy {

public:

 virtual bool pay(
 double amount
) = 0;

 virtual ~PaymentStrategy() {}
};

// =====
// UPI PAYMENT

```

```
// =====

class UPIPayment
 : public PaymentStrategy {

public:

 bool pay(double amount) override {

 cout << "Paid Rs."
 << amount
 << " using UPI\n";

 return true;
 }
};

// =====
// CARD PAYMENT
// =====

class CardPayment
 : public PaymentStrategy {

public:

 bool pay(double amount) override {

 cout << "Paid Rs."
 << amount
 << " using Card\n";

 return true;
 }
};

// =====
// BOOK MY SHOW SYSTEM
// =====

class BookMyShowSystem {

 vector<Movie*> movies;
 vector<Theatre*> theatres;
 vector<Show*> shows;
 vector<Booking*> bookings;

 int nextBookingId;

public:

 BookMyShowSystem() {
 nextBookingId = 1;
 }

 // -----
 // Add entities
 // -----

 void addMovie(Movie* movie) {
 movies.push_back(movie);
 }
};
```

```

}

void addTheatre(Theatre* theatre) {
 theatres.push_back(theatre);
}

void addShow>Show* show) {
 shows.push_back(show);
}

// -----
// Search movies by city
// -----

vector<Movie*> searchMovies(
 string city
) {

 set<int> movieIds;
 vector<Movie*> result;

 for (auto show : shows) {

 if (show->getTheatre()->getCity()
 != city) {

 continue;
 }

 int movieId =
 show->getMovie()->getId();

 if (movieIds.count(movieId))
 continue;

 movieIds.insert(movieId);

 result.push_back(
 show->getMovie()
);
 }

 return result;
}

// -----
// Get shows for movie + city
// -----

vector<Show*> getShows(
 Movie* movie,
 string city
) {

 vector<Show*> result;

 for (auto show : shows) {

 if (show->getMovie()->getId()
 == movie->getId()
 &&

```

```

 show->getTheatre()->getCity()
 == city) {

 result.push_back(show);
 }
 }

 return result;
}

// -----
// Display available seats
// -----

void showAvailableSeats(
 Show* show
) {

 cout << "\nAvailable seats:\n";

 for (auto showSeat :
 show->getAvailableSeats()) {

 cout << showSeat
 << "\n"
 << showSeat->getSeat()
 << showSeat->getSeatName()
 << " - Rs."
 << showSeat->getPrice()
 << "\n";
 }
}

// -----
// Create booking
// -----

Booking* createBooking(
 User* user,
 Show* show,
 vector<int> seatIds
) {

 vector<ShowSeat*> selectedSeats;

 double totalAmount = 0;

 /*
 Step 1:
 Validate + lock seats
 */

 for (int seatId : seatIds) {

 ShowSeat* showSeat =
 show->getShowSeatById(
 seatId
);

 if (showSeat == nullptr) {

 cout << "Invalid seat\n";

```



```

 releaseSeats(
 selectedSeats,
 user
);

 return nullptr;
 }

 bool locked =
 showSeat->lockSeat(user);

 if (!locked) {

 cout << "Seat "
 << showSeat
 ->getSeat()
 ->getSeatName()
 << " unavailable\n";

 releaseSeats(
 selectedSeats,
 user
);

 return nullptr;
 }

 selectedSeats.push_back(
 showSeat
);

 totalAmount +=
 showSeat->getPrice();
}

/*
 Step 2:
 Create booking
*/

Booking* booking =
 new Booking(
 nextBookingId++,
 user,
 show,
 selectedSeats,
 totalAmount
);

bookings.push_back(booking);

cout << "\nBooking created\n";

cout << "Booking ID: "
 << booking->getId()
 << "\n";

cout << "Amount: Rs."
 << totalAmount
 << "\n";

```

```

 return booking;
 }

// -----
// Make payment
// -----

bool makePayment(
 Booking* booking,
 PaymentStrategy* paymentStrategy
) {

 if (booking == nullptr)
 return false;

 if (booking->getStatus()
 != PAYMENT_PENDING) {

 cout << "Booking is not awaiting payment\n";

 return false;
 }

 /*
 Make sure locks still belong
 to this user.
 */

 for (auto seat :
 booking->getSeats()) {

 if (!seat->isLockedBy(
 booking->getUser()
)) {

 cout << "Seat lock expired\n";

 booking->fail();

 releaseSeats(
 booking->getSeats(),
 booking->getUser()
);

 return false;
 }
 }

 bool success =
 paymentStrategy->pay(
 booking->getAmount()
);

 if (!success) {

 booking->fail();

 releaseSeats(
 booking->getSeats(),
 booking->getUser()
);
 }
}

```

```

);

 return false;
}

/*
 Payment successful:
 LOCKED -> BOOKED
*/

for (auto seat :
 booking->getSeats()) {

 seat->bookSeat(
 booking->getUser()
);
}

booking->confirm();

cout << "Booking confirmed\n";

return true;
}

// -----
// Cancel confirmed booking
// -----

bool cancelBooking(
 Booking* booking
) {

 if (booking == nullptr)
 return false;

 if (booking->getStatus()
 != CONFIRMED) {

 cout << "Only confirmed booking can be cancelled\n";

 return false;
 }

 for (auto seat :
 booking->getSeats()) {

 seat->makeAvailable();
 }

 booking->cancel();

 cout << "Booking cancelled\n";

 return true;
}

// -----
// Display booking
// -----

```

```

void displayBooking(
 Booking* booking
) {

 if (booking == nullptr)
 return;

 cout << "\n===== BOOKING =====\n";

 cout << "Booking ID: "
 << booking->getId()
 << "\n";

 cout << "User: "
 << booking->getUser()->getName()
 << "\n";

 cout << "Movie: "
 << booking
 << booking->getShow()
 << booking->getMovie()
 << booking->getTitle()
 << "\n";

 cout << "Theatre: "
 << booking
 << booking->getShow()
 << booking->getTheatre()
 << booking->getName()
 << "\n";

 cout << "Seats: ";

 for (auto seat :
 booking->getSeats()) {

 cout << seat
 << seat->getSeat()
 << seat->getSeatName()
 << " ";
 }

 cout << "\nAmount: Rs."
 << booking->getAmount()
 << "\n";

 cout << "Status: ";

 if (booking->getStatus()
 == PAYMENT_PENDING)
 cout << "PAYMENT_PENDING";

 else if (booking->getStatus()
 == CONFIRMED)
 cout << "CONFIRMED";

 else if (booking->getStatus()
 == CANCELLED)
 cout << "CANCELLED";

 else

```

```

 cout << "FAILED";

 cout << "\n";
 }

private:

 void releaseSeats(
 vector<ShowSeat*>& seats,
 User* user
) {

 for (auto seat : seats) {

 if (seat->isLockedBy(user)) {
 seat->releaseLock();
 }
 }
 }
};

// =====
// MAIN
// =====

int main() {

 // -----
 // User
 // -----

 User* user =
 new User(
 1,
 "Anirudh"
);

 // -----
 // Movie
 // -----

 Movie* movie =
 new Movie(
 101,
 "Interstellar",
 169,
 "English"
);

 // -----
 // Theatre
 // -----

 Theatre* theatre =
 new Theatre(
 1,
 "PVR",
 "Prayagraj"
);

 // -----

```

```

// Screen
// -----

Screen* screen =
 new Screen(
 1,
 "Screen 1"
);

screen->addSeat(
 new Seat(
 1,
 "A",
 1,
 REGULAR
)
);

screen->addSeat(
 new Seat(
 2,
 "A",
 2,
 REGULAR
)
);

screen->addSeat(
 new Seat(
 3,
 "B",
 1,
 PREMIUM
)
);

screen->addSeat(
 new Seat(
 4,
 "B",
 2,
 PREMIUM
)
);

screen->addSeat(
 new Seat(
 5,
 "C",
 1,
 RECLINER
)
);

theatre->addScreen(screen);

// -----
// Show
// -----

long long showTime =

```

```

 time(nullptr) + 3600;

Show* show =
 new Show(
 1,
 movie,
 theatre,
 screen,
 showTime
);

// -----
// System
// -----

BookMyShowSystem system;

system.addMovie(movie);
system.addTheatre(theatre);
system.addShow(show);

// -----
// Show available seats
// -----

system.showAvailableSeats(
 show
);

// -----
// User selects A1 + A2
// -----

vector<int> selectedSeatIds = {
 1,
 2
};

Booking* booking =
 system.createBooking(
 user,
 show,
 selectedSeatIds
);

// -----
// Payment
// -----

PaymentStrategy* upi =
 new UPIPayment();

system.makePayment(
 booking,
 upi
);

// -----
// Show booking
// -----

```

```

system.displayBooking(
 booking
);

// -----
// Availability after booking
// -----

system.showAvailableSeats(
 show
);

// -----
// Cancel booking
// -----

system.cancelBooking(
 booking
);

// -----
// Availability after cancellation
// -----

system.showAvailableSeats(
 show
);

return 0;
}

```

The heart of the design is this:

```

Seat
→ physical seat only

ShowSeat
→ seat state for one show
→ AVAILABLE / LOCKED / BOOKED
→ price

```

And booking follows:

```

AVAILABLE
↓
lockSeat()
↓
LOCKED
↓
payment success
↓
BOOKED

```

If payment fails or the lock expires:

```

LOCKED → AVAILABLE

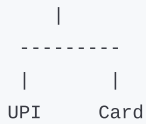
```

That is the part I would focus on most when explaining this LLD in an interview.

Also notice the Strategy Pattern:



PaymentStrategy



`BookMyShowSystem` doesn't care how payment happens:

```
paymentStrategy->pay(amount);
```

For a fresher interview, the classes you should be most comfortable implementing from scratch are `Seat` , `ShowSeat` , `Show` , `Booking` , and the booking/payment flow inside `BookMyShowSystem` .

13/09/2026, 13:58:28



why we created showseat class and how and when to think that showseat type classes needed ?

13/09/2026, 15:02:55



ShowSeat exists because a physical Seat and that seat's state **for a particular show** are not the same thing.

A physical seat like A1 belongs to a screen:

```
Screen 1
└─ Seat A1
```

That seat exists permanently. But its booking status changes per show:

```
3 PM show: A1 = BOOKED
6 PM show: A1 = AVAILABLE
9 PM show: A1 = LOCKED
```

So if you put:

```
bool booked;
double price;
```

inside Seat, you immediately get a problem. Which show does that booked value refer to?

That is why we split it into:

```
class Seat {
 int id;
 string row;
 int number;
 SeatType type;
};
```

and:

```
class ShowSeat {
 Seat* seat;
 SeatStatus status;
 double price;
};
```

Now the meaning is clean:

```
Seat
= physical thing

ShowSeat
= that physical seat in the context of one show
```

This pattern appears a lot in LLD.

The general rule is:

*If an object has some data/state that changes depending on another object or context, you may need an intermediate/context-specific class.*

For BookMyShow:

```
Seat + Show
↓
ShowSeat
```

Because `status` and `price` depend on both the seat and the show.

Another example from Splitwise:

```
User + Expense
↓
Split
```

A user does not globally have:

```
amount = 300
```

That amount only makes sense for a particular expense.

So we create:

```
class Split {
 User* user;
 double amount;
};
```

Same idea.

Another example from attendance:

```
Student + ClassSession
↓
AttendanceRecord
```

You should not store:

```
bool present;
```

inside `Student`, because:

```
Monday → PRESENT
Tuesday → ABSENT
Wednesday → LATE
```

So:

```
class AttendanceRecord {
 Student* student;
 AttendanceStatus status;
};
```

Again, context-specific state.

A very useful interview heuristic is:

*Ask: "Does this property belong permanently to this object, or only in relation to another object?"*

For `Seat`:

```
row, number, type
```

belong permanently.

But:

```
available/booked/locked
price
```

may depend on the show.

Therefore those go into `ShowSeat` .

You can recognize these “bridge” or “association” classes when you see a relationship like:

```
A and B have a many-to-many or context-dependent relationship,
and the relationship itself has data.
```

For example:

```
Student ↔ Course
```

If you only need enrollment, maybe a simple list is enough.

But if that relationship has:

```
grade
attendance
enrollmentDate
status
```

then you often create:

```
class Enrollment {
 Student* student;
 Course* course;

 string enrollmentDate;
 double grade;
};
```

That is the same design idea as `ShowSeat` .

So in interviews, remember this question:

*“Is there data that belongs to the relationship itself?”*

If yes, you may need another class.

For BookMyShow:

```
Show ↔ Seat

relationship data:
- status
- price
- lockedBy
- lockTime
```

Therefore:

```
class ShowSeat
```

is justified.

That is exactly how you should derive these kinds of classes instead of memorizing them.

13/09/2026, 15:02:55



why make movies when we have show ?

13/09/2026, 15:22:57



Because **Movie** and **Show** represent two different things.

**Movie** is the content itself:

```
Interstellar
169 minutes
English
Sci-Fi
```

**Show** means:

```
Interstellar
at PVR Screen 2
on 13 Sep
at 6:00 PM
```

So one movie can have many shows:

```
Movie: Interstellar
|
+-- 10 AM show at PVR
+-- 2 PM show at INOX
+-- 6 PM show at PVR
+-- 9 PM show at Cinepolis
```

If we remove **Movie** , then every **Show** would have to duplicate movie data:

```
class Show {
 string movieTitle;
 int duration;
 string language;
 ...
};
```

Then 20 shows of the same movie repeat the same information.

Better:

```
class Movie {
 int id;
 string title;
 int duration;
 string language;
};

class Show {
 int id;
 Movie* movie;
 Screen* screen;
 long long startTime;
};
```

The interview rule is:

**Movie** stores attributes common to all screenings.

**Show** stores attributes specific to one screening.

Think similarly:

Course vs ClassSession

Course:

DBMS

ClassSession:

DBMS lecture on Monday at 10 AM

or:

FlightRoute vs FlightInstance

Route:

Delhi → Bangalore

Flight:

that route on 15 Sep at 7 PM

So in BookMyShow:

Movie = WHAT is being played

Show = WHERE + WHEN that movie is being played

That's why both classes are needed.

13/09/2026, 15:22:57



explain each entity with this example

`Movie` is the content itself:  
```

Interstellar
169 minutes
English
Sci-Fi
```

`Show` means:  
```

Interstellar
at PVR Screen 2
on 13 Sep
at 6:00 PM
```

So one movie can have many shows:

13/09/2026, 15:23:53



Using the same **Interstellar at PVR** example, every important BookMyShow entity can be understood like this.

- **User** = the person booking

Example: Anirudh

- **Movie** = the content itself

Example:

Interstellar  
169 minutes  
English  
Sci-Fi

- **Theatre** = the physical cinema location

Example:

PVR Civil Lines  
Prayagraj

- **Screen** = one auditorium inside the theatre

Example:

Screen 2  
inside PVR Civil Lines

- **Seat** = one physical seat inside that screen

Example:

Seat A10  
Premium

This seat permanently belongs to **Screen 2**.

- **Show** = one particular screening of the movie

Example:

```
Interstellar
at PVR Civil Lines
Screen 2
on 13 Sep
at 6:00 PM
```

So one **Movie** can have many **Show**s:

```
Movie: Interstellar
|
+-- PVR Screen 2, 13 Sep, 6 PM
+-- PVR Screen 1, 13 Sep, 9 PM
+-- INOX Screen 3, 14 Sep, 4 PM
```

- **ShowSeat** = one physical seat **for one specific show**

Example:

```
Seat A10
for Interstellar, 13 Sep, 6 PM
status = LOCKED
price = ₹350
```

This is different from **Seat**.

```
Seat A10
= physical seat

ShowSeat(A10, 6 PM show)
= status/price of A10 for that particular show
```

So the same physical seat can be:

```
Interstellar 3 PM → A10 BOOKED
Interstellar 6 PM → A10 AVAILABLE
Interstellar 9 PM → A10 LOCKED
```

- **Booking** = one user's transaction/reservation

Example:

```
User: Anirudh
Show: Interstellar, 13 Sep, 6 PM
Seats: A10, A11
Amount: ₹700
Status: CONFIRMED
```

- **Payment** / **PaymentStrategy** = how that booking is paid

Example:

```
₹700
paid using UPI
```

So the full relationship becomes:



```
Anirudh
↓
Booking
↓
Interstellar Show
13 Sep, 6 PM
PVR Screen 2
↓
ShowSeats A10, A11
↓
physical Seats A10, A11
```

And the hierarchy is:

```
Theatre
↓
Screen
↓
Seat

Movie
↓
Show
↓
ShowSeat

User
↓
Booking
↓
Show + selected ShowSeats
```

The easiest way to remember them is:

Movie	= WHAT
Theatre	= WHERE
Screen	= WHICH auditorium
Seat	= WHICH physical chair
Show	= WHAT + WHERE + WHEN
ShowSeat	= seat state for that show
Booking	= WHO booked WHAT
Payment	= HOW they paid

13/09/2026, 15:23:52



`Movie` can have many `Show`  
why then movie does not have  
vector<show\*>shows

13/09/2026, 15:26:38



It **can** have `vector<Show*> shows`, but it does **not have to**.

This is an important LLD point: a relationship can exist without both sides storing references to each other.

For example, we already have:

```
class Show {
 Movie* movie;
};
```

So every `Show` knows which `Movie` it belongs to.

That is enough to represent:

Movie 1 ---- many Shows

because many different `Show` objects can all point to the same `Movie`.

Example:

```
Show* s1 = new Show(..., interstellar, ...);
Show* s2 = new Show(..., interstellar, ...);
Show* s3 = new Show(..., interstellar, ...);
```

All three have:

```
show->movie = interstellar;
```

So logically:

```
Interstellar
↑
|
+--- Show 1
+--- Show 2
+--- Show 3
```

Now, should `Movie` also store:

```
vector<Show*> shows;
```

You can do that if your use cases frequently need:

*"Give me all shows of this movie."*

Then this is reasonable:

```
class Movie {
 int id;
 string title;

 vector<Show*> shows;
```

```

public:
 void addShow>Show* show) {
 shows.push_back(show);
 }

 vector>Show*> getShows() {
 return shows;
 }
};

```

But there is a downside: now the relationship is stored in **two places**:

```

Show {
 Movie* movie;
}

Movie {
 vector>Show*> shows;
}

```

Whenever you create a show, you must update both:

```

Show* s = new Show(..., movie, ...);

movie->addShow(s);

```

If you forget one side, your data becomes inconsistent.

For example:

```

Show* s = new Show(..., interstellar, ...);

// forgot:
interstellar->addShow(s);

```

Now `Show` says:

```
I belong to Interstellar
```

but `Movie` says:

```
I have no shows
```

That is why in a simple design we often keep **one direction only**.

In our BookMyShow design:

```

class BookMyShowSystem {
 vector>Show*> shows;
};

```

So if we want all shows for a movie, we simply search:

```

vector>Show*> getShows(Movie* movie) {

 vector>Show*> result;

 for (auto show : shows) {
 if (show->getMovie()->getId() == movie->getId()) {
 result.push_back(show);
 }
 }

 return result;
}

```

```

 }
}

return result;
}

```

So the ownership/navigation becomes:

```

BookMyShowSystem
|
+-- all Shows
 |
 +-- Movie*

```

This is enough.

The general interview rule is:

***A relationship does not mean both classes must store each other. Store references only in the direction you actually need for navigation or ownership.***

For example:

```
Show → Movie
```

is definitely useful because when displaying a show we need to know which movie is playing.

But:

```
Movie → vector<Show*>
```

is optional. Add it only if it makes the required operations simpler.

This idea applies everywhere.

In Parking Lot:

```

ParkingFloor {
 vector<ParkingSpot*> spots;
}

```

We usually don't also put:

```

ParkingSpot {
 ParkingFloor* floor;
}

```

unless we need to navigate from a spot back to its floor.

So when deciding whether to add:

```
vector<Show*> shows;
```

ask:

*“Do I frequently need to navigate from `Movie` directly to all its `Show`s?”*

If yes → add it.

If no → let `BookMyShowSystem` manage shows and avoid redundant relationships.

For our fresher design, I prefer **not** putting `vector<Show*>` in `Movie` , because `BookMyShowSystem` already owns/searches all shows.

*13/09/2026, 15:26:37*



how you are calculating the pricing ?  
what if we want to give pricing with different ways

13/09/2026, 15:29:29



In the code I gave, pricing was very simple: I assigned the price while creating each `ShowSeat`.

For example:

```
if (seat->getType() == REGULAR)
 price = 200;

else if (seat->getType() == PREMIUM)
 price = 300;

else if (seat->getType() == RECLINER)
 price = 500;
```

That works for a basic system, but if pricing can vary in different ways, this logic should not stay hardcoded inside `Show`.

For example, price may depend on:

- seat type
- weekend vs weekday
- morning vs evening show
- blockbuster/surge pricing
- theatre
- discount/coupon

Then use **Strategy Pattern**.

```
class PricingStrategy {
public:
 virtual double calculatePrice(
 Seat* seat,
 Show* show
) = 0;

 virtual ~PricingStrategy() {}
};
```

Basic seat-type pricing:

```
class NormalPricingStrategy : public PricingStrategy {
public:
 double calculatePrice(
 Seat* seat,
 Show* show
) override {

 if (seat->getType() == REGULAR)
 return 200;

 if (seat->getType() == PREMIUM)
 return 300;
```

```

 return 500;
 }
};

```

Weekend pricing:

```

class WeekendPricingStrategy : public PricingStrategy {
public:
 double calculatePrice(
 Seat* seat,
 Show* show
) override {

 double basePrice;

 if (seat->getType() == REGULAR)
 basePrice = 200;
 else if (seat->getType() == PREMIUM)
 basePrice = 300;
 else
 basePrice = 500;

 return basePrice * 1.2;
 }
};

```

You could also have:

```

class MorningShowPricingStrategy : public PricingStrategy {
public:
 double calculatePrice(
 Seat* seat,
 Show* show
) override {

 // cheaper morning pricing
 if (seat->getType() == REGULAR)
 return 150;

 if (seat->getType() == PREMIUM)
 return 250;

 return 400;
 }
};

```

Then `Show` doesn't decide pricing itself.

Instead:

```

class Show {
 PricingStrategy* pricingStrategy;

public:
 Show(
 ...,
 PricingStrategy* pricingStrategy
) {
 this->pricingStrategy =
 pricingStrategy;
 }
};

```

```

 initializeShowSeats();
 }
};

```

Inside `initializeShowSeats()`:

```

void initializeShowSeats() {

 for (auto seat : screen->getSeats()) {

 double price =
 pricingStrategy->calculatePrice(
 seat,
 this
);

 showSeats.push_back(
 new ShowSeat(
 seat,
 price
)
);
 }
}

```

Now you can create different shows with different pricing:

```

PricingStrategy* normal =
 new NormalPricingStrategy();

Show* weekdayShow =
 new Show(
 1,
 movie,
 theatre,
 screen,
 showTime,
 normal
);

```

Weekend:

```

PricingStrategy* weekend =
 new WeekendPricingStrategy();

Show* weekendShow =
 new Show(
 2,
 movie,
 theatre,
 screen,
 saturdayTime,
 weekend
);

```

The key idea is:

```

PricingStrategy
|
+-- NormalPricing
+-- WeekendPricing

```



```
+-- MorningPricing
+-- SurgePricing
```

and `Show` simply asks:

```
pricingStrategy->calculatePrice(...)
```

It does not care which formula is being used.

In an interview, I would say:

*"If pricing is fixed, I can store price directly in `ShowSeat` . If pricing can vary based on seat type, timing, day, demand, or theatre, I'll extract the pricing logic into a `PricingStrategy` . Then different pricing algorithms can be added without modifying the booking flow."*

One more design point: even with Strategy Pattern, I would still store the **final calculated price** in `ShowSeat` or `Booking` at booking time. Otherwise if pricing rules change later, an old booking should not suddenly show a different amount.

13/09/2026, 15:29:28



the best algorithm for elevator design having  $n$  floors and  $m$  lifts then best algo for lift ?\\

13/09/2026, 15:37:04



For an interview, the best practical answer is usually not one “perfect” algorithm. A strong fresher answer is a **cost-based scheduler with direction awareness**, which is basically an improved elevator/SCAN approach.

For  $n$  floors and  $m$  lifts, when a request comes from floor  $f$  with direction  $d$ , evaluate every lift and assign a cost.

A simple and good cost rule is:

1. Prefer idle lifts
2. Prefer lifts already moving in the same direction
3. Request floor should lie ahead in that lift's current path
4. Among eligible lifts, choose minimum effective distance
5. If none are eligible, choose the nearest lift with a penalty for direction reversal

For example, request:

```
floor = 7
direction = UP
```

Lifts:

```
E1: floor 5, UP
E2: floor 6, DOWN
E3: floor 2, IDLE
```

Costs could be:

```
E1 → eligible, cost = |7 - 5| = 2
E2 → opposite direction, high penalty
E3 → idle, cost = |7 - 2| = 5
```

Choose **E1**.

A clean cost function can be:

```
int getCost(Elevator* e, int reqFloor, Direction reqDir) {

 int distance = abs(e->getCurrentFloor() - reqFloor);

 if (e->getDirection() == IDLE)
 return distance;

 if (e->getDirection() == reqDir) {

 if (reqDir == UP &&
 e->getCurrentFloor() <= reqFloor)
 return distance;

 if (reqDir == DOWN &&
 e->getCurrentFloor() >= reqFloor)
 return distance;

 }
}
```

```

 return distance + 1000; // direction-change penalty
}

```

Then scheduler:

```

Elevator* selectElevator(
 vector<Elevator*>& elevators,
 int reqFloor,
 Direction reqDir
) {
 Elevator* best = nullptr;
 int bestCost = INT_MAX;

 for (auto e : elevators) {

 int cost = getCost(
 e,
 reqFloor,
 reqDir
);

 if (cost < bestCost) {
 bestCost = cost;
 best = e;
 }
 }

 return best;
}

```

Inside each elevator, use two ordered request sets:

```

set<int> upRequests;
set<int, greater<int>> downRequests;

```

So if elevator is moving up:

```

5 → 7 → 8 → 10

```

then later serves downward requests:

```

10 → 6 → 3

```

This is very similar to **SCAN / LOOK disk scheduling**.

For a fresher interview, I would say:

*“For multiple elevators, I’ll use a cost-based scheduler. I’ll prefer an idle elevator or one already moving in the same direction with the requested floor ahead of it. Among those, I’ll choose the one with minimum estimated travel cost. Each elevator itself will process its assigned stops using two ordered sets for upward and downward requests, similar to the LOOK algorithm.”*

Time complexity for choosing a lift is:

```

O(m)

```

per external request, because you evaluate all  $m$  lifts.

If you also consider number of pending stops, a slightly better cost is:

```
cost =
distance
+ alpha * pendingStops
+ beta * directionChange
```

For example:

```
cost =
 abs(currentFloor - requestFloor)
 + 2 * pendingRequests
 + directionPenalty;
```

That is better than pure nearest-lift scheduling because the closest lift may already have many stops.

*13/09/2026, 15:37:03*



now fetch all the lld problems we have discussed till now

14/09/2026, 23:06:22



### Thinking

- **Identified and reviewed practiced LLD interview problems**

✓ Worked for 11s

So far, the **LLD problems we have actually discussed/practiced in depth** are:

#### 1. Vending Machine

- `Item`, `Inventory`, `Payment`, `VendingMachine`
- State Pattern
- `IdleState`, `PaymentState`, `DispenseState`
- We spent extra time understanding how State Pattern works.

#### 2. Splitwise

- `User`, `Group`, `Expense`, `Split`, `SplitwiseSystem`
- `balance[user1][user2]`
- Equal / Exact / Percentage splitting
- Strategy Pattern for split calculation.

#### 3. Elevator System

- `Elevator`, `Request`, `ExternalRequest`, `InternalRequest`, `ElevatorController`
- UP/DOWN request structures
- Direction-aware elevator selection
- Cost-based scheduling across `m` elevators
- SCAN/LOOK-style servicing.

#### 4. Car Rental System

- `User`, `Vehicle`, `Branch`, `VehicleInventory`, `Reservation`, `Payment`
- Availability using date-range overlap
- Pricing Strategy
- Payment Strategy
- Booking → pickup → return flow.

#### 5. Parking Lot

- `Vehicle`, `ParkingSpot`, `ParkingFloor`, `Ticket`, `ParkingLot`
- We specifically decided **not to use Gate classes**
- Parking Spot Allocation Strategy
- Pricing Strategy
- Payment Strategy.

## 6. Attendance Tracker

- Student , Course , ClassSession , AttendanceRecord , AttendanceSystem
- Important idea: attendance belongs to Student + Session , not directly to Student
- Manual / QR attendance using Strategy Pattern.

## 7. BookMyShow

- User , Movie , Theatre , Screen , Seat , Show , ShowSeat , Booking
- Why Movie and Show are separate
- Why ShowSeat is needed
- AVAILABLE → LOCKED → BOOKED
- Seat-lock expiry / double-booking prevention
- Payment Strategy
- Pricing Strategy.

We've also touched on related LLD/design prompts during interview research, such as **Shopping Cart / Order System, Inventory System, generic Booking/Ticket Booking System, Rule Engine, Product/Catalog hierarchy, and customer-support/leaderboard/feed-style systems**, but we have **not worked through those as fully as the 7 above**.

For our nightly LLD game, the main completed set so far is therefore:

**Vending Machine → Splitwise → Elevator → Car Rental → Parking Lot → Attendance Tracker → BookMyShow.**

14/09/2026, 23:06:23



start with car rental system explain everything I should explain to interviewer and then at the last tell me the full code .act as if you are giving the interview ?

14/09/2026, 23:09:14



Absolutely. I'll treat this like a real in-person LLD interview and show you exactly how I would expect a fresher to proceed.

We'll do **Car Rental System** from scratch.

## 1. How you should start in the interview

Do **not** directly write classes.

First say:

*"Before designing, I'd like to clarify the scope. I'll assume a user can search available cars for a time range, select a car, create a reservation, make payment, pick up the car, and return it. I'll also assume the company can have multiple branches. I'll ignore insurance, coupons, GPS tracking, and maintenance scheduling initially unless you want me to include them."*

This tells the interviewer that you understand LLD is requirement-driven.

Then establish the core flow:

```
Search cars
 ↓
Check availability
 ↓
Select vehicle
 ↓
Create reservation
 ↓
Calculate price
 ↓
Payment
 ↓
Pickup
 ↓
Return
```

## 2. Interviewer: "What are your main entities?"

You should answer:

*"My main entities would be `User`, `Vehicle`, `Branch`, `Reservation`, `Payment`, and `CarRentalSystem`. I may add `VehicleInventory`, `PricingStrategy`, and `PaymentStrategy` to separate changing behaviors."*

So:

```
User
Vehicle
Branch
VehicleInventory
Reservation
Payment
CarRentalSystem
```

And strategies:

PricingStrategy  
PaymentStrategy

### 3. Interviewer: “Why do you need Vehicle?”

A vehicle represents the actual rentable car.

You can say:

*“Vehicle stores properties that belong permanently to the car itself, such as registration number, brand, type, price per day, and current operational status.”*

```
enum VehicleType {
 SEDAN,
 SUV,
 HATCHBACK
};

enum VehicleStatus {
 AVAILABLE,
 RENTED,
 MAINTENANCE
};

class Vehicle {
 int id;
 string numberPlate;
 string brand;

 VehicleType type;
 VehicleStatus status;

 double pricePerDay;
};
```

One good interview point:

I would **not necessarily create** Sedan , SUV , Hatchback **subclasses**.

Why?

If they only differ by type:

```
VehicleType type;
```

is enough.

Use subclasses only if their behavior differs.

### 4. Interviewer: “What does Branch do?”

Suppose company operates in:

Bangalore Airport  
Koramangala  
Electronic City

Each branch contains vehicles.

So:



```
class Branch {
 int id;
 string name;
 string location;

 VehicleInventory* inventory;
};
```

Say:

*"A branch represents one physical rental location and owns/manages the inventory available at that branch."*

## 5. Why `VehicleInventory`?

You could directly do:

```
class Branch {
 vector<Vehicle*> vehicles;
};
```

and for a small problem that's completely acceptable.

But if searching/filtering vehicles becomes important, separating inventory is cleaner:

```
class VehicleInventory {
 vector<Vehicle*> vehicles;

public:
 void addVehicle(Vehicle* vehicle);
 Vehicle* getVehicleById(int id);
 vector<Vehicle*> getVehicles();
};
```

Responsibility:

```
Branch
 → location

VehicleInventory
 → vehicles and vehicle lookup
```

This follows SRP better.

## 6. Interviewer: "What is the most important class?"

Probably `Reservation`.

A reservation connects:

```
User
Vehicle
Start date
End date
Amount
Status
```

So:

```
enum ReservationStatus {
 CREATED,
```

```

 CONFIRMED,
 CANCELLED,
 COMPLETED
};

class Reservation {
 int id;

 User* user;
 Vehicle* vehicle;

 int startDay;
 int endDay;

 double amount;

 ReservationStatus status;
};

```

Say:

*“Reservation represents one booking of one vehicle by one user for a specific time interval.”*

## 7. Interviewer: “How will you check car availability?”

This is one of the most important Car Rental questions.

Do **not** say only:

```
vehicle->status == AVAILABLE
```

That is not sufficient for future bookings.

Example:

```

Car C1 already reserved:
20 Sep - 22 Sep

```

User asks:

```
15 Sep - 18 Sep
```

Car should still be available.

So availability depends on **reservation overlap**.

Two intervals overlap when:

```

newStart <= existingEnd
AND
newEnd >= existingStart

```

Example:

```

Existing: 10 - 15
New: 14 - 18

```

They overlap.

But:

```
Existing: 10 - 15
New: 16 - 18
```

does not.

Interview answer:

*"I will determine future availability based on overlapping reservations rather than only using vehicle status."*

That is a strong answer.

## 8. Then why do we need `VehicleStatus` ?

Good interviewer follow-up.

Because status represents the **current operational condition**, for example:

```
AVAILABLE
RENTED
MAINTENANCE
```

It answers:

*"What is happening to this vehicle right now?"*

Whereas reservations answer:

*"Is it booked during this requested future interval?"*

So these are different.

## 9. Interviewer: "How do you calculate price?"

Initially:

```
price = pricePerDay × numberOfDays
```

But then interviewer may say:

*"What if weekend pricing differs?"*

Then Strategy Pattern becomes useful.

```
class PricingStrategy {
public:
 virtual double calculatePrice(
 Vehicle* vehicle,
 int days
) = 0;

 virtual ~PricingStrategy() {}
};
```

Normal pricing:

```
class NormalPricingStrategy : public PricingStrategy {
public:
 double calculatePrice(
 Vehicle* vehicle,
```

```

 int days
) override {
 return vehicle->getPricePerDay() * days;
 }
};

```

Weekend pricing could be another strategy.

Interview definition:

*“Strategy Pattern lets me swap pricing algorithms without changing reservation creation logic.”*

## 10. Interviewer: “How do you handle payment?”

Same idea.

```

class PaymentStrategy {
public:
 virtual bool pay(double amount) = 0;
};

```

Then:

```

class UPIPayment : public PaymentStrategy {};
class CardPayment : public PaymentStrategy {};
class CashPayment : public PaymentStrategy {};

```

Why?

Because:

```

same goal = make payment
different implementations = UPI/Card/Cash

```

That is Strategy Pattern.

## 11. Interviewer: “What does CarRentalSystem do?”

This is the coordinator.

```

class CarRentalSystem {
 vector<User*> users;
 vector<Branch*> branches;
 vector<Reservation*> reservations;
};

```

Important methods:

```

searchVehicles()
createReservation()
makePayment()
pickupVehicle()
returnVehicle()
cancelReservation()

```

Say:

*“CarRentalSystem coordinates the use cases, but delegates pricing to `PricingStrategy`, payment to `PaymentStrategy`, and vehicle management to inventory.”*

## 12. Full booking flow you should explain verbally

Suppose:

```
User = A
Wants SUV
Branch = Bangalore Airport
Dates = 10 to 12
```

Flow:

```
searchVehicles(branch, SUV, 10, 12)
 ↓
iterate inventory
 ↓
check type
 ↓
check reservation overlap
 ↓
return available SUVs
 ↓
user selects vehicle
 ↓
createReservation()
 ↓
PricingStrategy calculates amount
 ↓
Reservation CREATED
 ↓
PaymentStrategy pays
 ↓
Reservation CONFIRMED
```

Pickup:

```
CONFIRMED
 ↓
vehicle status = RENTED
```

Return:

```
vehicle status = AVAILABLE
reservation status = COMPLETED
```

## 13. Interviewer: “What if two users book the same car at exactly the same time?”

Strong answer:

*“The availability check and reservation creation must be atomic. Otherwise both users could read the car as available and both create reservations. In production I’d use a transaction, row-level lock, or another atomic booking mechanism.”*

For fresher LLD, you usually don't need to implement DB locking in C++.

But mention it.

## 14. Interviewer: “What if user returns late?”

Say:

“Reservation can also store actual return time. During return, I would calculate additional late charges using pricing logic.”

Possible fields:

```
int actualReturnDay;
```

or real timestamps.

---

## 15. Interviewer: “What if a vehicle is under maintenance?”

Set:

```
vehicle->setStatus(MAINTENANCE);
```

Then `isVehicleAvailable()` immediately returns false.

---

## 16. Interviewer: “What if pickup and drop branch are different?”

Extend `Reservation` :

```
Branch* pickupBranch;
Branch* dropBranch;
```

No need initially unless requirement asks.

---

## 17. SOLID points

You don't need to recite all five. Mention useful ones.

### SRP

```
Vehicle → vehicle information
Inventory → manages vehicles
Reservation → booking information
PricingStrategy → pricing
PaymentStrategy → payment
CarRentalSystem → orchestration
```

### OCP

New pricing:

```
class HolidayPricingStrategy : public PricingStrategy
```

without modifying reservation code.

### DIP

`CarRentalSystem` depends on:

```
PricingStrategy*
PaymentStrategy*
```

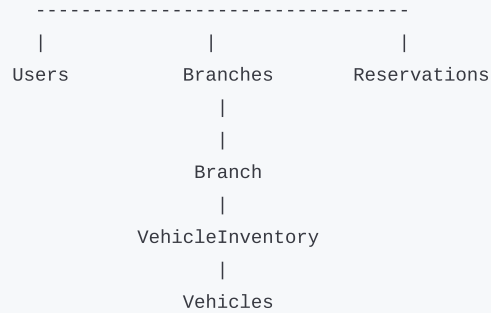
rather than concrete classes.

---

## 18. What class diagram should you draw?

Something like:

```
CarRentalSystem
|
```



#### Reservation

```

|
+-- User
+-- Vehicle
+-- start/end date
+-- amount
+-- status

```

#### PricingStrategy

```

|
+-- NormalPricing
+-- WeekendPricing

```

#### PaymentStrategy

```

|
+-- UPI
+-- Card
+-- Cash

```

## 19. What should you code first in an interview?

Start with these:

```

class User;
class Vehicle;
class Branch;
class Reservation;
class CarRentalSystem;

```

Then fill data members.

Only after that introduce strategies.

Don't start with patterns.

## 20. Full interview-ready code

```

#include <bits/stdc++.h>
using namespace std;

// =====
// ENUMS
// =====

enum VehicleType {
 SEDAN,
 SUV,

```

```

 HATCHBACK
 };

 enum VehicleStatus {
 AVAILABLE,
 RENTED,
 MAINTENANCE
 };

 enum ReservationStatus {
 CREATED,
 CONFIRMED,
 CANCELLED,
 COMPLETED
 };

// =====
// USER
// =====

class User {
 int id;
 string name;
 string drivingLicense;

public:
 User(
 int id,
 string name,
 string drivingLicense
) {
 this->id = id;
 this->name = name;
 this->drivingLicense = drivingLicense;
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }

 string getDrivingLicense() {
 return drivingLicense;
 }
};

// =====
// VEHICLE
// =====

class Vehicle {
 int id;
 string numberPlate;
 string brand;

 VehicleType type;
 VehicleStatus status;

```



```

 double pricePerDay;

public:
 Vehicle(
 int id,
 string numberPlate,
 string brand,
 VehicleType type,
 double pricePerDay
) {
 this->id = id;
 this->numberPlate = numberPlate;
 this->brand = brand;
 this->type = type;
 this->pricePerDay = pricePerDay;

 status = AVAILABLE;
 }

 int getId() {
 return id;
 }

 string getNumberPlate() {
 return numberPlate;
 }

 string getBrand() {
 return brand;
 }

 VehicleType getType() {
 return type;
 }

 VehicleStatus getStatus() {
 return status;
 }

 double getPricePerDay() {
 return pricePerDay;
 }

 void setStatus(VehicleStatus status) {
 this->status = status;
 }
};

// =====
// VEHICLE INVENTORY
// =====

class VehicleInventory {
 vector<Vehicle*> vehicles;

public:
 void addVehicle(Vehicle* vehicle) {
 vehicles.push_back(vehicle);
 }

 vector<Vehicle*>& getVehicles() {

```

```

 return vehicles;
 }

 Vehicle* getVehicleById(int id) {

 for (auto vehicle : vehicles) {

 if (vehicle->getId() == id)
 return vehicle;
 }

 return nullptr;
 }
};

// =====
// BRANCH
// =====

class Branch {
 int id;
 string name;
 string location;

 VehicleInventory* inventory;

public:
 Branch(
 int id,
 string name,
 string location
) {
 this->id = id;
 this->name = name;
 this->location = location;

 inventory = new VehicleInventory();
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }

 string getLocation() {
 return location;
 }

 VehicleInventory* getInventory() {
 return inventory;
 }

 void addVehicle(Vehicle* vehicle) {
 inventory->addVehicle(vehicle);
 }
};

// =====

```

```
// RESERVATION
// =====

class Reservation {
 int id;

 User* user;
 Vehicle* vehicle;

 int startDay;
 int endDay;

 double amount;

 ReservationStatus status;

public:
 Reservation(
 int id,
 User* user,
 Vehicle* vehicle,
 int startDay,
 int endDay,
 double amount
) {
 this->id = id;
 this->user = user;
 this->vehicle = vehicle;
 this->startDay = startDay;
 this->endDay = endDay;
 this->amount = amount;

 status = CREATED;
 }

 int getId() {
 return id;
 }

 User* getUser() {
 return user;
 }

 Vehicle* getVehicle() {
 return vehicle;
 }

 int getStartDay() {
 return startDay;
 }

 int getEndDay() {
 return endDay;
 }

 double getAmount() {
 return amount;
 }

 ReservationStatus getStatus() {
 return status;
 }
}
```

```

 }

 void confirm() {
 status = CONFIRMED;
 }

 void cancel() {
 status = CANCELLED;
 }

 void complete() {
 status = COMPLETED;
 }
};

// =====
// PRICING STRATEGY
// =====

class PricingStrategy {
public:
 virtual double calculatePrice(
 Vehicle* vehicle,
 int days
) = 0;

 virtual ~PricingStrategy() {}
};

class NormalPricingStrategy
: public PricingStrategy {

public:
 double calculatePrice(
 Vehicle* vehicle,
 int days
) override {

 return vehicle->getPricePerDay()
 * days;
 }
};

class WeekendPricingStrategy
: public PricingStrategy {

public:
 double calculatePrice(
 Vehicle* vehicle,
 int days
) override {

 return vehicle->getPricePerDay()
 * days
 * 1.2;
 }
};

// =====
// PAYMENT STRATEGY
// =====

```

```

class PaymentStrategy {
public:
 virtual bool pay(double amount) = 0;

 virtual ~PaymentStrategy() {}
};

class UPIPayment
 : public PaymentStrategy {

public:
 bool pay(double amount) override {

 cout << "Paid Rs."
 << amount
 << " using UPI\n";

 return true;
 }
};

class CardPayment
 : public PaymentStrategy {

public:
 bool pay(double amount) override {

 cout << "Paid Rs."
 << amount
 << " using Card\n";

 return true;
 }
};

class CashPayment
 : public PaymentStrategy {

public:
 bool pay(double amount) override {

 cout << "Paid Rs."
 << amount
 << " using Cash\n";

 return true;
 }
};

// =====
// CAR RENTAL SYSTEM
// =====

class CarRentalSystem {
 vector<User*> users;
 vector<Branch*> branches;
 vector<Reservation*> reservations;

 int nextReservationId;

```

```

public:
 CarRentalSystem() {
 nextReservationId = 1;
 }

 void addUser(User* user) {
 users.push_back(user);
 }

 void addBranch(Branch* branch) {
 branches.push_back(branch);
 }

 bool isVehicleAvailable(
 Vehicle* vehicle,
 int startDay,
 int endDay
) {
 if (vehicle->getStatus()
 == MAINTENANCE) {

 return false;
 }

 for (auto reservation : reservations) {

 if (reservation
 ->getVehicle()
 ->getId()
 != vehicle->getId()) {

 continue;
 }

 if (reservation->getStatus()
 == CANCELLED ||
 reservation->getStatus()
 == COMPLETED) {

 continue;
 }

 int existingStart =
 reservation->getStartDay();

 int existingEnd =
 reservation->getEndDay();

 bool overlap =
 startDay <= existingEnd &&
 endDay >= existingStart;

 if (overlap)
 return false;
 }

 return true;
 }

 vector<Vehicle*> searchVehicles(
 Branch* branch,

```

```

 VehicleType type,
 int startDay,
 int endDay
) {
 vector<Vehicle*> result;

 for (auto vehicle :
 branch->getInventory()
 ->getVehicles()) {

 if (vehicle->getType()
 != type) {

 continue;
 }

 if (isVehicleAvailable(
 vehicle,
 startDay,
 endDay)) {

 result.push_back(vehicle);
 }
 }

 return result;
 }
}

```

```

Reservation* createReservation(
 User* user,
 Vehicle* vehicle,
 int startDay,
 int endDay,
 PricingStrategy* pricingStrategy
) {
 if (startDay > endDay) {
 cout << "Invalid dates\n";
 return nullptr;
 }

 if (!isVehicleAvailable(
 vehicle,
 startDay,
 endDay)) {

 cout << "Vehicle unavailable\n";
 return nullptr;
 }

 int days =
 endDay - startDay + 1;

 double amount =
 pricingStrategy->calculatePrice(
 vehicle,
 days
);

 Reservation* reservation =
 new Reservation(
 nextReservationId++,

```

```

 user,
 vehicle,
 startDay,
 endDay,
 amount
);

 reservations.push_back(
 reservation
);

 cout << "Reservation created\n";

 cout << "Reservation ID: "
 << reservation->getId()
 << "\n";

 cout << "Amount: Rs."
 << amount
 << "\n";

 return reservation;
}

bool makePayment(
 Reservation* reservation,
 PaymentStrategy* paymentStrategy
) {
 if (reservation == nullptr)
 return false;

 if (reservation->getStatus()
 != CREATED) {

 cout << "Invalid reservation state\n";
 return false;
 }

 bool success =
 paymentStrategy->pay(
 reservation->getAmount()
);

 if (!success) {
 cout << "Payment failed\n";
 return false;
 }

 reservation->confirm();

 cout << "Reservation confirmed\n";

 return true;
}

bool pickupVehicle(
 Reservation* reservation
) {
 if (reservation == nullptr)
 return false;

```



```

 if (reservation->getStatus()
 != CONFIRMED) {

 cout << "Reservation not confirmed\n";

 return false;
 }

 Vehicle* vehicle =
 reservation->getVehicle();

 if (vehicle->getStatus()
 == MAINTENANCE) {

 cout << "Vehicle under maintenance\n";

 return false;
 }

 vehicle->setStatus(RENTED);

 cout << "Vehicle "
 << vehicle->getNumberPlate()
 << " picked up by "
 << reservation
 ->getUser()
 ->getName()
 << "\n";

 return true;
}

bool returnVehicle(
 Reservation* reservation
) {
 if (reservation == nullptr)
 return false;

 Vehicle* vehicle =
 reservation->getVehicle();

 if (vehicle->getStatus()
 != RENTED) {

 cout << "Vehicle is not rented\n";

 return false;
 }

 vehicle->setStatus(AVAILABLE);

 reservation->complete();

 cout << "Vehicle returned\n";
 cout << "Reservation completed\n";

 return true;
}

bool cancelReservation(
 Reservation* reservation

```

```

) {
 if (reservation == nullptr)
 return false;

 if (reservation->getStatus()
 == COMPLETED ||
 reservation->getStatus()
 == CANCELLED) {

 cout << "Reservation cannot be cancelled\n";

 return false;
 }

 reservation->cancel();

 cout << "Reservation cancelled\n";

 return true;
 }
}

```

```

void showReservations() {

 cout << "\n==== RESERVATIONS =====\n";

 for (auto reservation :
 reservations) {

 cout << "ID: "
 << reservation->getId()
 << "\n";

 cout << "User: "
 << reservation
 ->getUser()
 ->getName()
 << "\n";

 cout << "Vehicle: "
 << reservation
 ->getVehicle()
 ->getNumberPlate()
 << "\n";

 cout << "Days: "
 << reservation->getStartDay()
 << " to "
 << reservation->getEndDay()
 << "\n";

 cout << "Amount: Rs."
 << reservation->getAmount()
 << "\n\n";
 }
}
};

```

```

// =====
// MAIN
// =====

```

```

int main() {

 CarRentalSystem system;

 // User
 User* user =
 new User(
 1,
 "Anirudh",
 "DL12345"
);

 system.addUser(user);

 // Branch
 Branch* branch =
 new Branch(
 1,
 "Bangalore Airport",
 "Kempegowda Airport"
);

 system.addBranch(branch);

 // Vehicles
 Vehicle* car1 =
 new Vehicle(
 1,
 "KA01AA1111",
 "Hyundai Creta",
 SUV,
 3000
);

 Vehicle* car2 =
 new Vehicle(
 2,
 "KA01BB2222",
 "Honda City",
 SEDAN,
 2000
);

 Vehicle* car3 =
 new Vehicle(
 3,
 "KA01CC3333",
 "Tata Nexon",
 SUV,
 2500
);

 branch->addVehicle(car1);
 branch->addVehicle(car2);
 branch->addVehicle(car3);

 // Search
 vector<Vehicle*> available =
 system.searchVehicles(
 branch,
 SUV,

```

```

 10,
 12
);

 cout << "Available SUVs: "
 << available.size()
 << "\n";

 if (available.empty())
 return 0;

 Vehicle* selected =
 available[0];

 // Pricing Strategy
 PricingStrategy* pricing =
 new NormalPricingStrategy();

 Reservation* reservation =
 system.createReservation(
 user,
 selected,
 10,
 12,
 pricing
);

 // Payment
 PaymentStrategy* payment =
 new UPIPayment();

 system.makePayment(
 reservation,
 payment
);

 // Pickup
 system.pickupVehicle(
 reservation
);

 // Return
 system.returnVehicle(
 reservation
);

 system.showReservations();

 return 0;
}

```

The most important things to remember for the interview are not every line of code. Focus on these:

1. Vehicle vs Reservation responsibilities
2. Future availability = date overlap checking
3. Branch owns inventory
4. Reservation connects User + Vehicle + time interval
5. PricingStrategy handles varying pricing
6. PaymentStrategy handles varying payment methods
7. CarRentalSystem coordinates the workflow
8. Concurrent reservation creation must be atomic

If the interviewer stops you after the class diagram and says, “Looks good, now implement only availability and reservation creation,” that is normal. You do not need to force the entire code into the interview.

*14/09/2026, 23:09:14*



don't over engineer thinks defing pattern is good but like in brach its good to have vehicles so be it do not make new class

14/09/2026, 23:42:30



Agreed. For fresher LLD, that's the better default: **keep the object model simple, and introduce patterns only where behavior genuinely varies.**

So for Car Rental, I would simplify `Branch` to:

```
class Branch {
 int id;
 string name;
 string location;

 vector<Vehicle*> vehicles;

public:
 void addVehicle(Vehicle* vehicle) {
 vehicles.push_back(vehicle);
 }

 vector<Vehicle*>& getVehicles() {
 return vehicles;
 }
};
```

No separate `VehicleInventory` class unless the interviewer specifically asks for complex inventory behavior.

The rule we'll follow from now on is:

- If something is just naturally owned data, keep it in the owning class.
- Use a separate class when it has a **clear independent responsibility**.
- Use Strategy/State/Factory only when the requirement actually benefits from it.
- Don't create wrapper classes just to make the diagram look "designed."

So for Car Rental, the cleaner core becomes:

```
CarRentalSystem
|
+-- Users
+-- Branches
+-- Reservations

Branch
|
+-- vector<Vehicle*>

Reservation
|
+-- User*
+-- Vehicle*
+-- start/end
+-- amount
+-- status
```

```
PricingStrategy
PaymentStrategy
```

That's the style I'll use for your LLDs going forward: **simple first, extensible where justified, no unnecessary abstractions.**

14/09/2026, 23:42:30



when its necessary for having a carinventory ?

14/09/2026, 23:43:24



You need a separate `CarInventory` / `VehicleInventory` class only when **managing vehicles itself becomes a meaningful responsibility**, not just storing them.

If `Branch` only needs:

```
vector<Vehicle*> vehicles;
```

plus simple methods like:

```
addVehicle()
getVehicles()
```

then keep it inside `Branch`. That is simpler and better.

Create `CarInventory` when you start needing logic such as:

- search/filter by `VehicleType`, price, brand, fuel type
- track available/reserved/maintenance counts
- maintain fast lookup by vehicle id/number
- support multiple allocation/search strategies
- manage a large fleet efficiently
- centralize vehicle add/remove/update rules
- reuse inventory logic across many branches

Then it starts becoming its own responsibility:

```
class VehicleInventory {
 vector<Vehicle*> vehicles;

public:
 void addVehicle(Vehicle* vehicle);
 void removeVehicle(int vehicleId);

 Vehicle* getVehicleById(int id);

 vector<Vehicle*> searchByType(VehicleType type);

 vector<Vehicle*> getAvailableVehicles(
 int startDay,
 int endDay
);
};
```

And `Branch` becomes:

```
class Branch {
 int id;
 string name;
 string location;
```



```
VehicleInventory* inventory;
};
```

A good interview rule is:

*If the class is only wrapping a `vector`, don't create it.*

*If that collection has substantial business logic of its own, extract it.*

For our current fresher-level Car Rental design, I would **not** create `VehicleInventory` initially. I'd keep:

```
class Branch {
 vector<Vehicle*> vehicles;
};
```

and extract an inventory class only if the interviewer adds more inventory-related requirements.

14/09/2026, 23:43:23



Pasted text.txt

give full code without vehicleInventory ?

15/09/2026, 00:01:34



Yes. I'll keep the same design you already had, remove `VehicleInventory`, and let `Branch` directly own `vector<Vehicle*> vehicles`. Your current version had a separate `VehicleInventory` wrapping the vehicle vector, while `Branch` only stored a pointer to that inventory. `Pasted text.txt`

This is the cleaner fresher-level version.

```
#include <bits/stdc++.h>
using namespace std;

// =====
// ENUMS
// =====

enum VehicleType {
 SEDAN,
 SUV,
 HATCHBACK
};

enum VehicleStatus {
 AVAILABLE,
 RENTED,
 MAINTENANCE
};

enum ReservationStatus {
 CREATED,
 CONFIRMED,
 CANCELLED,
 COMPLETED
};

// =====
// USER
// =====

class User {
 int id;
 string name;
 string drivingLicense;

public:
 User(
 int id,
 string name,
 string drivingLicense
) {
 this->id = id;
 this->name = name;
 this->drivingLicense = drivingLicense;
 }
};
```

```

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }

 string getDrivingLicense() {
 return drivingLicense;
 }
};

// =====
// VEHICLE
// =====

class Vehicle {
 int id;
 string numberPlate;
 string brand;

 VehicleType type;
 VehicleStatus status;

 double pricePerDay;

public:
 Vehicle(
 int id,
 string numberPlate,
 string brand,
 VehicleType type,
 double pricePerDay
) {
 this->id = id;
 this->numberPlate = numberPlate;
 this->brand = brand;
 this->type = type;
 this->pricePerDay = pricePerDay;

 status = AVAILABLE;
 }

 int getId() {
 return id;
 }

 string getNumberPlate() {
 return numberPlate;
 }

 string getBrand() {
 return brand;
 }

 VehicleType getType() {
 return type;
 }
}

```

```

VehicleStatus getStatus() {
 return status;
}

double getPricePerDay() {
 return pricePerDay;
}

void setStatus(VehicleStatus status) {
 this->status = status;
}
};

// =====
// BRANCH
// =====

class Branch {
 int id;
 string name;
 string location;

 // Branch directly owns vehicles
 vector<Vehicle*> vehicles;

public:
 Branch(
 int id,
 string name,
 string location
) {
 this->id = id;
 this->name = name;
 this->location = location;
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }

 string getLocation() {
 return location;
 }

 void addVehicle(Vehicle* vehicle) {
 vehicles.push_back(vehicle);
 }

 vector<Vehicle*>& getVehicles() {
 return vehicles;
 }

 Vehicle* getVehicleById(int vehicleId) {

 for (auto vehicle : vehicles) {

 if (vehicle->getId() == vehicleId)

```

```

 return vehicle;
 }

 return nullptr;
}
};

// =====
// RESERVATION
// =====

class Reservation {
 int id;

 User* user;
 Vehicle* vehicle;

 int startDay;
 int endDay;

 double amount;

 ReservationStatus status;

public:
 Reservation(
 int id,
 User* user,
 Vehicle* vehicle,
 int startDay,
 int endDay,
 double amount
) {
 this->id = id;
 this->user = user;
 this->vehicle = vehicle;
 this->startDay = startDay;
 this->endDay = endDay;
 this->amount = amount;

 status = CREATED;
 }

 int getId() {
 return id;
 }

 User* getUser() {
 return user;
 }

 Vehicle* getVehicle() {
 return vehicle;
 }

 int getStartDay() {
 return startDay;
 }

 int getEndDay() {
 return endDay;
 }

```

```

 }

 double getAmount() {
 return amount;
 }

 ReservationStatus getStatus() {
 return status;
 }

 void confirm() {
 status = CONFIRMED;
 }

 void cancel() {
 status = CANCELLED;
 }

 void complete() {
 status = COMPLETED;
 }
};

// =====
// PRICING STRATEGY
// =====

class PricingStrategy {
public:
 virtual double calculatePrice(
 Vehicle* vehicle,
 int days
) = 0;

 virtual ~PricingStrategy() {}
};

// -----
// NORMAL PRICING
// -----

class NormalPricingStrategy : public PricingStrategy {

public:
 double calculatePrice(
 Vehicle* vehicle,
 int days
) override {

 return vehicle->getPricePerDay()
 * days;
 }
};

// -----
// WEEKEND PRICING
// -----

class WeekendPricingStrategy : public PricingStrategy {

public:

```

```

double calculatePrice(
 Vehicle* vehicle,
 int days
) override {

 return vehicle->getPricePerDay()
 * days
 * 1.2;

}

};

// =====
// PAYMENT STRATEGY
// =====

class PaymentStrategy {
public:
 virtual bool pay(double amount) = 0;

 virtual ~PaymentStrategy() {}
};

// -----
// UPI
// -----

class UPIPayment : public PaymentStrategy {

public:
 bool pay(double amount) override {

 cout << "Paid Rs."
 << amount
 << " using UPI\n";

 return true;
 }
};

// -----
// CARD
// -----

class CardPayment : public PaymentStrategy {

public:
 bool pay(double amount) override {

 cout << "Paid Rs."
 << amount
 << " using Card\n";

 return true;
 }
};

// -----
// CASH
// -----

class CashPayment : public PaymentStrategy {

```

```

public:
 bool pay(double amount) override {

 cout << "Paid Rs."
 << amount
 << " using Cash\n";

 return true;
 }
};

// =====
// CAR RENTAL SYSTEM
// =====

class CarRentalSystem {

 vector<User*> users;
 vector<Branch*> branches;
 vector<Reservation*> reservations;

 int nextReservationId;

public:
 CarRentalSystem() {
 nextReservationId = 1;
 }

 // -----
 // ADD USER
 // -----

 void addUser(User* user) {
 users.push_back(user);
 }

 // -----
 // ADD BRANCH
 // -----

 void addBranch(Branch* branch) {
 branches.push_back(branch);
 }

 // -----
 // CHECK VEHICLE AVAILABILITY
 // -----

 bool isVehicleAvailable(
 Vehicle* vehicle,
 int startDay,
 int endDay
) {

 // If vehicle is under maintenance,
 // we don't allow booking.
 if (vehicle->getStatus() == MAINTENANCE)
 return false;

 /*

```



```

 Check existing reservations
 for this vehicle.

 Two ranges overlap if:

 newStart <= existingEnd
 &&
 newEnd >= existingStart
 */

 for (auto reservation : reservations) {

 if (
 reservation->getVehicle()->getId()
 != vehicle->getId()
) {
 continue;
 }

 // Cancelled reservation doesn't block vehicle
 if (
 reservation->getStatus() == CANCELLED
) {
 continue;
 }

 int existingStart =
 reservation->getStartDay();

 int existingEnd =
 reservation->getEndDay();

 bool overlap =
 startDay <= existingEnd &&
 endDay >= existingStart;

 if (overlap)
 return false;
 }

 return true;
}

// -----
// SEARCH VEHICLES
// -----

vector<Vehicle*> searchVehicles(
 Branch* branch,
 VehicleType type,
 int startDay,
 int endDay
) {

 vector<Vehicle*> result;

 for (auto vehicle :
 branch->getVehicles()) {

 // filter by vehicle type
 if (vehicle->getType() != type)

```

```

 continue;

 // check reservation overlap
 if (
 isVehicleAvailable(
 vehicle,
 startDay,
 endDay
)
) {
 result.push_back(vehicle);
 }
 }

 return result;
}

// -----
// CREATE RESERVATION
// -----

Reservation* createReservation(
 User* user,
 Vehicle* vehicle,
 int startDay,
 int endDay,
 PricingStrategy* pricingStrategy
) {

 if (startDay > endDay) {

 cout << "Invalid dates\n";
 return nullptr;
 }

 if (
 !isVehicleAvailable(
 vehicle,
 startDay,
 endDay
)
) {

 cout << "Vehicle unavailable\n";
 return nullptr;
 }

 int numberOfDays =
 endDay - startDay + 1;

 double amount =
 pricingStrategy->calculatePrice(
 vehicle,
 numberOfDays
);

 Reservation* reservation =
 new Reservation(
 nextReservationId++,
 user,
 vehicle,

```

```

 startDay,
 endDay,
 amount
);

 reservations.push_back(
 reservation
);

 cout << "\nReservation created\n";

 cout << "Reservation ID: "
 << reservation->getId()
 << "\n";

 cout << "Vehicle: "
 << vehicle->getNumberPlate()
 << "\n";

 cout << "Amount: Rs."
 << amount
 << "\n";

 return reservation;
}

// -----
// PAYMENT
// -----

bool makePayment(
 Reservation* reservation,
 PaymentStrategy* paymentStrategy
) {

 if (reservation == nullptr)
 return false;

 if (
 reservation->getStatus()
 != CREATED
) {

 cout << "Reservation is not awaiting payment\n";
 return false;
 }

 bool success =
 paymentStrategy->pay(
 reservation->getAmount()
);

 if (!success) {

 cout << "Payment failed\n";
 return false;
 }

 reservation->confirm();

 cout << "Reservation confirmed\n";

```

```

 return true;
 }

// -----
// PICKUP VEHICLE
// -----

bool pickupVehicle(
 Reservation* reservation
) {

 if (reservation == nullptr)
 return false;

 if (
 reservation->getStatus()
 != CONFIRMED
) {

 cout << "Reservation not confirmed\n";
 return false;
 }

 Vehicle* vehicle =
 reservation->getVehicle();

 if (
 vehicle->getStatus()
 == MAINTENANCE
) {

 cout << "Vehicle is under maintenance\n";
 return false;
 }

 if (
 vehicle->getStatus()
 == RENTED
) {

 cout << "Vehicle already rented\n";
 return false;
 }

 vehicle->setStatus(
 RENTED
);

 cout << "Vehicle "
 << vehicle->getNumberPlate()
 << " picked up by "
 << reservation
 ->getUser()
 ->getName()
 << "\n";

 return true;
}

// -----

```

```

// RETURN VEHICLE
// -----

bool returnVehicle(
 Reservation* reservation
) {

 if (reservation == nullptr)
 return false;

 Vehicle* vehicle =
 reservation->getVehicle();

 if (
 vehicle->getStatus()
 != RENTED
) {

 cout << "Vehicle is not currently rented\n";
 return false;
 }

 vehicle->setStatus(
 AVAILABLE
);

 reservation->complete();

 cout << "Vehicle returned successfully\n";
 cout << "Reservation completed\n";

 return true;
}

// -----
// CANCEL RESERVATION
// -----

bool cancelReservation(
 Reservation* reservation
) {

 if (reservation == nullptr)
 return false;

 if (
 reservation->getStatus()
 == COMPLETED ||
 reservation->getStatus()
 == CANCELLED
) {

 cout << "Reservation cannot be cancelled\n";
 return false;
 }

 reservation->cancel();

 cout << "Reservation cancelled\n";

 return true;
}

```

```

}

// -----
// DISPLAY RESERVATIONS
// -----

void showReservations() {

 cout << "\n==== RESERVATIONS =====\n";

 for (auto reservation :
 reservations) {

 cout << "\nReservation ID: "
 << reservation->getId()
 << "\n";

 cout << "User: "
 << reservation
 << reservation->getUser()
 << reservation->getName()
 << "\n";

 cout << "Vehicle: "
 << reservation
 << reservation->getVehicle()
 << reservation->getNumberPlate()
 << "\n";

 cout << "Booking days: "
 << reservation->getStartDay()
 << " to "
 << reservation->getEndDay()
 << "\n";

 cout << "Amount: Rs."
 << reservation->getAmount()
 << "\n";

 cout << "Status: ";

 if (
 reservation->getStatus()
 == CREATED
) {
 cout << "CREATED";
 }

 else if (
 reservation->getStatus()
 == CONFIRMED
) {
 cout << "CONFIRMED";
 }

 else if (
 reservation->getStatus()
 == CANCELLED
) {
 cout << "CANCELLED";
 }
 }
}

```

```

 else {
 cout << "COMPLETED";
 }

 cout << "\n";
 }
}

};

// =====
// MAIN
// =====

int main() {

 CarRentalSystem system;

 // -----
 // USER
 // -----

 User* user =
 new User(
 1,
 "Anirudh",
 "DL12345"
);

 system.addUser(user);

 // -----
 // BRANCH
 // -----

 Branch* branch =
 new Branch(
 1,
 "Bangalore Airport",
 "Kempegowda Airport"
);

 system.addBranch(branch);

 // -----
 // VEHICLES
 // -----

 Vehicle* car1 =
 new Vehicle(
 1,
 "KA01AA1111",
 "Hyundai Creta",
 SUV,
 3000
);

 Vehicle* car2 =
 new Vehicle(
 2,
 "KA01BB2222",

```

```

 "Honda City",
 SEDAN,
 2000
);

Vehicle* car3 =
 new Vehicle(
 3,
 "KA01CC3333",
 "Tata Nexon",
 SUV,
 2500
);

// Branch directly owns vehicles

branch->addVehicle(car1);
branch->addVehicle(car2);
branch->addVehicle(car3);

// -----
// SEARCH SUV
// -----

vector<Vehicle*> available =
 system.searchVehicles(
 branch,
 SUV,
 10,
 12
);

cout << "\nAvailable SUVs: "
 << available.size()
 << "\n";

for (auto vehicle : available) {

 cout << vehicle->getBrand()
 << " - "
 << vehicle->getNumberPlate()
 << " - Rs."
 << vehicle->getPricePerDay()
 << "/day\n";
}

if (available.empty())
 return 0;

Vehicle* selectedVehicle =
 available[0];

// -----
// PRICING STRATEGY
// -----

PricingStrategy* pricingStrategy =
 new NormalPricingStrategy();

// -----
// CREATE RESERVATION

```



```

// -----

Reservation* reservation =
 system.createReservation(
 user,
 selectedVehicle,
 10,
 12,
 pricingStrategy
);

// -----
// PAYMENT STRATEGY
// -----

PaymentStrategy* paymentStrategy =
 new UPIPayment();

system.makePayment(
 reservation,
 paymentStrategy
);

// -----
// PICKUP
// -----

system.pickupVehicle(
 reservation
);

// -----
// RETURN
// -----

system.returnVehicle(
 reservation
);

// -----
// DISPLAY RESERVATIONS
// -----

system.showReservations();

return 0;
}

```

The important simplification is now:

```

Branch
|
+-- vector<Vehicle*> vehicles

```

instead of:

```

Branch
|
+-- VehicleInventory

```

```
|
+-- vector<Vehicle*>
```

And this is what I would prefer in a fresher interview unless the interviewer specifically adds substantial inventory-management requirements.

The patterns we **did keep** are justified:

```
PricingStrategy
 → because pricing can vary

PaymentStrategy
 → because payment methods can vary
```

But `VehicleInventory` was mostly wrapping a vector in this version, so removing it makes the design cleaner rather than less object-oriented.

*15/09/2026, 00:01:34*



```

...
newStart <= existingEnd
 &&
 newEnd >= existingStar

why newend>=existing start ?
...

```

15/09/2026, 00:08:26



Because for two date ranges to overlap, the **new reservation must not finish before the existing reservation starts.**

Suppose:

```

Existing booking: 10 ----- 15
New booking: 12 ----- 18

```

Here:

```

newStart = 12
newEnd = 18
existingStart = 10
existingEnd = 15

```

Check:

```

newStart <= existingEnd
12 <= 15 ✓

```

and:

```

newEnd >= existingStart
18 >= 10 ✓

```

So they overlap.

Now look at a non-overlapping case:

```

Existing: 10 ----- 15
New: 16 ----- 18

```

Here:

```

newStart <= existingEnd
16 <= 15 ✗

```

So no overlap.

Another non-overlapping case:

```

New: 5 ----- 9
Existing: 10 ----- 15

```

Here:

```

newEnd >= existingStart
9 >= 10 ✗

```

So no overlap.

That second condition:

```
newEnd >= existingStart
```

is needed to catch the case where the **new booking is completely before the existing booking**.

The easiest way to remember it is actually through the **non-overlap condition**:

Two intervals do **not** overlap only if:

```
newEnd < existingStart
OR
newStart > existingEnd
```

So overlap is just the opposite:

```
newEnd >= existingStart
&&
newStart <= existingEnd
```

Visually:

```
Overlap:
Existing: |-----|
New: |-----|

No overlap:
New: |----|
Existing: |-----|
```

So both conditions are necessary.

15/09/2026, 00:08:26



what for 5-9 and 6-8 ?

15/09/2026, 00:09:16



For:

```
Existing: 5 ----- 9
New: 6 ---- 8
```

they clearly overlap because the new interval is completely inside the existing one.

Check the formula:

```
newStart <= existingEnd
```

means:

```
6 <= 9 ✓
```

and:

```
newEnd >= existingStart
```

means:

```
8 >= 5 ✓
```

Both are true, so overlap exists.

This condition works for all overlap cases:

```
newStart <= existingEnd &&
newEnd >= existingStart
```

including:

- partial overlap
- one interval inside another
- exact same interval
- touching endpoints, if your business logic treats endpoints as overlapping.

15/09/2026, 00:09:16



I am just saying second condition is not essential giving case where we use only first condition and it gives wrong answer\

15/09/2026, 00:10:19



Yes — the **second condition is essential**.

If you only use:

```
newStart <= existingEnd
```

you get a wrong answer when the **new interval is completely before the existing interval**.

Example:

```
Existing: 10 ----- 15
New: 5 ---- 9
```

Using only the first condition:

```
newStart <= existingEnd
5 <= 15 ✓
```

So you would incorrectly say **overlap exists**.

But actually:

```
5 ---- 9 10 ---- 15
```

There is no overlap.

Now the second condition catches it:

```
newEnd >= existingStart
9 >= 10 ✗
```

Therefore:

```
newStart <= existingEnd &&
newEnd >= existingStart
```

becomes:

```
true && false
= false
```

Correct answer: **no overlap**.

So each condition prevents one non-overlap case:

```
newStart > existingEnd
```

means new interval is completely **after** existing.

And:

```
newEnd < existingStart
```

means new interval is completely **before** existing.

That's why we need both.

*15/09/2026, 00:10:19*



**\*\*Elevator**; there are **n** floors and **m** elevators and now do the same as in previous case

15/09/2026, 00:21:17



For an **Elevator System with **n** floors and **m** elevators**, I'd handle it exactly like the Car Rental discussion: first show what you should say in the interview, then derive entities, responsibilities, scheduling logic, and finally give you the full code.

## 1. How to start in the interview

Say:

*"I'll assume the building has **n** floors and **m** elevators. A user can make an external request by pressing UP/DOWN on a floor, and once inside an elevator can make an internal request by selecting a destination floor. The system should choose the best elevator for an external request, while each elevator manages its own pending stops and movement."*

That immediately defines the scope.

The main flow is:

```
External request
 ↓
ElevatorController
 ↓
select best elevator
 ↓
assign request to elevator
 ↓
elevator stores request
 ↓
elevator processes stops

Internal request
 ↓
directly goes to that elevator
 ↓
elevator stores request
 ↓
elevator processes stops
```

## 2. Main entities

For a fresher, keep this simple:

```
Elevator
Request
ExternalRequest
InternalRequest
ElevatorController
ElevatorSystem
```

You do **not** need extra classes like `FloorManager`, `ElevatorManager`, `RequestManager`, etc. unless there is a real requirement.

## 3. Request design

There are two kinds of requests.



## External request

User is outside elevator:

```
Floor 7
Pressed UP
```

So we need:

```
class ExternalRequest {
 int floor;
 Direction direction;
};
```

## Internal request

User is already inside elevator:

```
Pressed floor 12
```

So:

```
class InternalRequest {
 int destinationFloor;
};
```

You can use a common base:

```
class Request {
protected:
 int floor;
};
```

Then derive both.

That is reasonable because both are kinds of requests.

---

## 4. Elevator state

Each elevator needs to remember:

```
id
current floor
current direction
current state
pending upward stops
pending downward stops
```

So:

```
class Elevator {
 int id;
 int currentFloor;

 Direction direction;
 ElevatorState state;

 set<int> upRequests;
 set<int, greater<int>> downRequests;
};
```

Why two sets?

Suppose elevator is at 5.

Requests:

```
7
10
3
8
```

Then:

```
upRequests = {7, 8, 10}
downRequests = {3}
```

While moving up:

```
5 → 7 → 8 → 10
```

Then reverse:

```
10 → 3
```

This is similar to LOOK/SCAN-style servicing.

---

## 5. Why `set` instead of vector?

Because we want sorted stops automatically.

For up:

```
set<int> upRequests;
```

gives:

```
7, 8, 10
```

For down:

```
set<int, greater<int>> downRequests;
```

gives:

```
9, 6, 3
```

Also duplicate floor requests disappear automatically.

For interview purposes, this is a clean choice.

---

## 6. ElevatorController responsibility

This is very important.

Say:

*“The controller decides **which elevator should serve an external request**. Once selected, the elevator itself handles its movement and pending stops.”*

So:

```
ElevatorController
 → scheduling / assignment

Elevator
 → execution / movement
```

Do not make controller move every elevator floor-by-floor.

## 7. Scheduling algorithm

For `m` elevators, a practical fresher answer is:

1. Prefer elevator already moving in same direction.
2. Requested floor must lie ahead in its current direction.
3. Idle elevators are also eligible.
4. Among eligible elevators, choose minimum cost.
5. If none are ideal, fall back to minimum overall estimated cost.

For request:

```
floor = 7
direction = UP
```

Elevators:

```
E1 = floor 5, UP
E2 = floor 6, DOWN
E3 = floor 2, IDLE
```

Then:

```
E1 → eligible, cost 2
E2 → opposite direction, penalize
E3 → eligible, cost 5
```

Choose `E1`.

## 8. Cost function

Simplest:

```
cost = abs(currentFloor - requestFloor);
```

But direction matters.

So:

```
int getCost(Elevator* e, ExternalRequest* req) {
 int distance =
 abs(e->getCurrentFloor() - req->getFloor());

 if (e->getDirection() == IDLE)
 return distance;

 if (
 e->getDirection() == req->getDirection()
) {
 if (
```

```

 req->getDirection() == UP &&
 e->getCurrentFloor() <= req->getFloor()
)
 return distance;

 if (
 req->getDirection() == DOWN &&
 e->getCurrentFloor() >= req->getFloor()
)
 return distance;
 }

 return distance + 1000;
}

```

The `1000` is just a simple penalty for reversing direction.

In a more realistic system:

```

cost =
distance
+ pendingStopsPenalty
+ directionChangePenalty

```

But don't overcomplicate unless interviewer asks.

## 9. Internal request

If user is already inside `E2` and presses floor `12`, no scheduling is needed.

Just:

```

InternalRequest(12)
 ↓
E2.addInternalRequest()

```

That is the key difference:

```

External request
→ needs controller

Internal request
→ directly goes to elevator

```

## 10. Elevator movement

Suppose:

```

currentFloor = 5
direction = UP

upRequests = {7, 8, 10}
downRequests = {3}

```

Then elevator processes:

```

5 → 7 → 8 → 10

```

Once up requests finish:

```
direction = DOWN
```

Then:

```
10 → 3
```

Finally:

```
direction = IDLE
state = STOPPED
```

## 11. Important edge case

Suppose elevator is moving UP:

```
current = 5
target = 10
```

At floor 7 someone presses DOWN.

Should this elevator pick them immediately?

Usually no.

It can serve them later when it reverses, or another elevator can take the request.

That's why external request direction matters.

## 12. Complexity

For each external request, controller checks all  $m$  elevators:

```
 $O(m)$
```

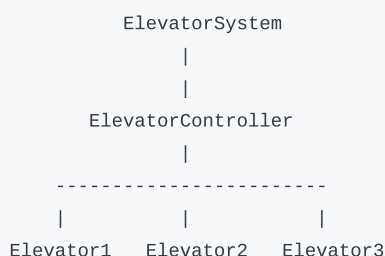
Adding a stop to a set :

```
 $O(\log k)$
```

where  $k$  is number of pending stops.

That is a good interview answer.

## 13. Class diagram



```
Elevator
|
+-- currentFloor
+-- direction
+-- state
+-- upRequests
+-- downRequests
```

```

Request
|
+-- ExternalRequest
|
+-- InternalRequest

```

## 14. Full code

```

#include <bits/stdc++.h>
using namespace std;

// =====
// ENUMS
// =====

enum Direction {
 UP,
 DOWN,
 IDLE
};

enum ElevatorState {
 MOVING,
 STOPPED
};

// =====
// REQUEST
// =====

class Request {
protected:
 int floor;

public:
 Request(int floor) {
 this->floor = floor;
 }

 int getFloor() {
 return floor;
 }

 virtual ~Request() {}
};

// =====
// EXTERNAL REQUEST
// =====

class ExternalRequest : public Request {
 Direction direction;

public:
 ExternalRequest(
 int floor,
 Direction direction
) : Request(floor) {

```

```

 this->direction = direction;
 }

 Direction getDirection() {
 return direction;
 }
};

// =====
// INTERNAL REQUEST
// =====

class InternalRequest : public Request {

public:
 InternalRequest(
 int destinationFloor
) : Request(destinationFloor) {}
};

// =====
// ELEVATOR
// =====

class Elevator {
 int id;
 int currentFloor;

 Direction direction;
 ElevatorState state;

 set<int> upRequests;
 set<int, greater<int>> downRequests;

public:

 Elevator(
 int id,
 int currentFloor = 0
) {
 this->id = id;
 this->currentFloor = currentFloor;

 direction = IDLE;
 state = STOPPED;
 }

 int getId() {
 return id;
 }

 int getCurrentFloor() {
 return currentFloor;
 }

 Direction getDirection() {
 return direction;
 }

 ElevatorState getState() {

```

```

 return state;
 }

 int getPendingRequestCount() {
 return upRequests.size()
 + downRequests.size();
 }

 // -----
 // INTERNAL REQUEST
 // -----

 void addInternalRequest(
 InternalRequest* request
) {

 int floor =
 request->getFloor();

 if (floor > currentFloor) {

 upRequests.insert(floor);

 if (direction == IDLE)
 direction = UP;
 }

 else if (floor < currentFloor) {

 downRequests.insert(floor);

 if (direction == IDLE)
 direction = DOWN;
 }

 else {

 cout << "Elevator "
 << id
 << " already at floor "
 << floor
 << "\n";

 openDoor();
 closeDoor();
 }
 }

 // -----
 // EXTERNAL REQUEST
 // -----

 void addExternalRequest(
 ExternalRequest* request
) {

 int floor =
 request->getFloor();

 /*
 We store based mainly on

```



```

 where the requested floor lies
 relative to current elevator.
 */

 if (floor > currentFloor) {

 upRequests.insert(floor);

 if (direction == IDLE)
 direction = UP;
 }

 else if (floor < currentFloor) {

 downRequests.insert(floor);

 if (direction == IDLE)
 direction = DOWN;
 }

 else {

 cout << "Elevator "
 << id
 << " already at requested floor\n";

 openDoor();
 closeDoor();
 }
}

// -----
// PROCESS REQUESTS
// -----

void processRequests() {

 while (
 !upRequests.empty() ||
 !downRequests.empty()
) {

 if (direction == UP) {

 if (!upRequests.empty()) {

 int nextFloor =
 *upRequests.begin();

 upRequests.erase(
 upRequests.begin()
);

 moveToFloor(
 nextFloor
);
 }

 else if (
 !downRequests.empty()
) {

```

```

 direction = DOWN;
 }

 else {
 direction = IDLE;
 }
}

else if (direction == DOWN) {

 if (!downRequests.empty()) {

 int nextFloor =
 *downRequests.begin();

 downRequests.erase(
 downRequests.begin()
);

 moveToFloor(
 nextFloor
);
 }

 else if (
 !upRequests.empty()
) {
 direction = UP;
 }

 else {
 direction = IDLE;
 }
}

else {

 /*
 Elevator is IDLE.
 Choose a direction based
 on available requests.
 */

 if (!upRequests.empty())
 direction = UP;

 else if (!downRequests.empty())
 direction = DOWN;
}

}

direction = IDLE;
state = STOPPED;

cout << "Elevator "
 << id
 << " is IDLE at floor "
 << currentFloor
 << "\n";
}

```

```

private:

// -----
// MOVE
// -----

void moveToFloor(
 int destination
) {

 state = MOVING;

 if (destination > currentFloor) {

 direction = UP;

 while (
 currentFloor < destination
) {

 currentFloor++;

 cout << "Elevator "
 << id
 << " moving UP -> "
 << currentFloor
 << "\n";

 }

 }

 else if (
 destination < currentFloor
) {

 direction = DOWN;

 while (
 currentFloor > destination
) {

 currentFloor--;

 cout << "Elevator "
 << id
 << " moving DOWN -> "
 << currentFloor
 << "\n";

 }

 }

 state = STOPPED;

 cout << "Elevator "
 << id
 << " reached floor "
 << currentFloor
 << "\n";

 openDoor();
 closeDoor();

}

```

```

void openDoor() {

 cout << "Elevator "
 << id
 << " door opened\n";

}

void closeDoor() {

 cout << "Elevator "
 << id
 << " door closed\n";

}

};

// =====
// ELEVATOR CONTROLLER
// =====

class ElevatorController {

public:

 int getCost(
 Elevator* elevator,
 ExternalRequest* request
) {

 int distance =
 abs(
 elevator->getCurrentFloor()
 - request->getFloor()
);

 // -----
 // IDLE elevator
 // -----

 if (
 elevator->getDirection()
 == IDLE
) {
 return distance;
 }

 // -----
 // Same direction
 // -----

 if (
 elevator->getDirection()
 == request->getDirection()
) {

 /*
 Elevator going UP
 and request is ahead
 */

 if (

```

```

 request->getDirection()
 == UP &&
 elevator->getCurrentFloor()
 <= request->getFloor()
) {

 return distance;
 }

 /*
 Elevator going DOWN
 and request is ahead
 in downward direction
 */

 if (
 request->getDirection()
 == DOWN &&
 elevator->getCurrentFloor()
 >= request->getFloor()
) {

 return distance;
 }
}

/*
 Opposite direction
 or request behind elevator.

 Penalize it.
*/

return distance + 1000;
}

// -----
// SELECT BEST ELEVATOR
// -----

Elevator* selectElevator(
 vector<Elevator*>& elevators,
 ExternalRequest* request
) {

 Elevator* bestElevator =
 nullptr;

 int minimumCost =
 INT_MAX;

 for (
 auto elevator :
 elevators
) {

 int cost =
 getCost(
 elevator,
 request
);

```

```

 /*
 Optional small penalty
 for many pending stops.
 */

 cost +=
 elevator
 ->getPendingRequestCount();

 if (
 cost < minimumCost
) {

 minimumCost = cost;
 bestElevator = elevator;
 }
 }

 return bestElevator;
}

// -----
// HANDLE EXTERNAL REQUEST
// -----

void handleRequest(
 vector<Elevator*>& elevators,
 ExternalRequest* request
) {

 Elevator* elevator =
 selectElevator(
 elevators,
 request
);

 if (
 elevator == nullptr
) {

 cout << "No elevator available\n";
 return;
 }

 cout << "Elevator "
 << elevator->getId()
 << " selected for floor "
 << request->getFloor()
 << "\n";

 elevator->addExternalRequest(
 request
);
}

};

// =====
// ELEVATOR SYSTEM
// =====

```

```

class ElevatorSystem {

 int totalFloors;

 vector<Elevator*> elevators;

 ElevatorController* controller;

public:

 ElevatorSystem(
 int totalFloors
) {
 this->totalFloors =
 totalFloors;

 controller =
 new ElevatorController();
 }

 // -----
 // ADD ELEVATOR
 // -----

 void addElevator(
 Elevator* elevator
) {

 elevators.push_back(
 elevator
);
 }

 // -----
 // EXTERNAL REQUEST
 // -----

 void requestElevator(
 int floor,
 Direction direction
) {

 if (
 floor < 0 ||
 floor >= totalFloors
) {

 cout << "Invalid floor\n";
 return;
 }

 ExternalRequest* request =
 new ExternalRequest(
 floor,
 direction
);

 controller->handleRequest(
 elevators,
 request
);
 }
}

```

```

}

// -----
// INTERNAL REQUEST
// -----

void selectFloor(
 int elevatorId,
 int destinationFloor
) {

 if (
 destinationFloor < 0 ||
 destinationFloor >= totalFloors
) {

 cout << "Invalid floor\n";
 return;
 }

 Elevator* elevator =
 getElevatorById(
 elevatorId
);

 if (
 elevator == nullptr
) {

 cout << "Invalid elevator\n";
 return;
 }

 InternalRequest* request =
 new InternalRequest(
 destinationFloor
);

 elevator
 ->addInternalRequest(
 request
);
}

// -----
// RUN ELEVATOR
// -----

void runElevator(
 int elevatorId
) {

 Elevator* elevator =
 getElevatorById(
 elevatorId
);

 if (
 elevator != nullptr
) {

```



```

 elevator
 ->processRequests();
 }
}

// -----
// RUN ALL
// -----

void runAll() {

 for (
 auto elevator :
 elevators
) {

 elevator
 ->processRequests();
 }
}

private:

Elevator* getElevatorById(
 int id
) {

 for (
 auto elevator :
 elevators
) {

 if (
 elevator->getId()
 == id
) {

 return elevator;
 }
 }

 return nullptr;
}

};

// =====
// MAIN
// =====

int main() {

 int n = 20; // floors
 int m = 3; // elevators

 ElevatorSystem system(n);

 Elevator* e1 =
 new Elevator(
 1,
 2
);

```

```

Elevator* e2 =
 new Elevator(
 2,
 6
);

Elevator* e3 =
 new Elevator(
 3,
 12
);

system.addElevator(e1);
system.addElevator(e2);
system.addElevator(e3);

// -----
// External request:
// Floor 7 presses UP
// -----

system.requestElevator(
 7,
 UP
);

// -----
// Suppose passenger gets into elevator 2
// and presses 15
// -----

system.selectFloor(
 2,
 15
);

// -----
// Another external request
// Floor 4 presses DOWN
// -----

system.requestElevator(
 4,
 DOWN
);

// -----
// Process all elevator requests
// -----

system.runAll();

return 0;
}

```

The core interview summary you should remember is:

```

External request
→ ElevatorController selects elevator

```

Internal request  
→ directly assigned to existing elevator

Controller  
→ decides WHICH elevator

Elevator  
→ decides HOW to process its stops

upRequests  
→ ascending order

downRequests  
→ descending order

And for  $m$  elevators, your scheduler checks all lifts in:

$O(m)$

which is a perfectly reasonable starting solution for an interview.

15/09/2026, 00:21:18



```
...
 ExternalRequest(
 int floor,
 Direction direction
) : Request(floor)
 this is constructor delegation and we can use super also ?
...
```

15/09/2026, 00:36:45



In **C++**, this:

```
ExternalRequest(
 int floor,
 Direction direction
) : Request(floor)
```

is a **base-class constructor call through the initializer list**.

It means:

*"Before constructing `ExternalRequest`, call the constructor of `Request` with `floor`."*

So if:

```
class Request {
protected:
 int floor;

public:
 Request(int floor) {
 this->floor = floor;
 }
};
```

then:

```
class ExternalRequest : public Request {
 Direction direction;

public:
 ExternalRequest(int floor, Direction direction)
 : Request(floor) {
 this->direction = direction;
 }
};
```

construction happens roughly like:

1. `Request(floor)` runs first
2. then `ExternalRequest` constructor body runs

For example:

```
ExternalRequest(7, UP)
```

does:

```
Request(7)
 ↓
floor = 7

then

ExternalRequest constructor
 ↓
direction = UP
```

But **no, C++ does not use** `super` .

`super` is used in languages like Java:

```
class ExternalRequest extends Request {
 ExternalRequest(int floor, Direction direction) {
 super(floor);
 this.direction = direction;
 }
}
```

In C++, equivalent syntax is:

```
ExternalRequest(int floor, Direction direction)
 : Request(floor) {
 this->direction = direction;
}
```

One terminology nuance: I would call this a **constructor initializer list**, specifically calling the **base-class constructor**.

“Constructor delegation” in C++ more specifically often means one constructor of the **same class** calling another constructor of the same class, like:

```
class A {
public:
 A(int x) {
 }

 A() : A(10) {
 }
};
```

Here:

```
A() : A(10)
```

is true constructor delegation.

Whereas:

```
ExternalRequest(...) : Request(floor)
```

is **base-class initialization**.

15/09/2026, 00:36:44



find parking lot full solution from the chat and then present it here

15/09/2026, 02:44:46



Yes — here is the **Parking Lot full solution we settled on earlier**:

- NO EntranceGate / ExitGate
- ParkingLot directly coordinates park/unpark
- Branch -style overengineering avoided
- Strategy Pattern only where it actually helps:
  - parking spot selection
  - pricing
  - payment

### What you should explain first in the interview

*"I'll model the system using Vehicle, ParkingSpot, ParkingFloor, Ticket, and ParkingLot. A parking lot has multiple floors, each floor has multiple parking spots, and each spot can park a compatible vehicle. On entry, the system finds a spot using a parking-allocation strategy and creates a ticket. On exit, it calculates the fee using a pricing strategy, processes payment using a payment strategy, releases the spot, and closes the ticket."*

The important relationships are:

```
ParkingLot
|
+-- ParkingFloor
 |
 +-- ParkingSpot
 |
 +-- Vehicle

Ticket
|
+-- Vehicle
+-- ParkingSpot
+-- ParkingFloor
+-- entryTime

ParkingSpotStrategy
|
+-- FirstAvailableSpotStrategy
+-- LowestFloorSpotStrategy

PricingStrategy
|
+-- HourlyPricingStrategy
+-- FlatPricingStrategy

PaymentStrategy
|
```

```
+-- UPIPayment
+-- CardPayment
+-- CashPayment
```

And the final code:

```
#include <bits/stdc++.h>
using namespace std;

// =====
// ENUMS
// =====

enum VehicleType {
 BIKE,
 CAR,
 TRUCK
};

enum SpotType {
 BIKE_SPOT,
 CAR_SPOT,
 TRUCK_SPOT
};

enum TicketStatus {
 ACTIVE,
 CLOSED
};

// =====
// VEHICLE
// =====

class Vehicle {
 string number;
 VehicleType type;

public:
 Vehicle(string number, VehicleType type) {
 this->number = number;
 this->type = type;
 }

 string getNumber() {
 return number;
 }

 VehicleType getType() {
 return type;
 }
};

// =====
// PARKING SPOT
// =====

class ParkingSpot {
 int id;
 SpotType type;
```

```

// nullptr means the spot is free
Vehicle* vehicle;

public:
 ParkingSpot(int id, SpotType type) {
 this->id = id;
 this->type = type;
 this->vehicle = nullptr;
 }

 int getId() {
 return id;
 }

 SpotType getType() {
 return type;
 }

 Vehicle* getVehicle() {
 return vehicle;
 }

 bool isAvailable() {
 return vehicle == nullptr;
 }

 bool canPark(Vehicle* v) {

 if (!isAvailable())
 return false;

 if (v->getType() == BIKE &&
 type == BIKE_SPOT)
 return true;

 if (v->getType() == CAR &&
 type == CAR_SPOT)
 return true;

 if (v->getType() == TRUCK &&
 type == TRUCK_SPOT)
 return true;

 return false;
 }

 bool parkVehicle(Vehicle* v) {

 if (!canPark(v))
 return false;

 vehicle = v;

 return true;
 }

 void removeVehicle() {
 vehicle = nullptr;
 }
};

```



```

// =====
// PARKING FLOOR
// =====

class ParkingFloor {
 int floorNumber;

 vector<ParkingSpot*> spots;

public:
 ParkingFloor(int floorNumber) {
 this->floorNumber = floorNumber;
 }

 int getFloorNumber() {
 return floorNumber;
 }

 void addSpot(ParkingSpot* spot) {
 spots.push_back(spot);
 }

 vector<ParkingSpot*>& getSpots() {
 return spots;
 }

 void displayAvailability() {

 cout << "\nFloor " << floorNumber << "\n";

 for (auto spot : spots) {

 cout << "Spot "
 << spot->getId()
 << " -> ";

 if (spot->isAvailable())
 cout << "FREE";
 else
 cout << "OCCUPIED";

 cout << "\n";
 }
 }
};

// =====
// TICKET
// =====

class Ticket {
 int id;

 Vehicle* vehicle;
 ParkingSpot* spot;
 ParkingFloor* floor;

 long long entryTime;

 TicketStatus status;

```

```

public:
 Ticket(
 int id,
 Vehicle* vehicle,
 ParkingSpot* spot,
 ParkingFloor* floor,
 long long entryTime
) {
 this->id = id;
 this->vehicle = vehicle;
 this->spot = spot;
 this->floor = floor;
 this->entryTime = entryTime;

 this->status = ACTIVE;
 }

 int getId() {
 return id;
 }

 Vehicle* getVehicle() {
 return vehicle;
 }

 ParkingSpot* getSpot() {
 return spot;
 }

 ParkingFloor* getFloor() {
 return floor;
 }

 long long getEntryTime() {
 return entryTime;
 }

 TicketStatus getStatus() {
 return status;
 }

 void close() {
 status = CLOSED;
 }
};

// =====
// PARKING SPOT STRATEGY
// =====

class ParkingSpotStrategy {
public:

 virtual pair<ParkingFloor*, ParkingSpot*>
 findSpot(
 vector<ParkingFloor*>& floors,
 Vehicle* vehicle
) = 0;

 virtual ~ParkingSpotStrategy() {}
};

```

```
// =====
// FIRST AVAILABLE STRATEGY
// =====

class FirstAvailableSpotStrategy
 : public ParkingSpotStrategy {

public:

 pair<ParkingFloor*, ParkingSpot*>
 findSpot(
 vector<ParkingFloor*>& floors,
 Vehicle* vehicle
) override {

 for (auto floor : floors) {

 for (auto spot : floor->getSpots()) {

 if (spot->canPark(vehicle)) {

 return {
 floor,
 spot
 };
 }
 }
 }

 return {nullptr, nullptr};
 }
};

// =====
// LOWEST FLOOR STRATEGY
// =====

class LowestFloorSpotStrategy
 : public ParkingSpotStrategy {

public:

 pair<ParkingFloor*, ParkingSpot*>
 findSpot(
 vector<ParkingFloor*>& floors,
 Vehicle* vehicle
) override {

 ParkingFloor* selectedFloor = nullptr;
 ParkingSpot* selectedSpot = nullptr;

 int bestFloor = INT_MAX;

 for (auto floor : floors) {

 for (auto spot : floor->getSpots()) {

 if (!spot->canPark(vehicle))
 continue;
 }
 }
 }
};
```

```

 if (floor->getFloorNumber()
 < bestFloor) {

 bestFloor =
 floor->getFloorNumber();

 selectedFloor = floor;
 selectedSpot = spot;
 }
 }
}

return {
 selectedFloor,
 selectedSpot
};
}
};

// =====
// PRICING STRATEGY
// =====

class PricingStrategy {
public:

 virtual double calculateFee(
 Ticket* ticket,
 long long exitTime
) = 0;

 virtual ~PricingStrategy() {}
};

// =====
// HOURLY PRICING
// =====

class HourlyPricingStrategy
 : public PricingStrategy {
public:

 double calculateFee(
 Ticket* ticket,
 long long exitTime
) override {

 long long duration =
 exitTime - ticket->getEntryTime();

 int hours =
 max(
 1LL,
 (duration + 3599) / 3600
);

 int rate = 0;

 VehicleType type =
 ticket->getVehicle()->getType();

```

```

 if (type == BIKE)
 rate = 10;

 else if (type == CAR)
 rate = 20;

 else if (type == TRUCK)
 rate = 30;

 return hours * rate;
 }
};

// =====
// FLAT PRICING
// =====

class FlatPricingStrategy
 : public PricingStrategy {

public:

 double calculateFee(
 Ticket* ticket,
 long long exitTime
) override {

 VehicleType type =
 ticket->getVehicle()->getType();

 if (type == BIKE)
 return 50;

 if (type == CAR)
 return 100;

 return 200;
 }
};

// =====
// PAYMENT STRATEGY
// =====

class PaymentStrategy {

public:

 virtual bool pay(double amount) = 0;

 virtual ~PaymentStrategy() {}
};

// =====
// UPI
// =====

class UPIPayment
 : public PaymentStrategy {

public:

```

```

 bool pay(double amount) override {

 cout << "Paid Rs."
 << amount
 << " using UPI\n";

 return true;
 }
};

// =====
// CARD
// =====

class CardPayment
 : public PaymentStrategy {

public:

 bool pay(double amount) override {

 cout << "Paid Rs."
 << amount
 << " using Card\n";

 return true;
 }
};

// =====
// CASH
// =====

class CashPayment
 : public PaymentStrategy {

public:

 bool pay(double amount) override {

 cout << "Paid Rs."
 << amount
 << " using Cash\n";

 return true;
 }
};

// =====
// PARKING LOT
// =====

class ParkingLot {

 vector<ParkingFloor*> floors;
 vector<Ticket*> tickets;

 ParkingSpotStrategy* spotStrategy;
 PricingStrategy* pricingStrategy;

```

```

 int nextTicketId;

public:

 ParkingLot(
 ParkingSpotStrategy* spotStrategy,
 PricingStrategy* pricingStrategy
) {
 this->spotStrategy =
 spotStrategy;

 this->pricingStrategy =
 pricingStrategy;

 nextTicketId = 1;
 }

 // -----
 // ADD FLOOR
 // -----

 void addFloor(
 ParkingFloor* floor
) {
 floors.push_back(floor);
 }

 // -----
 // CHANGE SPOT STRATEGY
 // -----

 void setSpotStrategy(
 ParkingSpotStrategy* strategy
) {
 spotStrategy = strategy;
 }

 // -----
 // CHANGE PRICING STRATEGY
 // -----

 void setPricingStrategy(
 PricingStrategy* strategy
) {
 pricingStrategy = strategy;
 }

 // -----
 // PARK VEHICLE
 // -----

 Ticket* parkVehicle(
 Vehicle* vehicle
) {

 pair<ParkingFloor*, ParkingSpot*> result =
 spotStrategy->findSpot(
 floors,
 vehicle
);
 }

```

```

ParkingFloor* floor =
 result.first;

ParkingSpot* spot =
 result.second;

if (spot == nullptr) {

 cout << "No compatible parking spot available\n";

 return nullptr;
}

bool parked =
 spot->parkVehicle(vehicle);

if (!parked) {

 cout << "Unable to park vehicle\n";

 return nullptr;
}

long long entryTime =
 time(nullptr);

Ticket* ticket =
 new Ticket(
 nextTicketId++,
 vehicle,
 spot,
 floor,
 entryTime
);

tickets.push_back(ticket);

cout << "\nVehicle "
 << vehicle->getNumber()
 << " parked successfully\n";

cout << "Ticket ID: "
 << ticket->getId()
 << "\n";

cout << "Floor: "
 << floor->getFloorNumber()
 << "\n";

cout << "Spot: "
 << spot->getId()
 << "\n";

return ticket;
}

// -----
// UNPARK VEHICLE
// -----

bool unparkVehicle(

```



```

Ticket* ticket,
PaymentStrategy* paymentStrategy
) {

 if (ticket == nullptr) {

 cout << "Invalid ticket\n";

 return false;
 }

 if (ticket->getStatus()
 == CLOSED) {

 cout << "Ticket already closed\n";

 return false;
 }

 long long exitTime =
 time(nullptr);

 double fee =
 pricingStrategy
 ->calculateFee(
 ticket,
 exitTime
);

 cout << "\nParking fee = Rs."
 << fee
 << "\n";

 bool success =
 paymentStrategy->pay(fee);

 if (!success) {

 cout << "Payment failed\n";

 return false;
 }

 ticket
 ->getSpot()
 ->removeVehicle();

 ticket->close();

 cout << "Vehicle "
 << ticket
 ->getVehicle()
 ->getNumber()
 << " removed successfully\n";

 cout << "Ticket closed\n";

 return true;
}

// -----

```

```

// DISPLAY AVAILABILITY
// -----

void displayAvailability() {

 cout << "\n==== PARKING STATUS ==== \n";

 for (auto floor : floors) {
 floor->displayAvailability();
 }
}

// -----
// GET TICKET
// -----

Ticket* getTicketById(
 int ticketId
) {

 for (auto ticket : tickets) {

 if (ticket->getId()
 == ticketId) {

 return ticket;
 }
 }

 return nullptr;
}

};

// =====
// MAIN
// =====

int main() {

 // -----
 // FLOORS
 // -----

 ParkingFloor* floor1 =
 new ParkingFloor(1);

 ParkingFloor* floor2 =
 new ParkingFloor(2);

 // -----
 // FLOOR 1 SPOTS
 // -----

 floor1->addSpot(
 new ParkingSpot(
 101,
 BIKE_SPOT
)
);

 floor1->addSpot(

```

```

 new ParkingSpot(
 102,
 CAR_SPOT
)
);

 floor1->addSpot(
 new ParkingSpot(
 103,
 CAR_SPOT
)
);

 floor1->addSpot(
 new ParkingSpot(
 104,
 TRUCK_SPOT
)
);

 // -----
 // FLOOR 2 SPOTS
 // -----

 floor2->addSpot(
 new ParkingSpot(
 201,
 BIKE_SPOT
)
);

 floor2->addSpot(
 new ParkingSpot(
 202,
 CAR_SPOT
)
);

 floor2->addSpot(
 new ParkingSpot(
 203,
 TRUCK_SPOT
)
);

 // -----
 // STRATEGIES
 // -----

 ParkingSpotStrategy* spotStrategy =
 new FirstAvailableSpotStrategy();

 PricingStrategy* pricingStrategy =
 new HourlyPricingStrategy();

 // -----
 // PARKING LOT
 // -----

 ParkingLot parkingLot(
 spotStrategy,

```

```

 pricingStrategy
);

 parkingLot.addFloor(floor1);
 parkingLot.addFloor(floor2);

 // -----
 // VEHICLES
 // -----

 Vehicle* bike =
 new Vehicle(
 "UP70-BIKE-01",
 BIKE
);

 Vehicle* car1 =
 new Vehicle(
 "UP70-CAR-01",
 CAR
);

 Vehicle* car2 =
 new Vehicle(
 "UP70-CAR-02",
 CAR
);

 Vehicle* truck =
 new Vehicle(
 "UP70-TRUCK-01",
 TRUCK
);

 // -----
 // INITIAL STATUS
 // -----

 parkingLot.displayAvailability();

 // -----
 // PARK
 // -----

 Ticket* bikeTicket =
 parkingLot.parkVehicle(
 bike
);

 Ticket* carTicket1 =
 parkingLot.parkVehicle(
 car1
);

 Ticket* carTicket2 =
 parkingLot.parkVehicle(
 car2
);

 Ticket* truckTicket =
 parkingLot.parkVehicle(

```

```

 truck
);

// -----
// STATUS
// -----

parkingLot.displayAvailability();

// -----
// EXIT USING UPI
// -----

PaymentStrategy* upi =
 new UPIPayment();

parkingLot.unparkVehicle(
 carTicket1,
 upi
);

// -----
// STATUS AGAIN
// -----

parkingLot.displayAvailability();

// -----
// CHANGE SPOT STRATEGY
// -----

ParkingSpotStrategy* lowestFloor =
 new LowestFloorSpotStrategy();

parkingLot.setSpotStrategy(
 lowestFloor
);

// -----
// CHANGE PRICING STRATEGY
// -----

PricingStrategy* flatPricing =
 new FlatPricingStrategy();

parkingLot.setPricingStrategy(
 flatPricing
);

Vehicle* car3 =
 new Vehicle(
 "UP70-CAR-03",
 CAR
);

Ticket* carTicket3 =
 parkingLot.parkVehicle(
 car3
);

PaymentStrategy* card =

```

```

 new CardPayment();

 parkingLot.unparkVehicle(
 carTicket3,
 card
);

 return 0;
 }

```

The three Strategy Pattern uses are:

```

ParkingSpotStrategy
→ how do I choose the spot?

PricingStrategy
→ how do I calculate parking fee?

PaymentStrategy
→ how does the customer pay?

```

And the main flow you should explain is:

```

parkVehicle()
↓
spotStrategy->findSpot()
↓
spot->parkVehicle()
↓
create Ticket

```

Then:

```

unparkVehicle()
↓
pricingStrategy->calculateFee()
↓
paymentStrategy->pay()
↓
spot->removeVehicle()
↓
ticket->close()

```

And remember the design choice we specifically made earlier:

```
Vehicle* vehicle;
```

inside `ParkingSpot` is enough. We don't also need:

```
bool occupied;
```

because:

```

bool isAvailable() {
 return vehicle == nullptr;
}

```

avoids duplicated state.

15/09/2026, 02:44:46



do the same fetching for book my show

15/09/2026, 02:55:25



Yes — here is the **BookMyShow full solution we settled on earlier**, fetched back into one clean version.

The design choices we used were:

- `Movie` and `Show` are separate.
- `Seat` is only the physical seat.
- `ShowSeat` stores show-specific seat state like `AVAILABLE` / `LOCKED` / `BOOKED`.
- `Booking` connects user + show + selected seats.
- `PaymentStrategy` handles UPI/Card.
- `PricingStrategy` can be added when pricing varies.
- Seat locking prevents double booking.

### What you should explain first in the interview

*"I'll model the system using `User`, `Movie`, `Theatre`, `Screen`, `Seat`, `Show`, `ShowSeat`, `Booking`, and `BookMyShowSystem`.*

*A theatre has screens, screens have physical seats, and a show represents one movie playing on one screen at a particular time.*

*Since seat availability changes per show, I'll keep booking state in `ShowSeat`, not directly in `Seat`.*

*During booking, seats move from `AVAILABLE` → `LOCKED` → `BOOKED`. If payment fails or the lock expires, they become `AVAILABLE` again."*

The core relationships are:

```
Theatre
|
+-- Screen
 |
 +-- Seat

Movie
|
+-- Show
 |
 +-- ShowSeat
 |
 +-- Seat

Booking
|
+-- User
+-- Show
+-- selected ShowSeats
```

And the final code:

```
#include <bits/stdc++.h>
using namespace std;
```

```
// =====
// ENUMS
// =====

enum SeatType {
 REGULAR,
 PREMIUM,
 RECLINER
};

enum SeatStatus {
 AVAILABLE,
 LOCKED,
 BOOKED
};

enum BookingStatus {
 PAYMENT_PENDING,
 CONFIRMED,
 CANCELLED,
 FAILED
};

// =====
// USER
// =====

class User {
 int id;
 string name;

public:
 User(int id, string name) {
 this->id = id;
 this->name = name;
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }
};

// =====
// MOVIE
// =====

class Movie {
 int id;
 string title;
 int durationMinutes;
 string language;

public:
 Movie(
 int id,
 string title,
 int durationMinutes,
```



```

 string language
) {
 this->id = id;
 this->title = title;
 this->durationMinutes = durationMinutes;
 this->language = language;
 }

 int getId() {
 return id;
 }

 string getTitle() {
 return title;
 }

 int getDurationMinutes() {
 return durationMinutes;
 }

 string getLanguage() {
 return language;
 }
};

// =====
// SEAT
// Physical seat only
// =====

class Seat {
 int id;
 string row;
 int number;
 SeatType type;

public:
 Seat(
 int id,
 string row,
 int number,
 SeatType type
) {
 this->id = id;
 this->row = row;
 this->number = number;
 this->type = type;
 }

 int getId() {
 return id;
 }

 string getRow() {
 return row;
 }

 int getNumber() {
 return number;
 }
}

```

```

 SeatType getType() {
 return type;
 }

 string getSeatName() {
 return row + to_string(number);
 }
};

// =====
// SCREEN
// =====

class Screen {
 int id;
 string name;

 vector<Seat*> seats;

public:
 Screen(int id, string name) {
 this->id = id;
 this->name = name;
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }

 void addSeat(Seat* seat) {
 seats.push_back(seat);
 }

 vector<Seat*>& getSeats() {
 return seats;
 }
};

// =====
// THEATRE
// =====

class Theatre {
 int id;
 string name;
 string city;

 vector<Screen*> screens;

public:
 Theatre(
 int id,
 string name,
 string city
) {
 this->id = id;
 this->name = name;
 }
};

```

```

 this->city = city;
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }

 string getCity() {
 return city;
 }

 void addScreen(Screen* screen) {
 screens.push_back(screen);
 }

 vector<Screen*>& getScreens() {
 return screens;
 }
};

// =====
// FORWARD DECLARATION
// =====

class Show;

// =====
// PRICING STRATEGY
// =====

class PricingStrategy {
public:
 virtual double calculatePrice(
 Seat* seat,
 Show* show
) = 0;

 virtual ~PricingStrategy() {}
};

// =====
// NORMAL PRICING
// =====

class NormalPricingStrategy
 : public PricingStrategy {
public:
 double calculatePrice(
 Seat* seat,
 Show* show
) override {

 if (seat->getType() == REGULAR)
 return 200;

 if (seat->getType() == PREMIUM)

```

```

 return 300;

 return 500;
 }
};

// =====
// WEEKEND PRICING
// =====

class WeekendPricingStrategy
: public PricingStrategy {

public:
 double calculatePrice(
 Seat* seat,
 Show* show
) override {

 double basePrice;

 if (seat->getType() == REGULAR)
 basePrice = 200;

 else if (seat->getType() == PREMIUM)
 basePrice = 300;

 else
 basePrice = 500;

 return basePrice * 1.2;
 }
};

// =====
// SHOW SEAT
// Seat state for one particular show
// =====

class ShowSeat {
 Seat* seat;

 SeatStatus status;

 double price;

 User* lockedBy;

 long long lockTime;

 int lockDurationSeconds;

public:
 ShowSeat(
 Seat* seat,
 double price
) {
 this->seat = seat;
 this->price = price;

 status = AVAILABLE;
 }
};

```

```

 lockedBy = nullptr;
 lockTime = 0;

 lockDurationSeconds = 300;
 }

 Seat* getSeat() {
 return seat;
 }

 double getPrice() {
 return price;
 }

 SeatStatus getStatus() {

 if (
 status == LOCKED &&
 isLockExpired()
) {
 releaseLock();
 }

 return status;
 }

 bool isAvailable() {
 return getStatus() == AVAILABLE;
 }

 bool lockSeat(User* user) {

 if (!isAvailable())
 return false;

 status = LOCKED;

 lockedBy = user;

 lockTime = time(nullptr);

 return true;
 }

 bool isLockedBy(User* user) {

 if (getStatus() != LOCKED)
 return false;

 if (lockedBy == nullptr)
 return false;

 return lockedBy->getId()
 == user->getId();
 }

 bool bookSeat(User* user) {

 if (!isLockedBy(user))
 return false;
 }

```

```

 status = BOOKED;

 lockedBy = nullptr;

 lockTime = 0;

 return true;
 }

 void releaseLock() {

 if (status == LOCKED) {

 status = AVAILABLE;

 lockedBy = nullptr;

 lockTime = 0;
 }
 }

 void makeAvailable() {

 status = AVAILABLE;

 lockedBy = nullptr;

 lockTime = 0;
 }

private:

 bool isLockExpired() {

 if (status != LOCKED)
 return false;

 long long currentTime =
 time(nullptr);

 return
 currentTime - lockTime
 >= lockDurationSeconds;
 }
};

// =====
// SHOW
// =====

class Show {
 int id;

 Movie* movie;

 Theatre* theatre;

 Screen* screen;

 long long startTime;

```

```

vector<ShowSeat*> showSeats;

PricingStrategy* pricingStrategy;

public:
 Show(
 int id,
 Movie* movie,
 Theatre* theatre,
 Screen* screen,
 long long startTime,
 PricingStrategy* pricingStrategy
) {
 this->id = id;
 this->movie = movie;
 this->theatre = theatre;
 this->screen = screen;
 this->startTime = startTime;
 this->pricingStrategy = pricingStrategy;

 initializeShowSeats();
 }

 int getId() {
 return id;
 }

 Movie* getMovie() {
 return movie;
 }

 Theatre* getTheatre() {
 return theatre;
 }

 Screen* getScreen() {
 return screen;
 }

 long long getStartTime() {
 return startTime;
 }

 vector<ShowSeat*>& getShowSeats() {
 return showSeats;
 }

 ShowSeat* getShowSeatById(
 int seatId
) {
 for (auto showSeat : showSeats) {

 if (
 showSeat
 ->getSeat()
 ->getId()
 == seatId
) {
 return showSeat;
 }
 }
 }

```

```

 }
}

return nullptr;
}

vector<ShowSeat*> getAvailableSeats() {

 vector<ShowSeat*> available;

 for (auto showSeat : showSeats) {

 if (showSeat->isAvailable()) {
 available.push_back(showSeat);
 }
 }

 return available;
}

private:

void initializeShowSeats() {

 for (auto seat :
 screen->getSeats()) {

 double price =
 pricingStrategy
 ->calculatePrice(
 seat,
 this
);

 showSeats.push_back(
 new ShowSeat(
 seat,
 price
)
);
 }
};

// =====
// BOOKING
// =====

class Booking {
 int id;

 User* user;

 Show* show;

 vector<ShowSeat*> seats;

 double amount;

 BookingStatus status;

```



```

public:
 Booking(
 int id,
 User* user,
 Show* show,
 vector<ShowSeat*> seats,
 double amount
) {
 this->id = id;
 this->user = user;
 this->show = show;
 this->seats = seats;
 this->amount = amount;

 status = PAYMENT_PENDING;
 }

 int getId() {
 return id;
 }

 User* getUser() {
 return user;
 }

 Show* getShow() {
 return show;
 }

 vector<ShowSeat*>& getSeats() {
 return seats;
 }

 double getAmount() {
 return amount;
 }

 BookingStatus getStatus() {
 return status;
 }

 void confirm() {
 status = CONFIRMED;
 }

 void cancel() {
 status = CANCELLED;
 }

 void fail() {
 status = FAILED;
 }
};

```

```

// =====
// PAYMENT STRATEGY
// =====

```

```

class PaymentStrategy {
public:

```

```

 virtual bool pay(
 double amount
) = 0;

 virtual ~PaymentStrategy() {}
};

// =====
// UPI PAYMENT
// =====

class UPIPayment
 : public PaymentStrategy {

public:

 bool pay(
 double amount
) override {

 cout << "Paid Rs."
 << amount
 << " using UPI\n";

 return true;
 }
};

// =====
// CARD PAYMENT
// =====

class CardPayment
 : public PaymentStrategy {

public:

 bool pay(
 double amount
) override {

 cout << "Paid Rs."
 << amount
 << " using Card\n";

 return true;
 }
};

// =====
// BOOKMYSHOW SYSTEM
// =====

class BookMyShowSystem {

 vector<Movie*> movies;

 vector<Theatre*> theatres;

 vector<Show*> shows;

```

```

vector<Booking*> bookings;

int nextBookingId;

public:

BookMyShowSystem() {
 nextBookingId = 1;
}

// -----
// ADD MOVIE
// -----

void addMovie(
 Movie* movie
) {
 movies.push_back(movie);
}

// -----
// ADD THEATRE
// -----

void addTheatre(
 Theatre* theatre
) {
 theatres.push_back(theatre);
}

// -----
// ADD SHOW
// -----

void addShow(
 Show* show
) {
 shows.push_back(show);
}

// -----
// SEARCH MOVIES BY CITY
// -----

vector<Movie*> searchMovies(
 string city
) {
 vector<Movie*> result;

 set<int> addedMovies;

 for (auto show : shows) {

 if (
 show
 ->getTheatre()
 ->getCity()
 != city
) {
 continue;

```

```

 }

 int movieId =
 show
 ->getMovie()
 ->getId();

 if (
 addedMovies.count(movieId)
) {
 continue;
 }

 addedMovies.insert(movieId);

 result.push_back(
 show->getMovie()
);
}

return result;
}

// -----
// GET SHOWS FOR MOVIE + CITY
// -----

vector<Show*> getShows(
 Movie* movie,
 string city
) {

 vector<Show*> result;

 for (auto show : shows) {

 if (
 show->getMovie()->getId()
 == movie->getId()
 &&
 show->getTheatre()->getCity()
 == city
) {
 result.push_back(show);
 }
 }

 return result;
}

// -----
// DISPLAY AVAILABLE SEATS
// -----

void showAvailableSeats(
 Show* show
) {

 cout << "\nAvailable seats:\n";

 for (

```

```

 auto showSeat :
 show->getAvailableSeats()
) {

 cout << showSeat
 ->getSeat()
 ->getSeatName()
 << " - Rs."
 << showSeat->getPrice()
 << "\n";

 }
}

// -----
// CREATE BOOKING
// -----

Booking* createBooking(
 User* user,
 Show* show,
 vector<int> seatIds
) {

 vector<ShowSeat*>
 selectedSeats;

 double totalAmount = 0;

 /*
 Step 1:
 Try locking all selected seats.
 */

 for (int seatId : seatIds) {

 ShowSeat* showSeat =
 show->getShowSeatById(
 seatId
);

 if (
 showSeat == nullptr
) {

 cout << "Invalid seat\n";

 releaseSeats(
 selectedSeats,
 user
);

 return nullptr;
 }

 bool locked =
 showSeat->lockSeat(
 user
);

 if (!locked) {

```

```

 cout << "Seat "
 << showSeat
 ->getSeat()
 ->getSeatName()
 << " unavailable\n";

 releaseSeats(
 selectedSeats,
 user
);

 return nullptr;
 }

 selectedSeats.push_back(
 showSeat
);

 totalAmount +=
 showSeat->getPrice();
}

/*
 Step 2:
 Create booking.
*/

Booking* booking =
 new Booking(
 nextBookingId++,
 user,
 show,
 selectedSeats,
 totalAmount
);

bookings.push_back(
 booking
);

cout << "\nBooking created\n";

cout << "Booking ID: "
 << booking->getId()
 << "\n";

cout << "Amount: Rs."
 << totalAmount
 << "\n";

return booking;
}

// -----
// PAYMENT
// -----

bool makePayment(
 Booking* booking,
 PaymentStrategy* paymentStrategy
) {

```

```

 if (
 booking == nullptr
) {
 return false;
 }

 if (
 booking->getStatus()
 != PAYMENT_PENDING
) {

 cout << "Booking is not awaiting payment\n";

 return false;
 }

 /*
 Verify that locks have
 not expired.
 */

 for (
 auto seat :
 booking->getSeats()
) {

 if (
 !seat->isLockedBy(
 booking->getUser()
)
) {

 cout << "Seat lock expired\n";

 booking->fail();

 releaseSeats(
 booking->getSeats(),
 booking->getUser()
);

 return false;
 }
 }

 bool paymentSuccess =
 paymentStrategy->pay(
 booking->getAmount()
);

 if (
 !paymentSuccess
) {

 booking->fail();

 releaseSeats(
 booking->getSeats(),
 booking->getUser()
);
 }

```

```

 return false;
 }

 /*
 Payment succeeded:

 LOCKED -> BOOKED
 */

 for (
 auto seat :
 booking->getSeats()
) {

 seat->bookSeat(
 booking->getUser()
);
 }

 booking->confirm();

 cout << "Booking confirmed\n";

 return true;
}

// -----
// CANCEL BOOKING
// -----

bool cancelBooking(
 Booking* booking
) {

 if (
 booking == nullptr
) {
 return false;
 }

 if (
 booking->getStatus()
 != CONFIRMED
) {

 cout << "Only confirmed booking can be cancelled\n";

 return false;
 }

 for (
 auto seat :
 booking->getSeats()
) {

 seat->makeAvailable();
 }

 booking->cancel();
}

```



```

 cout << "Booking cancelled\n";

 return true;
 }

 // -----
 // DISPLAY BOOKING
 // -----

void displayBooking(
 Booking* booking
) {

 if (
 booking == nullptr
) {
 return;
 }

 cout << "\n===== BOOKING =====\n";

 cout << "Booking ID: "
 << booking->getId()
 << "\n";

 cout << "User: "
 << booking
 << booking->getUser()
 << booking->getName()
 << "\n";

 cout << "Movie: "
 << booking
 << booking->getShow()
 << booking->getMovie()
 << booking->getTitle()
 << "\n";

 cout << "Theatre: "
 << booking
 << booking->getShow()
 << booking->getTheatre()
 << booking->getName()
 << "\n";

 cout << "Seats: ";

 for (
 auto seat :
 booking->getSeats()
) {

 cout << seat
 << booking->getSeat()
 << booking->getSeatName()
 << " ";
 }

 cout << "\nAmount: Rs."
 << booking->getAmount()
 << "\n";
}

```

```

 cout << "Status: ";

 if (
 booking->getStatus()
 == PAYMENT_PENDING
) {
 cout << "PAYMENT_PENDING";
 }

 else if (
 booking->getStatus()
 == CONFIRMED
) {
 cout << "CONFIRMED";
 }

 else if (
 booking->getStatus()
 == CANCELLED
) {
 cout << "CANCELLED";
 }

 else {
 cout << "FAILED";
 }

 cout << "\n";
 }

private:

 void releaseSeats(
 vector<ShowSeat*>& seats,
 User* user
) {

 for (
 auto seat :
 seats
) {

 if (
 seat->isLockedBy(user)
) {
 seat->releaseLock();
 }
 }
 }
};

// =====
// MAIN
// =====

int main() {

 // -----
 // USER
 // -----

```

```

User* user =
 new User(
 1,
 "Anirudh"
);

// -----
// MOVIE
// -----

Movie* movie =
 new Movie(
 101,
 "Interstellar",
 169,
 "English"
);

// -----
// THEATRE
// -----

Theatre* theatre =
 new Theatre(
 1,
 "PVR Civil Lines",
 "Prayagraj"
);

// -----
// SCREEN
// -----

Screen* screen =
 new Screen(
 1,
 "Screen 1"
);

screen->addSeat(
 new Seat(
 1,
 "A",
 1,
 REGULAR
)
);

screen->addSeat(
 new Seat(
 2,
 "A",
 2,
 REGULAR
)
);

screen->addSeat(
 new Seat(
 3,

```

```

 "B",
 1,
 PREMIUM
)
};

screen->addSeat(
 new Seat(
 4,
 "B",
 2,
 PREMIUM
)
);

screen->addSeat(
 new Seat(
 5,
 "C",
 1,
 RECLINER
)
);

theatre->addScreen(
 screen
);

// -----
// PRICING STRATEGY
// -----

PricingStrategy* pricing =
 new NormalPricingStrategy();

// -----
// SHOW
// -----

long long showTime =
 time(nullptr) + 3600;

Show* show =
 new Show(
 1,
 movie,
 theatre,
 screen,
 showTime,
 pricing
);

// -----
// SYSTEM
// -----

BookMyShowSystem system;

system.addMovie(movie);
system.addTheatre(theatre);
system.addShow(show);

```

```

// -----
// AVAILABLE SEATS
// -----

system.showAvailableSeats(
 show
);

// -----
// USER SELECTS A1 + A2
// -----

vector<int> selectedSeatIds = {
 1,
 2
};

Booking* booking =
 system.createBooking(
 user,
 show,
 selectedSeatIds
);

// -----
// PAYMENT STRATEGY
// -----

PaymentStrategy* upi =
 new UPIPayment();

system.makePayment(
 booking,
 upi
);

// -----
// DISPLAY BOOKING
// -----

system.displayBooking(
 booking
);

// -----
// SHOW AVAILABLE SEATS AFTER BOOKING
// -----

system.showAvailableSeats(
 show
);

// -----
// CANCEL BOOKING
// -----

system.cancelBooking(
 booking
);

```

```
// -----
// AVAILABLE SEATS AGAIN
// -----

system.showAvailableSeats(
 show
);

return 0;
}
```

The main interview logic to remember is:

```
Seat
= physical seat

ShowSeat
= that seat for one specific show
= status + price + lock info
```

and:

```
AVAILABLE
↓
lockSeat()
↓
LOCKED
↓
payment success
↓
BOOKED
```

If payment fails or the lock expires:

```
LOCKED → AVAILABLE
```

The two patterns we kept are justified:

```
PricingStrategy
→ because pricing can vary

PaymentStrategy
→ because payment methods can vary
```

And the single most important design decision is:

*Do not put `booked` directly inside `Seat` ; seat status belongs to `ShowSeat` because the same physical seat can be booked for one show and available for another.*

15/09/2026, 02:55:24



for parking lot get all the entities, class diagram functionality and tradeoffs do not give the whole code and tell where solid is used ?

15/09/2026, 09:14:43



For a fresher interview, this is exactly what you should cover for **Parking Lot LLD** without jumping into full code.

## 1. First explain the scope

Start with something like:

*"I'll assume the parking lot has multiple floors, supports different vehicle and spot types, allocates a compatible parking spot, generates a ticket, calculates parking charges on exit, accepts payment, and frees the spot. I'll keep entry/exit gates out unless specifically required."*

That keeps the design simple and matches the version we settled on.

## 2. Main entities

The core entities are:

```
Vehicle
ParkingSpot
ParkingFloor
Ticket
ParkingLot
```

Then behavior that can vary:

```
ParkingSpotStrategy
PricingStrategy
PaymentStrategy
```

That is enough.

## 3. Vehicle

Represents the actual incoming vehicle.

```
class Vehicle {
 string number;
 VehicleType type;
};
```

Example:

```
number = "UP70AB1234"
type = CAR
```

Use enum:

```
enum VehicleType {
 BIKE,
 CAR,
 TRUCK
};
```

Why enum instead of subclasses?

Because if Bike/Car/Truck differ only by category and not by meaningful behavior, subclasses would be unnecessary.

---

#### 4. `ParkingSpot`

Represents one physical parking slot.

```
class ParkingSpot {
 int id;
 SpotType type;
 Vehicle* vehicle;
};
```

Example:

```
Spot 102
CAR_SPOT
currently holding car UP70AB1234
```

Important methods:

```
bool isAvailable();
bool canPark(Vehicle* vehicle);
bool parkVehicle(Vehicle* vehicle);
void removeVehicle();
```

Important design choice:

Do **not** keep both:

```
bool occupied;
Vehicle* vehicle;
```

because that duplicates state.

Prefer:

```
bool isAvailable() {
 return vehicle == nullptr;
}
```

So the actual source of truth is `vehicle`.

---

#### 5. `ParkingFloor`

Represents one floor.

```
class ParkingFloor {
 int floorNumber;
 vector<ParkingSpot*> spots;
};
```

Responsibilities:

```
void addSpot(ParkingSpot* spot);
vector<ParkingSpot*>& getSpots();
void displayAvailability();
```

You could also put:



```
ParkingSpot* findFreeSpot(Vehicle* vehicle);
```

inside `ParkingFloor`, but because we want multiple spot-allocation algorithms, we move selection logic to `ParkingSpotStrategy`.

## 6. Ticket

Represents one parking session.

```
class Ticket {
 int id;

 Vehicle* vehicle;
 ParkingSpot* spot;
 ParkingFloor* floor;

 long long entryTime;

 TicketStatus status;
};
```

Example:

```
Ticket 501

Vehicle = UP70AB1234
Floor = 1
Spot = 102
Entry time = 10:00 AM
Status = ACTIVE
```

Why does Ticket need the spot?

Because at exit we need to know exactly which spot to free.

Why entry time?

Because pricing may depend on duration.

## 7. ParkingLot

This is the coordinator.

```
class ParkingLot {
 vector<ParkingFloor*> floors;
 vector<Ticket*> tickets;

 ParkingSpotStrategy* spotStrategy;
 PricingStrategy* pricingStrategy;
};
```

Main responsibilities:

```
Ticket* parkVehicle(Vehicle* vehicle);

bool unparkVehicle(
 Ticket* ticket,
 PaymentStrategy* paymentStrategy
);

void addFloor(ParkingFloor* floor);
```

```
void displayAvailability();
```

Important:

`ParkingLot` coordinates the flow, but it should not contain every algorithm itself.

For example, it should not hardcode:

```
if (spotRule == FIRST_AVAILABLE) ...
else if (spotRule == LOWEST_FLOOR) ...
```

Instead:

```
spotStrategy->findSpot(...)
```

Same for pricing/payment.

---

## 8. Parking spot allocation strategy

This answers:

*"Which spot should I assign?"*

Base:

```
class ParkingSpotStrategy {
public:
 virtual pair<ParkingFloor*, ParkingSpot*>
 findSpot(
 vector<ParkingFloor*>& floors,
 Vehicle* vehicle
) = 0;
};
```

Possible implementations:

```
FirstAvailableSpotStrategy
LowestFloorSpotStrategy
NearestSpotStrategy
```

For example:

```
Vehicle = CAR

Floor 1:
101 bike
102 occupied car
103 free car

Floor 2:
201 free car
```

`FirstAvailable` might choose:

```
Floor 1, Spot 103
```

---

## 9. Pricing strategy

This answers:

*"How much should the customer pay?"*

Base:

```
class PricingStrategy {
public:
 virtual double calculateFee(
 Ticket* ticket,
 long long exitTime
) = 0;
};
```

Possible implementations:

```
HourlyPricingStrategy
FlatPricingStrategy
WeekendPricingStrategy
```

Example hourly pricing:

```
Bike = ₹10/hour
Car = ₹20/hour
Truck = ₹30/hour
```

## 10. Payment strategy

This answers:

*"How is payment performed?"*

```
class PaymentStrategy {
public:
 virtual bool pay(double amount) = 0;
};
```

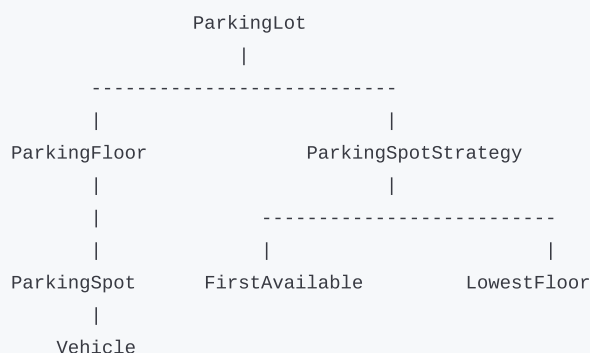
Implementations:

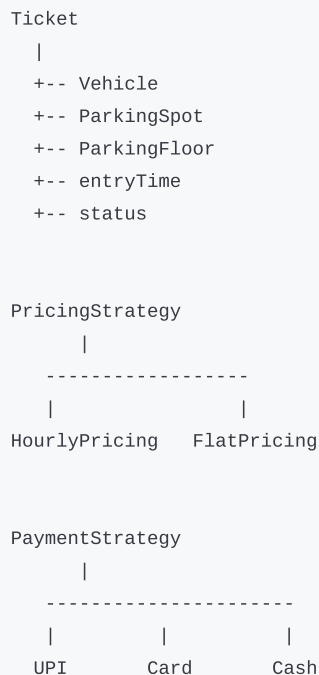
```
UPIPayment
CardPayment
CashPayment
```

Again, same action, different implementation → Strategy Pattern.

## 11. Class diagram

This is a good interview-level diagram:





Relationship summary:

ParkingLot has many ParkingFloors

ParkingFloor has many ParkingSpots

ParkingSpot may contain one Vehicle

Ticket connects:  
Vehicle + Spot + Floor + EntryTime

## 12. Main functionality / flow

### Vehicle entry

```
Vehicle enters
↓
ParkingLot::parkVehicle()
↓
spotStrategy->findSpot()
↓
find compatible available spot
↓
spot->parkVehicle(vehicle)
↓
create Ticket
↓
return Ticket
```

Example:

```
Car arrives
↓
FirstAvailableStrategy
↓
Floor 1 Spot 103
```

```
↓
park car
↓
Ticket #10 created
```

---

## Vehicle exit

```
Ticket submitted
↓
ParkingLot::unparkVehicle()
↓
pricingStrategy->calculateFee()
↓
paymentStrategy->pay()
↓
spot->removeVehicle()
↓
ticket->close()
```

If payment fails:

```
Do NOT:
- free the spot
- close the ticket
```

That is an important rule.

---

## 13. Where SOLID is used

### S — Single Responsibility Principle

Each class has one main job.

```
Vehicle
→ vehicle information

ParkingSpot
→ state of one parking spot

ParkingFloor
→ manages spots on one floor

Ticket
→ represents one parking session

ParkingSpotStrategy
→ spot selection

PricingStrategy
→ fee calculation

PaymentStrategy
→ payment behavior

ParkingLot
→ coordinates the workflow
```

This is the strongest SOLID principle visible here.

## O — Open/Closed Principle

Suppose tomorrow interviewer says:

*"Add nearest-to-exit parking."*

You can create:

```
class NearestSpotStrategy
 : public ParkingSpotStrategy {
};
```

without changing `ParkingLot`.

Similarly:

```
class WeekendPricingStrategy
 : public PricingStrategy {
};
```

or:

```
class WalletPayment
 : public PaymentStrategy {
};
```

So the system is:

*open for extension, closed for modification.*

---

## L — Liskov Substitution Principle

Wherever the system expects:

```
PricingStrategy*
```

you can pass:

```
HourlyPricingStrategy*
FlatPricingStrategy*
```

and the program should still behave correctly.

Same with:

```
PaymentStrategy*
```

and:

```
UPIPayment
CardPayment
CashPayment
```

---

## I — Interface Segregation Principle

Our strategy interfaces are small:

```
class PaymentStrategy {
 virtual bool pay(double amount) = 0;
};
```

We don't force a payment strategy to implement unrelated things like:

```
findParkingSpot()
calculatePricing()
```

Each abstraction contains only relevant behavior.

---

## D — Dependency Inversion Principle

`ParkingLot` depends on abstractions:

```
ParkingSpotStrategy*
PricingStrategy*
```

not on:

```
FirstAvailableSpotStrategy
HourlyPricingStrategy
```

Likewise `unparkVehicle()` accepts:

```
PaymentStrategy*
```

instead of directly creating:

```
UPIPayment
```

That is DIP.

---

## 14. Tradeoffs you should mention

This is where you can sound much stronger in an interview.

### Tradeoff 1: `vector` of spots vs optimized structure

Simple design:

```
vector<ParkingSpot*> spots;
```

Finding a free spot may scan many spots:

```
O(total spots)
```

Good enough for fresher LLD.

For a huge parking lot, optimize with:

```
set/free-list/priority queue
```

per spot type.

For example:

```
freeCarSpots
freeBikeSpots
freeTruckSpots
```

Then allocation can be faster.

---

## 15. Tradeoff 2: exact-type parking

Current simple rule:

```
Bike → BIKE_SPOT
Car → CAR_SPOT
Truck → TRUCK_SPOT
```

This wastes space sometimes.

Maybe a bike could park in a car spot.

Then compatibility could become:

```
Bike → Bike/Car/Truck
Car → Car/Truck
Truck → Truck
```

But that introduces allocation tradeoffs:

*Should I use a large spot for a small vehicle or save it for larger vehicles?*

For basic design, exact matching is simpler.

---

## 16. Tradeoff 3: strategy objects

Why use strategies?

Benefit:

```
easy extensibility
less if/else
clean OCP
```

Cost:

```
more classes
more indirection
```

For `Payment`, `Pricing`, `SpotAllocation`, the flexibility justifies it.

But don't create strategies for trivial behavior that never varies.

---

## 17. Tradeoff 4: storing all tickets in vector

Simple:

```
vector<Ticket*> tickets;
```

Lookup:

```
O(number of tickets)
```

For a real system:

```
unordered_map<int, Ticket*> tickets;
```

gives roughly:

```
O(1)
```

lookup by ticket ID.

For interview first-pass, vector is fine unless lookup performance is emphasized.



---

## 18. Tradeoff 5: concurrency

Suppose one free spot remains.

Two cars enter simultaneously:

```
Car A checks spot 103 → free
Car B checks spot 103 → free
```

Both could try to park there.

In a real system:

*spot selection + marking it occupied must be atomic.*

You'd use:

```
lock / transaction / compare-and-set
```

But for basic in-memory LLD, mention this rather than implementing complex concurrency.

---

## 19. Tradeoff 6: `Vehicle* vehicle` **vs** `bool occupied`

We chose:

```
Vehicle* vehicle;
```

and:

```
bool isAvailable() {
 return vehicle == nullptr;
}
```

instead of:

```
bool occupied;
Vehicle* vehicle;
```

Benefit:

*no duplicate state.*

Otherwise this invalid situation could happen:

```
occupied = false
vehicle = Car123
```

So deriving availability from `vehicle` is cleaner.

---

## 20. What you should say at the end of the interview

A strong summary would be:

*"The main design keeps physical entities simple: `ParkingLot` → `ParkingFloor` → `ParkingSpot`. `Ticket` represents the parking session. The parking lot coordinates entry and exit, while spot allocation, pricing, and payment are separated using Strategy Pattern because those algorithms can vary. The initial implementation uses vectors and linear scans for simplicity; for a larger system I could optimize free-spot lookup and ticket lookup and add concurrency protection around spot allocation."*

That is about the right level for a fresher: simple core model, justified patterns, clear tradeoffs, no unnecessary manager/service classes.

15/09/2026, 09:14:42



do the same for atm design as in prev example

15/09/2026, 09:15:46



For a fresher interview, **ATM LLD** should be kept simple and structured. The most important things are: card/account validation, PIN authentication, transaction selection, cash withdrawal flow, balance update, and ATM cash inventory.

## 1. First explain the scope

Start with:

*"I'll design a simple ATM that allows a user to insert a card, authenticate using PIN, check balance, withdraw cash, and deposit cash. The ATM also maintains its own cash inventory. I'll keep networking with bank servers abstracted behind a banking service."*

That is enough for scope.

Main flow:

```
Insert card
↓
Authenticate PIN
↓
Choose transaction
↓
Check balance / Withdraw / Deposit
↓
Update account
↓
Eject card
```

## 2. Main entities

Core entities:

```
ATM
Card
Account
User
BankService
Transaction
```

Useful transaction types:

```
WithdrawalTransaction
DepositTransaction
BalanceInquiryTransaction
```

And one more important supporting entity:

```
CashDispenser
```

because cash management is a real ATM-specific responsibility.

I would not add 15 manager classes.

## 3. User

Represents the bank customer.

```
class User {
 int id;
 string name;
};
```

Simple responsibility:

User → customer identity

## 4. Card

Represents the physical ATM/debit card.

```
class Card {
 string cardNumber;
 int pin;
 int accountId;
};
```

You can say:

*"The card identifies the account and is used during authentication."*

In real systems, PIN would not be stored like plain `int`, but for LLD we can simplify.

## 5. Account

Represents the bank account.

```
class Account {
 int id;
 double balance;
};
```

Important methods:

```
double getBalance();

bool debit(double amount);

void credit(double amount);
```

Example:

```
Account balance = ₹10,000

Withdraw ₹2,000
→ debit(2000)
→ new balance = ₹8,000
```

## 6. BankService

This is useful because ATM should not directly own all bank accounts and authentication logic.

```
class BankService {
 vector<Account*> accounts;
```

```

public:
 bool authenticate(Card* card, int enteredPin);

 Account* getAccount(Card* card);

 bool withdraw(Account* account, double amount);

 void deposit(Account* account, double amount);
};

```

Responsibility:

`BankService` represents communication with the bank/backend.

This is a justified abstraction, not overengineering.

## 7. ATM

This is the coordinator.

```

class ATM {
 Card* currentCard;

 BankService* bankService;

 CashDispenser* cashDispenser;

 ATMState state;
};

```

Responsibilities:

```

accept card
authenticate user
select transaction
coordinate bank service
coordinate cash dispenser
eject card

```

ATM should not itself contain all denomination logic.

## 8. ATM state

ATM naturally has states:

```

IDLE
CARD_INSERTED
AUTHENTICATED
PROCESSING

```

You can start with an enum:

```

enum ATMState {
 IDLE,
 CARD_INSERTED,
 AUTHENTICATED
};

```

For a fresher interview, enum is enough initially.

If interviewer asks to handle many state-specific behaviors cleanly, then you can mention **State Pattern**.

Example:

```
IdleState
CardInsertedState
AuthenticatedState
```

But don't force State Pattern unless needed.

## 9. Transaction

A transaction base class is justified because different transactions share a common idea but have different behavior.

```
class Transaction {
protected:
 int id;
 Account* account;

public:
 virtual void execute() = 0;
};
```

Then:

```
class WithdrawalTransaction : public Transaction {};
class DepositTransaction : public Transaction {};
class BalanceInquiryTransaction : public Transaction {};
```

This is a good use of polymorphism.

## 10. Withdrawal transaction

This is the most important flow.

Suppose:

```
Account balance = ₹10,000
ATM has ₹5,000
User requests ₹2,000
```

Steps:

1. Validate amount
2. Check account balance
3. Check ATM has enough cash
4. Check denomination combination possible
5. Debit account
6. Dispense cash
7. Complete transaction

Very important:

*Both account balance and ATM cash must be sufficient.*

## 11. CashDispenser

ATM has actual notes.

For example:

```
class CashDispenser {
 map<int, int> cash;

 // denomination -> count
};
```

Example:

```
₹500 → 10 notes
₹200 → 20 notes
₹100 → 30 notes
```

Total ATM cash:

```
500*10 + 200*20 + 100*30
```

Methods:

```
bool canDispense(int amount);

map<int,int> dispense(int amount);

int getTotalCash();
```

## 12. How to dispense notes

For a simple interview version, use greedy with descending denominations.

Suppose:

```
Amount = ₹1800

Available:
₹500
₹200
₹100
```

Then:

```
₹500 × 3 = 1500
₹200 × 1 = 200
₹100 × 1 = 100

Total = 1800
```

Pseudo:

```
for denomination from largest to smallest:
 take = min(
 amount / denomination,
 availableCount
);

 amount -= take * denomination;
```

If at end:

```
amount != 0
```

then ATM cannot dispense exact cash.

Important interview caveat:

*Greedy works for common ATM denomination systems, but for arbitrary denominations it may not always find a valid combination.*

That is a good tradeoff point.

## 13. Deposit transaction

Simple version:

```
User deposits ₹3000
↓
validate
↓
account.credit(3000)
```

If interviewer asks for cash-deposit ATM, then ATM inventory can also increase based on deposited notes.

For basic design, you can keep deposit abstract.

## 14. Balance inquiry

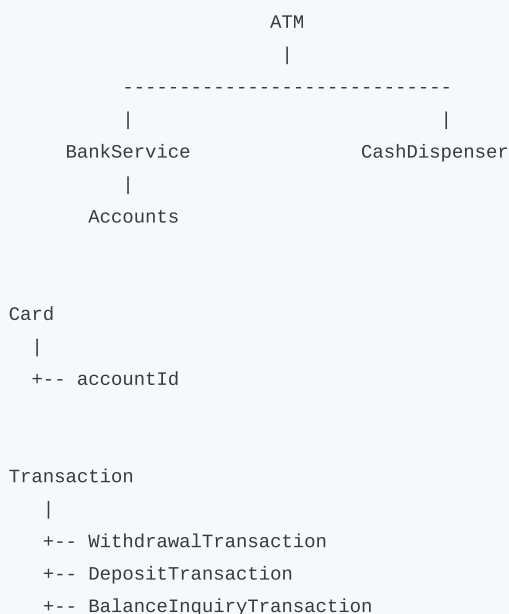
Very simple:

```
cout << account->getBalance();
```

This is why a separate `BalanceInquiryTransaction` can be useful.

## 15. Class diagram

A clean fresher-level diagram:



Relationships:

ATM uses BankService

ATM uses CashDispenser



Card identifies Account

Transaction operates on Account

## 16. Main functionality

### Insert card

```
ATM::insertCard(card)
 ↓
currentCard = card
 ↓
state = CARD_INSERTED
```

### Authenticate

```
ATM::authenticate(pin)
 ↓
BankService::authenticate()
 ↓
if valid
state = AUTHENTICATED
```

### Withdraw

```
ATM::withdraw(2000)
 ↓
get Account
 ↓
check balance
 ↓
cashDispenser.canDispense(2000)
 ↓
account.debit(2000)
 ↓
cashDispenser.dispense(2000)
```

### Deposit

```
ATM::deposit(3000)
 ↓
account.credit(3000)
```

### Balance inquiry

```
ATM::checkBalance()
 ↓
account.getBalance()
```

## 17. Where SOLID is used

### S — Single Responsibility

```
Card
→ card information
```

```
Account
→ balance and account operations

BankService
→ authentication + bank interaction

CashDispenser
→ ATM cash inventory

Transaction
→ transaction behavior

ATM
→ coordinates user interaction
```

That is clean SRP.

---

## O — Open/Closed

Suppose tomorrow we add:

```
MiniStatementTransaction
FundTransferTransaction
```

We can derive from:

```
Transaction
```

without rewriting existing transaction classes.

---

## L — Liskov

Anywhere we expect:

```
Transaction*
```

we can use:

```
WithdrawalTransaction
DepositTransaction
BalanceInquiryTransaction
```

without breaking behavior.

---

## I — Interface Segregation

Transaction abstractions should stay small.

Don't make every transaction implement irrelevant functions like:

```
dispenseCash()
acceptCash()
printBalance()
```

Instead, each concrete transaction implements only its behavior.

---

## D — Dependency Inversion

ATM can depend on abstractions/services like:

```
BankService*
CashDispenser*
```

rather than directly embedding all banking logic.

If you introduce:

```
PaymentProcessor / BankingInterface
```

later, ATM can depend on that abstraction.

---

## 18. Tradeoffs

### Tradeoff 1: enum state vs State Pattern

Simple:

```
ATMState state;
```

Benefit:

```
easy
less code
good for fresher interview
```

Problem:

```
many if/else checks as ATM grows
```

If behaviors differ heavily by state, then State Pattern is cleaner.

---

### Tradeoff 2: transaction subclasses vs enum + switch

You could do:

```
enum TransactionType {
 WITHDRAW,
 DEPOSIT,
 BALANCE
};
```

and then:

```
switch(type)
```

Simpler, but harder to extend.

Subclassing:

```
WithdrawalTransaction
DepositTransaction
BalanceInquiryTransaction
```

is better if transaction behavior is substantial.

---

### Tradeoff 3: greedy cash dispensing

Benefit:

```
simple
fast
```

```
natural for standard ATM denominations
```

Problem:

For arbitrary denominations, greedy may fail even if a valid combination exists.

Then you may need DP/backtracking.

---

### Tradeoff 4: debit first or dispense first?

Very important.

Bad sequence:

```
debit account
↓
ATM fails to dispense
```

Now user lost money.

So in production:

```
validate both first
↓
reserve cash
↓
perform bank debit atomically
↓
dispense
```

or have rollback/reconciliation.

For interview:

*"Withdrawal should be treated atomically from a business perspective."*

---

### Tradeoff 5: storing PIN in Card

Simple LLD:

```
int pin;
```

Real system:

```
PIN validation happens securely on bank backend,
not stored as plain text on card.
```

Mention this if interviewer pushes security.

---

## 19. Concurrency issue

Suppose ATM has only ₹2000 left.

Two simultaneous withdrawal requests for ₹2000 should not both succeed.

The check:

```
canDispense()
```

and reservation/removal of ATM cash must be atomic.

Same principle as Parking Lot's last spot or BookMyShow's last seat.

## 20. What you should say at the end

A strong summary:

*"The ATM coordinates card authentication and transactions. `BankService` handles account-level banking operations, while `CashDispenser` manages physical notes. Transactions are modeled separately because withdrawal, deposit, and balance inquiry have different behavior. I'd start with an enum for ATM state and only introduce State Pattern if state-dependent behavior becomes complex. The main correctness concern is making withdrawal atomic across account debit and physical cash dispensing."*

That is exactly the right fresher-level depth: simple model, clear responsibilities, justified polymorphism, and awareness of consistency/security/concurrency.

15/09/2026, 09:15:46



now full code for atm ?

15/09/2026, 10:00:47



Yes. I'll keep it at the same level as our Parking Lot / Car Rental versions: complete enough for an interview, but not overengineered.

```
#include <bits/stdc++.h>
using namespace std;

// =====
// ENUMS
// =====

enum ATMState {
 IDLE,
 CARD_INSERTED,
 AUTHENTICATED
};

enum TransactionType {
 WITHDRAWAL,
 DEPOSIT,
 BALANCE_INQUIRY
};

// =====
// USER
// =====

class User {
 int id;
 string name;

public:
 User(int id, string name) {
 this->id = id;
 this->name = name;
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }
};

// =====
// ACCOUNT
// =====

class Account {
 int id;
 double balance;

public:
```

```

Account(int id, double balance) {
 this->id = id;
 this->balance = balance;
}

int getId() {
 return id;
}

double getBalance() {
 return balance;
}

bool debit(double amount) {

 if (amount <= 0)
 return false;

 if (balance < amount)
 return false;

 balance -= amount;

 return true;
}

bool credit(double amount) {

 if (amount <= 0)
 return false;

 balance += amount;

 return true;
}
};

// =====
// CARD
// =====

class Card {
 string cardNumber;
 int pin;
 int accountId;

public:
 Card(
 string cardNumber,
 int pin,
 int accountId
) {
 this->cardNumber = cardNumber;
 this->pin = pin;
 this->accountId = accountId;
 }

 string getCardNumber() {
 return cardNumber;
 }
}

```

```

 int getPin() {
 return pin;
 }

 int getAccountId() {
 return accountId;
 }
};

// =====
// BANK SERVICE
// =====

class BankService {
 vector<Account*> accounts;

public:

 void addAccount(Account* account) {
 accounts.push_back(account);
 }

 Account* getAccountById(int id) {

 for (auto account : accounts) {

 if (account->getId() == id)
 return account;
 }

 return nullptr;
 }

 bool authenticate(
 Card* card,
 int enteredPin
) {

 if (card == nullptr)
 return false;

 return card->getPin() == enteredPin;
 }

 Account* getAccount(
 Card* card
) {

 if (card == nullptr)
 return nullptr;

 return getAccountById(
 card->getAccountId()
);
 }

 bool withdraw(
 Account* account,
 double amount
) {

```



```

 if (account == nullptr)
 return false;

 return account->debit(amount);
 }

 bool deposit(
 Account* account,
 double amount
) {

 if (account == nullptr)
 return false;

 return account->credit(amount);
 }
};

// =====
// CASH DISPENSER
// =====

class CashDispenser {

 // denomination -> number of notes
 map<int, int, greater<int>> cash;

public:

 void addCash(
 int denomination,
 int count
) {

 cash[denomination] += count;
 }

 int getTotalCash() {

 int total = 0;

 for (auto &entry : cash) {

 total +=
 entry.first *
 entry.second;
 }

 return total;
 }

 // -----
 // Check whether exact amount can be dispensed
 // -----

 bool canDispense(int amount) {

 if (amount <= 0)
 return false;

 if (getTotalCash() < amount)

```

```

 return false;

 int remaining = amount;

 for (auto &entry : cash) {

 int denomination =
 entry.first;

 int availableCount =
 entry.second;

 int needed =
 remaining /
 denomination;

 int take =
 min(
 needed,
 availableCount
);

 remaining -=
 take * denomination;
 }

 return remaining == 0;
}

// -----
// Dispense cash
// -----

bool dispense(int amount) {

 if (!canDispense(amount))
 return false;

 int remaining = amount;

 map<int, int, greater<int>>
 notesToDispense;

 /*
 First calculate notes.
 Don't modify actual inventory yet.
 */

 for (auto &entry : cash) {

 int denomination =
 entry.first;

 int availableCount =
 entry.second;

 int needed =
 remaining /
 denomination;

 int take =

```

```

 min(
 needed,
 availableCount
);

 if (take > 0) {

 notesToDispense[
 denomination
] = take;

 remaining -=
 take * denomination;
 }
 }

 if (remaining != 0)
 return false;

 /*
 Now actually remove notes.
 */

 for (
 auto &entry :
 notesToDispense
) {

 cash[entry.first]
 -= entry.second;
 }

 cout << "\nCash dispensed:\n";

 for (
 auto &entry :
 notesToDispense
) {

 cout << "Rs."
 << entry.first
 << " x "
 << entry.second
 << "\n";
 }

 return true;
}

void showCashInventory() {

 cout << "\n==== ATM CASH =====\n";

 for (auto &entry : cash) {

 cout << "Rs."
 << entry.first
 << " -> "
 << entry.second
 << " notes\n";
 }
}

```

```

 cout << "Total cash: Rs."
 << getTotalCash()
 << "\n";
 }
};

// =====
// TRANSACTION BASE CLASS
// =====

class Transaction {
protected:
 int id;
 Account* account;

public:
 Transaction(
 int id,
 Account* account
) {
 this->id = id;
 this->account = account;
 }

 virtual bool execute() = 0;

 virtual ~Transaction() {}
};

// =====
// BALANCE INQUIRY TRANSACTION
// =====

class BalanceInquiryTransaction
 : public Transaction {

public:

 BalanceInquiryTransaction(
 int id,
 Account* account
)
 : Transaction(
 id,
 account
) {}

 bool execute() override {

 if (account == nullptr)
 return false;

 cout << "Current balance: Rs."
 << account->getBalance()
 << "\n";

 return true;
 }
};

```

```

// =====
// DEPOSIT TRANSACTION
// =====

class DepositTransaction
: public Transaction {

 double amount;
 BankService* bankService;

public:

 DepositTransaction(
 int id,
 Account* account,
 double amount,
 BankService* bankService
)
 : Transaction(
 id,
 account
) {

 this->amount = amount;
 this->bankService =
 bankService;
 }

 bool execute() override {

 if (amount <= 0)
 return false;

 bool success =
 bankService->deposit(
 account,
 amount
);

 if (!success)
 return false;

 cout << "Deposited Rs."
 << amount
 << "\n";

 cout << "New balance: Rs."
 << account->getBalance()
 << "\n";

 return true;
 }
};

// =====
// WITHDRAWAL TRANSACTION
// =====

class WithdrawalTransaction
: public Transaction {

```

```

int amount;

BankService* bankService;

CashDispenser* cashDispenser;

public:

WithdrawalTransaction(
 int id,
 Account* account,
 int amount,
 BankService* bankService,
 CashDispenser* cashDispenser
)
: Transaction(
 id,
 account
) {

 this->amount = amount;

 this->bankService =
 bankService;

 this->cashDispenser =
 cashDispenser;
}

bool execute() override {

 if (amount <= 0) {

 cout << "Invalid amount\n";
 return false;
 }

 // -----
 // Check account balance
 // -----

 if (
 account->getBalance()
 < amount
) {

 cout << "Insufficient account balance\n";

 return false;
 }

 // -----
 // Check ATM cash
 // -----

 if (
 !cashDispenser
 ->canDispense(amount)
) {

 cout << "ATM cannot dispense requested amount\n";
 }
}

```

```

 return false;
 }

 // -----
 // Debit account
 // -----

 bool debitSuccess =
 bankService->withdraw(
 account,
 amount
);

 if (!debitSuccess) {

 cout << "Debit failed\n";

 return false;
 }

 // -----
 // Dispense physical cash
 // -----

 bool dispenseSuccess =
 cashDispenser->dispense(
 amount
);

 /*
 In production:
 if cash dispensing fails after debit,
 rollback/reconciliation is required.
 */

 if (!dispenseSuccess) {

 cout << "Cash dispensing failed\n";

 // simple rollback for demo
 account->credit(amount);

 return false;
 }

 cout << "Withdrawal successful\n";

 cout << "Remaining balance: Rs."
 << account->getBalance()
 << "\n";

 return true;
}
};

// =====
// ATM
// =====

class ATM {

```

```

 ATMState state;

 Card* currentCard;

 Account* currentAccount;

 BankService* bankService;

 CashDispenser* cashDispenser;

 int nextTransactionId;

public:

 ATM(
 BankService* bankService,
 CashDispenser* cashDispenser
) {

 this->bankService =
 bankService;

 this->cashDispenser =
 cashDispenser;

 this->state = IDLE;

 this->currentCard =
 nullptr;

 this->currentAccount =
 nullptr;

 nextTransactionId = 1;
 }

 // -----
 // INSERT CARD
 // -----

 bool insertCard(
 Card* card
) {

 if (state != IDLE) {

 cout << "ATM is already in use\n";

 return false;
 }

 currentCard = card;

 state = CARD_INSERTED;

 cout << "Card inserted\n";

 return true;
 }

```



```

// -----
// AUTHENTICATE
// -----

bool authenticate(
 int pin
) {

 if (
 state != CARD_INSERTED
) {

 cout << "Insert card first\n";

 return false;
 }

 bool valid =
 bankService->authenticate(
 currentCard,
 pin
);

 if (!valid) {

 cout << "Invalid PIN\n";

 return false;
 }

 currentAccount =
 bankService->getAccount(
 currentCard
);

 if (
 currentAccount == nullptr
) {

 cout << "Account not found\n";

 return false;
 }

 state = AUTHENTICATED;

 cout << "Authentication successful\n";

 return true;
}

// -----
// CHECK BALANCE
// -----

bool checkBalance() {

 if (
 state != AUTHENTICATED
) {

```

```

 cout << "Authentication required\n";

 return false;
 }

 BalanceInquiryTransaction
 transaction(
 nextTransactionId++,
 currentAccount
);

 return transaction.execute();
}

// -----
// WITHDRAW
// -----

bool withdraw(
 int amount
) {

 if (
 state != AUTHENTICATED
) {

 cout << "Authentication required\n";

 return false;
 }

 WithdrawalTransaction
 transaction(
 nextTransactionId++,
 currentAccount,
 amount,
 bankService,
 cashDispenser
);

 return transaction.execute();
}

// -----
// DEPOSIT
// -----

bool deposit(
 double amount
) {

 if (
 state != AUTHENTICATED
) {

 cout << "Authentication required\n";

 return false;
 }

 DepositTransaction

```

```

 transaction(
 nextTransactionId++,
 currentAccount,
 amount,
 bankService
);

 return transaction.execute();
 }

 // -----
 // EJECT CARD
 // -----

 void ejectCard() {

 if (state == IDLE) {

 cout << "No card inserted\n";
 return;
 }

 currentCard = nullptr;
 currentAccount = nullptr;

 state = IDLE;

 cout << "Card ejected\n";
 }

 ATMState getState() {
 return state;
 }
};

// =====
// MAIN
// =====

int main() {

 // -----
 // CREATE ACCOUNT
 // -----

 Account* account =
 new Account(
 101,
 10000
);

 // -----
 // CREATE CARD
 // -----

 Card* card =
 new Card(
 "1234-5678-9012",
 1234,
 101
);

```

```

// -----
// BANK SERVICE
// -----

BankService* bankService =
 new BankService();

bankService->addAccount(
 account
);

// -----
// CASH DISPENSER
// -----

CashDispenser* dispenser =
 new CashDispenser();

dispenser->addCash(
 500,
 10
);

dispenser->addCash(
 200,
 10
);

dispenser->addCash(
 100,
 20
);

dispenser->showCashInventory();

// -----
// ATM
// -----

ATM atm(
 bankService,
 dispenser
);

// -----
// INSERT CARD
// -----

atm.insertCard(
 card
);

// -----
// AUTHENTICATE
// -----

atm.authenticate(
 1234
);

```

```

// -----
// CHECK BALANCE
// -----

atm.checkBalance();

// -----
// WITHDRAW
// -----

atm.withdraw(
 1800
);

// -----
// CHECK BALANCE AGAIN
// -----

atm.checkBalance();

// -----
// DEPOSIT
// -----

atm.deposit(
 1000
);

// -----
// CHECK BALANCE AGAIN
// -----

atm.checkBalance();

// -----
// ATM CASH AFTER WITHDRAWAL
// -----

dispenser->showCashInventory();

// -----
// EJECT CARD
// -----

atm.ejectCard();

return 0;
}

```

The core architecture is:

```

ATM
|
+-- BankService
| |
| +-- Accounts
|
+-- CashDispenser
|
+-- current Card
+-- current Account

```

```
Transaction
|
+-- WithdrawalTransaction
+-- DepositTransaction
+-- BalanceInquiryTransaction
```

The most important withdrawal flow is:

```
withdraw(amount)
 ↓
check account balance
 ↓
check ATM can dispense exact amount
 ↓
debit account
 ↓
dispense cash
 ↓
success
```

And the most important design separation is:

```
Account
→ logical money in bank

CashDispenser
→ physical notes inside ATM

ATM
→ coordinates everything
```

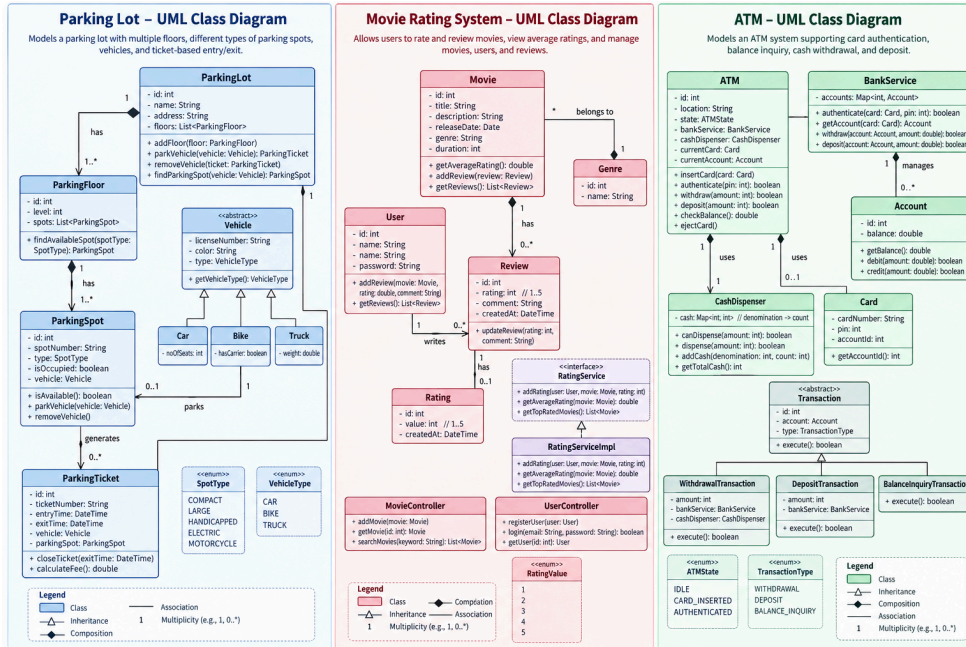
For a fresher interview, this is enough. I would not introduce a full State Pattern unless the interviewer specifically asks you to clean up state-specific `if/else` behavior.

15/09/2026, 10:00:48



## Uml diagrams for Parking lot uml, movie rating system and atm

15/09/2026, 11:42:33



15/09/2026, 11:42:32



For the **Vending Machine LLD**, this is the clean C++ version we discussed earlier using the **State Pattern**.

Key entities:

```
VendingMachine
Inventory
Item
Payment

VendingState
|
+-- IdleState
+-- PaymentState
+-- DispenseState
```

```
#include <bits/stdc++.h>
using namespace std;

// =====
// ITEM
// =====

class Item {
 int id;
 string name;
 int price;
 int quantity;

public:
 Item(int id, string name, int price, int quantity) {
 this->id = id;
 this->name = name;
 this->price = price;
 this->quantity = quantity;
 }

 int getId() {
 return id;
 }

 string getName() {
 return name;
 }

 int getPrice() {
 return price;
 }

 int getQuantity() {
 return quantity;
 }

 bool isAvailable() {
 return quantity > 0;
 }
}
```



```

 }

 void reduceQuantity() {
 if (quantity > 0)
 quantity--;
 }

 void addQuantity(int count) {
 quantity += count;
 }
};

// =====
// INVENTORY
// =====

class Inventory {
 vector<Item*> items;

public:

 void addItem(Item* item) {
 items.push_back(item);
 }

 Item* getItemById(int id) {

 for (auto item : items) {

 if (item->getId() == id)
 return item;
 }

 return nullptr;
 }

 bool isAvailable(int id) {

 Item* item = getItemById(id);

 return item != nullptr &&
 item->isAvailable();
 }

 void reduceItem(int id) {

 Item* item = getItemById(id);

 if (item != nullptr)
 item->reduceQuantity();
 }

 void displayItems() {

 cout << "\n==== ITEMS =====\n";

 for (auto item : items) {

 cout << "ID: "
 << item->getId()
 << " | "

```

```

 << item->getName()
 << " | Price: Rs."
 << item->getPrice()
 << " | Qty: "
 << item->getQuantity()
 << "\n";
 }
}

};

// =====
// PAYMENT
// =====

class Payment {
 int insertedAmount;

public:
 Payment() {
 insertedAmount = 0;
 }

 void addMoney(int amount) {

 insertedAmount += amount;

 cout << "Inserted Rs."
 << amount
 << "\n";

 cout << "Total inserted: Rs."
 << insertedAmount
 << "\n";
 }

 int getInsertedAmount() {
 return insertedAmount;
 }

 bool isEnough(int price) {
 return insertedAmount >= price;
 }

 int calculateChange(int price) {
 return insertedAmount - price;
 }

 void reset() {
 insertedAmount = 0;
 }
};

// =====
// FORWARD DECLARATIONS
// =====

class VendingMachine;
class VendingState;

// =====
// STATE BASE CLASS

```

```
// =====

class VendingState {
public:

 virtual void selectItem(
 VendingMachine* machine,
 int itemId
);

 virtual void insertMoney(
 VendingMachine* machine,
 int amount
);

 virtual void dispense(
 VendingMachine* machine
);

 virtual void cancel(
 VendingMachine* machine
);

 virtual ~VendingState() {}
};

// =====
// MACHINE
// =====

class VendingMachine {

 Inventory* inventory;
 Payment* payment;

 Item* selectedItem;

 VendingState* currentState;

public:

 VendingMachine(
 Inventory* inventory,
 Payment* payment
) {
 this->inventory = inventory;
 this->payment = payment;

 selectedItem = nullptr;
 currentState = nullptr;
 }

 void setState(VendingState* state) {
 currentState = state;
 }

 VendingState* getState() {
 return currentState;
 }

 Inventory* getInventory() {
```

```

 return inventory;
 }

 Payment* getPayment() {
 return payment;
 }

 Item* getSelectedItem() {
 return selectedItem;
 }

 void setSelectedItem(Item* item) {
 selectedItem = item;
 }

 void clearSelectedItem() {
 selectedItem = nullptr;
 }

 // -----
 // User operations delegate to current state
 // -----

 void selectItem(int itemId) {

 currentState->selectItem(
 this,
 itemId
);
 }

 void insertMoney(int amount) {

 currentState->insertMoney(
 this,
 amount
);
 }

 void dispense() {

 currentState->dispense(
 this
);
 }

 void cancel() {

 currentState->cancel(
 this
);
 }
};

// =====
// IDLE STATE
// =====

class IdleState
: public VendingState {

```

```

public:

 void selectItem(
 VendingMachine* machine,
 int itemId
) override;

 void insertMoney(
 VendingMachine* machine,
 int amount
) override {

 cout << "Select item first\n";
 }

 void dispense(
 VendingMachine* machine
) override {

 cout << "Select item first\n";
 }

 void cancel(
 VendingMachine* machine
) override {

 cout << "Nothing to cancel\n";
 }
};

// =====
// PAYMENT STATE
// =====

class PaymentState
 : public VendingState {

public:

 void selectItem(
 VendingMachine* machine,
 int itemId
) override {

 cout << "Item already selected\n";
 }

 void insertMoney(
 VendingMachine* machine,
 int amount
) override {

 machine
 ->getPayment()
 ->addMoney(amount);
 }

 void dispense(
 VendingMachine* machine
) override;

```

```

 void cancel(
 VendingMachine* machine
) override;
};

// =====
// DISPENSE STATE
// =====

class DispenseState
 : public VendingState {

public:

 void selectItem(
 VendingMachine* machine,
 int itemId
) override {

 cout << "Dispensing in progress\n";
 }

 void insertMoney(
 VendingMachine* machine,
 int amount
) override {

 cout << "Dispensing in progress\n";
 }

 void dispense(
 VendingMachine* machine
) override;

 void cancel(
 VendingMachine* machine
) override {

 cout << "Cannot cancel during dispense\n";
 }
};

// =====
// DEFAULT STATE BEHAVIOR
// =====

void VendingState::selectItem(
 VendingMachine* machine,
 int itemId
) {
 cout << "Operation not allowed\n";
}

void VendingState::insertMoney(
 VendingMachine* machine,
 int amount
) {
 cout << "Operation not allowed\n";
}

void VendingState::dispense(

```

```

 VendingMachine* machine
) {
 cout << "Operation not allowed\n";
 }

void VendingState::cancel(
 VendingMachine* machine
) {
 cout << "Operation not allowed\n";
}

// =====
// IDLE STATE IMPLEMENTATION
// =====

void IdleState::selectItem(
 VendingMachine* machine,
 int itemId
) {
 Item* item =
 machine
 ->getInventory()
 ->getItemById(itemId);

 if (item == nullptr) {

 cout << "Invalid item\n";
 return;
 }

 if (!item->isAvailable()) {

 cout << "Item out of stock\n";
 return;
 }

 machine->setSelectedItem(item);

 cout << "Selected: "
 << item->getName()
 << "\n";

 cout << "Price: Rs."
 << item->getPrice()
 << "\n";

 machine->setState(
 new PaymentState()
);
}

// =====
// PAYMENT STATE IMPLEMENTATION
// =====

void PaymentState::dispense(
 VendingMachine* machine
) {
 Item* item =

```

```

 machine->getSelectedItem();

 if (item == nullptr) {

 cout << "No item selected\n";
 return;
 }

 Payment* payment =
 machine->getPayment();

 if (
 !payment->isEnough(
 item->getPrice()
)
) {

 cout << "Insufficient money\n";

 cout << "Required: Rs."
 << item->getPrice()
 << "\n";

 cout << "Inserted: Rs."
 << payment
 ->getInsertedAmount()
 << "\n";

 return;
 }

 // -----
 // State change first
 // -----

 machine->setState(
 new DispenseState()
);

 // -----
 // This now calls DispenseState::dispense()
 // -----

 machine->dispense();
}

void PaymentState::cancel(
 VendingMachine* machine
) {

 int refund =
 machine
 ->getPayment()
 ->getInsertedAmount();

 if (refund > 0) {

 cout << "Refunded Rs."
 << refund
 << "\n";

 }
}

```



```

 machine
 ->getPayment()
 ->reset();

 machine
 ->clearSelectedItem();

 machine->setState(
 new IdleState()
);

 cout << "Transaction cancelled\n";
}

// =====
// DISPENSE STATE IMPLEMENTATION
// =====

void DispenseState::dispense(
 VendingMachine* machine
) {

 Item* item =
 machine->getSelectedItem();

 if (item == nullptr) {

 cout << "No item selected\n";

 machine->setState(
 new IdleState()
);

 return;
 }

 // -----
 // Reduce inventory
 // -----

 machine
 ->getInventory()
 ->reduceItem(
 item->getId()
);

 // -----
 // Calculate change
 // -----

 int change =
 machine
 ->getPayment()
 ->calculateChange(
 item->getPrice()
);

 // -----
 // Dispense item
 // -----

```

```

 cout << "\nDispensing "
 << item->getName()
 << "\n";

 if (change > 0) {

 cout << "Returning change: Rs."
 << change
 << "\n";
 }

 // -----
 // Reset transaction
 // -----

 machine
 ->getPayment()
 ->reset();

 machine
 ->clearSelectedItem();

 // -----
 // Back to Idle
 // -----

 machine->setState(
 new IdleState()
);

 cout << "Transaction completed\n";
}

// =====
// MAIN
// =====

int main() {

 // -----
 // INVENTORY
 // -----

 Inventory* inventory =
 new Inventory();

 inventory->addItem(
 new Item(
 1,
 "Coke",
 40,
 5
)
);

 inventory->addItem(
 new Item(
 2,
 "Chips",
 30,

```

```

 3
)
};

inventory->addItem(
 new Item(
 3,
 "Chocolate",
 50,
 4
)
);

inventory->displayItems();

// -----
// PAYMENT
// -----

Payment* payment =
 new Payment();

// -----
// MACHINE
// -----

VendingMachine machine(
 inventory,
 payment
);

machine.setState(
 new IdleState()
);

// -----
// FLOW
// -----

machine.selectItem(1);

machine.insertMoney(20);
machine.insertMoney(30);

machine.dispense();

// -----
// INVENTORY AFTER PURCHASE
// -----

inventory->displayItems();

return 0;
}

```

The most important part to explain to the interviewer is the state transition:

```

IdleState
|
| selectItem()
↓

```

```

PaymentState
 |
 | enough money + dispense()
 ↓
DispenseState
 |
 | dispense item
 ↓
IdleState

```

And this part often gets asked:

```

machine->setState(
 new DispenseState()
);

machine->dispense();

```

The second line does **not** call `PaymentState::dispense()` again.

`machine->dispense()` internally does:

```

currentState->dispense(this);

```

and `currentState` is now a `DispenseState`, so it calls:

```

DispenseState::dispense()

```

That is the central idea behind the State Pattern here.

15/09/2026, 12:07:12



...

```
class VendingMachine {

 Inventory* inventory;
 Payment* payment;

 Item* selectedItem;

 VendingState* currentState;

public:

 VendingMachine(
 Inventory* inventory,
 Payment* payment
) {
 this->inventory = inventory;
 this->payment = payment;

 selectedItem = nullptr;
 currentState = nullptr;
 }

 void setState(VendingState* state) {
 currentState = state;
 }

 VendingState* getState() {
 return currentState;
 }

 Inventory* getInventory() {
 return inventory;
 }

 Payment* getPayment() {
 return payment;
 }

 Item* getSelectedItem() {
 return selectedItem;
 }

 void setSelectedItem(Item* item) {
 selectedItem = item;
 }

 void clearSelectedItem() {
 selectedItem = nullptr;
 }

 // -----
 // User operations delegate to current state
 // -----

 void selectItem(int itemId) {

 currentState->selectItem(
```

```

 this,
 itemId
);
}

void insertMoney(int amount) {

 currentState->insertMoney(
 this,
 amount
);
}

void dispense() {

 currentState->dispense(
 this
);
}

void cancel() {

 currentState->cancel(
 this
);
}
};

//
=====
// IDLE STATE
//
=====

class IdleState
 : public VendingState {

public:

 void selectItem(
 VendingMachine* machine,
 int itemId
) override;

 void insertMoney(
 VendingMachine* machine,
 int amount
) override {

 cout << "Select item first\n";
 }

 void dispense(
 VendingMachine* machine
) override {

 cout << "Select item first\n";
 }

 void cancel(
 VendingMachine* machine

```

```

) override {

 cout << "Nothing to cancel\n";
 }
};

//
=====
// PAYMENT STATE
//
=====

class PaymentState
 : public VendingState {

public:

 void selectItem(
 VendingMachine* machine,
 int itemId
) override {

 cout << "Item already selected\n";
 }

 void insertMoney(
 VendingMachine* machine,
 int amount
) override {

 machine
 ->getPayment()
 ->addMoney(amount);
 }

 void dispense(
 VendingMachine* machine
) override;

 void cancel(
 VendingMachine* machine
) override;
};

//
=====
// DISPENSE STATE
//
=====

class DispenseState
 : public VendingState {

public:

 void selectItem(
 VendingMachine* machine,
 int itemId
) override {

 cout << "Dispensing in progress\n";

```

```

 }

 void insertMoney(
 VendingMachine* machine,
 int amount
) override {

 cout << "Dispensing in progress\n";
 }

 void dispense(
 VendingMachine* machine
) override;

 void cancel(
 VendingMachine* machine
) override {

 cout << "Cannot cancel during dispense\n";
 }
};

//
=====
// DEFAULT STATE BEHAVIOR
//
=====

void VendingState::selectItem(
 VendingMachine* machine,
 int itemId
) {
 cout << "Operation not allowed\n";
}

void VendingState::insertMoney(
 VendingMachine* machine,
 int amount
) {
 cout << "Operation not allowed\n";
}

void VendingState::dispense(
 VendingMachine* machine
) {
 cout << "Operation not allowed\n";
}

void VendingState::cancel(
 VendingMachine* machine
) {
 cout << "Operation not allowed\n";
}

//
=====
// IDLE STATE IMPLEMENTATION
//
=====

void IdleState::selectItem(

```



```

 VendingMachine* machine,
 int itemId
) {

 Item* item =
 machine
 ->getInventory()
 ->getItemById(itemId);

 if (item == nullptr) {

 cout << "Invalid item\n";
 return;
 }

 if (!item->isAvailable()) {

 cout << "Item out of stock\n";
 return;
 }

 machine->setSelectedItem(item);

 cout << "Selected: "
 << item->getName()
 << "\n";

 cout << "Price: Rs."
 << item->getPrice()
 << "\n";

 machine->setState(
 new PaymentState()
);
 }

//
=====
// PAYMENT STATE IMPLEMENTATION
//
=====

void PaymentState::dispense(
 VendingMachine* machine
) {

 Item* item =
 machine->getSelectedItem();

 if (item == nullptr) {

 cout << "No item selected\n";
 return;
 }

 Payment* payment =
 machine->getPayment();

 if (
 !payment->isEnough(
 item->getPrice()

```

```

)
} {

 cout << "Insufficient money\n";

 cout << "Required: Rs."
 << item->getPrice()
 << "\n";

 cout << "Inserted: Rs."
 << payment
 ->getInsertedAmount()
 << "\n";

 return;
}

// -----
// State change first
// -----

machine->setState(
 new DispenseState()
);

// -----
// This now calls DispenseState::dispense()
// -----

machine->dispense();
}

void PaymentState::cancel(
 VendingMachine* machine
) {

 int refund =
 machine
 ->getPayment()
 ->getInsertedAmount();

 if (refund > 0) {

 cout << "Refunded Rs."
 << refund
 << "\n";
 }

 machine
 ->getPayment()
 ->reset();

 machine
 ->clearSelectedItem();

 machine->setState(
 new IdleState()
);

 cout << "Transaction cancelled\n";
}

```

```

//
=====
// DISPENSE STATE IMPLEMENTATION
//
=====

void DispenseState::dispense(
 VendingMachine* machine
) {

 Item* item =
 machine->getSelectedItem();

 if (item == nullptr) {

 cout << "No item selected\n";

 machine->setState(
 new IdleState()
);

 return;
 }

 // -----
 // Reduce inventory
 // -----

 machine
 ->getInventory()
 ->reduceItem(
 item->getId()
);

 // -----
 // Calculate change
 // -----

 int change =
 machine
 ->getPayment()
 ->calculateChange(
 item->getPrice()
);

 // -----
 // Dispense item
 // -----

 cout << "\nDispensing "
 << item->getName()
 << "\n";

 if (change > 0) {

 cout << "Returning change: Rs."
 << change
 << "\n";
 }
}

```

```

// -----
// Reset transaction
// -----

machine
 ->getPayment()
 ->reset();

machine
 ->clearSelectedItem();

// -----
// Back to Idle
// -----

machine->setState(
 new IdleState()
);

cout << "Transaction completed\n";
}

//
=====
// MAIN
//
=====

int main() {

 // -----
 // INVENTORY
 // -----

 Inventory* inventory =
 new Inventory();

 inventory->addItem(
 new Item(
 1,
 "Coke",
 40,
 5
)
);

 inventory->addItem(
 new Item(
 2,
 "Chips",
 30,
 3
)
);

 inventory->addItem(
 new Item(
 3,
 "Chocolate",
 50,
 4

```

```

)
};

inventory->displayItems();

// -----
// PAYMENT
// -----

Payment* payment =
 new Payment();

// -----
// MACHINE
// -----

VendingMachine machine(
 inventory,
 payment
);

machine.setState(
 new IdleState()
);

// -----
// FLOW
// -----

machine.selectItem(1);

machine.insertMoney(20);
machine.insertMoney(30);

machine.dispense();

// -----
// INVENTORY AFTER PURCHASE
// -----

inventory->displayItems();

return 0;
}
explain this in detail
```

```

15/09/2026, 12:11:18



Yes. This code is mainly showing **State Pattern** in a vending machine. The easiest way to understand it is to follow one complete purchase from start to finish.

1. First understand the core idea

The machine can be in different states:

```

IdleState
  ↓ select item
PaymentState
  ↓ enough money

```

```
DispenseState
  ↓ item dispensed
IdleState
```

The important part is:

```
VendingState* currentState;
```

inside `VendingMachine`.

That pointer decides what `selectItem()`, `insertMoney()`, `dispense()`, and `cancel()` mean **right now**.

So the same call:

```
machine.dispense();
```

can behave differently depending on whether `currentState` is:

```
IdleState
PaymentState
DispenseState
```

2. Why `VendingMachine` contains these members

```
class VendingMachine {

    Inventory* inventory;
    Payment* payment;

    Item* selectedItem;

    VendingState* currentState;
};
```

Each field answers a different question.

```
Inventory* inventory
```

This stores all products and quantities.

Example:

| | | |
|-----------|----------|------------|
| Coke | price 40 | quantity 5 |
| Chips | price 30 | quantity 3 |
| Chocolate | price 50 | quantity 4 |

The machine needs access to inventory for:

```
checking whether item exists
checking quantity
reducing quantity after purchase
```

```
Payment* payment
```

This stores money inserted in the **current transaction**.

Example:

```
User inserts ₹20
User inserts ₹30
```

```
insertedAmount = ₹50
```

It can check:

```
payment->isEnough(price);
```

and calculate:

```
payment->calculateChange(price);
```

```
Item* selectedItem
```

This tells the machine:

Which product is the current customer trying to purchase?

Initially:

```
selectedItem = nullptr;
```

After:

```
machine.selectItem(1);
```

it might become:

```
selectedItem → Coke
```

After purchase/cancellation:

```
selectedItem = nullptr;
```

again.

```
VendingState* currentState
```

This is the main State Pattern variable.

Suppose:

```
currentState = new IdleState();
```

Then:

```
machine.insertMoney(20);
```

eventually calls:

```
IdleState::insertMoney()
```

which says:

```
Select item first
```

But once:

```
currentState = new PaymentState();
```

the exact same:

```
machine.insertMoney(20);
```

calls:

```
PaymentState::insertMoney()
```

which actually accepts money.

That is the heart of the design.

3. Why VendingMachine delegates operations

Look at:

```
void selectItem(int itemId) {  
  
    currentState->selectItem(  
        this,  
        itemId  
    );  
}
```

The machine does **not** itself write:

```
if(state == IDLE) ...  
else if(state == PAYMENT) ...  
else if(state == DISPENSE) ...
```

Instead it says:

“Current state, you decide whether this operation is valid and what should happen.”

Same for:

```
void insertMoney(int amount) {  
    currentState->insertMoney(this, amount);  
}  
  
void dispense() {  
    currentState->dispense(this);  
}  
  
void cancel() {  
    currentState->cancel(this);  
}
```

So `VendingMachine` becomes a **context object** in State Pattern terminology.

4. Why do we pass `this`?

For example:

```
currentState->selectItem(  
    this,  
    itemId  
);
```

Here `this` means:

pointer to the current `VendingMachine` object.

The state needs access to the actual machine so it can do things like:

```
machine->getInventory();
machine->setSelectedItem(item);
machine->getPayment();
machine->setState(...);
```

Without receiving the machine pointer, `IdleState` would know:

```
"someone selected an item"
```

but it would not know **which machine's inventory/state/selected item to modify**.

5. `IdleState`

When the machine starts:

```
No product selected
No transaction active
Waiting for customer
```

So it is in:

```
IdleState
```

Look at its valid operation:

```
void selectItem(
    VendingMachine* machine,
    int itemId
) override;
```

Selecting an item is allowed.

But:

```
void insertMoney(...) override {
    cout << "Select item first\n";
}
```

Money is rejected.

Likewise:

```
void dispense(...) override {
    cout << "Select item first\n";
}
```

and:

```
void cancel(...) override {
    cout << "Nothing to cancel\n";
}
```

This is much cleaner than scattering checks everywhere.

6. What happens inside `IdleState::selectItem()`

Let's use:

```
machine.selectItem(1);
```

Suppose item 1 is Coke.

The call chain is:

```
machine.selectItem(1)
  ↓
currentState->selectItem(this, 1)
  ↓
IdleState::selectItem(machine, 1)
```

Now:

```
Item* item =
  machine
    ->getInventory()
    ->getItemById(itemId);
```

This gets the item from inventory.

Equivalent mental model:

```
machine
  ↓
inventory
  ↓
find item 1
  ↓
Coke
```

Then:

```
if (item == nullptr)
```

checks invalid ID.

Then:

```
if (!item->isAvailable())
```

checks stock.

If valid:

```
machine->setSelectedItem(item);
```

Now:

```
selectedItem → Coke
```

Finally:

```
machine->setState(
  new PaymentState()
);
```

This is the state transition:

IdleState → PaymentState

Why?

Because once a product is selected, the next meaningful step is payment.

7. PaymentState

Now the machine behaves differently.

If user tries:

```
machine.selectItem(2);
```

the state says:

```
cout << "Item already selected\n";
```

But this is valid:

```
machine.insertMoney(20);
```

because:

```
void insertMoney(  
    VendingMachine* machine,  
    int amount  
) override {  
  
    machine  
        ->getPayment()  
        ->addMoney(amount);  
}
```

Suppose user inserts:

```
machine.insertMoney(20);  
machine.insertMoney(30);
```

Then `Payment` becomes:

```
insertedAmount = 50
```

8. Now the important part: `PaymentState::dispense()`

Suppose Coke costs ₹40.

User has inserted ₹50.

Then:

```
machine.dispense();
```

goes to:

```
PaymentState::dispense(machine)
```

First:

```
Item* item =  
    machine->getSelectedItem();
```

gets Coke.

Then:

```
Payment* payment =  
    machine->getPayment();
```

gets current payment.

Then:

```
if (  
    !payment->isEnough(  
        item->getPrice()  
    )  
)
```

checks:

```
insertedAmount >= price?  
50 >= 40?  
yes
```

If insufficient:

```
Required: 40  
Inserted: 20
```

and remain in `PaymentState`.

That means the customer can still insert more money.

9. Why do we do this?

```
machine->setState(  
    new DispenseState()  
);  
  
machine->dispense();
```

This is the part that initially looks strange.

Inside:

```
PaymentState::dispense()
```

we first change:

```
currentState:  
PaymentState → DispenseState
```

Then call:

```
machine->dispense();
```

Now remember:

```
void VendingMachine::dispense() {  
    currentState->dispense(this);  
}
```

But `currentState` is **no longer PaymentState**.

It is:

```
DispenseState
```

So this executes:

```
DispenseState::dispense(machine);
```

Not:

```
PaymentState::dispense(machine);
```

Again.

This is runtime polymorphism.

Flow:

```
PaymentState::dispense()  
    |  
    | enough money  
    ↓  
setState(DispenseState)  
    |  
    ↓  
machine->dispense()  
    |  
    ↓  
currentState->dispense()  
    |  
    ↓  
DispenseState::dispense()
```

10. Why have separate PaymentState and DispenseState?

Because their responsibilities are different.

PaymentState

Answers:

"Can we proceed with the transaction?"

It checks:

```
selected item  
enough money
```

DispenseState

Answers:

"Now actually complete the transaction."

It performs:

```
reduce stock
calculate change
give item
return change
reset transaction
return to idle
```

So:

```
PaymentState
= validation

DispenseState
= execution
```

That's a very useful way to explain it in interview.

11. `DispenseState::dispense()`

First:

```
Item* item =
    machine->getSelectedItem();
```

gets selected Coke.

Then:

```
machine
->getInventory()
->reduceItem(
    item->getId()
);
```

Suppose:

```
Coke quantity = 5
```

After this:

```
Coke quantity = 4
```

12. Calculate change

```
int change =
    machine
    ->getPayment()
    ->calculateChange(
        item->getPrice()
    );
```

Suppose:

```
inserted = 50
price    = 40
```

Then:

```
change = 50 - 40
        = 10
```

So:

```
cout << "Returning change: Rs."
      << change;
```

prints:

```
Returning change: Rs.10
```

13. Reset transaction

This is important.

After a successful purchase:

```
machine
    ->getPayment()
    ->reset();
```

sets:

```
insertedAmount = 0
```

Then:

```
machine
    ->clearSelectedItem();
```

sets:

```
selectedItem = nullptr
```

Then:

```
machine->setState(
    new IdleState()
);
```

Machine becomes ready for next customer.

So:

```
DispenseState → IdleState
```

14. Complete dry run

Start:

```
VendingMachine machine(...);

machine.setState(
    new IdleState()
);
```

State:

```
Idle
selectedItem = NULL
insertedAmount = 0
Coke qty = 5
```

Now:

```
machine.selectItem(1);
```

Call chain:

```
VendingMachine::selectItem()
    ↓
IdleState::selectItem()
```

Coke found.

Then:

```
selectedItem = Coke
state = PaymentState
```

Current situation:

```
State = Payment
Selected = Coke
Price = 40
Money = 0
```

Now:

```
machine.insertMoney(20);
```

Because current state is `PaymentState`:

```
PaymentState::insertMoney()
```

Money:

```
20
```

Then:

```
machine.insertMoney(30);
```

Money:

```
50
```

Now:

```
machine.dispense();
```

Current state:

```
PaymentState
```

so:


```
PaymentState::dispense()
```

checks:

```
50 >= 40
```

yes.

Then:

```
state = DispenseState
```

and:

```
machine->dispense();
```

Now calls:

```
DispenseState::dispense()
```

Then:

```
Coke quantity: 5 → 4  
change = 10
```

Print:

```
Dispensing Coke  
Returning change: Rs.10
```

Then:

```
insertedAmount = 0  
selectedItem = nullptr  
state = IdleState
```

Final:

```
State = IDLE  
Coke qty = 4  
Money = 0  
No selected item
```

15. What happens if money is insufficient?

Suppose:

```
Coke = ₹40  
Inserted = ₹20
```

Then:

```
machine.dispense();
```

enters:

```
PaymentState::dispense()
```

and:

```
payment->isEnough(40)
```

returns false.

It prints:

```
Insufficient money  
Required: Rs.40  
Inserted: Rs.20
```

Notice:

```
state remains PaymentState
```

This is important.

The user can now:

```
machine.insertMoney(20);
```

and try again.

16. What happens on cancel?

Suppose:

```
selected = Coke  
insertedAmount = ₹30  
state = PaymentState
```

Then:

```
machine.cancel();
```

calls:

```
PaymentState::cancel()
```

This does:

```
int refund =  
    machine  
        ->getPayment()  
        ->getInsertedAmount();
```

So:

```
refund = ₹30
```

Then:

```
Refunded Rs.30
```

Then:

```
payment->reset();
```

Money becomes 0.

Then:

```
clearSelectedItem();
```

Then:

```
setState(new IdleState());
```

So:

```
PaymentState → IdleState
```

17. What are these base implementations doing?

You have:

```
void VendingState::selectItem(...) {  
    cout << "Operation not allowed\n";  
}
```

and similar methods.

This gives a **default behavior**.

So a derived state does not necessarily need to implement everything.

However, in your current code most derived states override the functions anyway.

Another design would be to make them all pure virtual:

```
virtual void selectItem(...) = 0;  
virtual void insertMoney(...) = 0;  
virtual void dispense(...) = 0;  
virtual void cancel(...) = 0;
```

Then every state is forced to implement them.

Both approaches are possible.

18. Why declarations and definitions are separate here

Inside `IdleState` :

```
void selectItem(  
    VendingMachine* machine,  
    int itemId  
) override;
```

only declares the function.

Actual definition appears later:

```
void IdleState::selectItem(  
    VendingMachine* machine,  
    int itemId  
) {  
    ...  
}
```

Why?

Because the states refer to one another.

For example:

```
new PaymentState()
```

inside `IdleState` .

And:

```
new DispenseState()
```

inside `PaymentState` .

Splitting declaration and implementation helps deal with class-definition ordering.

19. One issue in this code

This code is interview-friendly, but:

```
machine->setState(  
    new PaymentState()  
);
```

and similar calls keep dynamically allocating state objects.

The old state pointer is never deleted.

So technically this creates **memory leaks**.

For interview simplification, this is often tolerated.

A cleaner version could keep one instance of each state:

```
IdleState idleState;  
PaymentState paymentState;  
DispenseState dispenseState;
```

and switch pointers between them.

But because you've preferred simple C++ without smart pointers, the current version is easier to explain.

If interviewer asks about memory management, mention this tradeoff.

20. Why State Pattern was actually useful here

Without State Pattern, your vending machine might look like:

```
void insertMoney(int amount) {  
  
    if (state == IDLE) {  
        cout << "Select item first";  
    }  
  
    else if (state == PAYMENT) {  
        payment.addMoney(amount);  
    }  
  
    else if (state == DISPENSING) {  
        cout << "Wait";  
    }  
}
```

And every method would have similar checks:

```
selectItem()  
insertMoney()
```

```
dispense()  
cancel()
```

As states increase:

```
IDLE  
PAYMENT  
DISPENSE  
OUT_OF_STOCK  
MAINTENANCE  
RETURN_CHANGE
```

your `if/else` blocks become huge.

State Pattern instead does:

```
Idle behavior  
  → IdleState  
  
Payment behavior  
  → PaymentState  
  
Dispensing behavior  
  → DispenseState
```

So the object's behavior changes when its state object changes.

Interview-ready explanation

You can say:

"VendingMachine acts as the context and contains a pointer to the current vendingState. User actions such as select item, insert money, dispense, and cancel are delegated to that current state. Each concrete state defines which operations are valid and can transition the machine to another state. For example, selecting an available item moves IdleState → PaymentState; successful payment moves PaymentState → DispenseState; after dispensing and resetting the transaction, the machine returns to IdleState."

And if the interviewer asks **why State Pattern**, say:

"Because the same operation should behave differently depending on the machine's current state, and State Pattern avoids large state-dependent if/else blocks."

15/09/2026, 12:11:17



parking lot full code

15/09/2026, 12:38:20



Here's the **full Parking Lot LLD code in C++**, matching the design we finalized earlier:

```
#include <bits/stdc++.h>
using namespace std;

// =====
// ENUMS
// =====

enum VehicleType {
    BIKE,
    CAR,
    TRUCK
};

enum SpotType {
    BIKE_SPOT,
    CAR_SPOT,
    TRUCK_SPOT
};

enum TicketStatus {
    ACTIVE,
    CLOSED
};

// =====
// VEHICLE
// =====

class Vehicle {
    string number;
    VehicleType type;

public:
    Vehicle(string number, VehicleType type) {
        this->number = number;
        this->type = type;
    }

    string getNumber() {
        return number;
    }

    VehicleType getType() {
        return type;
    }
};

// =====
// PARKING SPOT
// =====

class ParkingSpot {
```

```

    int id;
    SpotType type;
    Vehicle* vehicle;

public:
    ParkingSpot(int id, SpotType type) {
        this->id = id;
        this->type = type;
        this->vehicle = nullptr;
    }

    int getId() {
        return id;
    }

    SpotType getType() {
        return type;
    }

    Vehicle* getVehicle() {
        return vehicle;
    }

    bool isAvailable() {
        return vehicle == nullptr;
    }

    bool canPark(Vehicle* v) {

        if (!isAvailable())
            return false;

        if (v->getType() == BIKE &&
            type == BIKE_SPOT)
            return true;

        if (v->getType() == CAR &&
            type == CAR_SPOT)
            return true;

        if (v->getType() == TRUCK &&
            type == TRUCK_SPOT)
            return true;

        return false;
    }

    bool parkVehicle(Vehicle* v) {

        if (!canPark(v))
            return false;

        vehicle = v;
        return true;
    }

    void removeVehicle() {
        vehicle = nullptr;
    }
};

```



```

// =====
// PARKING FLOOR
// =====

class ParkingFloor {
    int floorNumber;
    vector<ParkingSpot*> spots;

public:
    ParkingFloor(int floorNumber) {
        this->floorNumber = floorNumber;
    }

    int getFloorNumber() {
        return floorNumber;
    }

    void addSpot(ParkingSpot* spot) {
        spots.push_back(spot);
    }

    vector<ParkingSpot*>& getSpots() {
        return spots;
    }

    void displayAvailability() {

        cout << "\nFloor "
              << floorNumber
              << "\n";

        for (auto spot : spots) {

            cout << "Spot "
                  << spot->getId()
                  << " -> ";

            if (spot->isAvailable())
                cout << "FREE";
            else
                cout << "OCCUPIED";

            cout << "\n";
        }
    }
};

// =====
// TICKET
// =====

class Ticket {
    int id;

    Vehicle* vehicle;
    ParkingSpot* spot;
    ParkingFloor* floor;

    long long entryTime;

    TicketStatus status;

```

```

public:
    Ticket(
        int id,
        Vehicle* vehicle,
        ParkingSpot* spot,
        ParkingFloor* floor,
        long long entryTime
    ) {
        this->id = id;
        this->vehicle = vehicle;
        this->spot = spot;
        this->floor = floor;
        this->entryTime = entryTime;

        status = ACTIVE;
    }

    int getId() {
        return id;
    }

    Vehicle* getVehicle() {
        return vehicle;
    }

    ParkingSpot* getSpot() {
        return spot;
    }

    ParkingFloor* getFloor() {
        return floor;
    }

    long long getEntryTime() {
        return entryTime;
    }

    TicketStatus getStatus() {
        return status;
    }

    void close() {
        status = CLOSED;
    }
};

// =====
// PARKING SPOT STRATEGY
// =====

class ParkingSpotStrategy {
public:
    virtual pair<ParkingFloor*, ParkingSpot*>
    findSpot(
        vector<ParkingFloor*>& floors,
        Vehicle* vehicle
    ) = 0;

    virtual ~ParkingSpotStrategy() {}
};

```

```

// =====
// FIRST AVAILABLE STRATEGY
// =====

class FirstAvailableSpotStrategy
    : public ParkingSpotStrategy {

public:
    pair<ParkingFloor*, ParkingSpot*>
    findSpot(
        vector<ParkingFloor*>& floors,
        Vehicle* vehicle
    ) override {

        for (auto floor : floors) {

            for (auto spot : floor->getSpots()) {

                if (spot->canPark(vehicle)) {

                    return {
                        floor,
                        spot
                    };
                }
            }
        }

        return {
            nullptr,
            nullptr
        };
    }
};

// =====
// LOWEST FLOOR STRATEGY
// =====

class LowestFloorSpotStrategy
    : public ParkingSpotStrategy {

public:
    pair<ParkingFloor*, ParkingSpot*>
    findSpot(
        vector<ParkingFloor*>& floors,
        Vehicle* vehicle
    ) override {

        ParkingFloor* selectedFloor =
            nullptr;

        ParkingSpot* selectedSpot =
            nullptr;

        int bestFloor =
            INT_MAX;

        for (auto floor : floors) {

```

```

        for (auto spot : floor->getSpots()) {

            if (!spot->canPark(vehicle))
                continue;

            if (
                floor->getFloorNumber()
                < bestFloor
            ) {
                bestFloor =
                    floor->getFloorNumber();

                selectedFloor = floor;
                selectedSpot = spot;
            }
        }

        return {
            selectedFloor,
            selectedSpot
        };
    }
};

// =====
// PRICING STRATEGY
// =====

class PricingStrategy {
public:
    virtual double calculateFee(
        Ticket* ticket,
        long long exitTime
    ) = 0;

    virtual ~PricingStrategy() {}
};

// =====
// HOURLY PRICING
// =====

class HourlyPricingStrategy
    : public PricingStrategy {
public:
    double calculateFee(
        Ticket* ticket,
        long long exitTime
    ) override {

        long long duration =
            exitTime -
            ticket->getEntryTime();

        int hours =
            max(
                1LL,
                (duration + 3599) / 3600
            );
    }
};

```

```

        int rate = 0;

        VehicleType type =
            ticket
                ->getVehicle()
                ->getType();

        if (type == BIKE)
            rate = 10;

        else if (type == CAR)
            rate = 20;

        else if (type == TRUCK)
            rate = 30;

        return hours * rate;
    }
};

// =====
// FLAT PRICING
// =====

class FlatPricingStrategy
    : public PricingStrategy {

public:
    double calculateFee(
        Ticket* ticket,
        long long exitTime
    ) override {

        VehicleType type =
            ticket
                ->getVehicle()
                ->getType();

        if (type == BIKE)
            return 50;

        if (type == CAR)
            return 100;

        return 200;
    }
};

// =====
// PAYMENT STRATEGY
// =====

class PaymentStrategy {
public:
    virtual bool pay(
        double amount
    ) = 0;

    virtual ~PaymentStrategy() {}
};

```

```

// =====
// UPI PAYMENT
// =====

class UPIPayment
    : public PaymentStrategy {

public:
    bool pay(
        double amount
    ) override {

        cout << "Paid Rs."
              << amount
              << " using UPI\n";

        return true;
    }
};

// =====
// CARD PAYMENT
// =====

class CardPayment
    : public PaymentStrategy {

public:
    bool pay(
        double amount
    ) override {

        cout << "Paid Rs."
              << amount
              << " using Card\n";

        return true;
    }
};

// =====
// CASH PAYMENT
// =====

class CashPayment
    : public PaymentStrategy {

public:
    bool pay(
        double amount
    ) override {

        cout << "Paid Rs."
              << amount
              << " using Cash\n";

        return true;
    }
};

```

```
// =====
// PARKING LOT
// =====

class ParkingLot {
    vector<ParkingFloor*> floors;
    vector<Ticket*> tickets;

    ParkingSpotStrategy* spotStrategy;
    PricingStrategy* pricingStrategy;

    int nextTicketId;

public:
    ParkingLot(
        ParkingSpotStrategy* spotStrategy,
        PricingStrategy* pricingStrategy
    ) {
        this->spotStrategy =
            spotStrategy;

        this->pricingStrategy =
            pricingStrategy;

        nextTicketId = 1;
    }

    void addFloor(
        ParkingFloor* floor
    ) {
        floors.push_back(floor);
    }

    void setSpotStrategy(
        ParkingSpotStrategy* strategy
    ) {
        spotStrategy = strategy;
    }

    void setPricingStrategy(
        PricingStrategy* strategy
    ) {
        pricingStrategy = strategy;
    }

// -----
// PARK VEHICLE
// -----

    Ticket* parkVehicle(
        Vehicle* vehicle
    ) {

        auto result =
            spotStrategy->findSpot(
                floors,
                vehicle
            );

        ParkingFloor* floor =
            result.first;
    }
}
```

```

ParkingSpot* spot =
    result.second;

if (spot == nullptr) {

    cout << "No compatible parking spot available\n";
    return nullptr;
}

bool success =
    spot->parkVehicle(vehicle);

if (!success) {

    cout << "Could not park vehicle\n";
    return nullptr;
}

long long entryTime =
    time(nullptr);

Ticket* ticket =
    new Ticket(
        nextTicketId++,
        vehicle,
        spot,
        floor,
        entryTime
    );

tickets.push_back(ticket);

cout << "\nVehicle "
    << vehicle->getNumber()
    << " parked successfully\n";

cout << "Ticket ID: "
    << ticket->getId()
    << "\n";

cout << "Floor: "
    << floor->getFloorNumber()
    << "\n";

cout << "Spot: "
    << spot->getId()
    << "\n";

return ticket;
}

// -----
// UNPARK VEHICLE
// -----

bool unparkVehicle(
    Ticket* ticket,
    PaymentStrategy* paymentStrategy
) {

```



```

    if (ticket == nullptr) {

        cout << "Invalid ticket\n";
        return false;
    }

    if (
        ticket->getStatus()
        == CLOSED
    ) {

        cout << "Ticket already closed\n";
        return false;
    }

    long long exitTime =
        time(nullptr);

    double fee =
        pricingStrategy
            ->calculateFee(
                ticket,
                exitTime
            );

    cout << "\nParking fee = Rs."
        << fee
        << "\n";

    bool paymentSuccess =
        paymentStrategy->pay(fee);

    if (!paymentSuccess) {

        cout << "Payment failed\n";
        return false;
    }

    ticket
        ->getSpot()
        ->removeVehicle();

    ticket->close();

    cout << "Vehicle "
        << ticket
            ->getVehicle()
            ->getNumber()
        << " removed successfully\n";

    return true;
}

// -----
// DISPLAY
// -----

void displayAvailability() {

    cout << "\n==== PARKING STATUS =====\n";

```

```

        for (auto floor : floors) {
            floor->displayAvailability();
        }
    }

// -----
// GET TICKET
// -----

Ticket* getTicketById(
    int ticketId
) {

    for (auto ticket : tickets) {

        if (
            ticket->getId()
            == ticketId
        ) {
            return ticket;
        }
    }

    return nullptr;
}

};

// =====
// MAIN
// =====

int main() {

    // -----
    // FLOORS
    // -----

    ParkingFloor* floor1 =
        new ParkingFloor(1);

    ParkingFloor* floor2 =
        new ParkingFloor(2);

    // -----
    // FLOOR 1 SPOTS
    // -----

    floor1->addSpot(
        new ParkingSpot(
            101,
            BIKE_SPOT
        )
    );

    floor1->addSpot(
        new ParkingSpot(
            102,
            CAR_SPOT
        )
    );
};

```

```

floor1->addSpot(
    new ParkingSpot(
        103,
        CAR_SPOT
    )
);

floor1->addSpot(
    new ParkingSpot(
        104,
        TRUCK_SPOT
    )
);

// -----
// FLOOR 2 SPOTS
// -----

floor2->addSpot(
    new ParkingSpot(
        201,
        BIKE_SPOT
    )
);

floor2->addSpot(
    new ParkingSpot(
        202,
        CAR_SPOT
    )
);

floor2->addSpot(
    new ParkingSpot(
        203,
        TRUCK_SPOT
    )
);

// -----
// STRATEGIES
// -----

ParkingSpotStrategy* spotStrategy =
    new FirstAvailableSpotStrategy();

PricingStrategy* pricingStrategy =
    new HourlyPricingStrategy();

// -----
// PARKING LOT
// -----

ParkingLot parkingLot(
    spotStrategy,
    pricingStrategy
);

parkingLot.addFloor(floor1);
parkingLot.addFloor(floor2);

```

```

// -----
// VEHICLES
// -----

Vehicle* bike =
    new Vehicle(
        "UP70-BIKE-01",
        BIKE
    );

Vehicle* car1 =
    new Vehicle(
        "UP70-CAR-01",
        CAR
    );

Vehicle* car2 =
    new Vehicle(
        "UP70-CAR-02",
        CAR
    );

Vehicle* truck =
    new Vehicle(
        "UP70-TRUCK-01",
        TRUCK
    );

// -----
// INITIAL STATUS
// -----

parkingLot.displayAvailability();

// -----
// PARK VEHICLES
// -----

Ticket* bikeTicket =
    parkingLot.parkVehicle(
        bike
    );

Ticket* carTicket1 =
    parkingLot.parkVehicle(
        car1
    );

Ticket* carTicket2 =
    parkingLot.parkVehicle(
        car2
    );

Ticket* truckTicket =
    parkingLot.parkVehicle(
        truck
    );

// -----
// STATUS AFTER PARKING
// -----

```

```

parkingLot.displayAvailability();

// -----
// UNPARK CAR 1
// -----

PaymentStrategy* upi =
    new UPIPayment();

parkingLot.unparkVehicle(
    carTicket1,
    upi
);

// -----
// STATUS AFTER EXIT
// -----

parkingLot.displayAvailability();

// -----
// CHANGE SPOT STRATEGY
// -----

ParkingSpotStrategy* lowestFloor =
    new LowestFloorSpotStrategy();

parkingLot.setSpotStrategy(
    lowestFloor
);

// -----
// CHANGE PRICING STRATEGY
// -----

PricingStrategy* flatPricing =
    new FlatPricingStrategy();

parkingLot.setPricingStrategy(
    flatPricing
);

// -----
// NEW CAR
// -----

Vehicle* car3 =
    new Vehicle(
        "UP70-CAR-03",
        CAR
    );

Ticket* carTicket3 =
    parkingLot.parkVehicle(
        car3
    );

PaymentStrategy* card =
    new CardPayment();

```

```

    parkingLot.unparkVehicle(
        carTicket3,
        card
    );

    return 0;
}

```

The important architecture to remember is:

```

ParkingLot
|
+-- ParkingFloor
    |
    +-- ParkingSpot
        |
        +-- Vehicle

Ticket
|
+-- Vehicle
+-- ParkingSpot
+-- ParkingFloor

```

And the three strategy abstractions are:

```

ParkingSpotStrategy
→ which spot should I allocate?

PricingStrategy
→ how much should I charge?

PaymentStrategy
→ how should the customer pay?

```

The two main flows are:

```

parkVehicle()
↓
findSpot()
↓
parkVehicle()
↓
create Ticket

```

and

```

unparkVehicle()
↓
calculateFee()
↓
pay()
↓
removeVehicle()
↓
close Ticket

```

Also remember one important design choice: `ParkingSpot` only stores `Vehicle* vehicle`; it does not separately store `bool occupied`. `vehicle == nullptr` itself tells us whether the spot is available.

15/09/2026, 12:39:04



Pasted text.txt

here why use vending state we can simply define differnt states ?

15/09/2026, 13:03:57



Yes, you **can simply keep one enum like** `IDLE`, `PAYMENT`, `DISPENSING` **inside** `VendingMachine` and handle everything with `if/else` or `switch`.

The reason we used `VendingState` is not because different states cannot be represented directly. It is because **the behavior changes a lot depending on state**, and State Pattern keeps that behavior separated. In your code, `VendingMachine` delegates every action to `currentState`. Pasted text.txt

For example, with a simple enum you could do:

```
enum State {
    IDLE,
    PAYMENT,
    DISPENSING
};

class VendingMachine {
    State state;

public:
    void insertMoney(int amount) {
        if (state == IDLE) {
            cout << "Select item first\n";
        }
        else if (state == PAYMENT) {
            payment->addMoney(amount);
        }
        else if (state == DISPENSING) {
            cout << "Dispensing in progress\n";
        }
    }
};
```

This is completely valid.

The problem is that you would need the same state checking in **every operation**:

```
selectItem()
insertMoney()
dispense()
cancel()
```

So eventually you get:

```
selectItem()
    if IDLE ...
    else if PAYMENT ...
    else if DISPENSING ...

insertMoney()
    if IDLE ...
    else if PAYMENT ...
    else if DISPENSING ...
```



```

dispense()
    if IDLE ...
    else if PAYMENT ...
    else if DISPENSING ...

cancel()
    if IDLE ...
    else if PAYMENT ...
    else if DISPENSING ...

```

That becomes messy when you add more states like:

```

OUT_OF_STOCK
MAINTENANCE
RETURN_CHANGE
ERROR

```

With State Pattern, each class owns only its own behavior:

```

IdleState
    selectItem → allowed
    insertMoney → reject
    dispense → reject
    cancel → nothing

PaymentState
    selectItem → reject
    insertMoney → allowed
    dispense → validate payment
    cancel → refund

DispenseState
    selectItem → reject
    insertMoney → reject
    dispense → actually dispense
    cancel → reject

```

That is exactly what your current `IdleState`, `PaymentState`, and `DispenseState` are doing. `Pasted text.txt`

So the distinction is:

```

Enum state
→ stores only "which state am I in?"

State Pattern
→ stores "which state am I in?"
+
puts that state's behavior inside its own class

```

For a **small vending machine**, an enum is actually simpler and perfectly fine.

For an LLD interview, I would say:

"If the machine has only a few simple states, an enum with switch cases is enough. I would use State Pattern when state-specific behavior grows, because it avoids repeated `if/else` checks and keeps each state's behavior isolated."

So yes - **you can simply define different states**. `VendingState` is mainly for cleaner extensibility and separation of behavior, not because it is mandatory.

15/09/2026, 13:03:57



Here's the **ATM LLD** in the same style as Parking Lot and Vending Machine: first the interview explanation, then entities, flow, SOLID, tradeoffs, and finally full C++ code.

1. What to say first in the interview

"I'll design an ATM that supports card insertion, PIN authentication, balance inquiry, cash withdrawal, cash deposit, and card ejection. I'll separate bank-account operations from physical cash handling, so `BankService` manages account-related actions while `CashDispenser` manages the notes inside the ATM."

The main flow is:

```
Insert Card
  ↓
Enter PIN
  ↓
Authenticate
  ↓
Choose Transaction
  ↓
Withdraw / Deposit / Balance Inquiry
  ↓
Eject Card
```

2. Main entities

Core classes:

```
ATM
Card
Account
BankService
CashDispenser
Transaction
```

Transaction subclasses:

```
WithdrawalTransaction
DepositTransaction
BalanceInquiryTransaction
```

For a fresher interview, this is enough.

3. `Account`

Represents the user's bank account.

```
class Account {
    int id;
    double balance;
};
```

Responsibilities:

```
getBalance()
debit()
credit()
```

Example:

```
balance = 10000

withdraw 2000
→ balance = 8000
```

4. Card

The card identifies the account and is used for authentication.

```
class Card {
    string cardNumber;
    int pin;
    int accountId;
};
```

In a real banking system, PIN would not be stored like plain text, but for LLD this is acceptable simplification.

5. BankService

This is the abstraction between ATM and the banking backend.

```
class BankService {
    vector<Account*> accounts;
};
```

Responsibilities:

```
authenticate card
find account
debit account
credit account
```

Why not let ATM directly maintain all accounts?

Because ATM should coordinate ATM behavior, not own the bank's account database.

6. CashDispenser

This is very important.

Account balance means:

logical money in bank.

CashDispenser means:

actual physical notes inside ATM.

Example:

```
₹500 → 10 notes
₹200 → 10 notes
```

₹100 → 20 notes

So:

```
map<int, int, greater<int>> cash;
```

means:

denomination → count

Important methods:

```
addCash()  
getTotalCash()  
canDispense()  
dispense()
```

7. ATM

ATM is the coordinator.

```
class ATM {  
    ATMState state;  
  
    Card* currentCard;  
    Account* currentAccount;  
  
    BankService* bankService;  
    CashDispenser* cashDispenser;  
};
```

Responsibilities:

```
insert card  
authenticate  
check balance  
withdraw  
deposit  
eject card
```

It does not calculate everything itself.

It delegates:

```
Account work  
→ BankService  
  
physical cash  
→ CashDispenser  
  
transaction-specific behavior  
→ Transaction subclasses
```

8. ATM states

For a simple ATM:

```
enum ATMState {  
    IDLE,
```

```
CARD_INSERTED,  
AUTHENTICATED  
};
```

Meaning:

```
IDLE  
→ waiting for card  
  
CARD_INSERTED  
→ card inserted but not authenticated  
  
AUTHENTICATED  
→ PIN verified, transactions allowed
```

Here I would use an enum first.

Unlike our Vending Machine, State Pattern is not mandatory unless interviewer wants complex state behavior.

9. Transaction hierarchy

Base:

```
class Transaction {  
protected:  
    int id;  
    Account* account;  
  
public:  
    virtual bool execute() = 0;  
};
```

Then:

```
Transaction  
|  
+-- WithdrawalTransaction  
+-- DepositTransaction  
+-- BalanceInquiryTransaction
```

Why?

Because every transaction:

```
works on an Account  
has an execute operation
```

but the actual behavior differs.

10. Withdrawal flow

This is the most important part.

Suppose:

```
Account balance = ₹10,000  
ATM cash = ₹6,000  
  
User requests ₹1,800
```

Steps:

1. Check authenticated
2. Check amount > 0
3. Check account balance
4. Check ATM can make exact ₹1,800
5. Debit account
6. Dispense notes
7. Show remaining balance

For ₹1800:

₹500 × 3 = 1500
₹200 × 1 = 200
₹100 × 1 = 100

Total = 1800

11. Full C++ code

```
#include <bits/stdc++.h>
using namespace std;

// =====
// ENUM
// =====

enum ATMState {
    IDLE,
    CARD_INSERTED,
    AUTHENTICATED
};

// =====
// ACCOUNT
// =====

class Account {
    int id;
    double balance;

public:
    Account(int id, double balance) {
        this->id = id;
        this->balance = balance;
    }

    int getId() {
        return id;
    }

    double getBalance() {
        return balance;
    }

    bool debit(double amount) {

        if (amount <= 0)
            return false;

        if (balance < amount)
```

```

        return false;

        balance -= amount;

        return true;
    }

    bool credit(double amount) {

        if (amount <= 0)
            return false;

        balance += amount;

        return true;
    }
};

// =====
// CARD
// =====

class Card {
    string cardNumber;
    int pin;
    int accountId;

public:
    Card(
        string cardNumber,
        int pin,
        int accountId
    ) {
        this->cardNumber = cardNumber;
        this->pin = pin;
        this->accountId = accountId;
    }

    string getCardNumber() {
        return cardNumber;
    }

    int getPin() {
        return pin;
    }

    int getAccountId() {
        return accountId;
    }
};

// =====
// BANK SERVICE
// =====

class BankService {
    vector<Account*> accounts;

public:

    void addAccount(Account* account) {

```

```

        accounts.push_back(account);
    }

    Account* getAccountById(int id) {

        for (auto account : accounts) {

            if (account->getId() == id)
                return account;
        }

        return nullptr;
    }

    bool authenticate(
        Card* card,
        int enteredPin
    ) {

        if (card == nullptr)
            return false;

        return card->getPin() == enteredPin;
    }

    Account* getAccount(Card* card) {

        if (card == nullptr)
            return nullptr;

        return getAccountById(
            card->getAccountId()
        );
    }

    bool withdraw(
        Account* account,
        double amount
    ) {

        if (account == nullptr)
            return false;

        return account->debit(amount);
    }

    bool deposit(
        Account* account,
        double amount
    ) {

        if (account == nullptr)
            return false;

        return account->credit(amount);
    }
};

```

```

// =====
// CASH DISPENSER
// =====

```

```

class CashDispenser {

    map<int, int, greater<int>> cash;

public:

    void addCash(
        int denomination,
        int count
    ) {
        cash[denomination] += count;
    }

    int getTotalCash() {

        int total = 0;

        for (auto &entry : cash) {

            total +=
                entry.first *
                entry.second;
        }

        return total;
    }

    bool canDispense(int amount) {

        if (amount <= 0)
            return false;

        if (getTotalCash() < amount)
            return false;

        int remaining = amount;

        for (auto &entry : cash) {

            int denomination =
                entry.first;

            int count =
                entry.second;

            int needed =
                remaining / denomination;

            int take =
                min(
                    needed,
                    count
                );

            remaining -=
                take * denomination;
        }

        return remaining == 0;
    }
}

```



```

bool dispense(int amount) {

    if (!canDispense(amount))
        return false;

    int remaining = amount;

    map<int, int, greater<int>>
        notesToDispense;

    for (auto &entry : cash) {

        int denomination =
            entry.first;

        int count =
            entry.second;

        int needed =
            remaining / denomination;

        int take =
            min(
                needed,
                count
            );

        if (take > 0) {

            notesToDispense[
                denomination
            ] = take;

            remaining -=
                take * denomination;
        }
    }

    if (remaining != 0)
        return false;

    for (
        auto &entry :
            notesToDispense
    ) {
        cash[entry.first]
            -= entry.second;
    }

    cout << "\nCash dispensed:\n";

    for (
        auto &entry :
            notesToDispense
    ) {

        cout << "Rs."
            << entry.first
            << " x "
            << entry.second

```

```

        << "\n";
    }

    return true;
}

void displayCash() {

    cout << "\n==== ATM CASH =====\n";

    for (auto &entry : cash) {

        cout << "Rs."
              << entry.first
              << " -> "
              << entry.second
              << " notes\n";
    }

    cout << "Total = Rs."
          << getTotalCash()
          << "\n";
    }
};

// =====
// TRANSACTION BASE CLASS
// =====

class Transaction {
protected:
    int id;
    Account* account;

public:

    Transaction(
        int id,
        Account* account
    ) {
        this->id = id;
        this->account = account;
    }

    virtual bool execute() = 0;

    virtual ~Transaction() {}
};

// =====
// BALANCE INQUIRY
// =====

class BalanceInquiryTransaction
    : public Transaction {

public:

    BalanceInquiryTransaction(
        int id,
        Account* account

```

```

    ) : Transaction(id, account) {}

    bool execute() override {

        if (account == nullptr)
            return false;

        cout << "Current balance: Rs."
              << account->getBalance()
              << "\n";

        return true;
    }
};

// =====
// DEPOSIT
// =====

class DepositTransaction
    : public Transaction {

    double amount;
    BankService* bankService;

public:

    DepositTransaction(
        int id,
        Account* account,
        double amount,
        BankService* bankService
    ) : Transaction(id, account) {

        this->amount = amount;

        this->bankService =
            bankService;
    }

    bool execute() override {

        if (amount <= 0) {

            cout << "Invalid amount\n";
            return false;
        }

        bool success =
            bankService->deposit(
                account,
                amount
            );

        if (!success)
            return false;

        cout << "Deposited Rs."
              << amount
              << "\n";
    }
};

```

```

        cout << "New balance: Rs."
              << account->getBalance()
              << "\n";

        return true;
    }
};

// =====
// WITHDRAWAL
// =====

class WithdrawalTransaction
    : public Transaction {

    int amount;

    BankService* bankService;

    CashDispenser* cashDispenser;

public:

    WithdrawalTransaction(
        int id,
        Account* account,
        int amount,
        BankService* bankService,
        CashDispenser* cashDispenser
    ) : Transaction(id, account) {

        this->amount = amount;

        this->bankService =
            bankService;

        this->cashDispenser =
            cashDispenser;
    }

    bool execute() override {

        if (amount <= 0) {

            cout << "Invalid amount\n";
            return false;
        }

        // -----
        // 1. CHECK ACCOUNT BALANCE
        // -----

        if (
            account->getBalance()
            < amount
        ) {

            cout << "Insufficient account balance\n";

            return false;
        }
    }
};

```

```

// -----
// 2. CHECK ATM CAN DISPENSE
// -----

if (
    !cashDispenser
        ->canDispense(amount)
) {

    cout << "ATM cannot dispense exact amount\n";

    return false;
}

// -----
// 3. DEBIT ACCOUNT
// -----

bool debitSuccess =
    bankService->withdraw(
        account,
        amount
    );

if (!debitSuccess) {

    cout << "Account debit failed\n";

    return false;
}

// -----
// 4. DISPENSE CASH
// -----

bool dispenseSuccess =
    cashDispenser->dispense(
        amount
    );

if (!dispenseSuccess) {

    /*
        Demo rollback.

        In real banking systems this would be
        handled through transaction/reconciliation.
    */

    account->credit(amount);

    cout << "Cash dispensing failed\n";

    return false;
}

cout << "Withdrawal successful\n";

cout << "Remaining balance: Rs."
    << account->getBalance()

```

```

        << "\n";

        return true;
    }
};

// =====
// ATM
// =====

class ATM {

    ATMState state;

    Card* currentCard;

    Account* currentAccount;

    BankService* bankService;

    CashDispenser* cashDispenser;

    int nextTransactionId;

public:

    ATM(
        BankService* bankService,
        CashDispenser* cashDispenser
    ) {

        this->bankService =
            bankService;

        this->cashDispenser =
            cashDispenser;

        state = IDLE;

        currentCard = nullptr;
        currentAccount = nullptr;

        nextTransactionId = 1;
    }

    // -----
    // INSERT CARD
    // -----

    bool insertCard(Card* card) {

        if (state != IDLE) {

            cout << "ATM already in use\n";
            return false;
        }

        currentCard = card;

        state = CARD_INSERTED;
    }
};

```

```

        cout << "Card inserted\n";

        return true;
    }

    // -----
    // AUTHENTICATE
    // -----

    bool authenticate(int pin) {

        if (
            state != CARD_INSERTED
        ) {

            cout << "Insert card first\n";
            return false;
        }

        bool valid =
            bankService->authenticate(
                currentCard,
                pin
            );

        if (!valid) {

            cout << "Invalid PIN\n";
            return false;
        }

        currentAccount =
            bankService->getAccount(
                currentCard
            );

        if (
            currentAccount == nullptr
        ) {

            cout << "Account not found\n";
            return false;
        }

        state = AUTHENTICATED;

        cout << "Authentication successful\n";

        return true;
    }

    // -----
    // BALANCE
    // -----

    bool checkBalance() {

        if (
            state != AUTHENTICATED
        ) {

```

```

        cout << "Authentication required\n";
        return false;
    }

    BalanceInquiryTransaction
        transaction(
            nextTransactionId++,
            currentAccount
        );

    return transaction.execute();
}

// -----
// WITHDRAW
// -----

bool withdraw(int amount) {

    if (
        state != AUTHENTICATED
    ) {

        cout << "Authentication required\n";
        return false;
    }

    WithdrawalTransaction
        transaction(
            nextTransactionId++,
            currentAccount,
            amount,
            bankService,
            cashDispenser
        );

    return transaction.execute();
}

// -----
// DEPOSIT
// -----

bool deposit(double amount) {

    if (
        state != AUTHENTICATED
    ) {

        cout << "Authentication required\n";
        return false;
    }

    DepositTransaction
        transaction(
            nextTransactionId++,
            currentAccount,
            amount,
            bankService
        );

```



```

        return transaction.execute();
    }

    // -----
    // EJECT CARD
    // -----

    void ejectCard() {

        if (state == IDLE) {

            cout << "No card inserted\n";
            return;
        }

        currentCard = nullptr;
        currentAccount = nullptr;

        state = IDLE;

        cout << "Card ejected\n";
    }
};

// =====
// MAIN
// =====

int main() {

    // -----
    // ACCOUNT
    // -----

    Account* account =
        new Account(
            101,
            10000
        );

    // -----
    // CARD
    // -----

    Card* card =
        new Card(
            "123456789",
            1234,
            101
        );

    // -----
    // BANK
    // -----

    BankService* bank =
        new BankService();

    bank->addAccount(
        account
    );

```

```

// -----
// CASH DISPENSER
// -----

CashDispenser* dispenser =
    new CashDispenser();

dispenser->addCash(
    500,
    10
);

dispenser->addCash(
    200,
    10
);

dispenser->addCash(
    100,
    20
);

dispenser->displayCash();

// -----
// ATM
// -----

ATM atm(
    bank,
    dispenser
);

// -----
// FLOW
// -----

atm.insertCard(card);

atm.authenticate(1234);

atm.checkBalance();

atm.withdraw(1800);

atm.checkBalance();

atm.deposit(1000);

atm.checkBalance();

dispenser->displayCash();

atm.ejectCard();

return 0;
}

```

12. Dry run

Initially:

```
Account balance = 10000
```

ATM:

```
500 × 10 = 5000
```

```
200 × 10 = 2000
```

```
100 × 20 = 2000
```

```
Total ATM cash = 9000
```

Now:

```
atm.insertCard(card);
```

State:

```
IDLE → CARD_INSERTED
```

Then:

```
atm.authenticate(1234);
```

PIN matches.

State:

```
CARD_INSERTED → AUTHENTICATED
```

Now:

```
atm.withdraw(1800);
```

First:

```
Account balance >= 1800?
```

```
10000 >= 1800 ✓
```

Then:

```
Can ATM create ₹1800?
```

CashDispenser tries:

```
500 → take 3  
remaining = 300
```

```
200 → take 1  
remaining = 100
```

```
100 → take 1  
remaining = 0
```

So:

```
500 × 3  
200 × 1  
100 × 1
```

Then account is debited:

10000 → 8200

and ATM note counts become:

500: 10 → 7
200: 10 → 9
100: 20 → 19

13. Why both Account and CashDispenser?

This is one of the most important interview questions.

Suppose:

Account balance = ₹1,00,000

but ATM contains only:

₹5,000

User cannot withdraw:

₹10,000

even though the bank account has enough money.

So:

Account
→ user's available bank balance

CashDispenser
→ physical ATM cash

Both must pass.

14. Why `BankService` ?

You might ask:

Why can't ATM directly call `account->debit()` ?

It can in a very small design.

But conceptually:

ATM
↓
Bank backend
↓
Account

is more realistic.

`BankService` provides that boundary:

```
bankService->authenticate(...)
bankService->withdraw(...)
bankService->deposit(...)
```

It also helps SRP.

15. Why transaction subclasses?

You could simply write:

```
ATM::withdraw()  
ATM::deposit()  
ATM::checkBalance()
```

and that is valid for a small system.

But transaction classes become useful because later you might add:

```
FundTransferTransaction  
MiniStatementTransaction  
ChangePinTransaction  
FastCashTransaction
```

Then:

```
Transaction  
|  
├─ Withdrawal  
├─ Deposit  
├─ BalanceInquiry  
├─ FundTransfer  
└─ MiniStatement
```

becomes cleaner.

So if interviewer asks why:

"I separated transactions because they have different execution logic and are likely to grow independently."

16. Why not use State Pattern here?

You certainly can.

Something like:

```
IdleState  
CardInsertedState  
AuthenticatedState  
TransactionState
```

But for this version:

```
ATMState state;
```

plus a few checks is simpler.

For example:

```
if (state != AUTHENTICATED)  
    return false;
```

There are only a few states.

So my interview answer would be:

"I'll start with an enum because ATM state behavior is currently simple. If states and transitions become more complex, I can refactor it into State Pattern."

Exactly the same tradeoff we discussed for the vending machine.

17. SOLID principles

S — Single Responsibility

```
Account
→ account balance

Card
→ card information

BankService
→ banking operations

CashDispenser
→ physical cash

Transaction subclasses
→ transaction-specific behavior

ATM
→ coordinates ATM workflow
```

O — Open/Closed

Add:

```
class FundTransferTransaction
: public Transaction {
};
```

without modifying existing transaction classes.

L — Liskov

Wherever:

```
Transaction*
```

is expected, you can use:

```
WithdrawalTransaction
DepositTransaction
BalanceInquiryTransaction
```

I — Interface Segregation

Transaction only requires:

```
execute()
```

We don't force every transaction to implement:

```
dispenseCash()  
depositCash()  
printStatement()
```

D — Dependency Inversion

Conceptually ATM depends on services:

```
BankService  
CashDispenser
```

rather than containing their implementation logic itself.

If we wanted stronger DIP, `BankService` itself could be an interface, but for fresher LLD this is sufficient.

18. Important tradeoffs

Greedy denomination selection

We use largest note first:

```
500  
200  
100
```

This is simple and efficient.

But for arbitrary denominations, greedy can fail even if a solution exists.

For standard ATM denominations, this is normally acceptable for interview design.

`vector<Account*>` vs hash map

Currently:

```
vector<Account*> accounts;
```

Searching account:

```
O(n)
```

A better production structure:

```
unordered_map<int, Account*> accounts;
```

gives approximately:

```
O(1)
```

lookup.

Start simple, mention optimization if asked.

Withdrawal atomicity

This is the strongest follow-up.

Bad case:

```
Account debited successfully
    ↓
ATM hardware fails
    ↓
No cash comes out
```

Now user has lost money logically.

So real systems need:

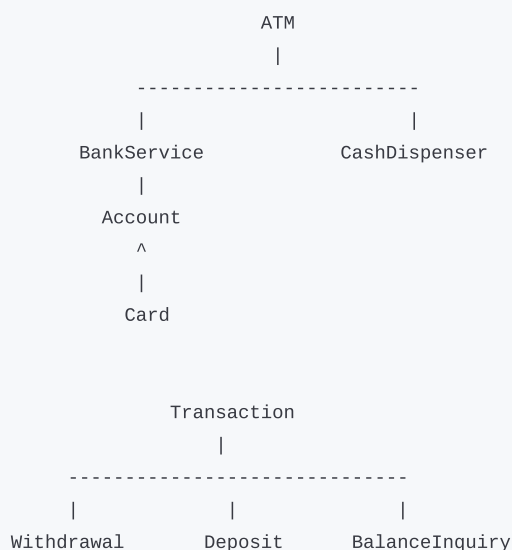
```
transaction
rollback
reconciliation
cash reservation
```

Our code does a simple rollback:

```
account->credit(amount);
```

but real ATM systems require much stronger guarantees.

19. Class diagram



And the core relationship is:

```
ATM
→ coordinates

BankService
→ logical bank money

CashDispenser
→ physical cash

Transaction
→ operation being performed
```

The shortest interview summary is:

*"ATM coordinates the session, **BankService** handles account operations, **CashDispenser** handles physical notes, and transaction subclasses encapsulate different transaction behaviors. The withdrawal flow validates both account balance and*

ATM cash before debiting and dispensing."

15/09/2026, 13:15:56



mars rover explain the question then its answer?

15/09/2026, 13:43:41



Absolutely. **Mars Rover** is a classic LLD/interview problem because it tests entity modeling, movement logic, direction handling, command processing, and extensibility.

1. First understand the question

A rover is placed on a 2D grid.

Example:

```
Grid = 5 x 5
```

```
Initial position:
```

```
x = 1
```

```
y = 2
```

```
Direction = NORTH
```

The rover receives commands such as:

```
L
```

```
R
```

```
M
```

where:

```
L = turn left
```

```
R = turn right
```

```
M = move one step forward
```

Example:

```
Initial:
```

```
(1,2), NORTH
```

```
Commands:
```

```
L M L M
```

Execution:

```
NORTH
```

```
L
```

```
↓
```

```
WEST
```

```
M
```

```
↓
```

```
(0,2)
```

```
L
```

```
↓
```

```
SOUTH
```

```
M
```

```
↓  
(0,1)
```

So final position:

```
(0,1), SOUTH
```

2. Typical requirements

For a basic Mars Rover problem:

1. Rover is placed on a rectangular grid.
2. Rover has x, y coordinates.
3. Rover has a direction:
NORTH, SOUTH, EAST, WEST.
4. Rover understands:
L = turn left
R = turn right
M = move forward.
5. Rover cannot leave the grid.
6. Optionally, grid can contain obstacles.

For an interview, first ask:

“Should the rover stop at grid boundaries or wrap around?”

and:

“Do we need obstacle handling?”

For now, we'll assume:

```
No wrapping.  
No obstacles initially.
```

3. What are the main entities?

The obvious entities are:

```
Rover  
Grid  
Position  
Direction  
Command
```

But we should avoid unnecessary classes.

A clean baseline is:

```
Rover  
Grid  
Direction enum
```

Position can simply be:

```
int x;  
int y;
```

inside Rover.

No need for a separate `Position` class unless position develops more responsibility.

4. Direction enum

Use:

```
enum Direction {  
    NORTH,  
    EAST,  
    SOUTH,  
    WEST  
};
```

Why this order?

Because turning right becomes easy:

```
NORTH → EAST → SOUTH → WEST → NORTH
```

Numerically:

```
0 → 1 → 2 → 3 → 0
```

So:

```
direction = (direction + 1) % 4;
```

for right turn.

For left:

```
direction = (direction + 3) % 4;
```

Why `+3`?

Because:

```
NORTH = 0  
  
0 - 1  
would become -1
```

So instead:

```
(0 + 3) % 4 = 3 = WEST
```

5. Grid

The grid knows its boundaries.

```
class Grid {  
    int width;  
    int height;  
};
```

Main responsibility:

```
bool isValid(int x, int y);
```

For example:

```
bool isValid(int x, int y) {  
    return x >= 0 &&  
           x < width &&  
           y >= 0 &&  
           y < height;  
}
```

If grid is:

```
5 x 5
```

valid coordinates are:

```
x = 0..4  
y = 0..4
```

6. Rover

Rover needs:

```
class Rover {  
    int x;  
    int y;  
  
    Direction direction;  
  
    Grid* grid;  
};
```

Its responsibilities:

```
turnLeft()  
turnRight()  
move()  
executeCommand()  
executeCommands()
```

This is the central class.

7. Movement logic

Suppose:

```
NORTH
```

Moving means:

```
y++
```

For:

```
SOUTH
```

means:

```
y--
```

For:

EAST

means:

x++

For:

WEST

means:

x--

So:

```
if (direction == NORTH)
    newY++;

else if (direction == SOUTH)
    newY--;

else if (direction == EAST)
    newX++;

else if (direction == WEST)
    newX--;
```

Then validate using Grid.

8. Complete basic answer

```
#include <bits/stdc++.h>
using namespace std;

// =====
// DIRECTION
// =====

enum Direction {
    NORTH,
    EAST,
    SOUTH,
    WEST
};

// =====
// GRID
// =====

class Grid {
    int width;
    int height;

public:

    Grid(int width, int height) {
        this->width = width;
```

```

        this->height = height;
    }

    bool isValid(
        int x,
        int y
    ) {

        return x >= 0 &&
            x < width &&
            y >= 0 &&
            y < height;
    }
};

// =====
// ROVER
// =====

class Rover {
    int x;
    int y;

    Direction direction;

    Grid* grid;

public:

    Rover(
        int x,
        int y,
        Direction direction,
        Grid* grid
    ) {
        this->x = x;
        this->y = y;
        this->direction = direction;
        this->grid = grid;
    }

    // -----
    // TURN LEFT
    // -----

    void turnLeft() {

        direction =
            static_cast<Direction>(
                (direction + 3) % 4
            );
    }

    // -----
    // TURN RIGHT
    // -----

    void turnRight() {

        direction =
            static_cast<Direction>(

```

```

        (direction + 1) % 4
    );
}

// -----
// MOVE FORWARD
// -----

void move() {

    int newX = x;
    int newY = y;

    if (direction == NORTH)
        newY++;

    else if (direction == SOUTH)
        newY--;

    else if (direction == EAST)
        newX++;

    else if (direction == WEST)
        newX--;

    if (
        grid->isValid(
            newX,
            newY
        )
    ) {
        x = newX;
        y = newY;
    }

    else {
        cout << "Cannot move outside grid\n";
    }
}

// -----
// EXECUTE ONE COMMAND
// -----

void executeCommand(char command) {

    if (command == 'L') {
        turnLeft();
    }

    else if (command == 'R') {
        turnRight();
    }

    else if (command == 'M') {
        move();
    }

    else {
        cout << "Invalid command\n";
    }
}

```



```

}

// -----
// EXECUTE MULTIPLE COMMANDS
// -----

void executeCommands(
    string commands
) {

    for (char command : commands) {
        executeCommand(command);
    }
}

// -----
// DISPLAY
// -----

void displayPosition() {

    cout << "("
         << x
         << ", "
         << y
         << ") ";

    if (direction == NORTH)
        cout << "NORTH";

    else if (direction == SOUTH)
        cout << "SOUTH";

    else if (direction == EAST)
        cout << "EAST";

    else
        cout << "WEST";

    cout << "\n";
}
};

// =====
// MAIN
// =====

int main() {

    Grid* grid =
        new Grid(
            5,
            5
        );

    Rover rover(
        1,
        2,
        NORTH,
        grid
    );
}

```

```
    rover.displayPosition();

    rover.executeCommands(
        "LMLMLMLMM"
    );

    rover.displayPosition();

    return 0;
}
```

9. Dry run

Initial:

(1,2), NORTH

Commands:

L M L M L M L M M

Step-by-step:

Start:
(1,2) NORTH

L

WEST

M

(0,2) WEST

L

SOUTH

M

(0,1) SOUTH

L

EAST

M

(1,1) EAST

L

NORTH

M

(1,2) NORTH

M

```
(1,3) NORTH
```

Final:

```
(1,3), NORTH
```

10. Is this enough for LLD?

For a fresher interview, yes as a first version.

But the interviewer may ask:

“What if tomorrow we add many commands?”

For example:

```
L
R
M
B = move backward
S = scan
P = take picture
```

Now:

```
if(command == ...)
else if(...)
else if(...)
```

starts growing.

This is where **Command Pattern** can be useful.

11. Command Pattern version idea

You can create:

```
class Command {
public:
    virtual void execute(
        Rover* rover
    ) = 0;
};
```

Then:

```
Command
|
+-- MoveCommand
+-- LeftCommand
+-- RightCommand
```

For example:

```
class MoveCommand
: public Command {

public:
```

```
void execute(
    Rover* rover
) override {

    rover->move();
}
};
```

Then a command processor maps:

```
'M' → MoveCommand
'L' → LeftCommand
'R' → RightCommand
```

But I would **not start with this** unless interviewer asks for extensibility.

Baseline:

```
char commands + switch/if
```

is simpler.

12. What if obstacles exist?

Now Grid can maintain:

```
set<pair<int,int>> obstacles;
```

Example:

```
Obstacle at (1,3)
```

Grid gets:

```
bool isBlocked(
    int x,
    int y
);
```

Then move checks:

```
if (
    grid->isValid(newX,newY) &&
    !grid->isBlocked(newX,newY)
)
```

Otherwise Rover stays where it is.

So responsibility remains correct:

```
Grid
→ knows valid terrain

Rover
→ knows how to move
```

13. What if grid wraps around?

Some Mars Rover variants say:

If rover crosses one edge, it appears on the opposite edge.

Example width 5:

```
x values:  
0 1 2 3 4
```

Moving east from:

```
x = 4
```

goes to:

```
x = 0
```

Then:

```
newX =  
(newX + width) % width;
```

Same for `y`.

This is a requirement question you should clarify.

14. SOLID

SRP

```
Grid  
→ boundaries and terrain  
  
Rover  
→ rover state + movement  
  
Command classes, if used  
→ individual command behavior
```

OCP

If using Command Pattern:

```
MoveCommand  
LeftCommand  
RightCommand
```

and tomorrow:

```
ScanCommand  
TakePictureCommand
```

can be added without changing existing commands.

LSP

Any concrete:

```
Command*
```

can be replaced by:

```
MoveCommand
LeftCommand
RightCommand
```

and still executed through the same interface.

ISP

Command interface stays tiny:

```
execute()
```

No unrelated methods.

DIP

If we introduce command abstractions, command processor depends on:

```
Command*
```

rather than directly hardcoding each command class.

15. Important tradeoffs

The simplest design:

```
Rover + Grid + enum
```

is easier and probably enough for a fresher interview.

Using classes for every direction:

```
NorthDirection
SouthDirection
EastDirection
WestDirection
```

would usually be overengineering unless each direction had substantial unique behavior.

Similarly, Command Pattern is justified only if command types are expected to grow.

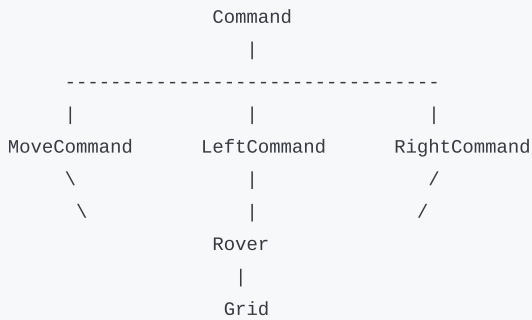
16. Class diagram

```

    Grid
    |
    |
    Rover
-----
x
y
direction
grid
-----
move()
turnLeft()
turnRight()
executeCommand()
```

```
Direction
-----
NORTH
EAST
SOUTH
WEST
```

If using Command Pattern:



17. Best interview explanation

You can say:

*"The core entities are **Rover** and **Grid**. **Rover** owns its current coordinates and direction, while **Grid** owns boundary and obstacle information. Turning only changes direction, while movement calculates the next coordinate and asks the grid whether that coordinate is valid. For a small fixed command set, I'd use simple command characters; if commands become extensible, I'd introduce the Command Pattern."*

That is a strong, simple fresher-level Mars Rover LLD answer.

15/09/2026, 13:43:40



```
...
static_cast<Direction>()
what is this >
...
```

15/09/2026, 13:54:36



`static_cast<Direction>()` is C++ syntax for **explicitly converting a value into the enum type** `Direction`.

In our Mars Rover code:

```
enum Direction {
    NORTH,
    EAST,
    SOUTH,
    WEST
};
```

internally these are basically:

```
NORTH = 0
EAST  = 1
SOUTH = 2
WEST  = 3
```

Now this expression:

```
(direction + 1) % 4
```

produces an **integer**.

Example:

```
direction = EAST = 1

(direction + 1) % 4
= (1 + 1) % 4
= 2
```

But `2` is an `int`, while our variable is:

```
Direction direction;
```

So we convert that integer back to the enum:

```
direction =
    static_cast<Direction>(2);
```

which means:

```
2 → SOUTH
```

That is why we write:

```
direction =
    static_cast<Direction>(
```



```
(direction + 1) % 4  
);
```

For turning right:

```
NORTH(0)  
↓  
EAST(1)  
↓  
SOUTH(2)  
↓  
WEST(3)  
↓  
NORTH(0)
```

Why not just write this?

```
direction = (direction + 1) % 4;
```

Because the right side is an `int`, while `direction` expects a `Direction`. C++ doesn't safely perform that conversion automatically here.

So:

```
static_cast<Direction>(value)
```

basically means:

"Compiler, I intentionally want to treat this value as a `Direction`."

A simpler alternative, if you don't like casting, is to write the turn logic explicitly:

```
if (direction == NORTH)  
    direction = EAST;  
else if (direction == EAST)  
    direction = SOUTH;  
else if (direction == SOUTH)  
    direction = WEST;  
else  
    direction = NORTH;
```

For an interview, both are completely fine.

15/09/2026, 13:54:35



```
resolve the problem using if (direction == NORTH)
&#x20; direction = EAST;
else if (direction == EAST)
&#x20; direction = SOUTH;
else if (direction == SOUTH)
&#x20; direction = WEST;
else
&#x20; direction = NORTH;
```

15/09/2026, 13:55:46



Yes — we can solve the Mars Rover problem without `static_cast` and just use explicit `if/else` for direction changes.

```
#include <bits/stdc++.h>
using namespace std;

// =====
// DIRECTION
// =====

enum Direction {
    NORTH,
    EAST,
    SOUTH,
    WEST
};

// =====
// GRID
// =====

class Grid {
    int width;
    int height;

public:

    Grid(int width, int height) {
        this->width = width;
        this->height = height;
    }

    bool isValid(int x, int y) {

        return x >= 0 &&
            x < width &&
            y >= 0 &&
            y < height;
    }
};

// =====
// ROVER
// =====

class Rover {
    int x;
    int y;
```

```

    Direction direction;

    Grid* grid;

public:

    Rover(
        int x,
        int y,
        Direction direction,
        Grid* grid
    ) {
        this->x = x;
        this->y = y;
        this->direction = direction;
        this->grid = grid;
    }

    // -----
    // TURN RIGHT
    // -----

    void turnRight() {

        if (direction == NORTH)
            direction = EAST;

        else if (direction == EAST)
            direction = SOUTH;

        else if (direction == SOUTH)
            direction = WEST;

        else if (direction == WEST)
            direction = NORTH;
    }

    // -----
    // TURN LEFT
    // -----

    void turnLeft() {

        if (direction == NORTH)
            direction = WEST;

        else if (direction == WEST)
            direction = SOUTH;

        else if (direction == SOUTH)
            direction = EAST;

        else if (direction == EAST)
            direction = NORTH;
    }

    // -----
    // MOVE FORWARD
    // -----

```

```

void move() {

    int newX = x;
    int newY = y;

    if (direction == NORTH) {
        newY++;
    }

    else if (direction == SOUTH) {
        newY--;
    }

    else if (direction == EAST) {
        newX++;
    }

    else if (direction == WEST) {
        newX--;
    }

    if (
        grid->isValid(
            newX,
            newY
        )
    ) {
        x = newX;
        y = newY;
    }

    else {
        cout << "Cannot move outside grid\n";
    }
}

// -----
// EXECUTE ONE COMMAND
// -----

void executeCommand(char command) {

    if (command == 'L') {
        turnLeft();
    }

    else if (command == 'R') {
        turnRight();
    }

    else if (command == 'M') {
        move();
    }

    else {
        cout << "Invalid command\n";
    }
}

// -----
// EXECUTE ALL COMMANDS

```

```

// -----

void executeCommands(string commands) {

    for (char command : commands) {
        executeCommand(command);
    }
}

// -----
// DISPLAY POSITION
// -----

void displayPosition() {

    cout << "("
         << x
         << ", "
         << y
         << ") ";

    if (direction == NORTH)
        cout << "NORTH";

    else if (direction == SOUTH)
        cout << "SOUTH";

    else if (direction == EAST)
        cout << "EAST";

    else if (direction == WEST)
        cout << "WEST";

    cout << "\n";
}
};

// =====
// MAIN
// =====

int main() {

    Grid* grid =
        new Grid(
            5,
            5
        );

    Rover rover(
        1,
        2,
        NORTH,
        grid
    );

    cout << "Initial position:\n";

    rover.displayPosition();

    string commands =

```

```

        "LMLMLMLMM";

    rover.executeCommands(
        commands
    );

    cout << "Final position:\n";

    rover.displayPosition();

    return 0;
}

```

For the input:

```

Start:
(1,2), NORTH

Commands:
LMLMLMLMM

```

the rover moves like this:

```

(1,2) NORTH
L → WEST
M → (0,2)

L → SOUTH
M → (0,1)

L → EAST
M → (1,1)

L → NORTH
M → (1,2)

M → (1,3)

```

So final answer:

```

(1,3), NORTH

```

For an interview, this `if/else` version is actually easier to explain than the `static_cast` version.

15/09/2026, 13:55:45



why using enum not the class for direction if we have to use class then how

15/09/2026, 14:03:56



We use an `enum` for `Direction` because in the simple Mars Rover design, direction is just a **fixed label/state**:

```
NORTH
EAST
SOUTH
WEST
```

There is no extra data or complex behavior attached to each direction yet.

So this is enough:

```
enum Direction {
    NORTH,
    EAST,
    SOUTH,
    WEST
};
```

Then Rover itself handles the behavior:

```
void turnRight();
void turnLeft();
void move();
```

So the responsibility is:

```
Direction enum
→ only tells which direction rover is facing

Rover
→ decides how movement changes for that direction
```

That is why enum is simpler.

If we make `Direction` a class, we normally do it because **each direction has its own behavior**.

For example:

```
NorthDirection
→ moving means y++
→ left becomes West
→ right becomes East

SouthDirection
→ moving means y--
→ left becomes East
→ right becomes West
```

Then we can model it like State/Strategy-style polymorphism.

Example:

```
#include <bits/stdc++.h>
using namespace std;
```

```

class Rover;

// =====
// DIRECTION BASE CLASS
// =====

class Direction {
public:

    virtual void move(
        Rover* rover
    ) = 0;

    virtual Direction* turnLeft() = 0;

    virtual Direction* turnRight() = 0;

    virtual string getName() = 0;

    virtual ~Direction() {}
};

// Forward declarations
class NorthDirection;
class SouthDirection;
class EastDirection;
class WestDirection;

// =====
// ROVER
// =====

class Rover {
    int x;
    int y;

    Direction* direction;

public:

    Rover(
        int x,
        int y,
        Direction* direction
    ) {
        this->x = x;
        this->y = y;
        this->direction = direction;
    }

    int getX() {
        return x;
    }

    int getY() {
        return y;
    }

    void setX(int x) {
        this->x = x;
    }
}

```



```

void setY(int y) {
    this->y = y;
}

void setDirection(
    Direction* direction
) {
    this->direction = direction;
}

void move() {
    direction->move(this);
}

void turnLeft() {
    direction =
        direction->turnLeft();
}

void turnRight() {
    direction =
        direction->turnRight();
}

void display() {

    cout << "("
        << x
        << ", "
        << y
        << ")"
        << direction->getName()
        << "\n";
}
};

```

Now each direction class owns its movement behavior.

```

class NorthDirection
    : public Direction {

public:

    void move(
        Rover* rover
    ) override {

        rover->setY(
            rover->getY() + 1
        );
    }

    Direction* turnLeft() override;

    Direction* turnRight() override;

    string getName() override {
        return "NORTH";
    }
};

```

```

class SouthDirection
: public Direction {

public:

    void move(
        Rover* rover
    ) override {

        rover->setY(
            rover->getY() - 1
        );
    }

    Direction* turnLeft() override;

    Direction* turnRight() override;

    string getName() override {
        return "SOUTH";
    }
};

```

```

class EastDirection
: public Direction {

public:

    void move(
        Rover* rover
    ) override {

        rover->setX(
            rover->getX() + 1
        );
    }

    Direction* turnLeft() override;

    Direction* turnRight() override;

    string getName() override {
        return "EAST";
    }
};

```

```

class WestDirection
: public Direction {

public:

    void move(
        Rover* rover
    ) override {

        rover->setX(
            rover->getX() - 1
        );
    }
};

```

```

    Direction* turnLeft() override;

    Direction* turnRight() override;

    string getName() override {
        return "WEST";
    }
};

```

Then define transitions:

```

Direction* NorthDirection::turnLeft() {
    return new WestDirection();
}

Direction* NorthDirection::turnRight() {
    return new EastDirection();
}

Direction* SouthDirection::turnLeft() {
    return new EastDirection();
}

Direction* SouthDirection::turnRight() {
    return new WestDirection();
}

Direction* EastDirection::turnLeft() {
    return new NorthDirection();
}

Direction* EastDirection::turnRight() {
    return new SouthDirection();
}

Direction* WestDirection::turnLeft() {
    return new SouthDirection();
}

Direction* WestDirection::turnRight() {
    return new NorthDirection();
}

```

Then:

```

int main() {

    Rover rover(
        1,
        2,
        new NorthDirection()
    );

    rover.display();

    rover.turnLeft();

    rover.move();

    rover.display();
}

```

```
    return 0;  
}
```

The key difference is:

ENUM approach

Direction = just data/state

NORTH

EAST

SOUTH

WEST

Rover contains:

if(direction == NORTH) ...

versus:

CLASS approach

Direction = behavior

NorthDirection

→ knows how north moves

→ knows left/right transitions

SouthDirection

→ knows how south moves

...

So in interview terms:

Use `enum` when direction is only a fixed value and the behavior is simple.

Use classes when each direction has meaningful behavior of its own and you want to avoid repeated `if/else` checks.

For Mars Rover, I would start with `enum`. If the interviewer asks, "Can you remove direction-based conditionals or make directions extensible?", then I would move to the class-based design.

15/09/2026, 14:03:57



```
\#include <bits/stdc++.h>
using namespace std;
```

```
//
```

```
// DIRECTION
```

```
//
```

```
enum Direction {
    NORTH,
    EAST,
    SOUTH,
    WEST
};
```

```
//
```

```
// GRID
```

```
//
```

```
class Grid {
    int width;
    int height;

public:
    Grid(int width, int height) {
        this->width = width;
        this->height = height;
    }
}
```

```
bool isValid(int x, int y) {
```

```
    return x >= 0 &&
           x < width &&
           y >= 0 &&
           y < height;
}
```

```
};
```

```
//
```

```
// ROVER
```

```
//
```

```
class Rover {
    int x;
    int y;
    Direction direction;

    Grid* grid;
}
```

```

public:
    ...

    Rover(
        int x,
        int y,
        Direction direction,
        Grid* grid
    ) {
        this->x = x;
        this->y = y;
        this->direction = direction;
        this->grid = grid;
    }

// -----
// TURN RIGHT
// -----

void turnRight() {

    if (direction == NORTH)
        direction = EAST;

    else if (direction == EAST)
        direction = SOUTH;

    else if (direction == SOUTH)
        direction = WEST;

    else if (direction == WEST)
        direction = NORTH;
}

// -----
// TURN LEFT
// -----

void turnLeft() {

    if (direction == NORTH)
        direction = WEST;

    else if (direction == WEST)
        direction = SOUTH;

    else if (direction == SOUTH)
        direction = EAST;

    else if (direction == EAST)
        direction = NORTH;
}

// -----
// MOVE FORWARD
// -----

void move() {

    int newX = x;
    int newY = y;

```

```

        if (direction == NORTH) {
            newY++;
        }

        else if (direction == SOUTH) {
            newY--;
        }

        else if (direction == EAST) {
            newX++;
        }

        else if (direction == WEST) {
            newX--;
        }

        if (
            grid->isValid(
                newX,
                newY
            )
        ) {
            x = newX;
            y = newY;
        }

        else {
            cout << "Cannot move outside grid\n";
        }
    }

// -----
// EXECUTE ONE COMMAND
// -----

void executeCommand(char command) {

    if (command == 'L') {
        turnLeft();
    }

    else if (command == 'R') {
        turnRight();
    }

    else if (command == 'M') {
        move();
    }

    else {
        cout << "Invalid command\n";
    }
}

// -----
// EXECUTE ALL COMMANDS
// -----

void executeCommands(string commands) {

    for (char command : commands) {

```

```

        executeCommand(command);
    }
}

// -----
// DISPLAY POSITION
// -----

void displayPosition() {

    cout << "("
         << x
         << ", "
         << y
         << ") ";

    if (direction == NORTH)
        cout << "NORTH";

    else if (direction == SOUTH)
        cout << "SOUTH";

    else if (direction == EAST)
        cout << "EAST";

    else if (direction == WEST)
        cout << "WEST";

    cout << "\n";
}
...

};

//
=====
// MAIN
//
=====

int main() {
    ...

    Grid* grid =
        new Grid(
            5,
            5
        );

    Rover rover(
        1,
        2,
        NORTH,
        grid
    );

    cout << "Initial position:\n";

    rover.displayPosition();

    string commands =
        "LMLMLMLMM";

```



```

rover.executeCommands(
    commands
);

cout << "Final position:\n";

rover.displayPosition();

return 0;
...

}

```

in this grid has a relationship with rover or rover has a with grid ? composition or aggregation

15/09/2026, 14:07:31



In **your code**, `Rover` has a relationship with `Grid` because `Rover` stores:

```
Grid* grid;
```

So:

```
Rover ----> Grid
```

The rover knows which grid it is moving on.

This is best modeled as **aggregation / association**, not composition.

Why not composition? Because the `Grid` can exist independently of a `Rover`.

In your `main()`:

```

Grid* grid = new Grid(5, 5);

Rover rover(
    1,
    2,
    NORTH,
    grid
);

```

Notice the grid is created **before** the rover and passed into it.

If the rover is destroyed, the grid can still exist.

That means lifecycle is independent:

```

Grid can exist without Rover
Rover uses Grid

```

So this is aggregation-like:

```
Rover o----- Grid
```

or more simply an association:

```
Rover -----> Grid
```

In UML, I would actually call it **association** first, because `Rover` just holds a pointer/reference to an externally created `Grid`. Composition would look more like this:

```
class Rover {
    Grid grid;

    public:
        Rover(...) : grid(5, 5) {
        }
    };
```

Now `Grid` is physically owned by `Rover`.

If `Rover` dies:

```
Rover destroyed
  ↓
Grid destroyed too
```

That would be:

```
Rover ◆----- Grid
```

But that design is actually less logical for Mars Rover, because usually:

```
Grid
→ represents the environment

Rover
→ exists inside / moves on that environment
```

So the better relationship is:

```
Grid
  ↑
  |
Rover
```

or:

```
Rover -----> Grid
```

with aggregation/association semantics.

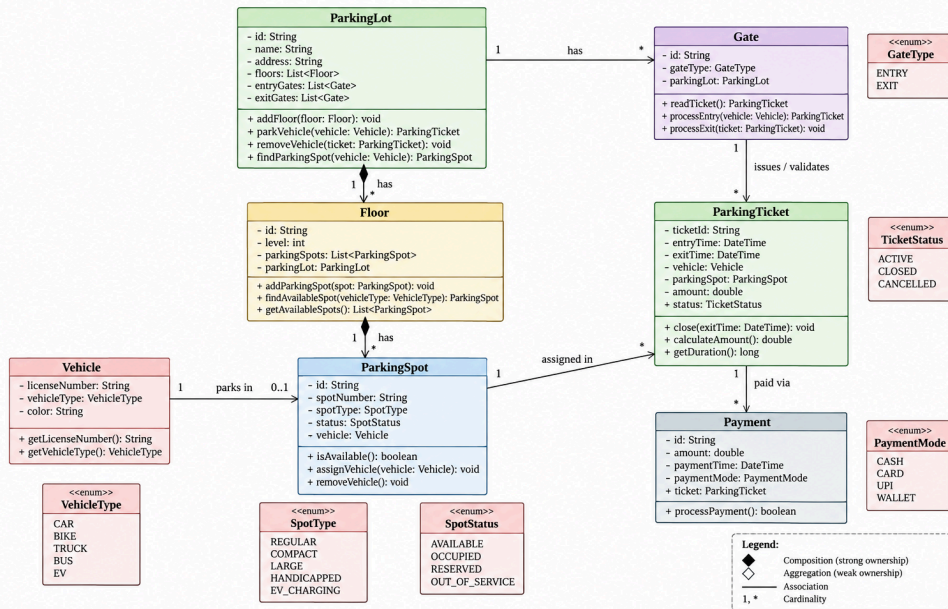
A good interview explanation is:

"Rover has an association with Grid because it needs the grid to validate movement, but the grid has an independent lifecycle and can exist without the rover. So I would not model it as composition."

15/09/2026, 14:07:30



Parking Lot - UML Class Diagram



15/09/2026, 15:04:02